FPT UNIVERSITY

FPT UNIVERSITY
SoftwareDesignDocument

Frevia

GroupBar	
Group Members	Pham Kha Vy - CE182254 Nguyen Tien Thuan - CE181024 Tran Phat Tai - CE181632 Vo Nhut Tin - CE180619 Nguyen Lu Nhat Huy - CE181196
Supervisor	Mrs. Cao Hong Nga
Ext Supervisor	
Capstone Project code	FRE

- Can Tho, October 2026 -

Table of Contents

1 Overall Description **9**

1.1 Assumptions 9

1.2 Design Constraints 10

1.2.1 Platform Compatibility 10

1.2.2 Integration Complexity 10

1.2.3 Performance Requirements 10

1.3 Technology Suggestion 11

1.3.1 NestJS Framework 12

1.3.2 Next.js and React 19 Frontend 12

1.3.3 Prisma ORM and PostgreSQL 12

1.3.4 Supporting Services 12

2 System Architecture Design **13**

2.1 Overall Architecture 13

2.2 System Architecture 14

2.2.1 API Module Structure 14

2.3 Package Diagram 16

3 System Detailed Design **17**

3.1 View Forum Category List 17

3.2 View Forum Category Detail 18

3.3 View Post List 19

3.4 View Post Detail 21

3.5 Create Post 23

3.6 Update Post 24

3.7 Delete Post 26

3.8 Like Post 28

3.9 Unlike Post 30

3.10 View Post Likes 32

3.11 View Post Comments 33

3.12 Create Comment 35

3.13 Update Comment 37

3.14 Delete Comment 38

3.15 Like Comment 40

3.16 Unlike Comment 41

3.17 Report Content 43

3.18 Freelancer Profile Management 46
3.18.1 UC-07.01 – View Freelancer Profile 47
3.18.2 UC-07.02 – Update Freelancer Profile 48
3.19 Freelancer Skill Management 49
3.19.1 UC-08.01 – View Freelancer Skill List 50
3.19.2 UC-08.02 – Add Freelancer Skill 51
3.19.3 UC-08.03 – Delete Freelancer Skill 52
3.20 Portfolio Management 53
3.20.1 UC-09.01 – View Portfolio List 54
3.20.2 UC-09.02 – Add Portfolio 55
3.20.3 UC-09.03 – View Portfolio Detail 56
3.20.4 UC-09.04 – Update Portfolio 57
3.20.5 UC-09.05 – Delete Portfolio 58
3.21 Authentication 59
3.21.1 UC-01 – Login with Google 60
3.21.2 UC-02 – Login 61
3.21.3 UC-03 – Logout 62
3.21.4 UC-04 – Register 63
3.21.5 UC-05 – Forgot Password 64
3.22 Role Management 64
3.22.1 UC-06.01 – View Role List 66
3.22.2 UC-06.02 – View Role Detail 67
3.22.3 UC-06.03 – Create Role 68
3.22.4 UC-06.04 – Update Role 69
3.22.5 UC-06.05 – Delete Role 70
3.22.6 UC-06.06 – Update Role Permissions 71
3.23 View Conversation List 71
3.24 Start Conversation 73
3.25 View Conversation Detail 75
3.26 Send Message 77
3.27 Mark Message as Read 79
3.28 Delete/Recall Message 81
3.29 Pin/Unpin Conversation 83
3.30 Send File 85
3.31 Identity Verification Management 87
3.31.1 UC-18.01 – Upload Identity Documents 88
3.31.2 UC-18.02 – View Verification Status 89

3.32 Client Profile Management 90
3.32.1 UC-19.01 – View Client Profile 91
3.32.2 UC-19.02 – Update Company Information 92
3.33 Social Link Management 93
3.33.1 UC-20.01 – View Social Link List 94
3.33.2 UC-20.02 – Add Social Link 95
3.33.3 UC-20.03 – Delete Social Link 96
3.34 Favorite Freelancer Management 97
3.34.1 UC-21.01 – View Favorite Freelancer List 98
3.34.2 UC-21.02 – Add Freelancer to Favorites 99
3.34.3 UC-21.03 – Remove Freelancer from Favorites 100
3.35 Permission Management 101
3.35.1 UC-15.01 – View Permission List 102
3.35.2 UC-15.02 – View Permission Detail 103
3.36 Session Management 103
3.36.1 UC-16.01 – View Session List 105
3.36.2 UC-16.02 – View Session Detail 106
3.36.3 UC-16.03 – Revoke Session 107
3.37 Forum Category Management 108
3.37.1 UC-17.01 – Create Forum Category 109
3.37.2 UC-17.02 – Update Forum Category 110
3.37.3 UC-17.03 – Delete Forum Category 111
3.37.4 UC-17.06 – Restore Forum Category 112
3.37.5 UC-14.01 – View Forum Category List 113
3.37.6 UC-14.02 – View Forum Category Detail 114
3.38 View Moderation Queue 114
3.39 View Moderation Item Detail 116
3.40 Resolve Moderation Item 118
3.41 View ID Verification Request List 119
3.42 View ID Verification Request Detail 121
3.43 Approve ID Verification 122
3.44 Reject ID Verification 124
3.45 View Deleted Posts/Comments List 126
3.46 View Deleted Posts/Comments Detail 128
3.47 Restore Deleted Posts/Comments 130
3.48 Following Freelancer Management 132
3.48.1 UC-29.01 – View Following List 134

3.48.2 UC-29.02 – Follow Freelancer . . . 135
3.48.3 UC-29.03 – Unfollow Freelancer . . . 136
3.49 Notification Management . . . 137
3.49.1 UC-30.01 – View Notification List . . . 138
3.49.2 UC-30.02 – Mark Notification as Read . . . 139
3.49.3 UC-30.03 – Mark All Notifications as Read . . . 140
3.49.4 UC-30.04 – Delete Notification . . . 141
3.50 Account Role Management . . . 141
3.50.1 UC-31.01 – Switch Role . . . 143
3.50.2 UC-31.02 – Join New Role . . . 144
3.51 User Management . . . 145
3.51.1 UC-28.01 – View User List . . . 146
3.51.2 UC-28.02 – View User Detail . . . 147
3.51.3 UC-28.03 – Create User . . . 148
3.51.4 UC-28.04 – Update User . . . 149
3.51.5 UC-28.05 – Update Client Profile . . . 150
3.51.6 UC-28.06 – Update Freelancer Basic Info . . . 151
3.51.7 UC-28.07 – Update Freelancer Skills . . . 152
3.51.8 UC-28.08 – Update Freelancer Portfolio . . . 153
3.52 Review Management . . . 154
3.52.1 UC-39.01 – View Review List . . . 155
3.52.2 UC-39.02 – View Review Detail . . . 156
3.52.3 UC-39.03 – Write Review . . . 157
3.52.4 UC-39.04 – Update Review . . . 158
3.52.5 UC-39.05 – Delete Review . . . 159
3.53 Review Response Management . . . 160
3.53.1 UC-40.01 – Respond to Review . . . 161
3.53.2 UC-40.02 – Update Review Response . . . 162
3.53.3 UC-40.03 – Delete Review Response . . . 163
3.54 General Account Profile Management . . . 164
3.54.1 UC-41.01 – View Profile . . . 165
3.54.2 UC-41.02 – Update Profile . . . 166
3.54.3 UC-41.03 – Change Password . . . 167
3.54.4 UC-41.04 – Upload Avatar . . . 168
3.55 Banner Management . . . 169
3.55.1 UC-37.01 – View Banner List . . . 170
3.55.2 UC-37.02 – View Banner Detail . . . 171
3.55.3 UC-37.03 – Create Banner . . . 172

3.55.4 UC-37.04 – Update Banner . . . 173
3.55.5 UC-37.05 – Delete Banner . . . 174
3.56 Skill Management . . . 175
3.56.1 UC-38.01 – View Skill List . . . 176
3.56.2 UC-38.02 – View Skill Detail . . . 177
3.56.3 UC-38.03 – Create Skill . . . 178
3.56.4 UC-38.04 – Update Skill . . . 179
3.56.5 UC-38.05 – Delete Skill . . . 180
3.56.6 UC-38.06 – Restore Skill . . . 181
3.57 Profile Update Request Management . . . 181
3.57.1 UC-47.01 – View Pending Profile Update Request List . . . 183
3.57.2 UC-47.02 – View Profile Update Request Detail . . . 184
3.57.3 UC-47.03 – Approve Profile Update Request . . . 185
3.57.4 UC-47.04 – Reject Profile Update Request . . . 186
3.57.5 UC-47.05 – View Profile Update Request History . . . 187
3.58 Expert Profile Management . . . 187
3.58.1 UC-47.06 – Update Profile Expert . . . 189
3.59 Freelancer CV Management . . . 190
3.59.1 UC-07.03 – Upload CV . . . 191
3.59.2 UC-07.04 – View CV . . . 192
3.59.3 UC-07.05 – Replace CV . . . 193
3.59.4 UC-07.06 – Delete CV . . . 194
3.60 Expert Profile Management . . . 195
3.60.1 UC-28.09 – Update Expert Profile . . . 196
3.61 Job Category Management . . . 197
3.61.1 UC-46.01 – View Job Category List . . . 198
3.61.2 UC-46.02 – View Job Category Detail . . . 199
3.61.3 UC-46.03 – Create Job Category . . . 200
3.61.4 UC-46.04 – Update Job Category . . . 201
3.61.5 UC-46.05 – Delete Job Category . . . 202
4 Class Specification . . . 203
4.1 ForumController Class . . . 203
4.2 ForumService Class . . . 205
4.3 ForumRepository Class . . . 209
4.4 ForumLikeController Class . . . 213
4.5 ForumLikeService Class . . . 213
4.6 ForumLikeRepository Class . . . 214

4.7 ForumCommentController Class 216
4.8 ForumCommentService Class 218
4.9 ForumCommentRepository Class 219
4.10 ForumReportController Class 222
4.11 ForumReportService Class 224
4.12 ForumReportRepository Class 226
Supporting DTO classes for reports 228
4.13 ProfileController Class 229
4.14 ProfileService Class 230
4.15 ProfileRepository Class 233
4.16 PortfolioController Class 235
4.17 PortfolioService Class 237
4.18 PortfolioRepository Class 239
4.19 AuthController Class 240
4.20 AuthService Class 243
4.21 AuthRepository Class 247
4.22 RolesController Class 250
4.23 RolesService Class 251
4.24 RolesRepository Class 253
4.25 ConversationController Class 255
4.26 ConversationService Class 257
4.27 ConversationRepository Class 261
4.28 ConversationsGateway Class 264
4.29 IdentityVerificationController Class 266
4.30 ClientProfileController Class 268
4.31 SocialLinkController Class 269
4.32 FavoriteFreelancerController Class 270
4.33 FollowingFreelancerController Class 272
4.34 AccountProfileService Class 273
4.35 AccountProfileRepository Class 279
4.36 PermissionsController Class 283
4.37 PermissionsService Class 283
4.38 PermissionsRepository Class 284
4.39 SessionsController Class 285
4.40 SessionsService Class 286

4.41 SessionsRepository Class 288
4.42 Forum Category Administration (Shared Classes) 289
4.43 ForumAdminController Class 289
4.44 ForumAdminService Class 293
4.45 ForumAdminRepository Class 297
4.46 IdentityVerificationsAdminController Class 301
4.47 IdentityVerificationsAdminService Class 303
4.48 IdentityVerificationsAdminRepository Class 305
4.49 FollowingFreelancerController Class 306
4.50 AccountProfileService Class 307
4.51 AccountProfileRepository Class 309
4.52 NotificationController Class 311
4.53 NotificationService Class 312
4.54 NotificationRepository Class 313
4.55 AuthController Class 315
4.56 AuthService Class 316
4.57 AuthRepository Class 317
4.58 UsersController Class 318
4.59 UsersService Class 321
4.60 UsersRepository Class 324
4.61 ReviewController Class 327
4.62 ReviewService Class 330
4.63 ReviewRepository Class 332
4.64 GeneralAccountProfileController Class 335
4.65 AccountProfileService Class 337
4.66 AccountProfileRepository Class 339
4.67 BannersAdminController Class 341
4.68 BannersAdminService Class 343
4.69 BannersAdminRepository Class 344
4.70 SkillsAdminController Class 346
4.71 SkillsAdminService Class 347
4.72 SkillsAdminRepository Class 349
4.73 ProfileRevisionController Class 350
4.74 ProfileRevisionAdminController Class 351
4.75 ProfileRevisionService Class 353

4.76 ProfileRevisionRepository Class . . . 356
4.77 ExpertProfileController Class . . . 358
4.78 ExpertProfileService Class . . . 359
4.79 ExpertProfileRepository Class . . . 360
4.80 ProfileController Class . . . 361
4.81 ProfileService Class . . . 363
4.82 ProfileRepository Class . . . 364
4.83 AdminExpertProfileController Class . . . 365
4.84 ExpertProfileService Class . . . 366
4.85 ExpertProfileRepository Class . . . 368
4.86 JobCategoriesAdminController Class . . . 369
4.87 JobCategoriesAdminService Class . . . 371
4.88 JobCategoriesAdminRepository Class . . . 373

5 Database Design . . . 375

5.1 Profile, Account and Review Persistence – Nguyen Lu Nhat Huy . . . 375
5.1.1 Profile . . . 375
5.1.2 FreelancerProfile . . . 375
5.1.3 FreelancerSkill . . . 376
5.1.4 PortfolioItem . . . 376
5.1.5 ClientProfile . . . 377
5.1.6 Expert . . . 377
5.1.7 SocialLink . . . 378
5.1.8 FavoriteFreelancer . . . 378
5.1.9 FollowFreelancer . . . 378
5.1.10 Notification . . . 379
5.1.11 Review . . . 379
5.1.12 ReviewResponse . . . 380
5.1.13 ProfileRevision . . . 380
5.1.14 IdVerificationDocument . . . 381
5.1.15 UserRole . . . 382

1 Overall Description

1.1 Assumptions

This system is designed based on the following assumptions:

- The Node.js runtime and the frameworks chosen for the project (NestJS 11 for the back-end and Next.js 16 for the frontend) will continue to be actively supported and maintained, ensuring long-term stability and compatibility with evolving technologies.
- The RESTful API architecture will remain a scalable and efficient choice for separating the Next.js presentation layer from the NestJS business layer.
- PostgreSQL, accessed through Prisma ORM 7, will continue to provide reliable, scalable, and secure relational data storage suitable for handling user identity, jobs, contracts, escrow payments, and forum data.
- Third-party services used by the platform — Stripe (payments and escrow), Cloudinary (file storage), Google (OAuth login), the AI moderation/analysis service, and mail delivery via Nodemailer — will stay available and API-compatible during the project lifetime.
- Modern web browsers (e.g., Chrome, Edge, Firefox, Safari) will continue to support the front-end standards required by Next.js/React 19, including HTML5, CSS3 (TailwindCSS), JavaScript ESM, and responsive rendering with React Server Components.
- The application will be deployed in environments with stable internet access to support real-time messaging (Socket.io) and online payment processing.
- Users (Guests, Clients, Freelancers, Experts, and Admins) are familiar with basic web application navigation: browsing job boards, filling forms, managing profiles, and following payment flows.

This system design is also based on the following risks:

- Any breaking change in the major versions of Next.js, NestJS, or Prisma could require refactoring of the user interface components or of the data access layer.
- Cross-browser inconsistencies or outdated device software may affect rendering or the real-time messaging experience.
- Cybersecurity threats such as credential stuffing, token hijacking, or database breaches could compromise sensitive personal and financial information if not proactively managed (the design therefore includes bcrypt-hashed passwords, JWT access/refresh tokens, session revocation, and role-based access control).

- Outages or policy changes of the integrated third-party services (Stripe, Cloudinary, Google OAuth, AI moderation) could temporarily disrupt payments, file uploads, login, or content moderation.
- Network connectivity issues could disrupt access to job proposals, contract collaboration, or checkout flows that rely on server interaction.

1.2 Design Constraints

This system should comply with the following constraints:

1.2.1 Platform Compatibility

The Frevia application must be compatible with modern web browsers (e.g., Chrome, Edge, Firefox, Safari) for all user roles including Guest, Client, Freelancer, Expert, and Admin. The frontend is implemented with Next.js 16 (App Router, React Server Components) styled with TailwindCSS 4 and shadcn/ui components; the backend is a NestJS 11 REST API. Responsive web design principles must be applied so that a consistent user experience is provided across desktop, tablet, and mobile browsers. Internationalization is achieved with next-intl so the interface can be served in multiple languages from the same code base.

1.2.2 Integration Complexity

The Frevia platform integrates a variety of components: the Next.js web application, the NestJS REST API, the PostgreSQL database (via Prisma 7 with the pg adapter), Stripe Checkout and Stripe Connect for escrow payments and payouts, Cloudinary for image and document storage, Socket.io for real-time chat and notifications, Google OAuth for federated login, e-mail for OTP delivery, and AI services for content moderation, CV parsing, and identity document verification. Communication between the client and the server is carried out through versioned REST endpoints secured with JWT access tokens and refresh tokens stored in dedicated session records; role-based access control restricts each operation to the allowed roles. Design decisions must keep these integrations loosely coupled behind services and repositories so that a provider can be replaced without rewriting business logic.

1.2.3 Performance Requirements

The application must remain responsive under concurrent usage by many freelancers and clients. Job browsing, proposal listing, forum feeds, and conversation histories must load promptly; therefore the design mandates server-side pagination for every list operation, database

indexes on frequently filtered columns, optimized Prisma queries executed inside transactions for counted lists, and streaming React Server Components to reduce time-to-first-byte. Real-time features (messaging, notifications) must propagate events with minimal delay via Socket.io. Payment operations must reconcile eventual states through Stripe webhooks and explicit synchronization endpoints.

1.3 Technology Suggestion

Frevia adopts a RESTful API architecture to support secure, stateless, and scalable communication between the Next.js frontend and the NestJS backend. This approach provides:

- Modular, reusable API endpoints for account management, job posting, proposals, contracting, milestone escrow, dispute resolution, community forum, and administration.
- Easy extension for mobile applications or third-party integrations in the future.
- Secure access using JWT tokens with role-based authorization for every endpoint.
- Uniform use of standard HTTP methods (GET, POST, PATCH, PUT, DELETE), making the system intuitive to maintain.
- A clear separation between the presentation layer and the business logic layer.

All request/response payloads are validated and typed with shared Zod schemas packaged in `@shared/types`, which is consumed by both the frontend and the backend so that a single source of truth prevents contract drift between the two applications.

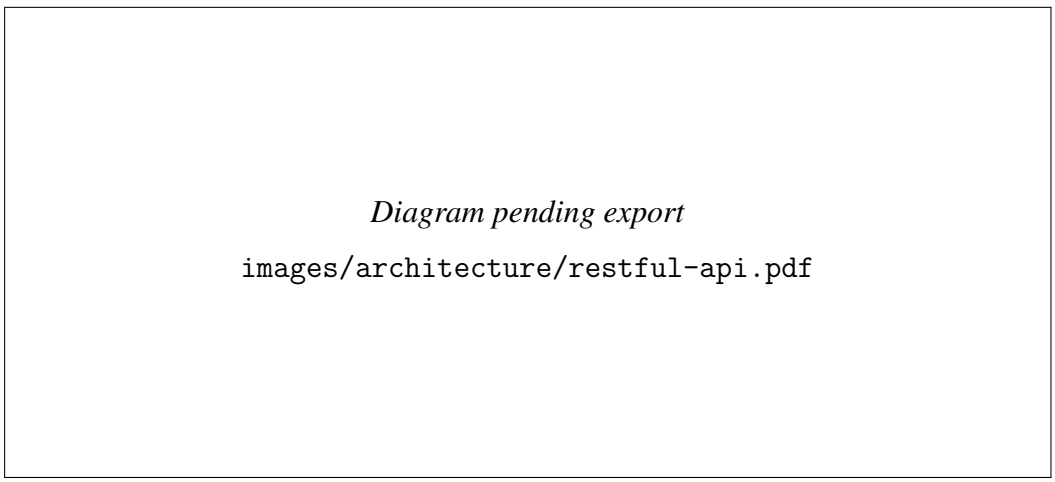

Figure 1: RESTful API communication between the Next.js frontend and the NestJS backend

1.3.1 NestJS Framework

The backend of Frevia is built with NestJS 11, a TypeScript-first, modular web framework running on Node.js. It is selected for its:

- Rich dependency-injection and module ecosystem, matching the Controller–Service–Repository layering.
- First-class decorator-based REST routing with guards and interceptors for authentication, roles, permissions, and response serialization.
- Excellent support for Zod schema validation (nestjs-zod) and typed DTOs.
- Component structure for WebSockets (Socket.io gateways) used by the messaging module.
- Straightforward integration with Prisma ORM and PostgreSQL for persistence.

1.3.2 Next.js and React 19 Frontend

The web client uses Next.js 16 with the App Router. React Server Components render public pages (job lists, job details, forum) directly from the server, while TanStack Query drives interactive islands (forms, likes, real-time chat). react-hook-form together with Zod resolvers validate user input with the same schemas the API uses.

1.3.3 Prisma ORM and PostgreSQL

Persistence relies on PostgreSQL accessed through Prisma 7. The schema defines 62 models covering identity and access, profiles, jobs, proposals, contracts and milestones, payments and disputes, reviews, messaging, the community forum, and platform configuration. Prisma's type-safe client keeps data access code verifiable at compile time.

1.3.4 Supporting Services

Payments and payouts use Stripe Checkout and Stripe Connect with webhook-driven reconciliation; binaries are stored in Cloudinary; emails (OTP, notifications) are sent with Nodemailer; Google login uses OAuth 2.0; environment configuration is loaded through Infisical so no secret is committed to the repository.

2 System Architecture Design

2.1 Overall Architecture

Frevia follows a three-tier, deployment-separated architecture (Figure 2):

- **Presentation tier — Next.js 16 web application:** server components render public pages (job board, job detail, community forum, expert directory) and private dashboards; client components handle interactivity such as forms, likes, and chat. All server access goes through a single typed HTTP client that attaches the JWT access token and refreshes it transparently when it expires.
- **Application tier — NestJS REST API:** a modular monolith organized into 31 route modules. Every module is internally layered as Controller → Service → Repository. Controllers validate and serialize with shared Zod DTOs; services hold business rules and authorization checks; repositories encapsulate all Prisma queries.
- **Data tier — PostgreSQL via Prisma 7:** the relational store for 62 domain models. Binary files are never stored in the database but in Cloudinary; real-time transport for chat and notifications is handled by a Socket.io gateway beside the REST API.

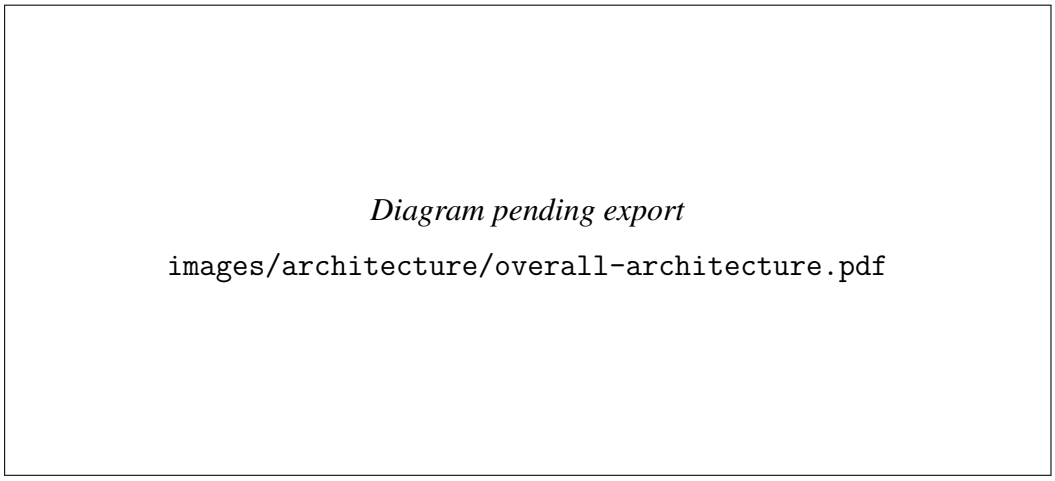

Figure 2: Overall architecture of the Frevia system

The two applications are deployed independently (the web app on Vercel, the API on a Node.js host) and communicate only through the documented REST contract validated by the shared Zod schemas package.

2.2 System Architecture

The internal architecture of the API follows a strict layered pattern, enforced identically in every module (Figure 3):

- **Controller layer** — thin Routing Layer. Exposes REST endpoints, applies guards (authentication, roles, permissions), binds validated request DTOs, and delegates to a service. Controllers never query the database directly.
- **Service layer** — business logic layer. Implements use cases, ownership/authorization checks, state-machine transitions (proposal, contract, dispute), and cross-aggregation logic; it throws typed HTTP exceptions (`NotFoundException 404`, `ForbiddenException 403`, `ConflictException 409`) that a global exception filter converts to a uniform error response.
- **Repository layer** — data access layer. The only component allowed to touch Prisma; complex list queries (pagination + counting) are wrapped in `$transaction` for consistency.
- **Shared kernel** — the `@shared/types` workspace package contains Zod models, constants, permission names, and error codes shared with the web app.

Security flow: every request carries a JWT access token; the auth guard validates it and resolves the active role; role and permission guards authorize module-specific operations; refresh tokens are stored server-side in a `Session` row and can be revoked from the session management feature.

AI-assisted pipeline: user-generated content (forum posts/comments, CVs, identity documents) is passed through AI moderation or AI analysis services; forum content can be auto-approved when clean, or queued in `ModerationQueue` for administrator approval when suspicious.

2.2.1 API Module Structure

Table 1 summarizes the 31 route modules of the API and their responsibilities.

Table 1: API route modules and responsibilities

Module	Responsibility
auth	registration, OTP, login, refresh token, logout, forgot password, Google OAuth, current account, role switching/joining
sessions	device session listing, detail, revocation

Module	Responsibility
account-profile	general account profile, avatar, identity verification upload/AI check, client profile, social links, favorite freelancers, following
users	administrator user management and skill catalog
roles, permissions	RBAC role and permission administration
profiles	professional profile, CV upload/AI analysis, skills
portfolios	portfolio item CRUD on profiles
expert-profile	expert profile management and public expert directory
profile-revisions	profile change request approval workflow
identity-verifications-admin	identity verification review
job-categories	public job category browsing
job-categories-admin	job category administration (soft delete + restore)
skills-admin	skill catalog administration (soft delete + restore)
browse-job	public job listing and job detail by slug
manage-job	client job management, bookmarks, skill search
proposals	proposal lifecycle: draft, submit, withdraw, interview, reject, accept
contract	contract lifecycle: create, update, sign, complete, cancel
milestone	milestone management inside a contract
milestone-file, milestone-submission	milestone attachments and work submission review
shared-file	contract shared workspace files
payments	Stripe Connect onboarding, platform fee, milestone funding, release, refund, dispute fee, transactions, webhook
disputes	dispute lifecycle (user side and admin arbitration side)
reviews	contract reviews and responses
conversations	direct messaging with real-time delivery
notifications	notification feed management
forums	forum categories and posts (public side)
forums-comment, forums-like, forums-reports	comments, likes, content reports
forums-admin	AI moderation queue, pending posts, trash restore, forum statistics
saved-search	saved job search filters
banners, banners-admin	advertisement banner delivery and administration

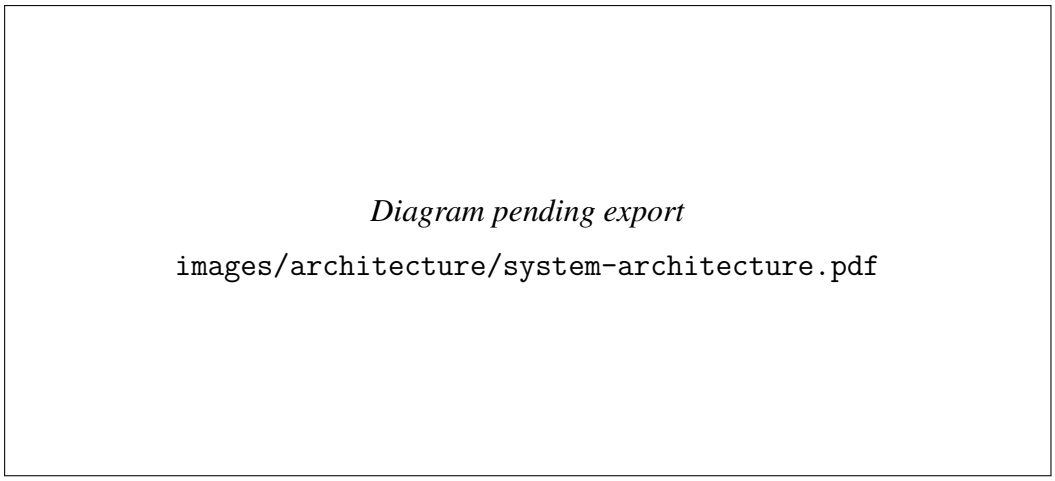

Figure 3: Layered system architecture with security and AI moderation flows

2.3 Package Diagram

The repository is a pnpm monorepo managed by Turborepo with three workspaces (Figure 4):

- **apps/web** — the Next.js application. Sub-packages group screens by actor (guest browsing, client, freelancer, admin), shared UI components (`@repo/ui`), server actions, and the i18n message catalogs used by next-intl.
- **apps/api** — the NestJS application. `src/routes/*` contains the 31 domain modules described above; `src/shared` contains cross-cutting infrastructure: authentication guards and decorators, permission constants, global HTTP exception filter, Zod serializer interceptor, Cloudinary storage service, and Prisma client setup. Each domain module folder is organized as `<module>.controller.ts`, `<module>.service.ts`, `<module>.repository.ts`, DTO classes, and module-specific error factories (e.g., `forums.error.ts`).
- **packages/shared-type** — the single source of truth for API contracts: Zod request/response models (`model/*`), enumeration/constant definitions, and permission name lists, imported by both `apps/web` and `apps/api`.

Dependencies only point downwards: `web` and `api` both depend on `shared-type`; no route module imports another route module — cross-module data needs flow through services at the Prisma schema level.

Diagram pending export

`images/architecture/package-diagram.pdf`

Figure 4: Package diagram of the Frevia monorepo

3 System Detailed Design

3.1 View Forum Category List

a. Class Diagram

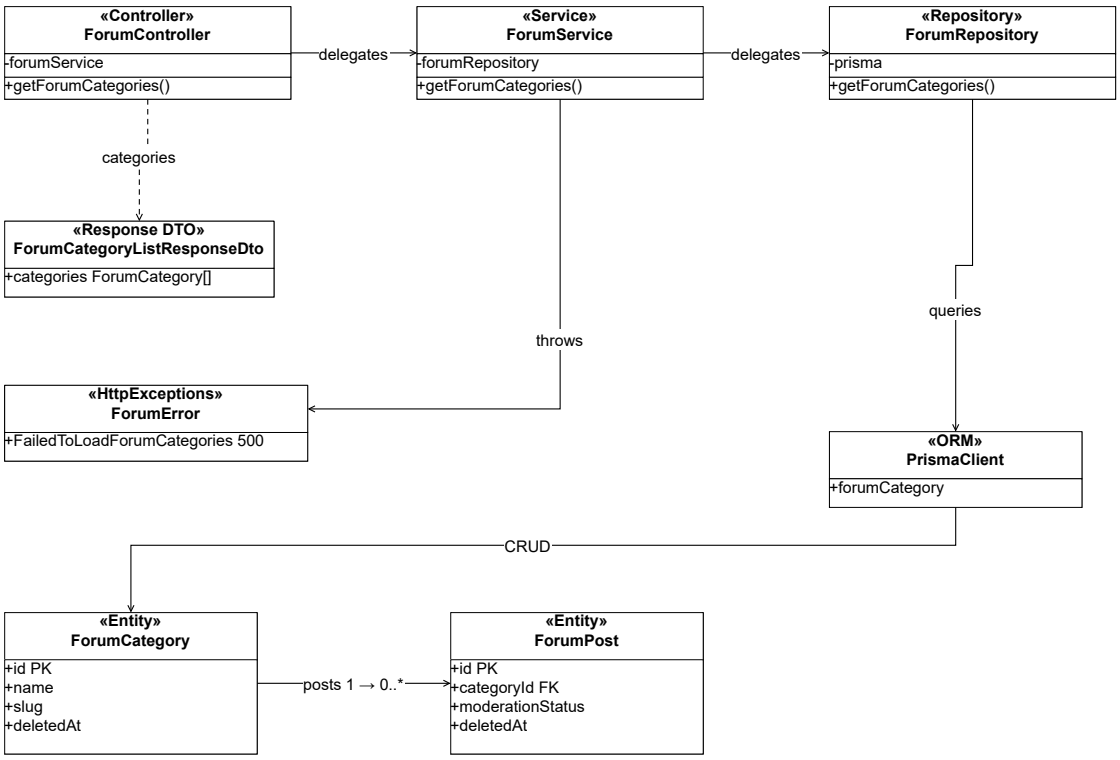

Figure 5: Class diagram of View Forum Category List

b. Sequence Diagram(s)

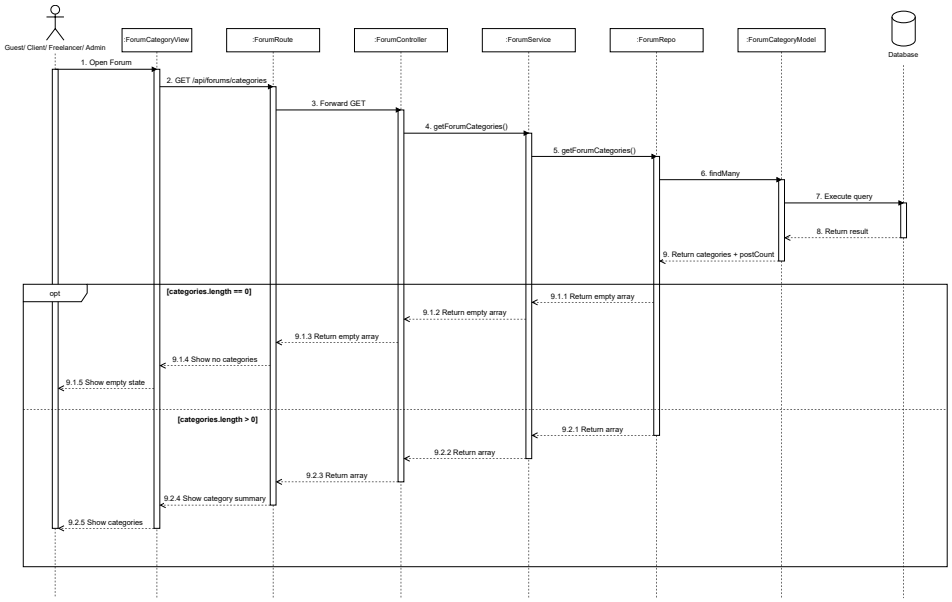

Figure 6: Sequence diagram of View Forum Category List

3.2 View Forum Category Detail

a. Class Diagram

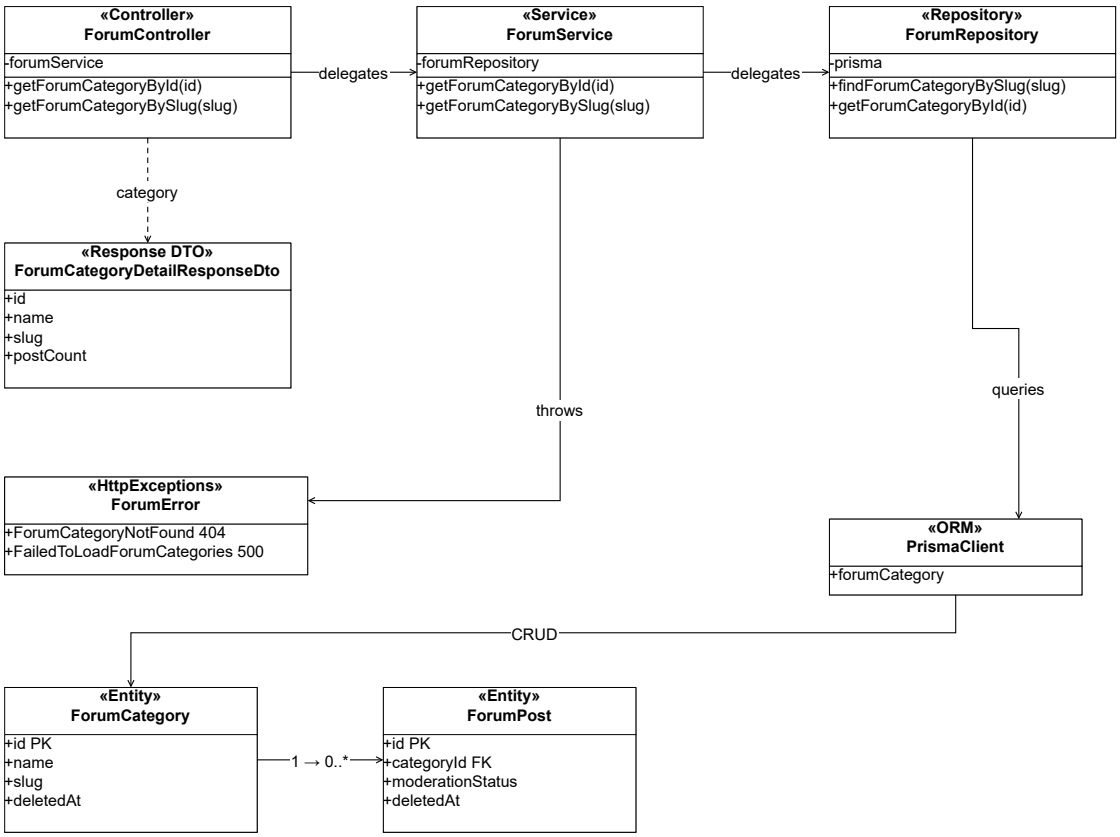

Figure 7: Class diagram of View Forum Category Detail

b. Sequence Diagram(s)

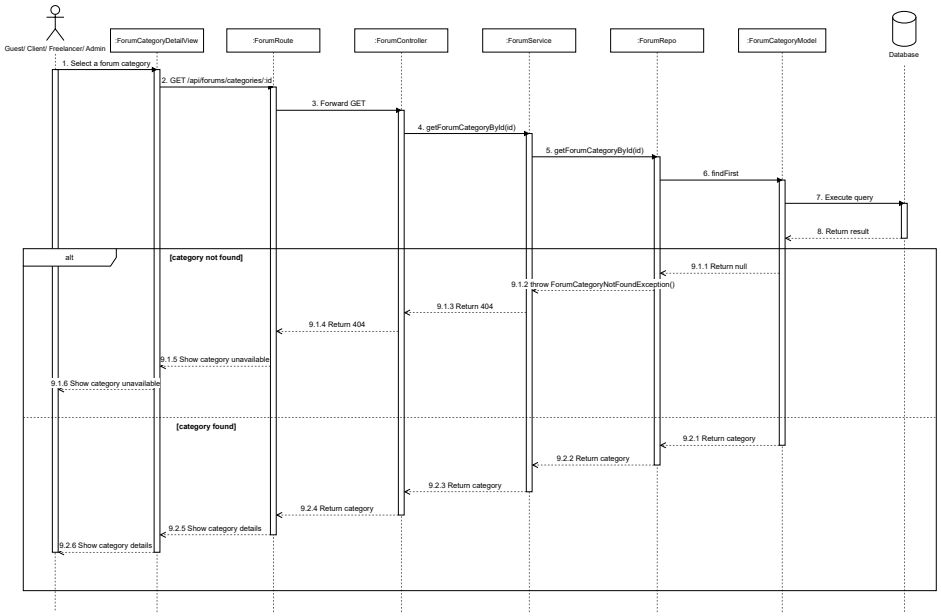

Figure 8: Sequence diagram of View Forum Category Detail

3.3 View Post List

a. Class Diagram

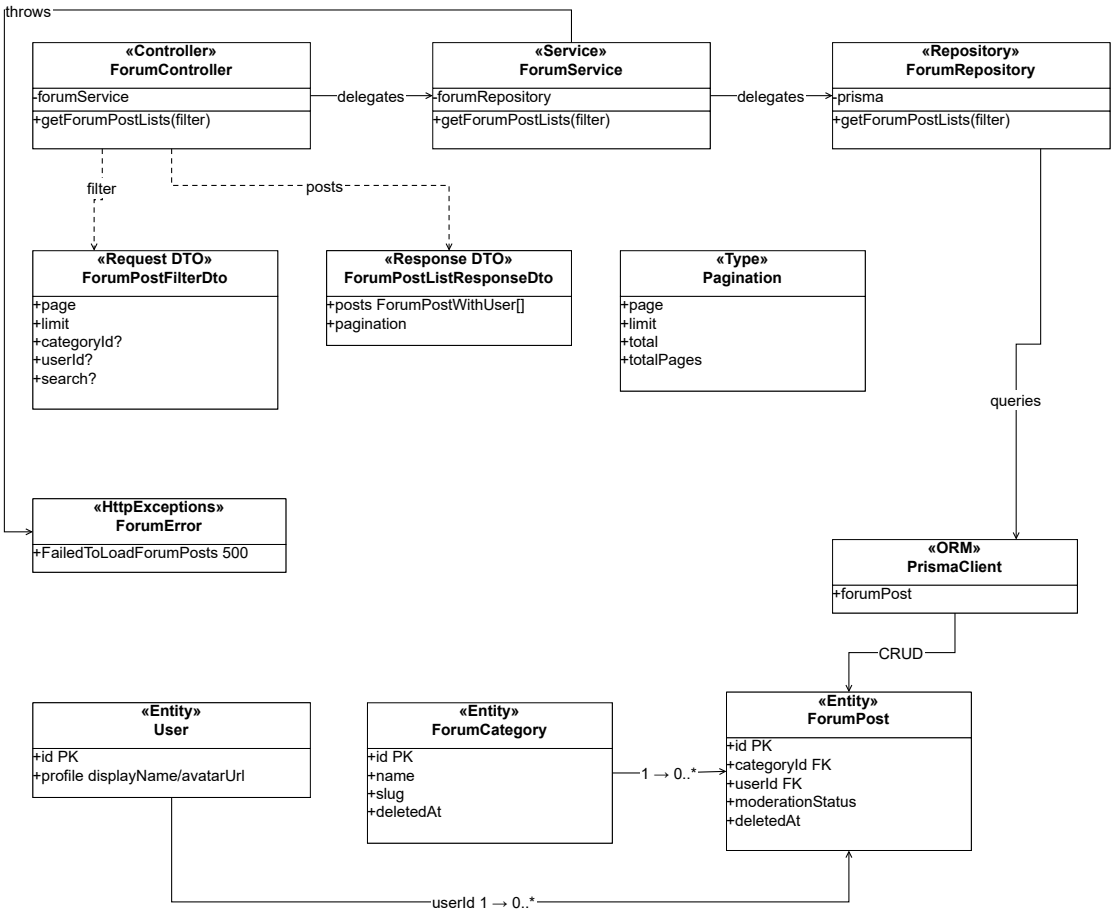

Figure 9: Class diagram of View Post List

b. Sequence Diagram(s)

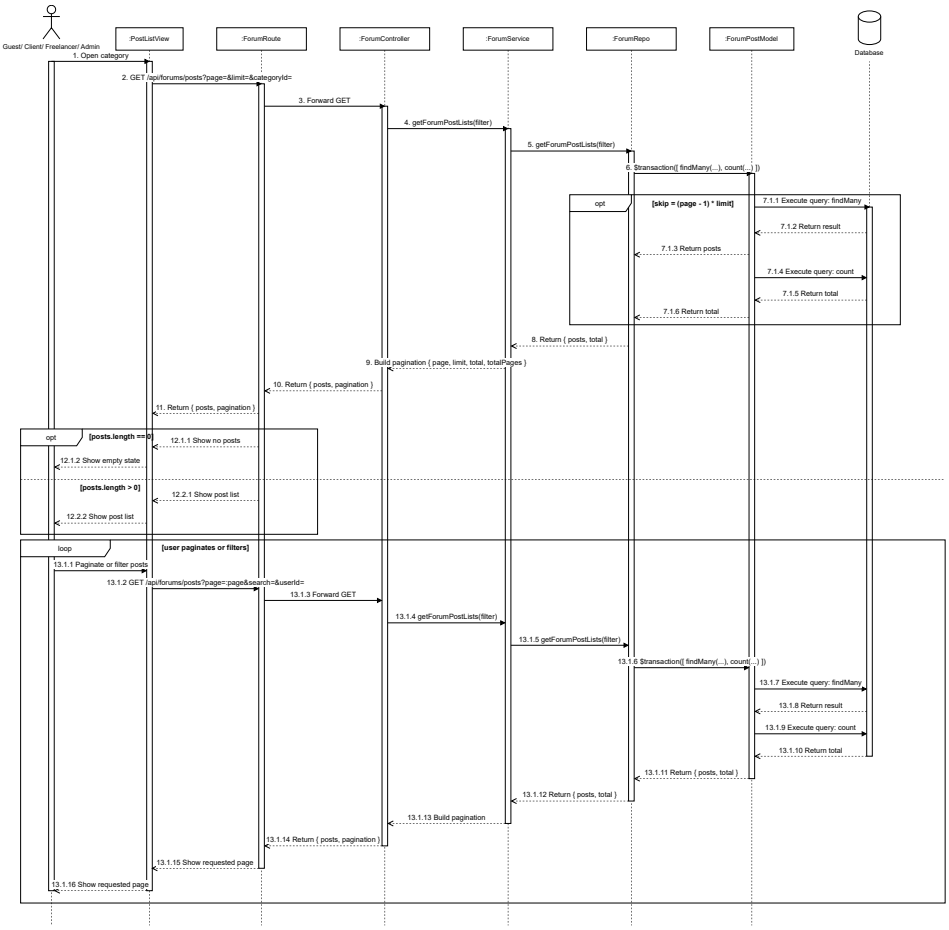

Figure 10: Sequence diagram of View Post List

3.4 View Post Detail

a. Class Diagram

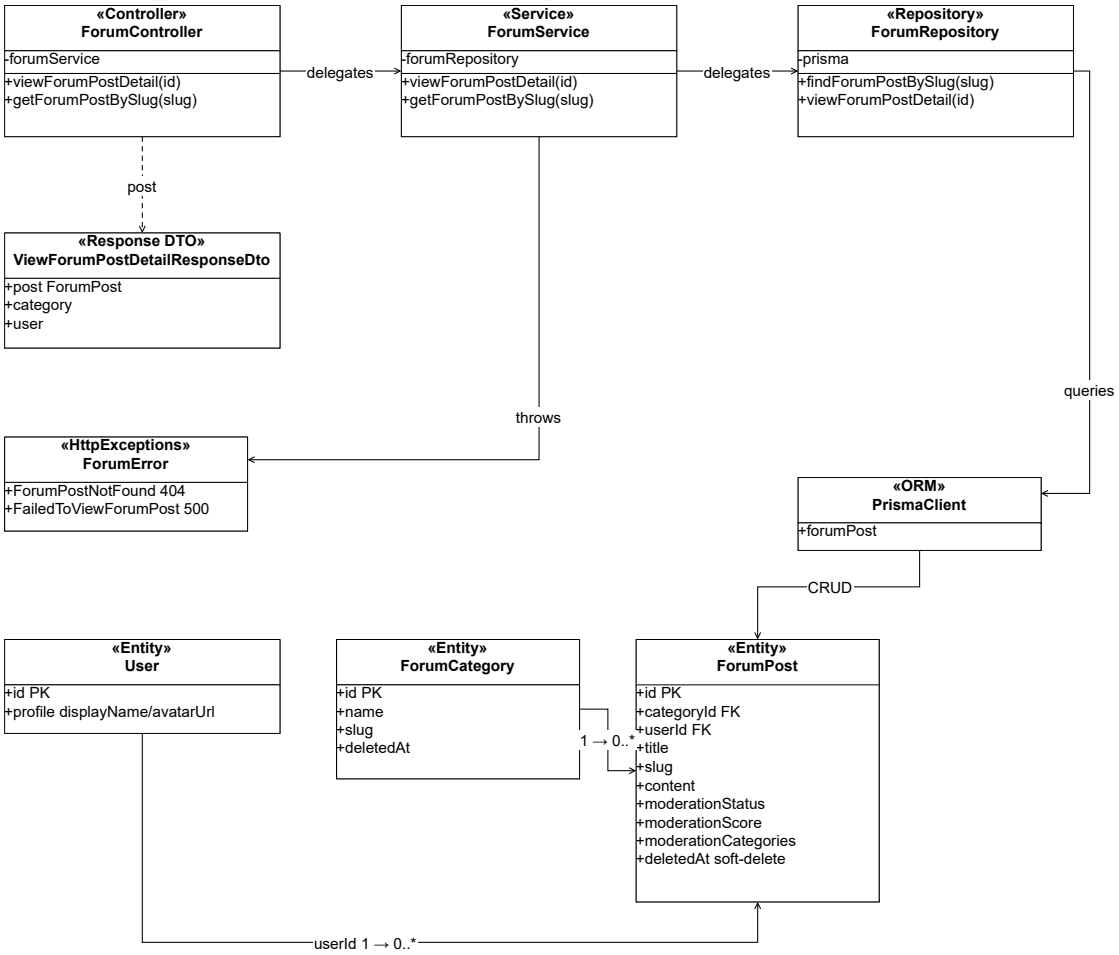

Figure 11: Class diagram of View Post Detail

b. Sequence Diagram(s)

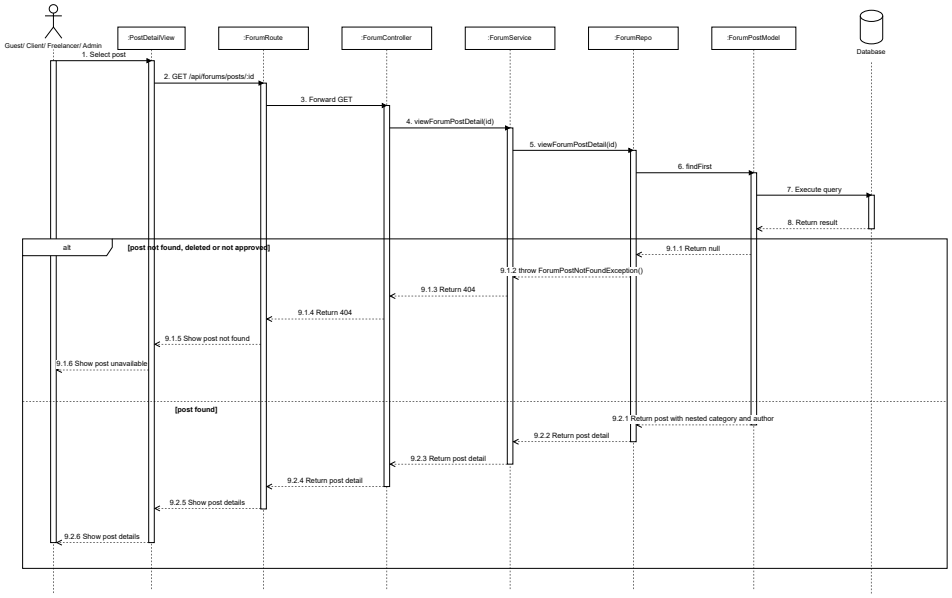

Figure 12: Sequence diagram of View Post Detail

3.5 Create Post

a. Class Diagram

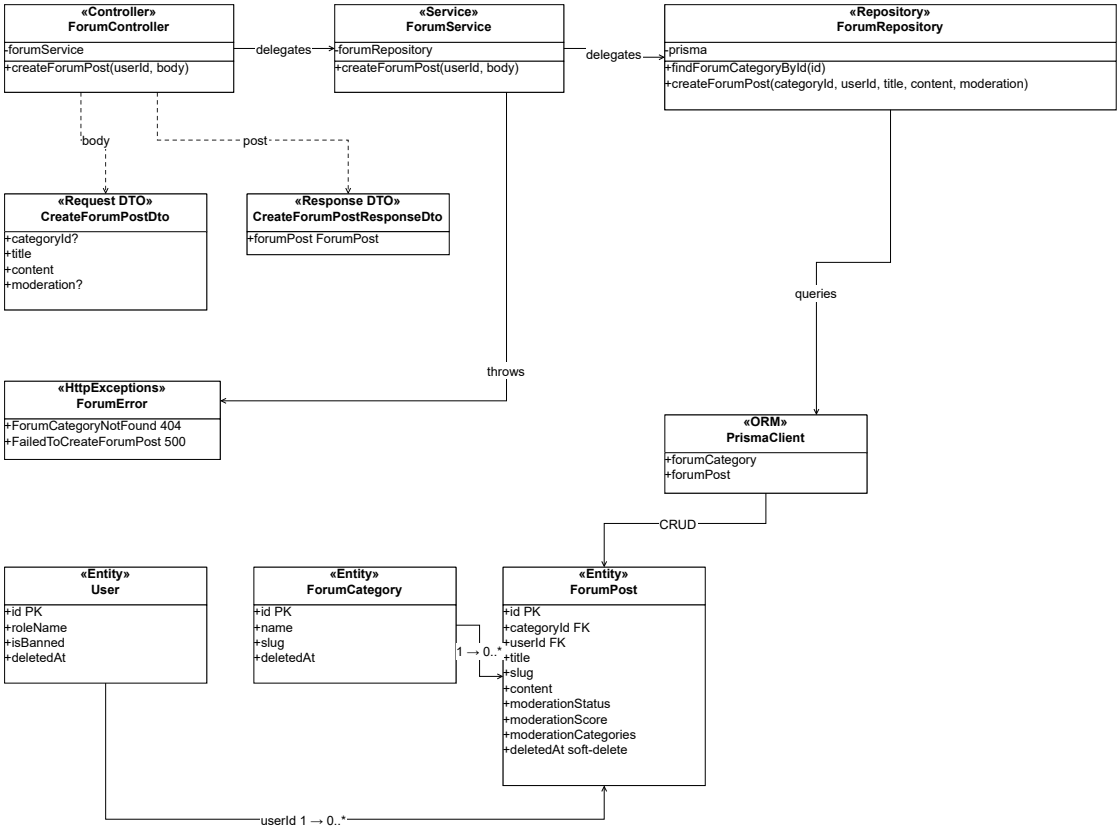

Figure 13: Class diagram of Create Post

b. Sequence Diagram(s)

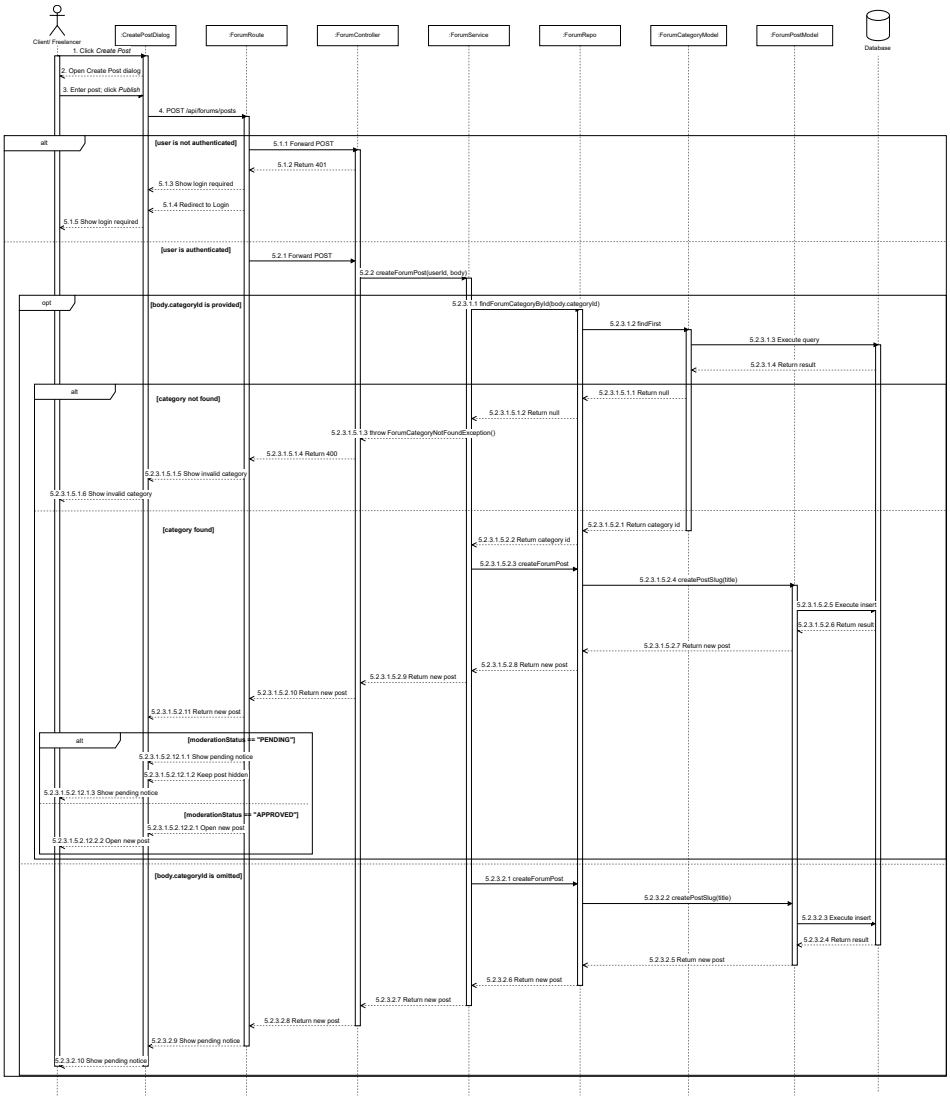

Figure 14: Sequence diagram of Create Post

3.6 Update Post

a. Class Diagram

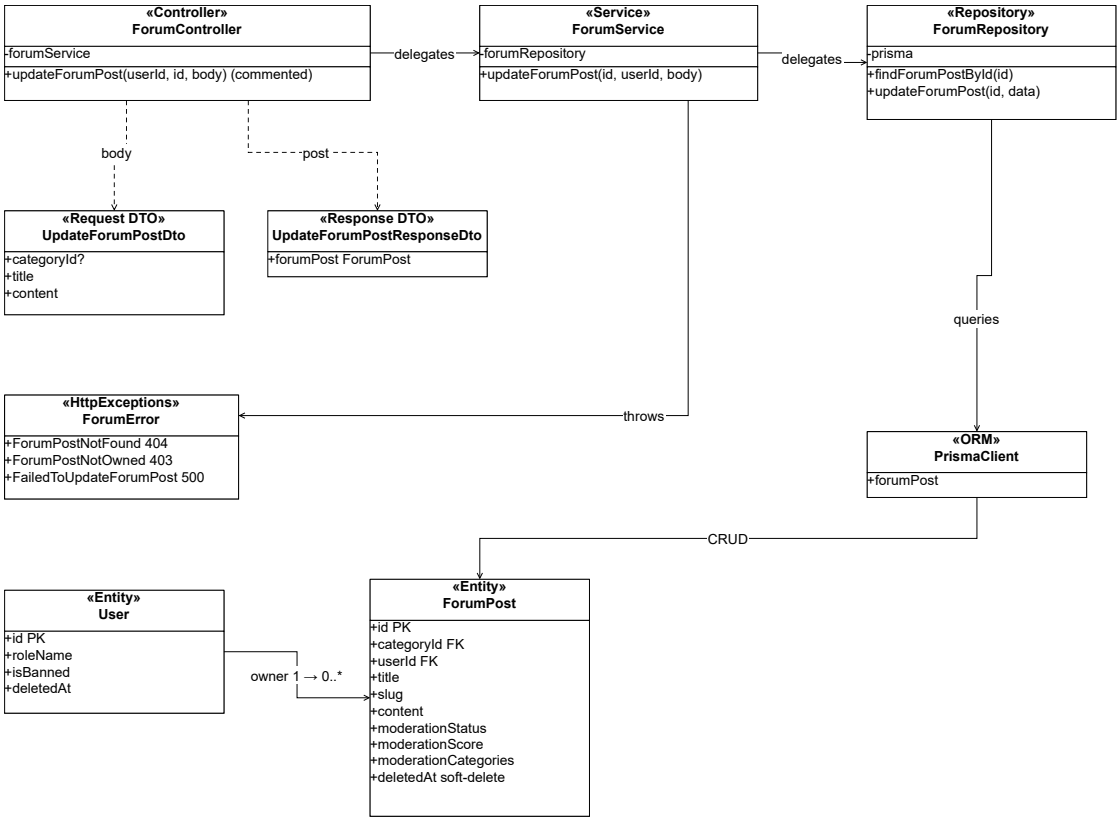

Figure 15: Class diagram of Update Post

b. Sequence Diagram(s)

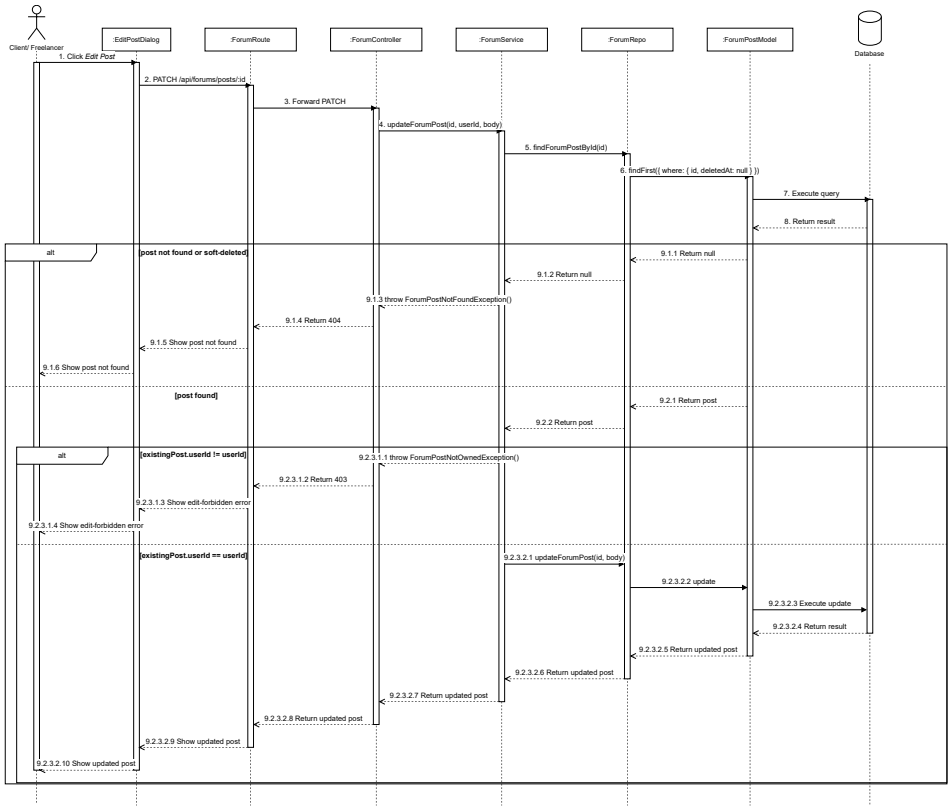

Figure 16: Sequence diagram of Update Post

3.7 Delete Post

a. Class Diagram

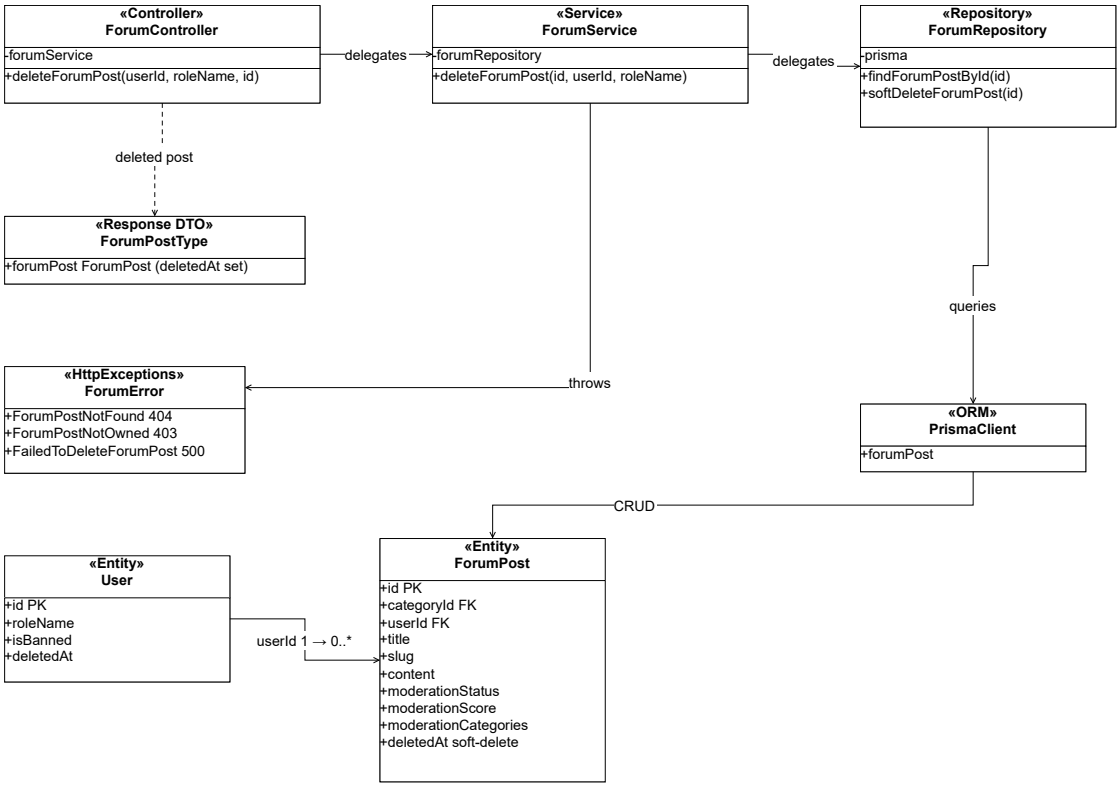

Figure 17: Class diagram of Delete Post

b. Sequence Diagram(s)

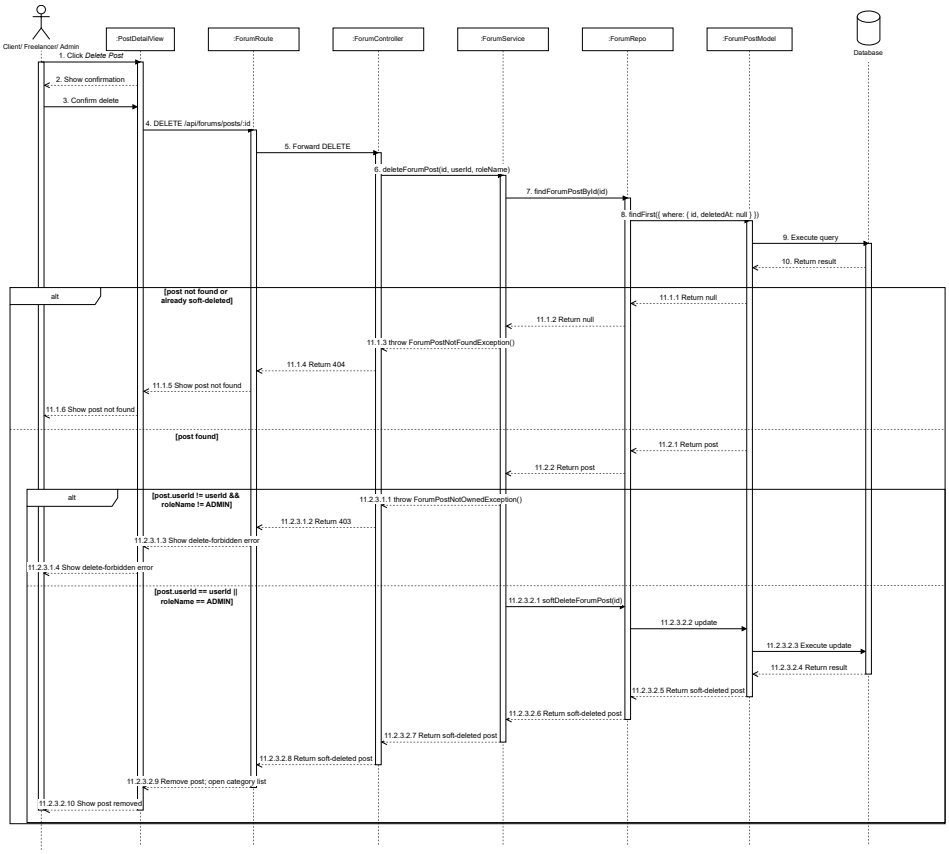

Figure 18: Sequence diagram of Delete Post

3.8 Like Post

a. Class Diagram

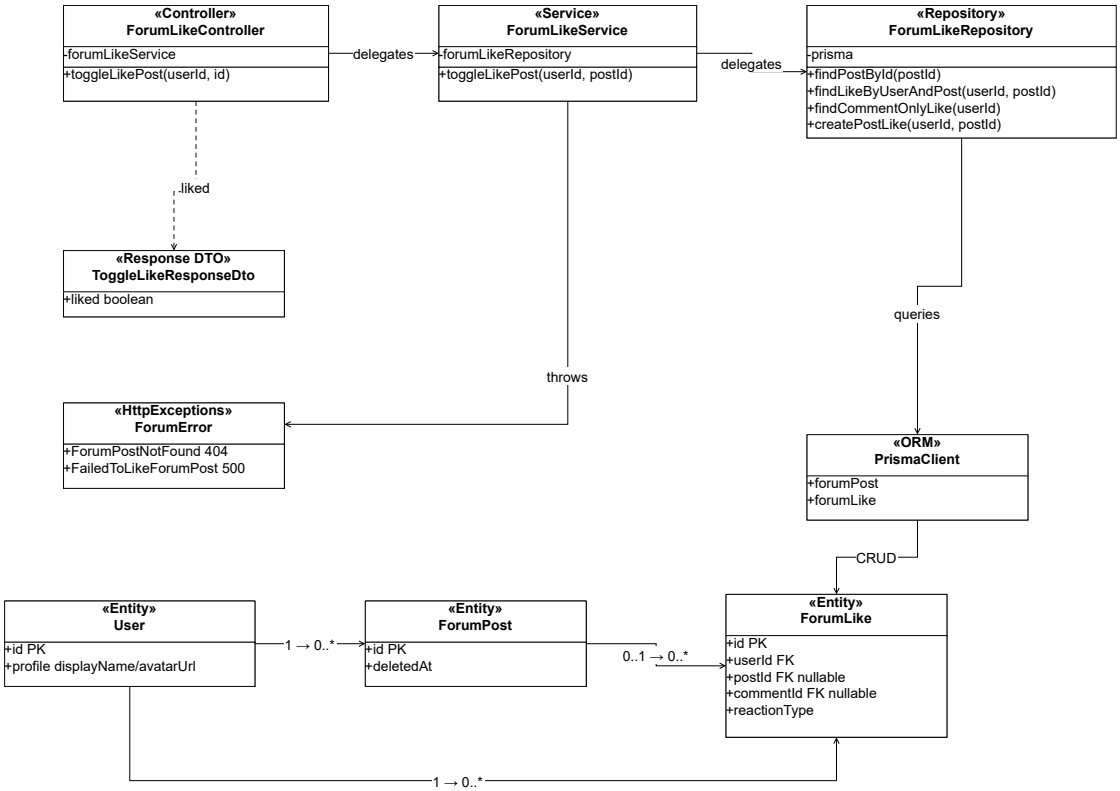

Figure 19: Class diagram of Like Post

b. Sequence Diagram(s)

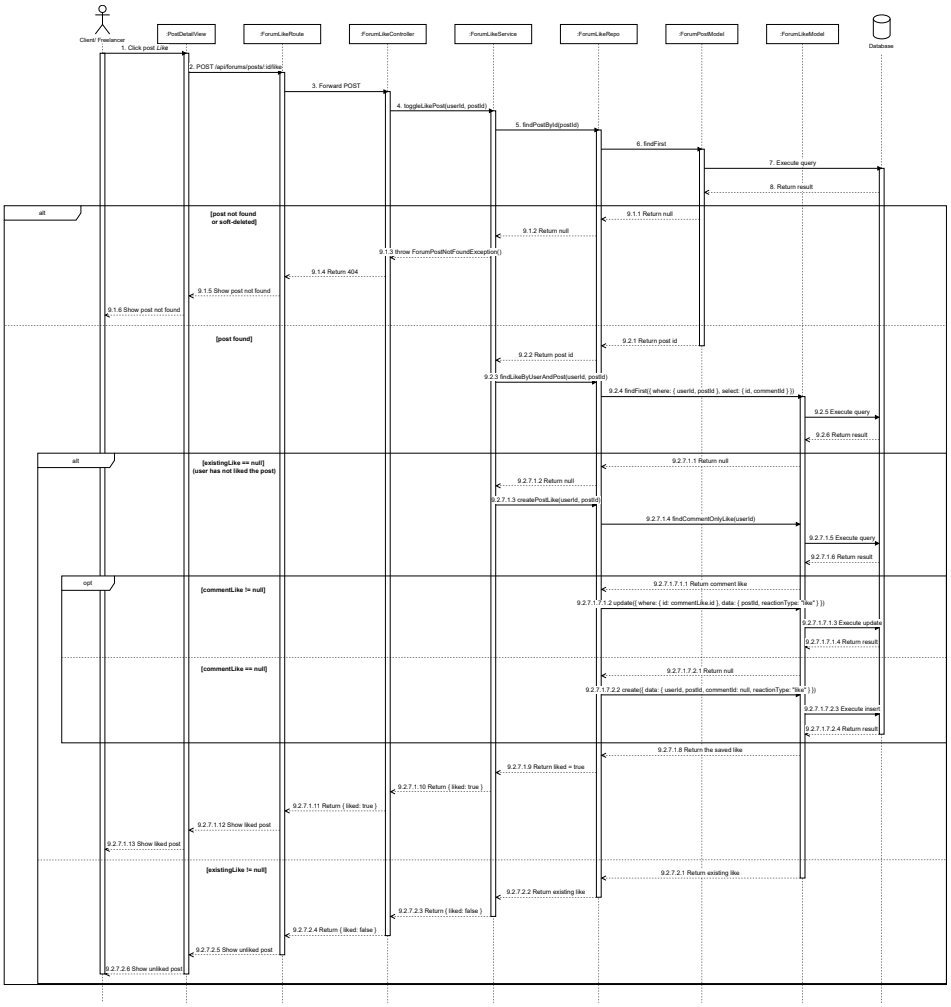

Figure 20: Sequence diagram of Like Post

3.9 Unlike Post

a. Class Diagram

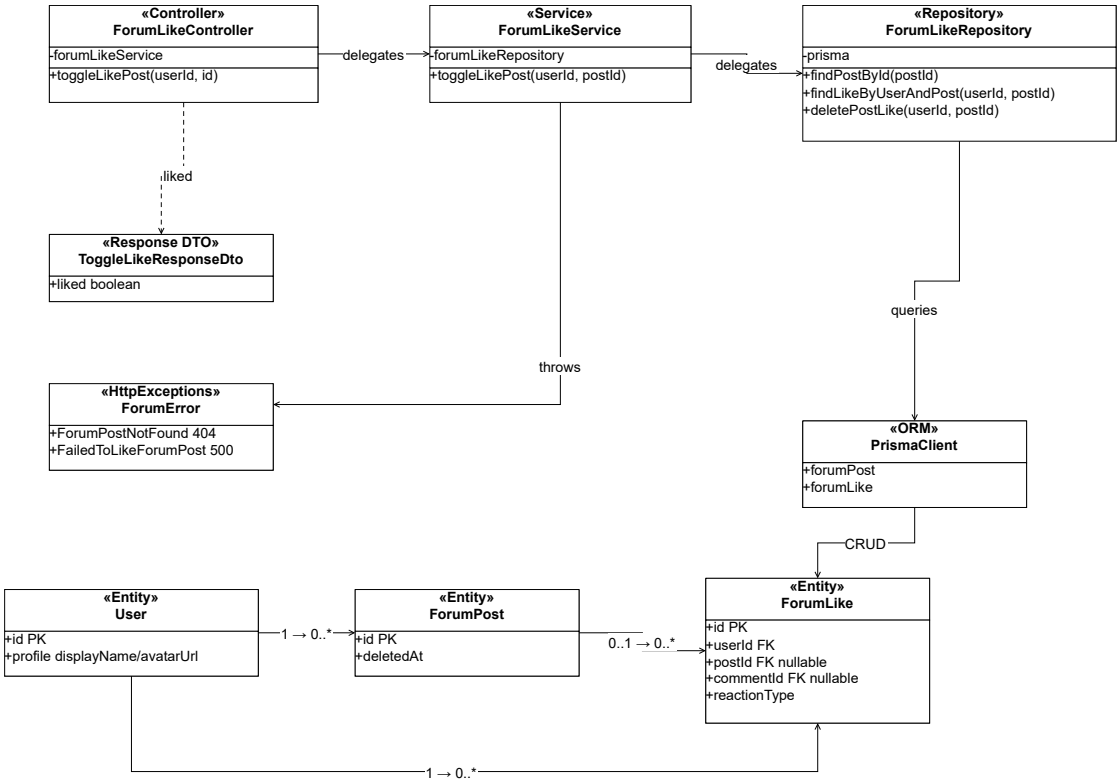

Figure 21: Class diagram of Unlike Post

b. Sequence Diagram(s)

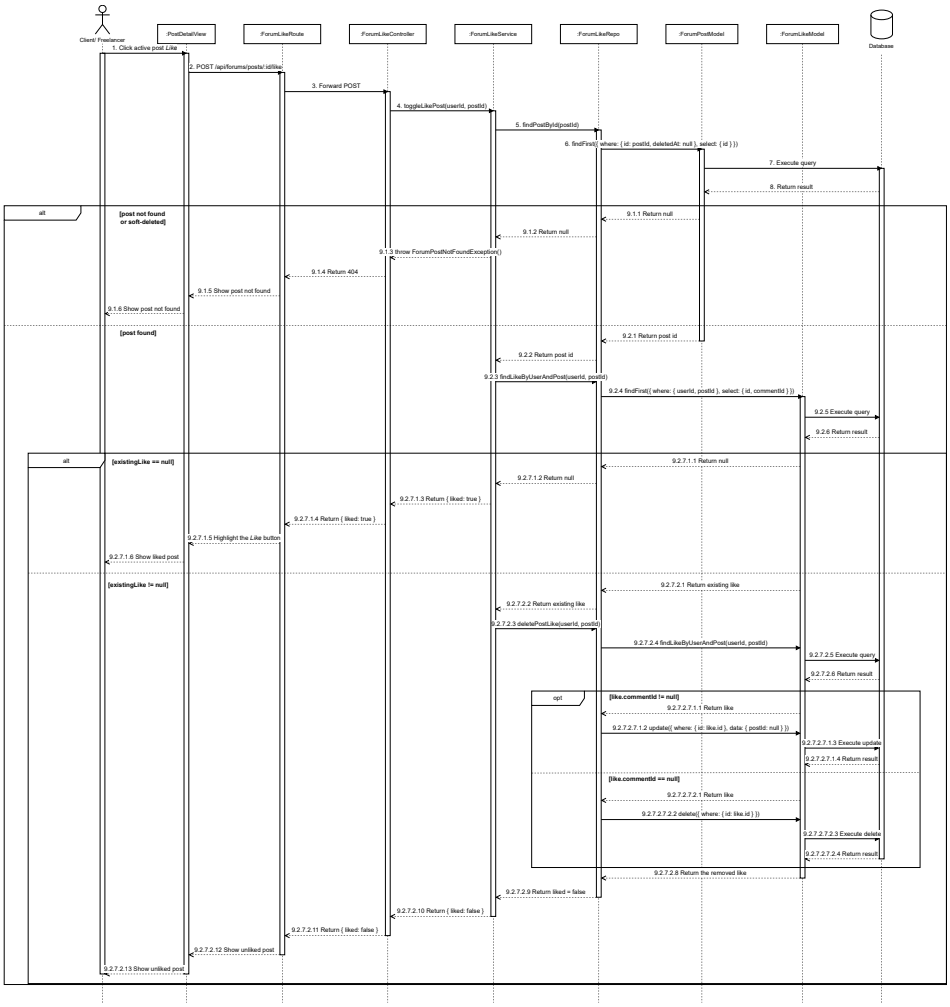

Figure 22: Sequence diagram of Unlike Post

3.10 View Post Likes

a. Class Diagram

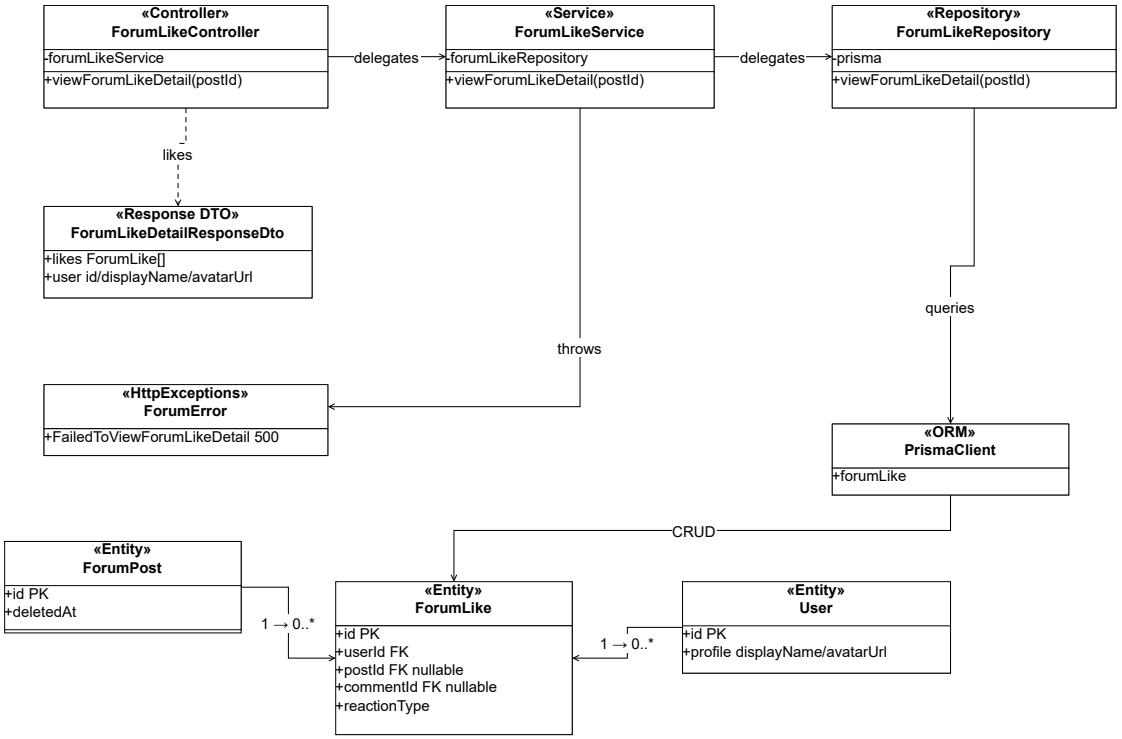

Figure 23: Class diagram of View Post Likes

b. Sequence Diagram(s)

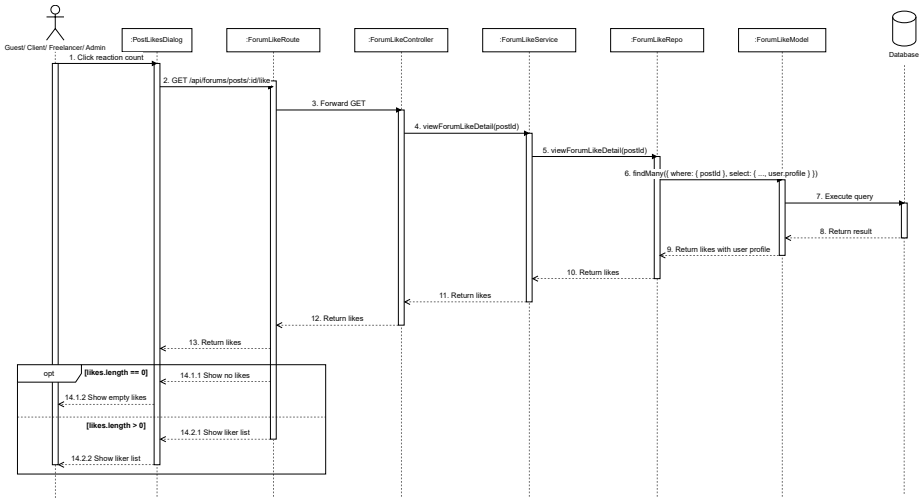

Figure 24: Sequence diagram of View Post Likes

3.11 View Post Comments

a. Class Diagram

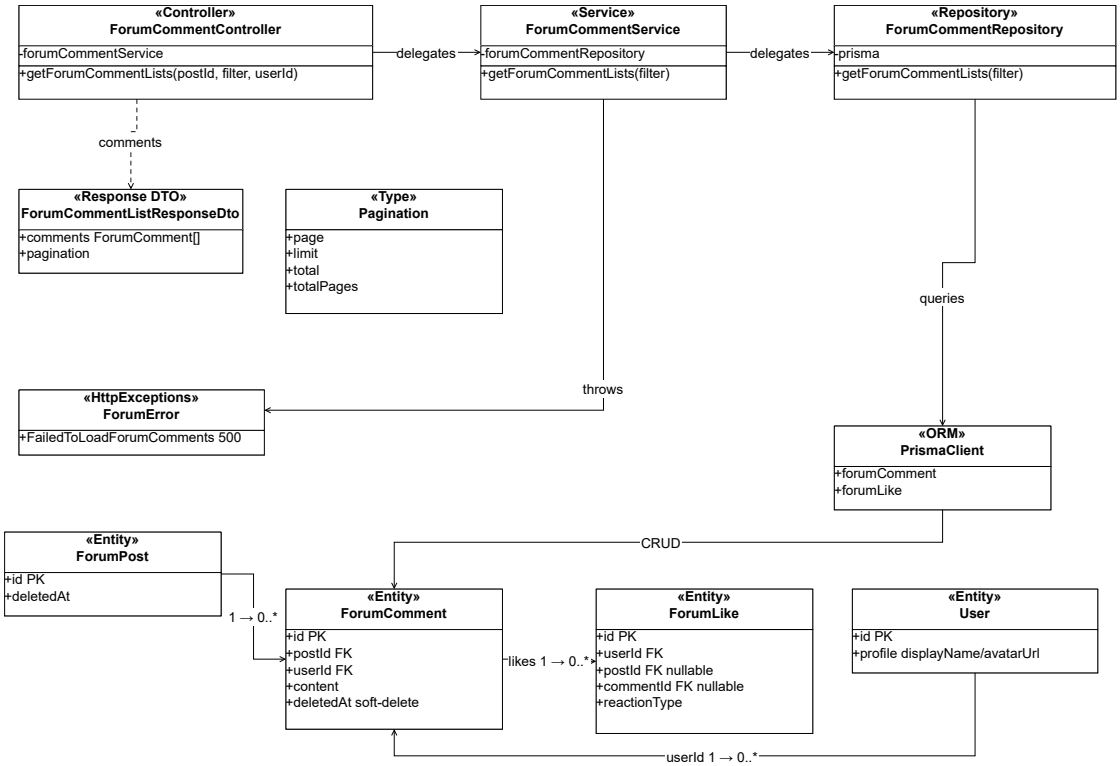

Figure 25: Class diagram of View Post Comments

b. Sequence Diagram(s)

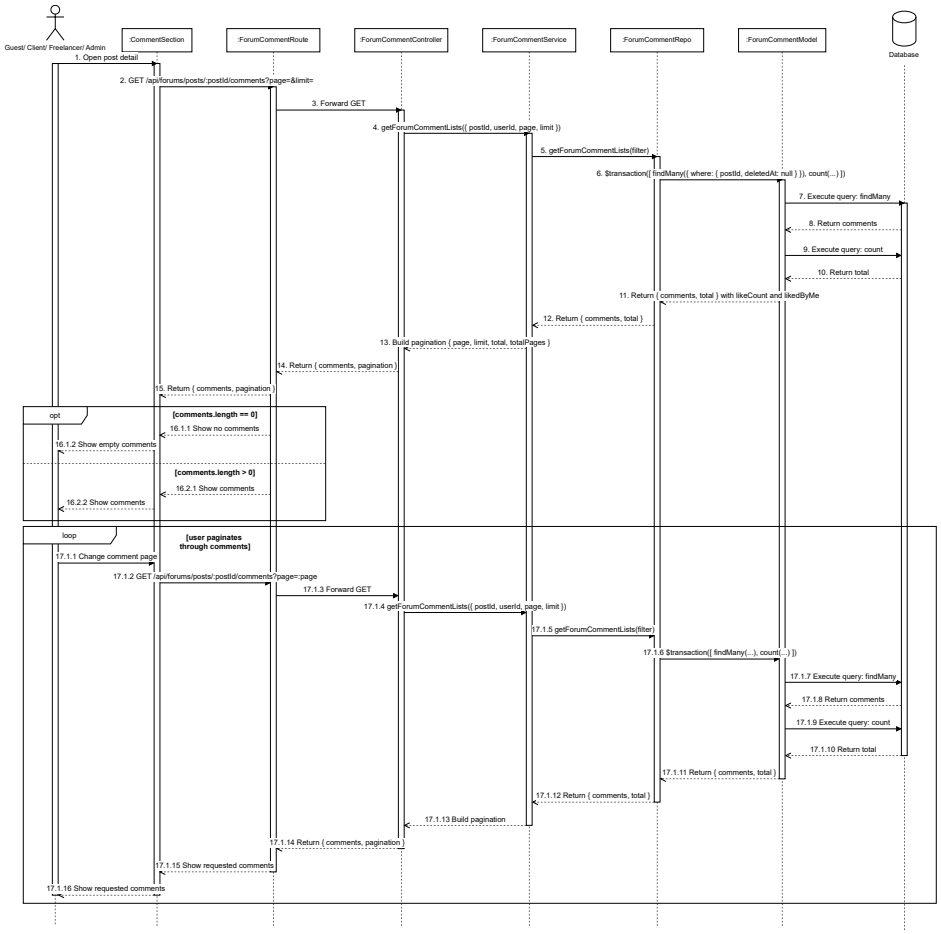

Figure 26: Sequence diagram of View Post Comments

3.12 Create Comment

a. Class Diagram

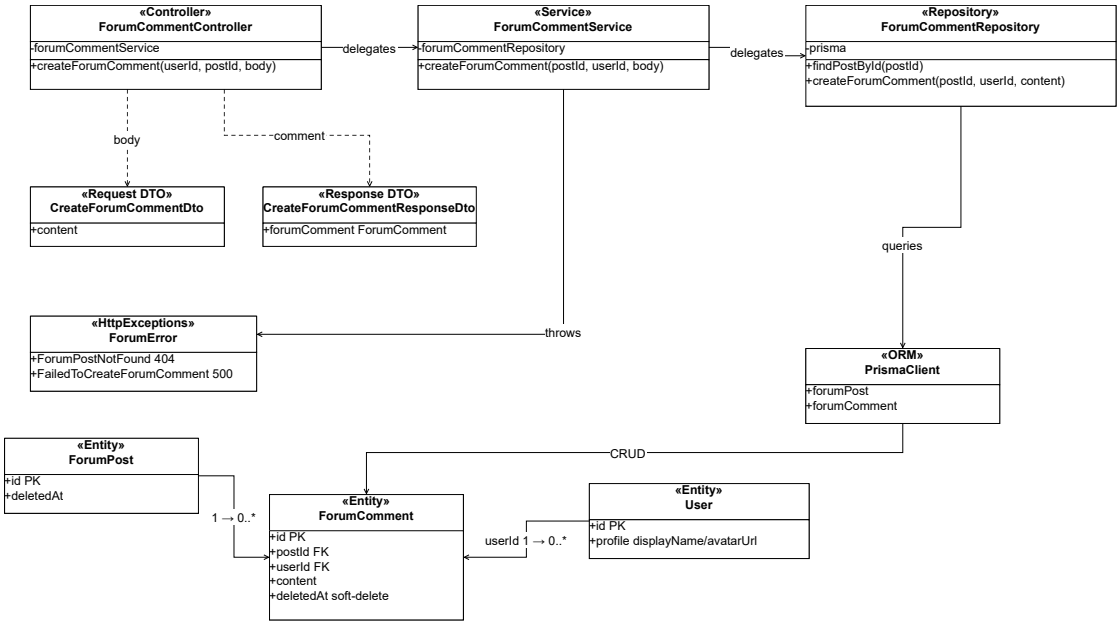

Figure 27: Class diagram of Create Comment

b. Sequence Diagram(s)

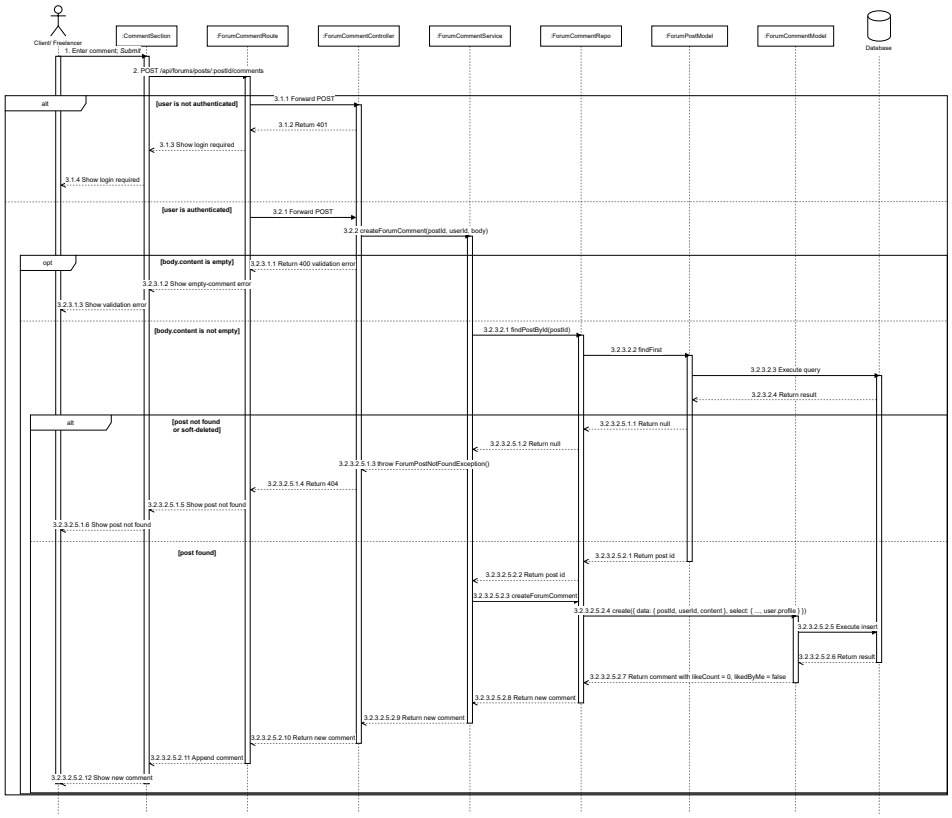

Figure 28: Sequence diagram of Create Comment

3.13 Update Comment

a. Class Diagram

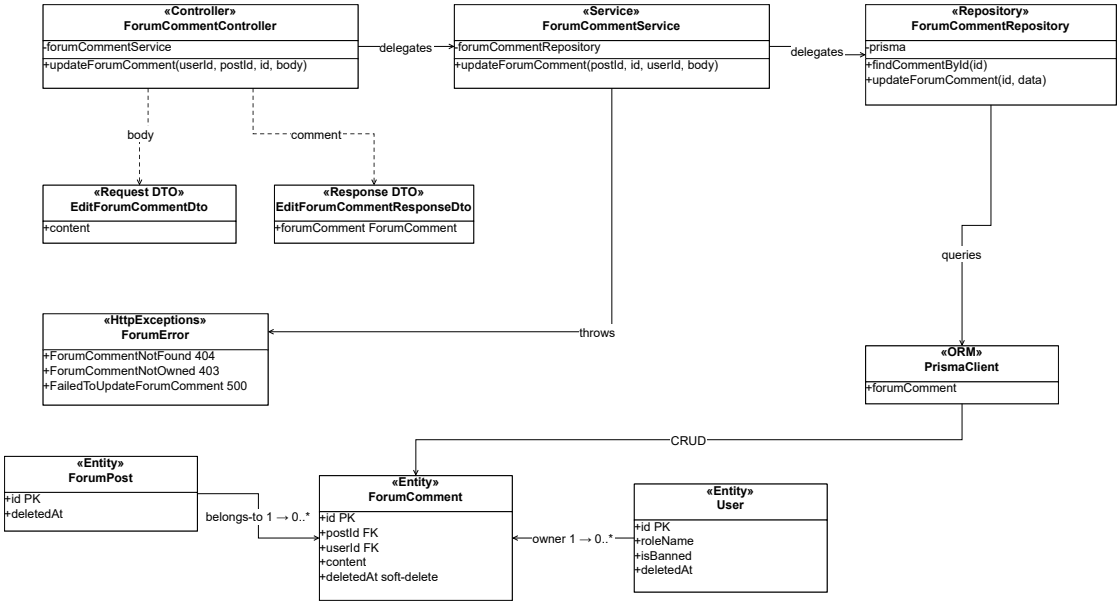

Figure 29: Class diagram of Update Comment

b. Sequence Diagram(s)

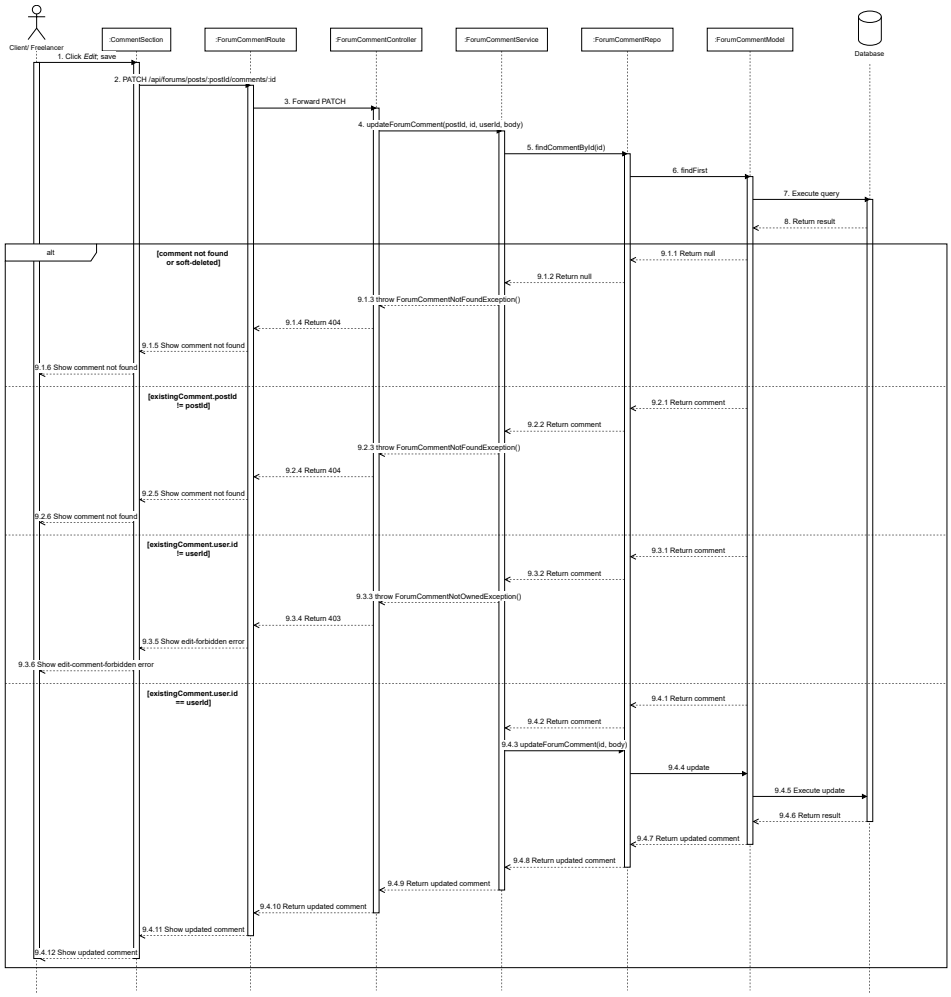

Figure 30: Sequence diagram of Update Comment

3.14 Delete Comment

a. Class Diagram

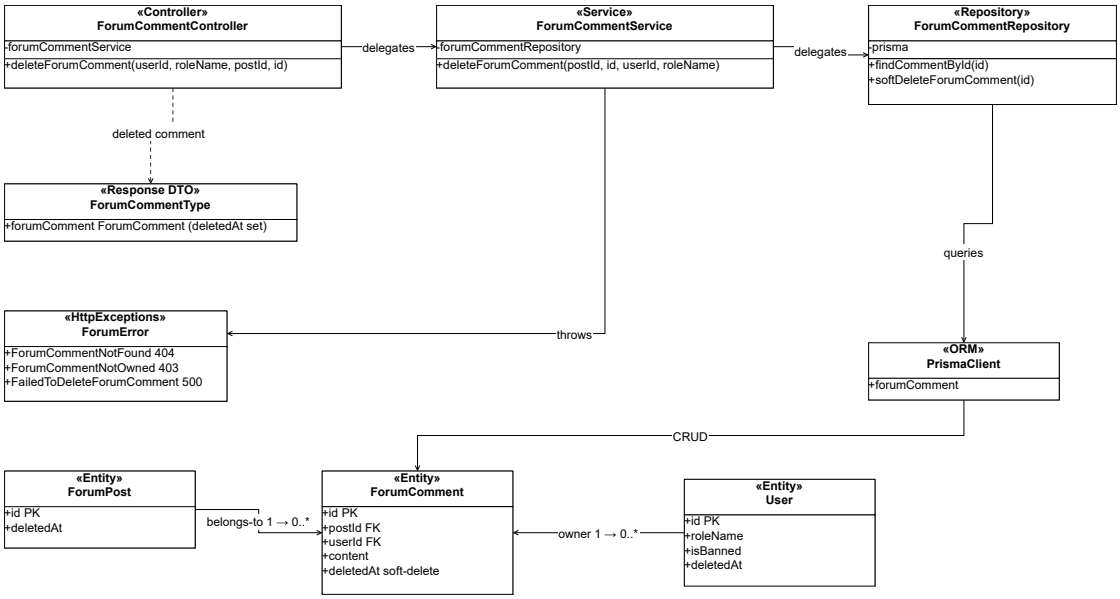

Figure 31: Class diagram of Delete Comment

b. Sequence Diagram(s)

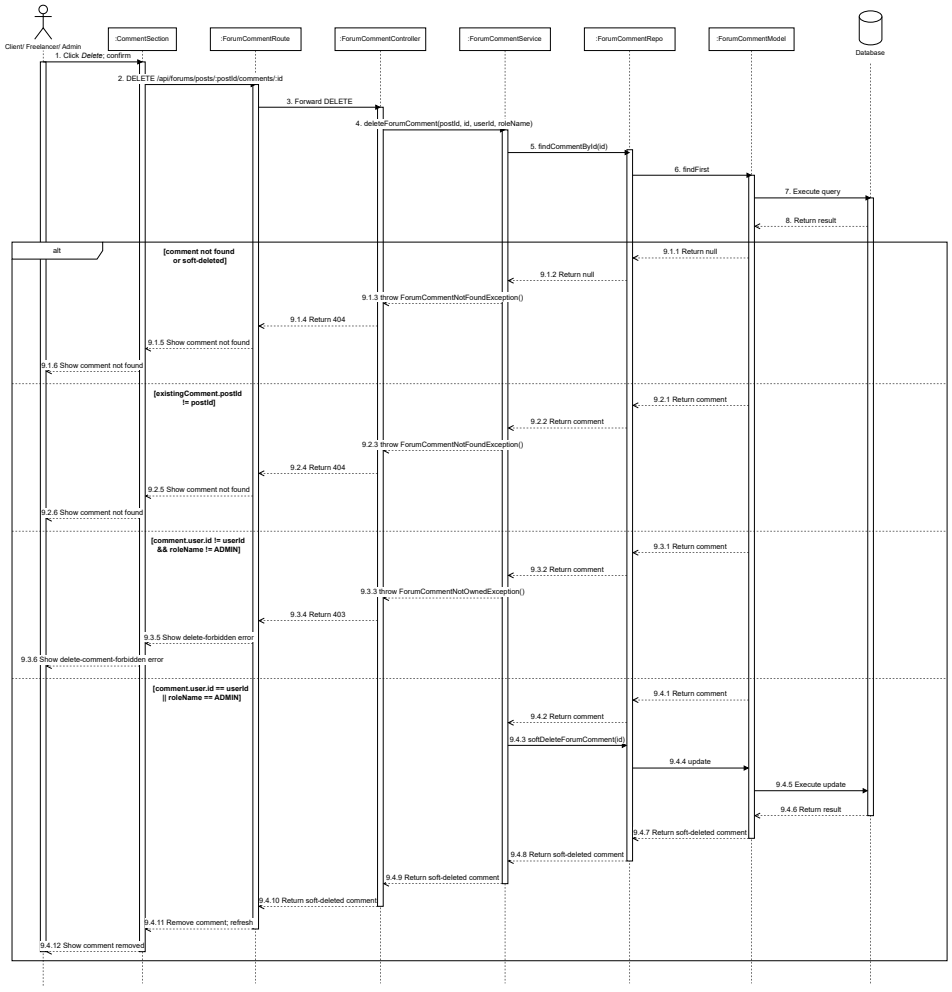

Figure 32: Sequence diagram of Delete Comment

3.15 Like Comment

a. Class Diagram

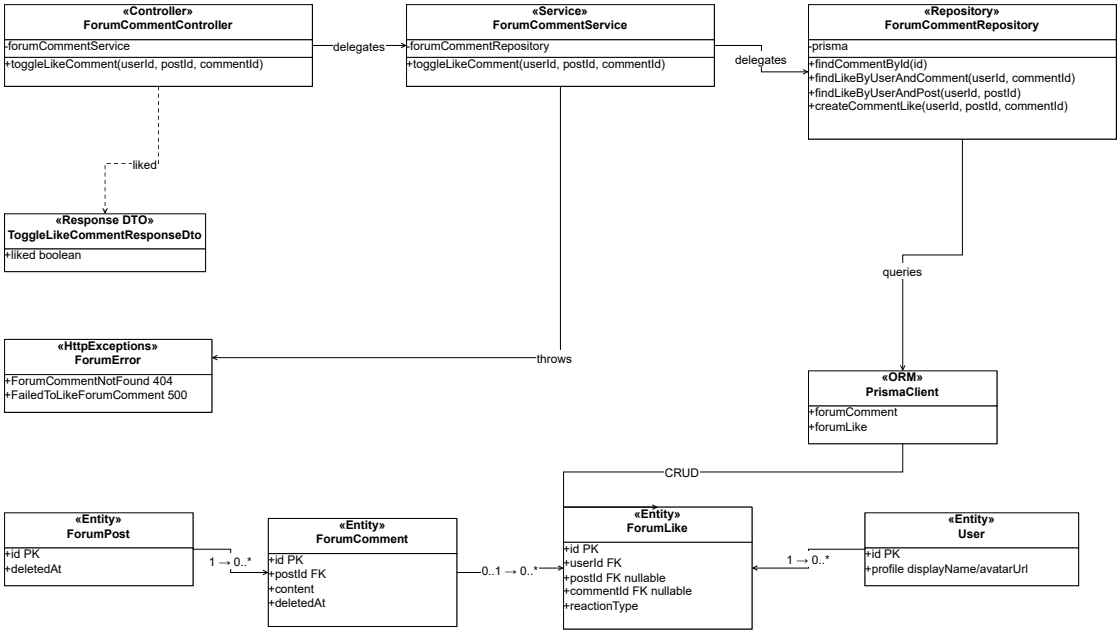

Figure 33: Class diagram of Like Comment

b. Sequence Diagram(s)

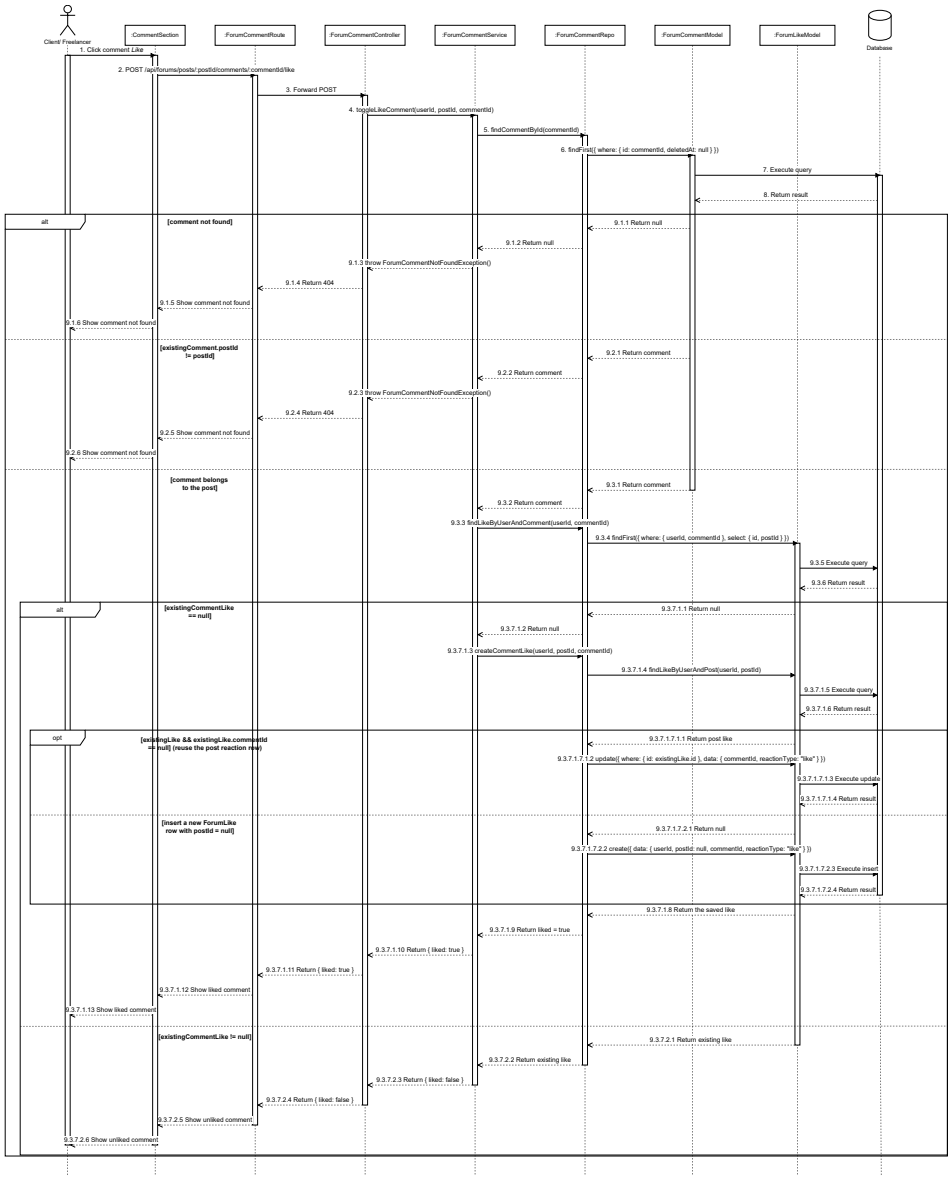

Figure 34: Sequence diagram of Like Comment

3.16 Unlike Comment

a. Class Diagram

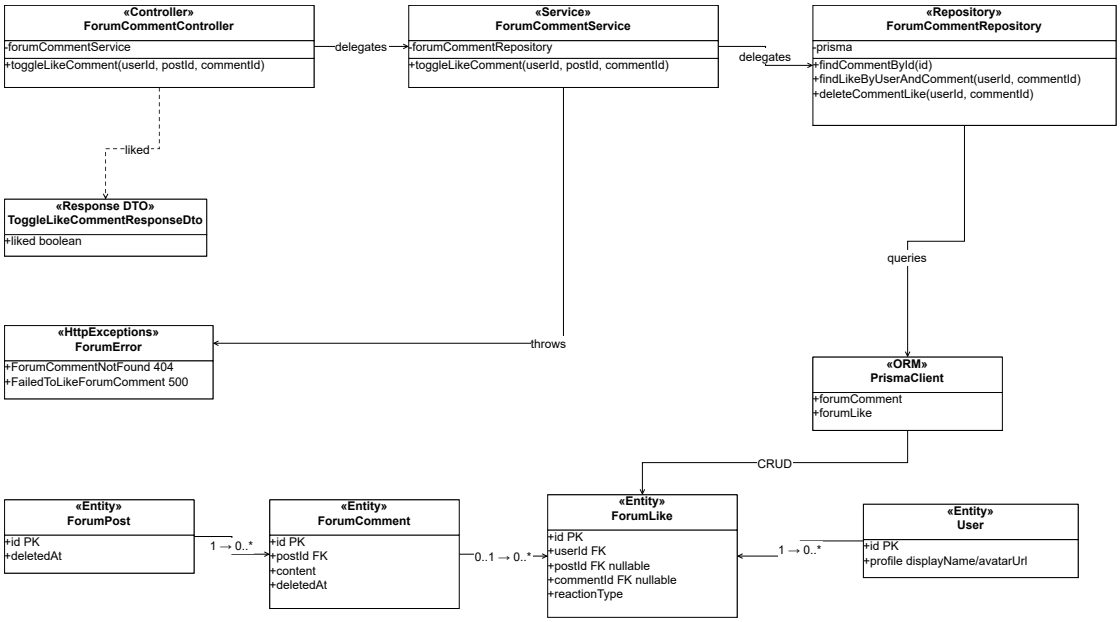

Figure 35: Class diagram of Unlike Comment

b. Sequence Diagram(s)

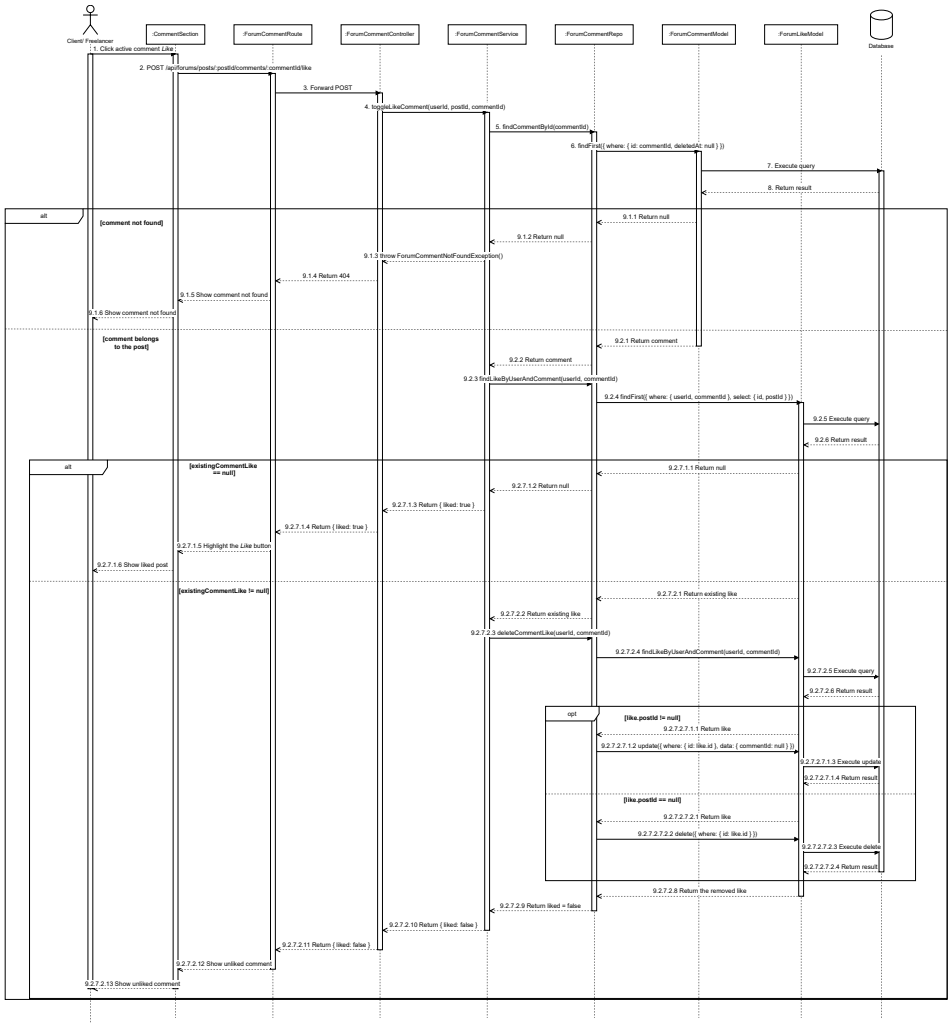

Figure 36: Sequence diagram of Unlike Comment

3.17 Report Content

a. Class Diagram

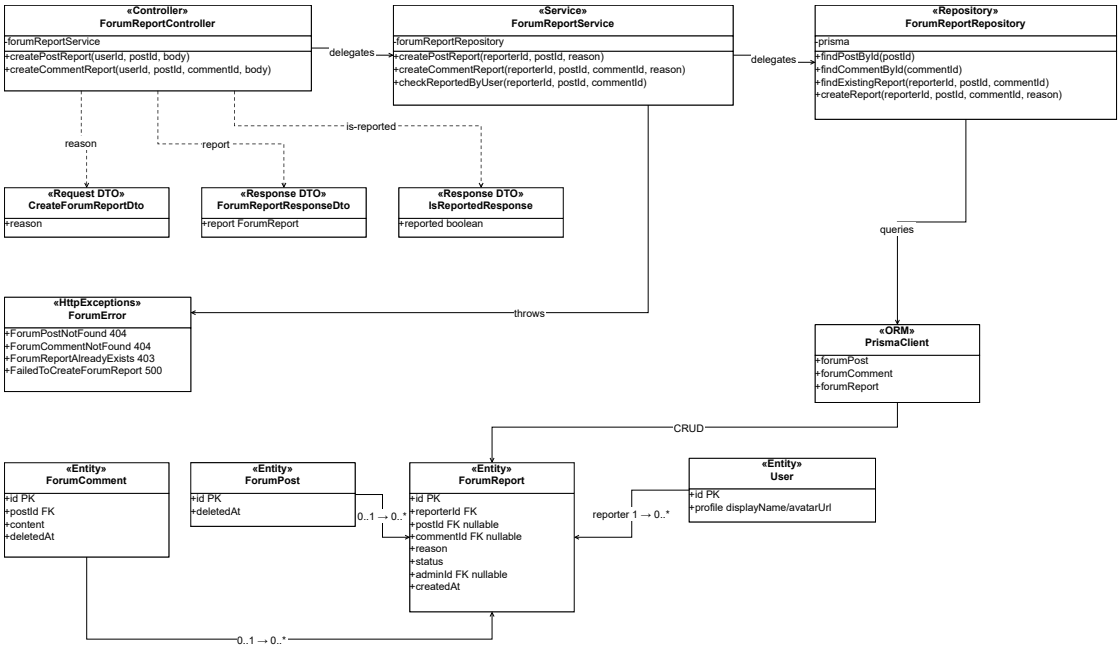

Figure 37: Class diagram of Report Content

b. Sequence Diagram(s)

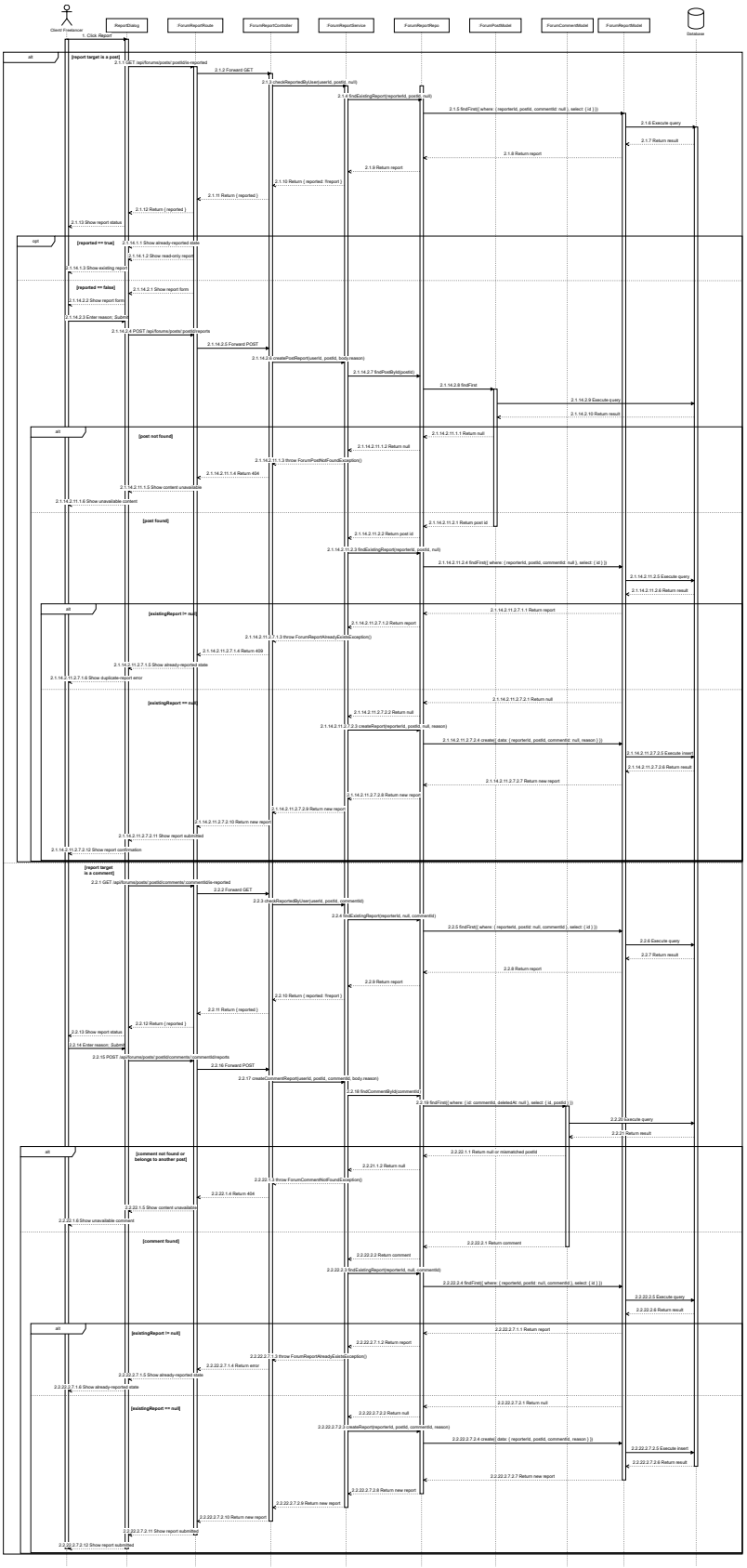

Figure 38: Sequence diagram of Report Content

3.18 Freelancer Profile Management

Public freelancer profile viewing and ownership-checked profile updates. Low-strength changes are submitted for review.

a. Class Diagram

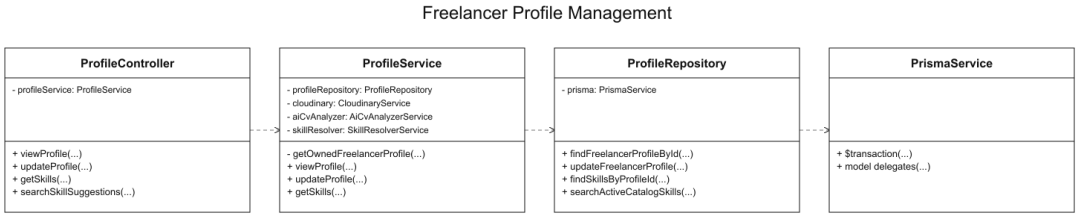

Figure 39: Class diagram of Freelancer Profile Management

b. Sequence Diagram(s)

3.18.1 UC-07.01 – View Freelancer Profile

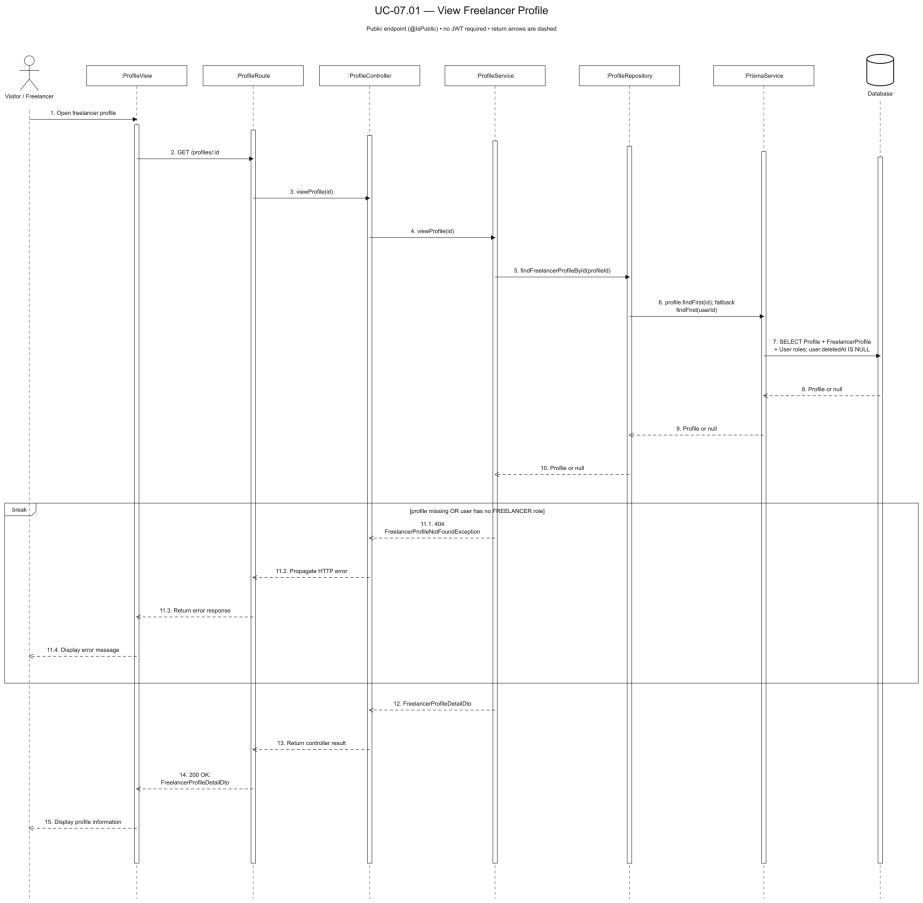

Figure 40: Sequence diagram of UC-07.01 View Freelancer Profile

3.18.2 UC-07.02 – Update Freelancer Profile

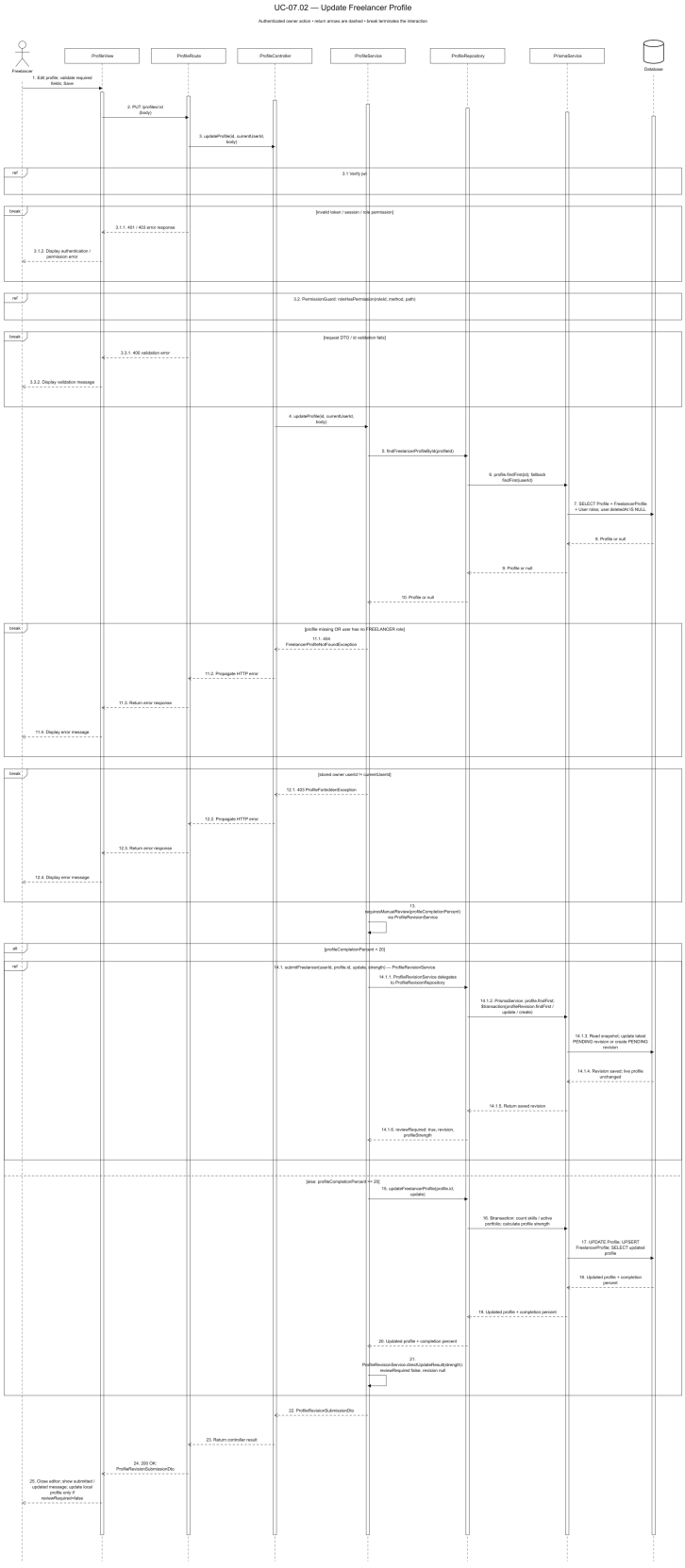

Figure 41: Sequence diagram of UC-07.02 Update Freelancer Profile

3.19 Freelancer Skill Management

The freelancer skill list supports catalog lookup, duplicate checking and ownership-checked association removal.

a. Class Diagram

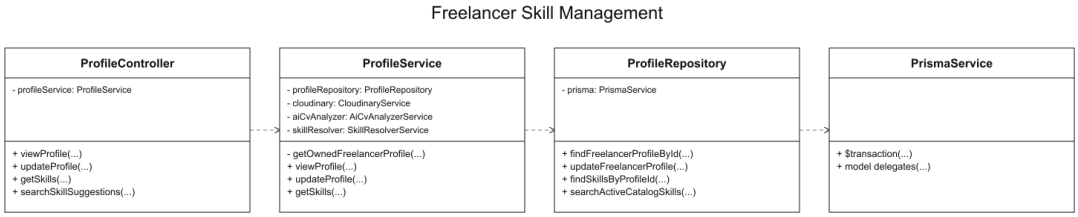

Figure 42: Class diagram of Freelancer Skill Management

b. Sequence Diagram(s)

3.19.1 UC-08.01 – View Freelancer Skill List

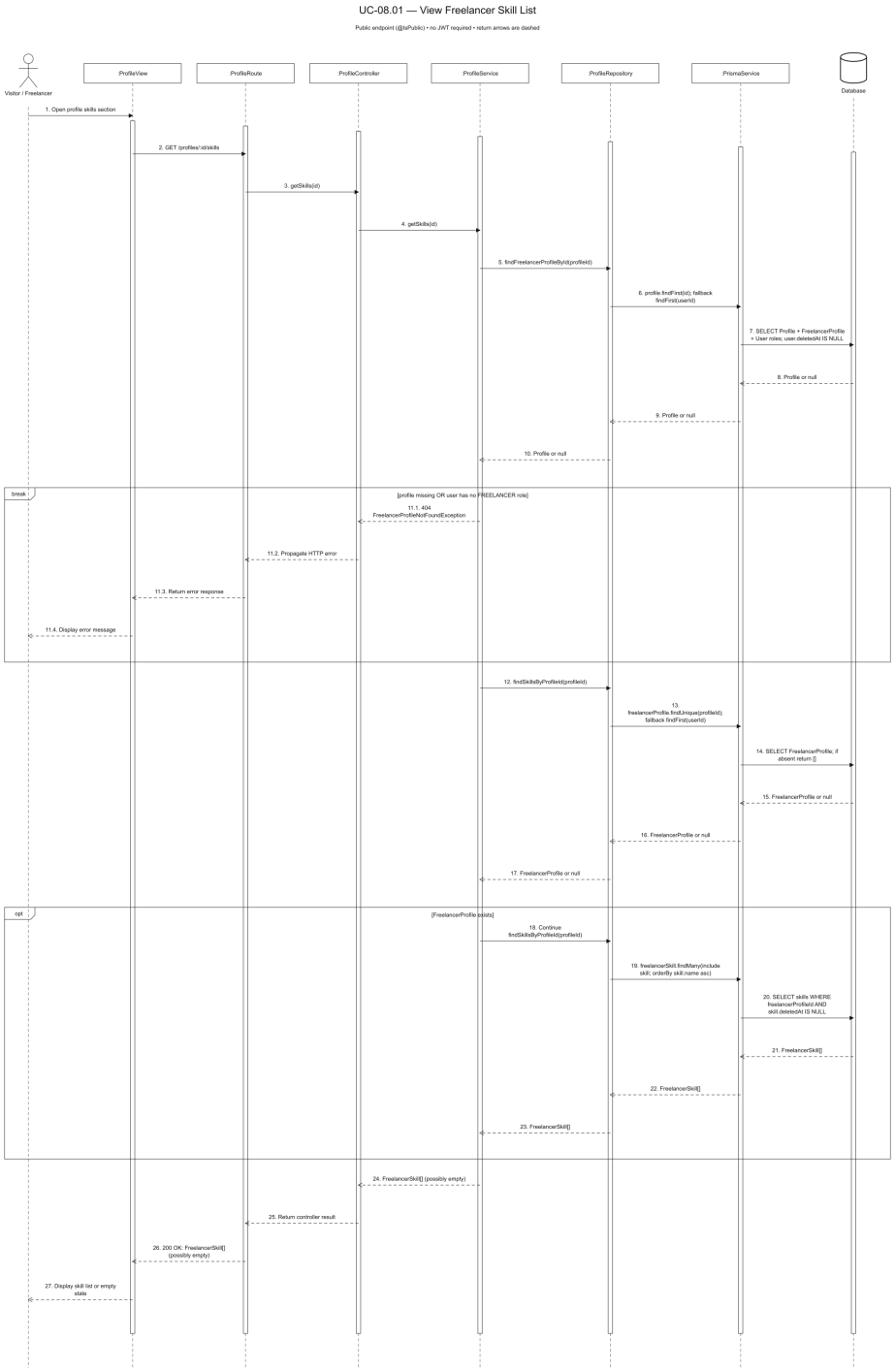

Figure 43: Sequence diagram of UC-08.01 View Freelancer Skill List

3.19.2 UC-08.02 – Add Freelancer Skill

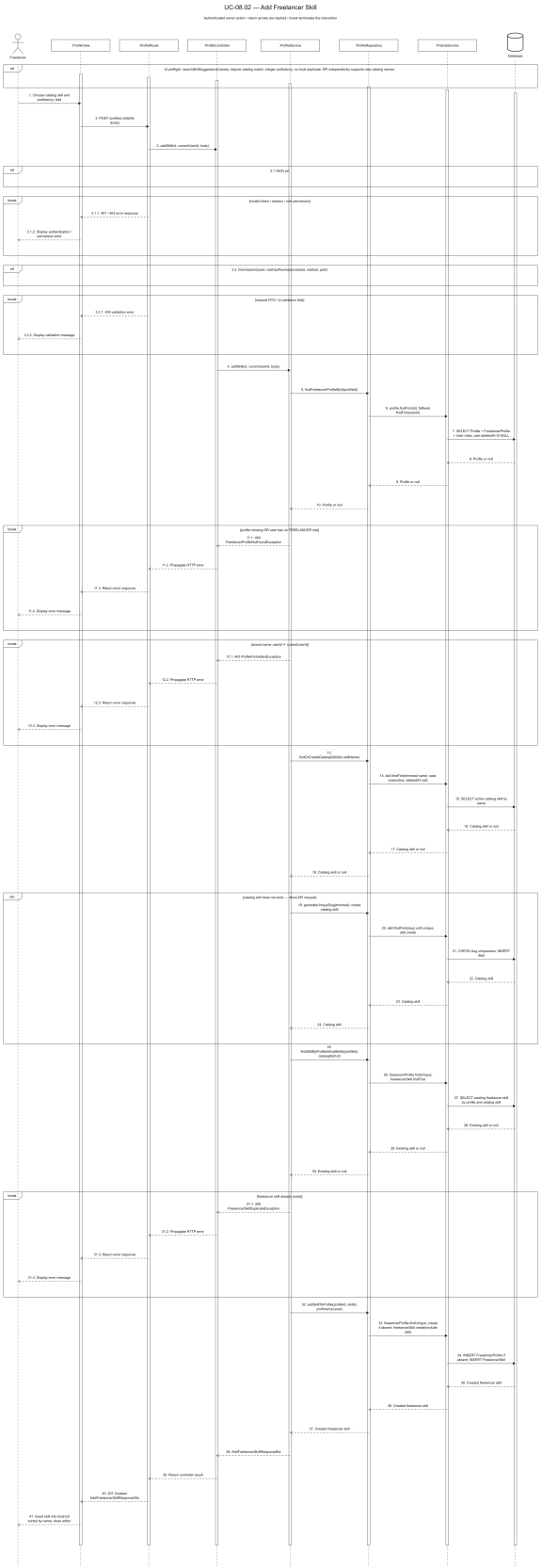

Figure 44: Sequence diagram of UC-08.02 Add Freelancer Skill

3.19.3 UC-08.03 – Delete Freelancer Skill

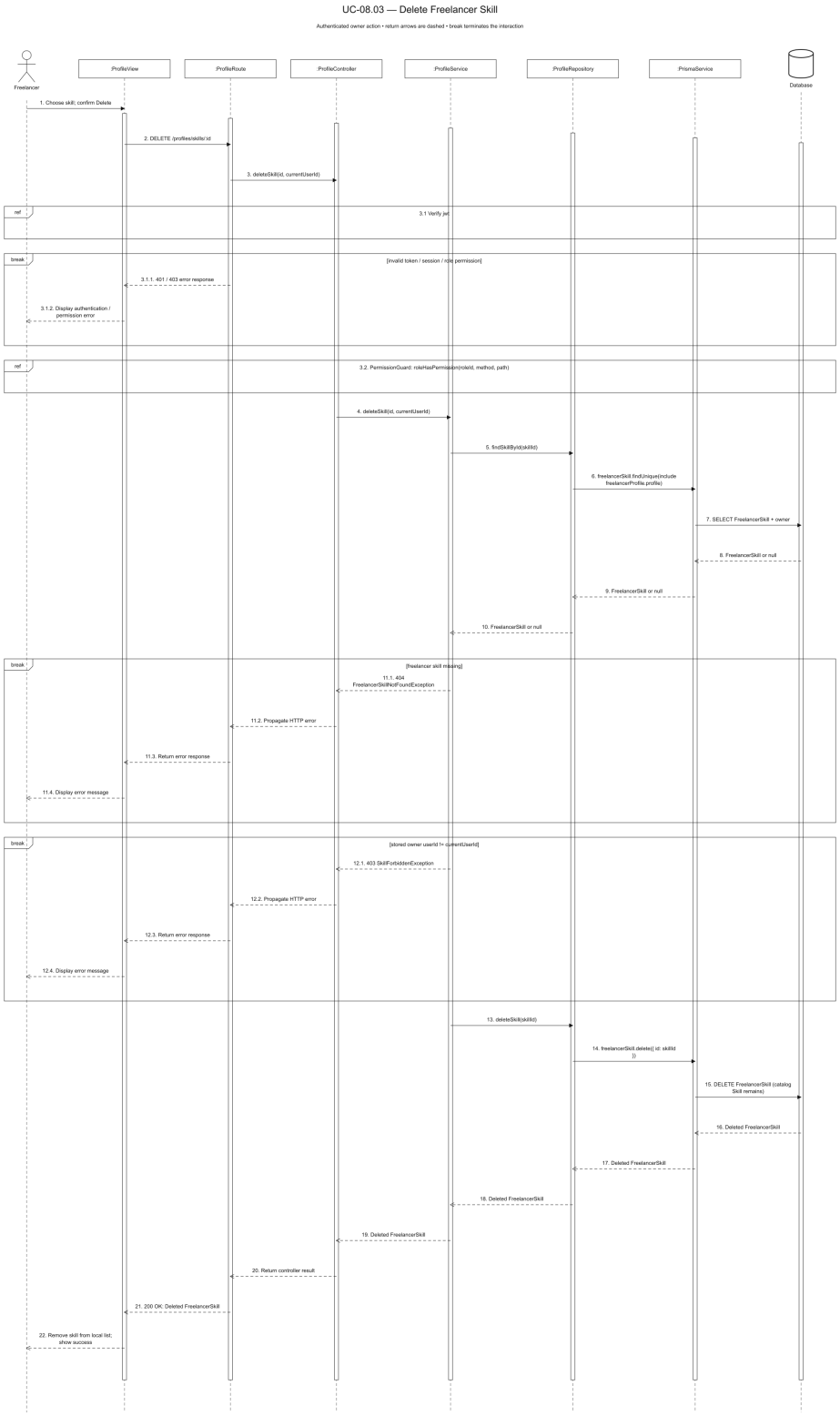

Figure 45: Sequence diagram of UC-08.03 Delete Freelancer Skill

3.20 Portfolio Management

Portfolio entries can be viewed individually or as a list. Owners can add, update and soft-delete their portfolio items.

a. Class Diagram

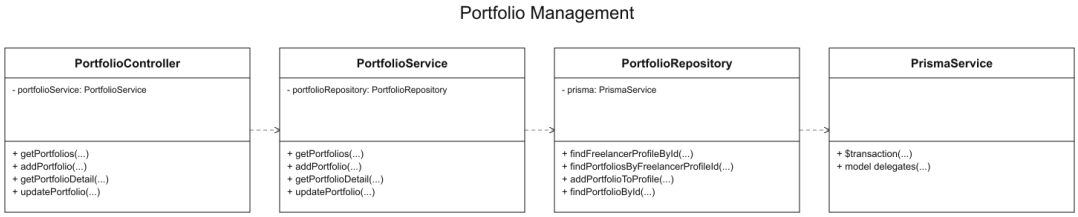

Figure 46: Class diagram of Portfolio Management

b. Sequence Diagram(s)

3.20.1 UC-09.01 – View Portfolio List

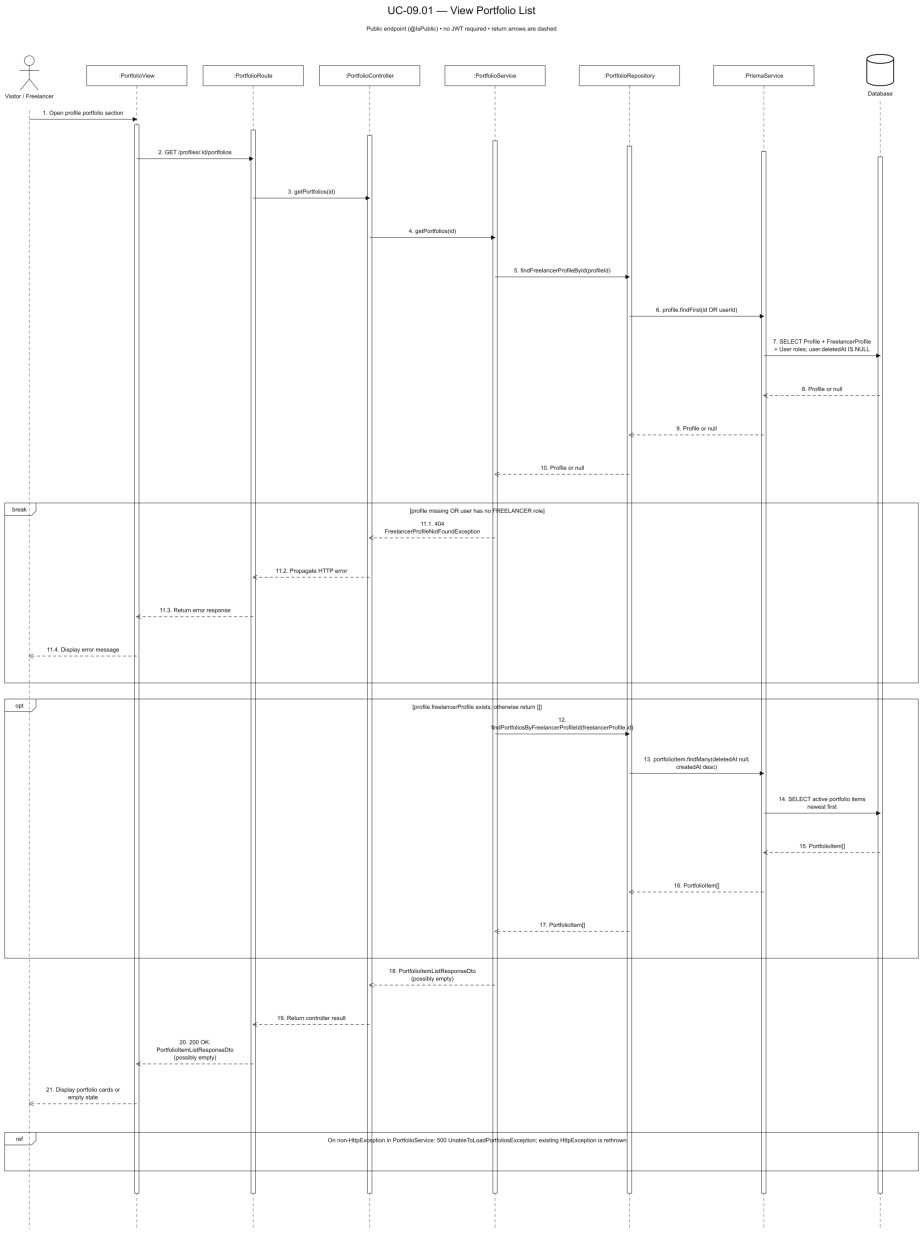

Figure 47: Sequence diagram of UC-09.01 View Portfolio List

3.20.2 UC-09.02 – Add Portfolio

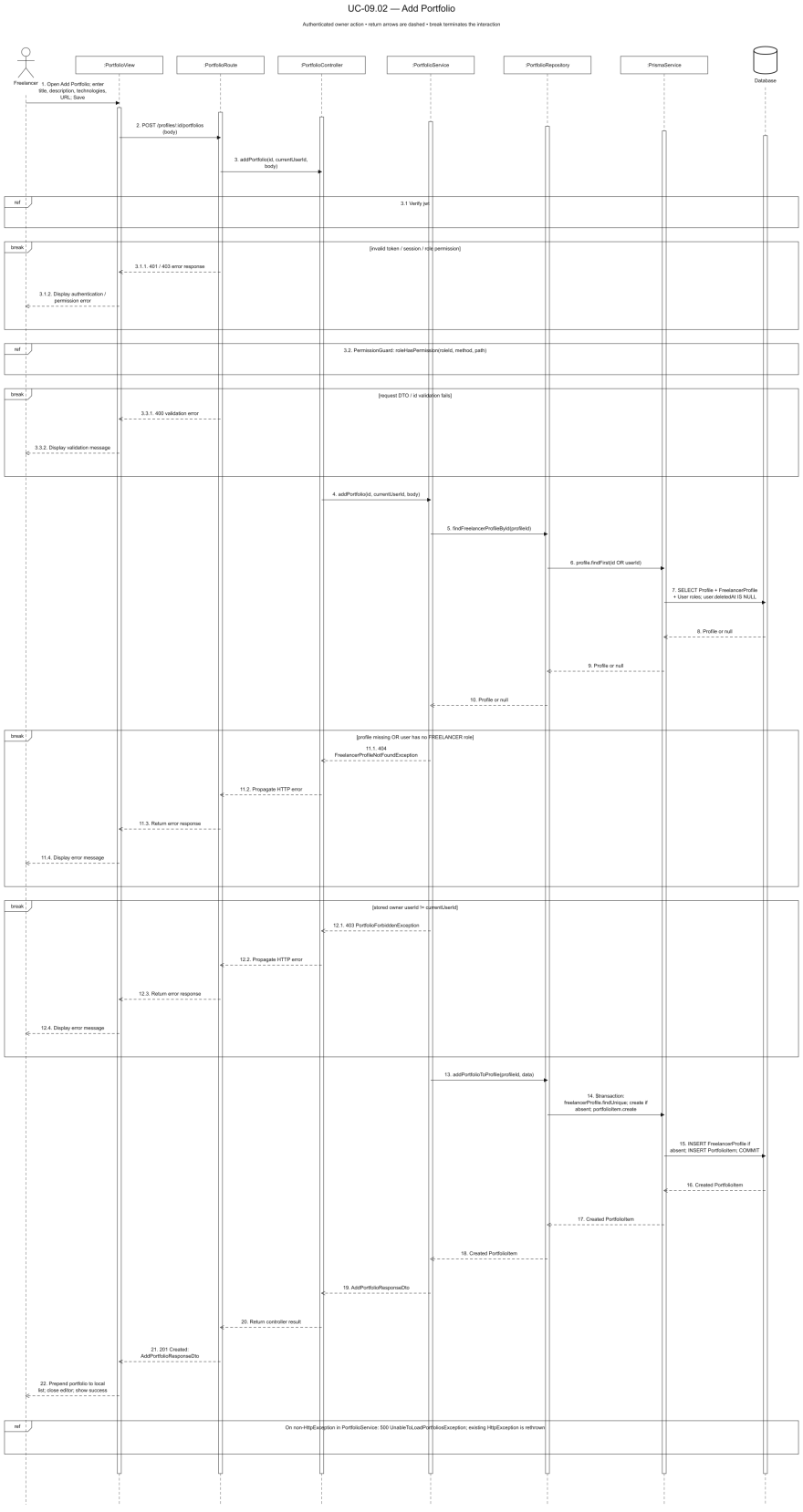

Figure 48: Sequence diagram of UC-09.02 Add Portfolio

3.20.3 UC-09.03 – View Portfolio Detail

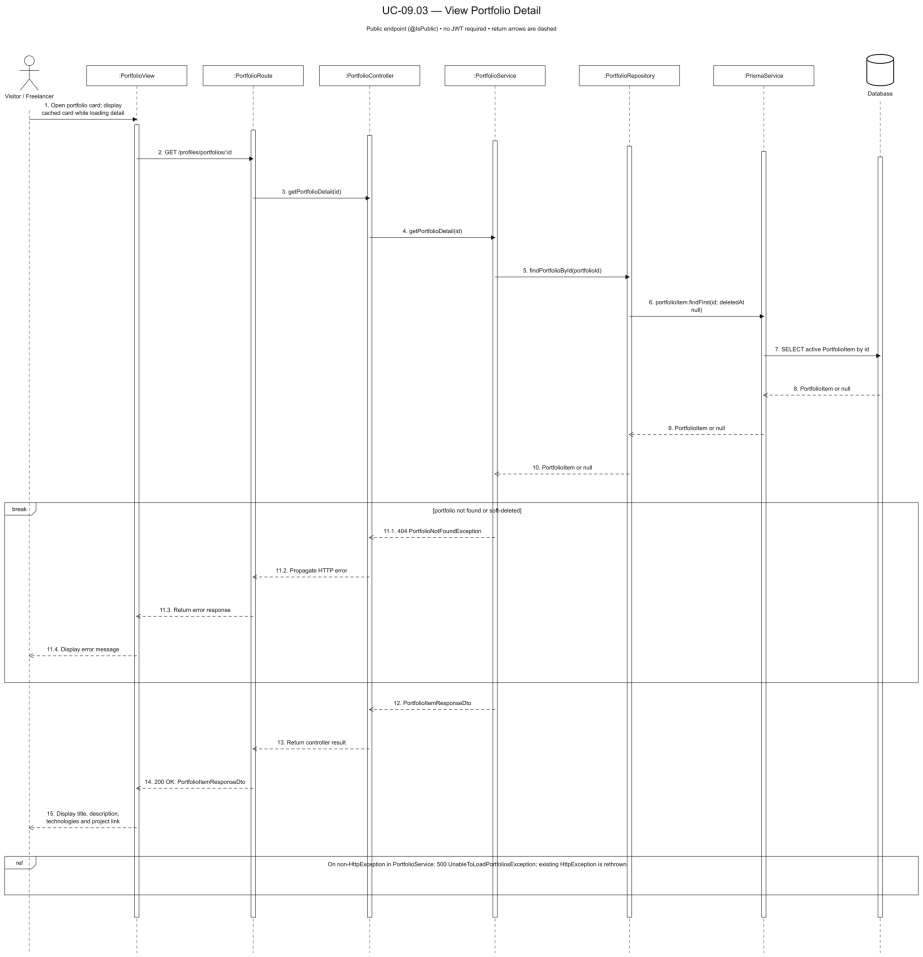

Figure 49: Sequence diagram of UC-09.03 View Portfolio Detail

3.20.4 UC-09.04 – Update Portfolio

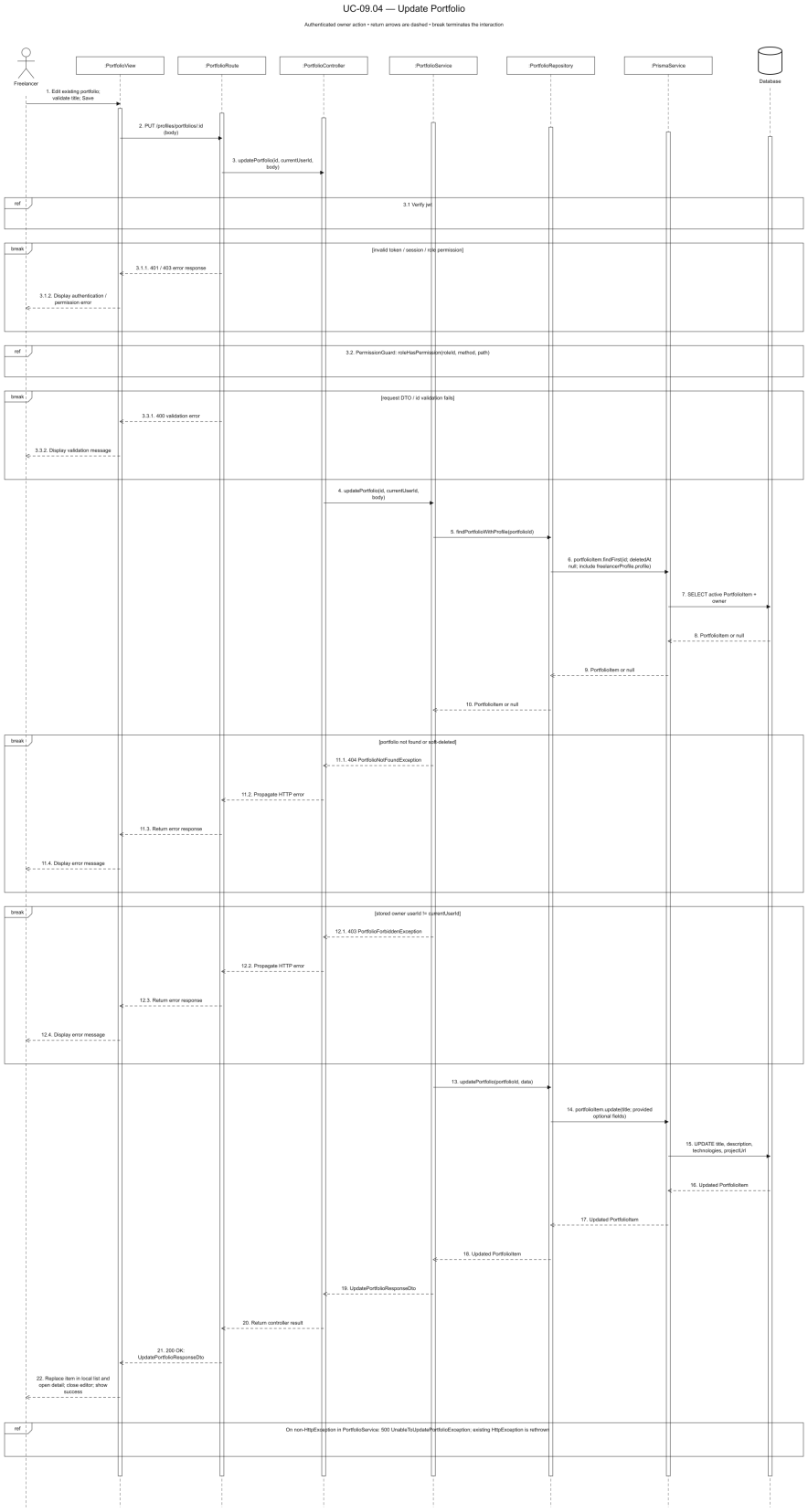

Figure 50: Sequence diagram of UC-09.04 Update Portfolio

3.20.5 UC-09.05 – Delete Portfolio

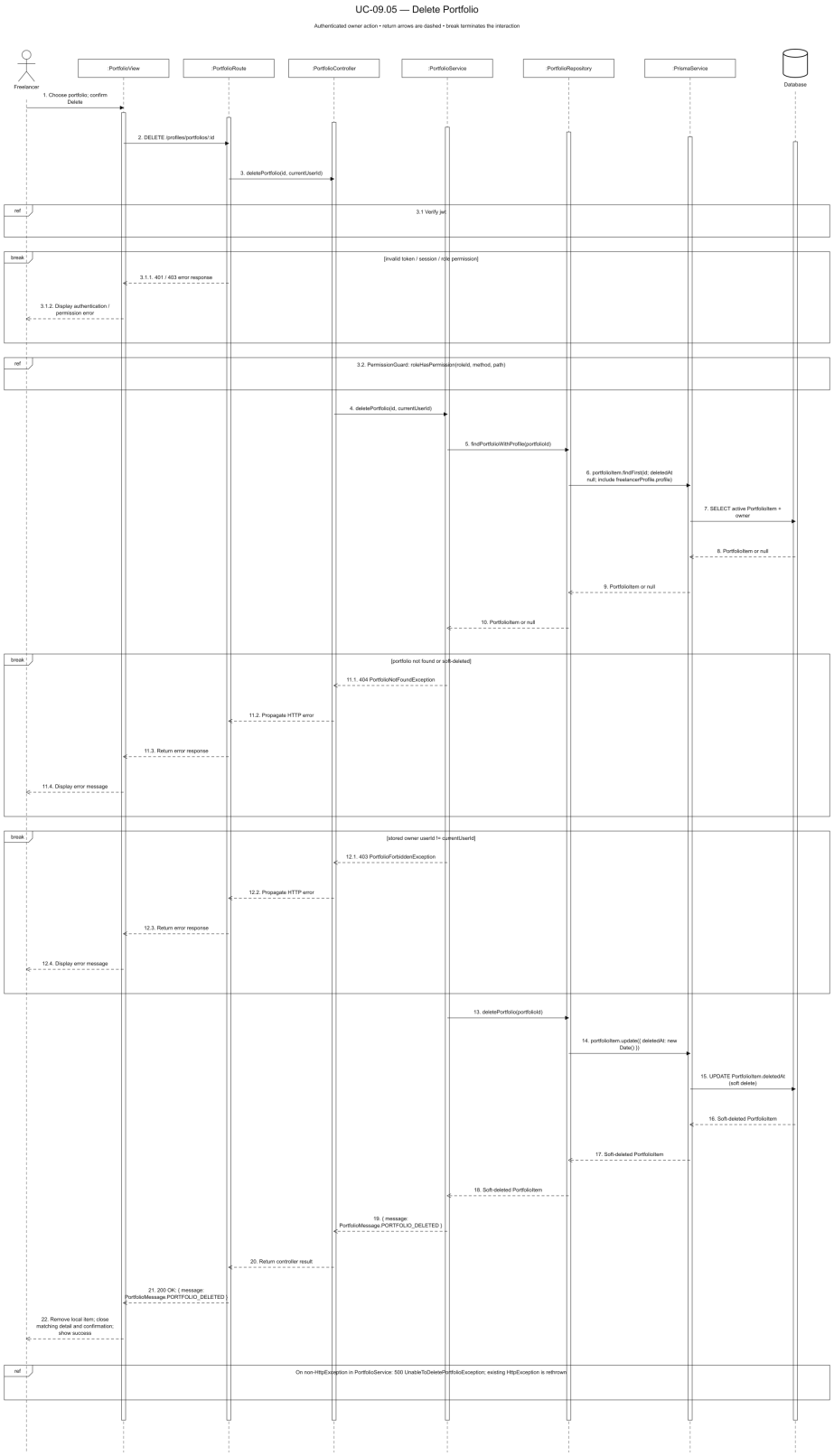

Figure 51: Sequence diagram of UC-09.05 Delete Portfolio

3.21 Authentication

The authentication module covers account registration, email/password and Google OAuth login, logout, password recovery, and the JWT/session flow shared by all protected endpoints.

a. Class Diagram

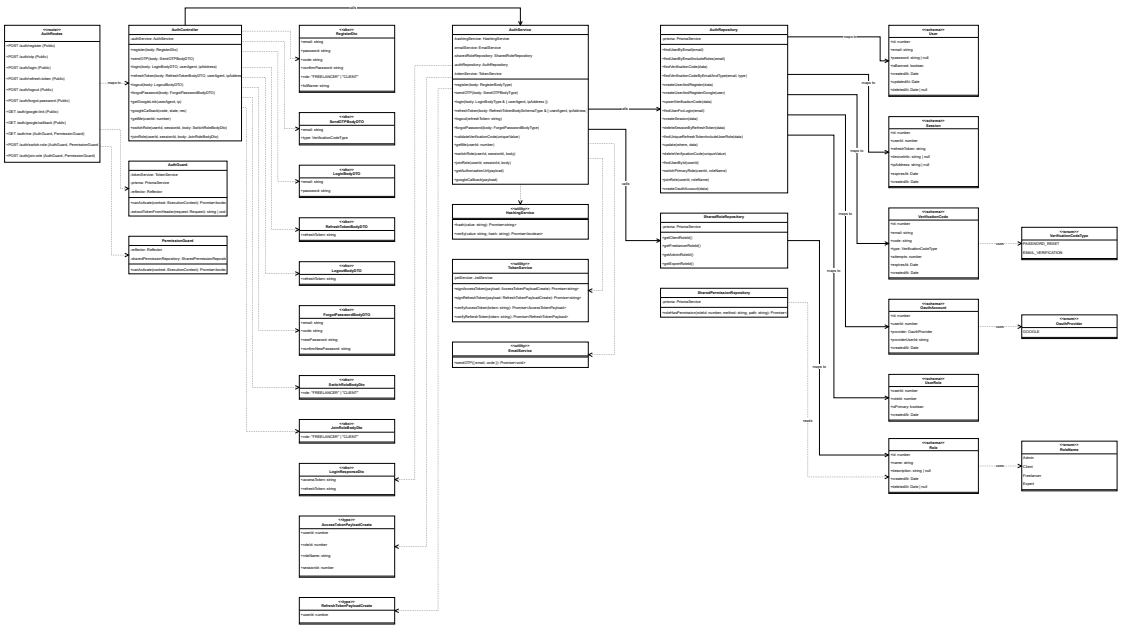

Figure 52: Class diagram of Authentication

b. Sequence Diagram(s)

3.21.1 UC-01 – Login with Google

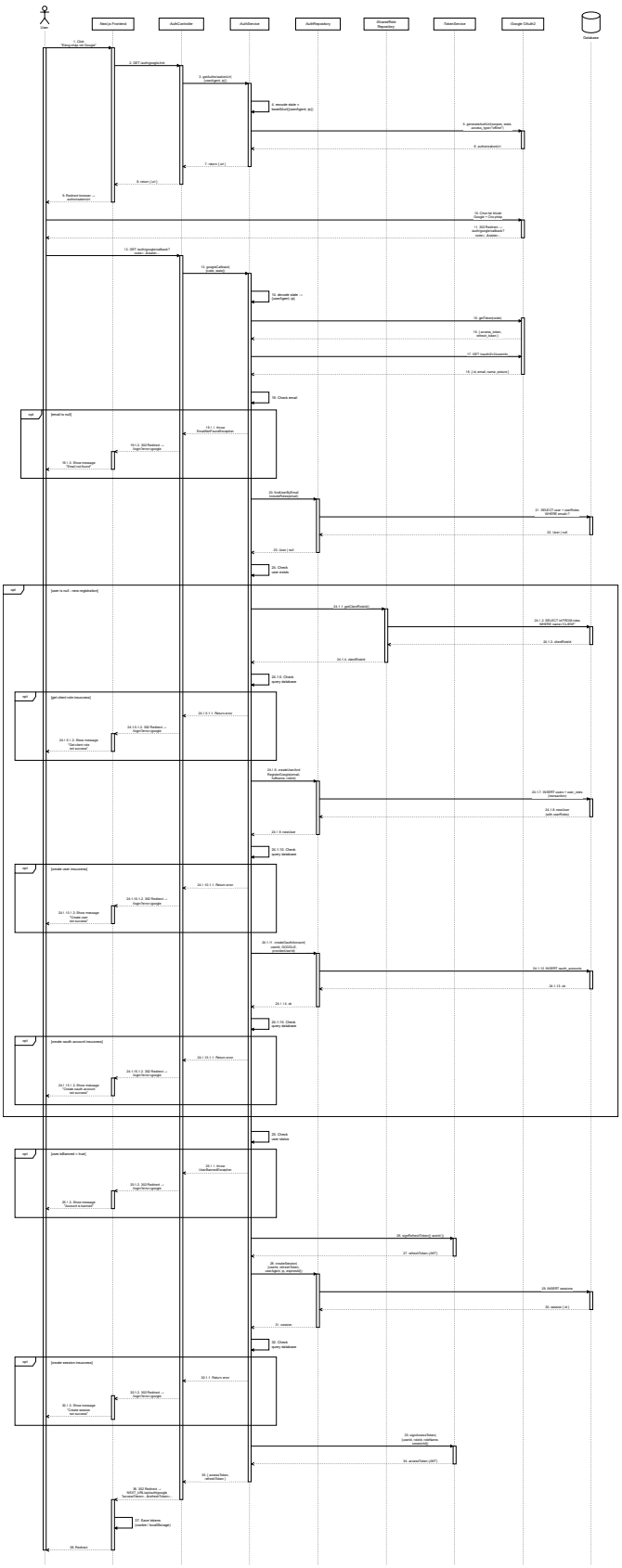

Figure 53: Sequence diagram of UC-01 Login with Google

3.21.2 UC-02 – Login

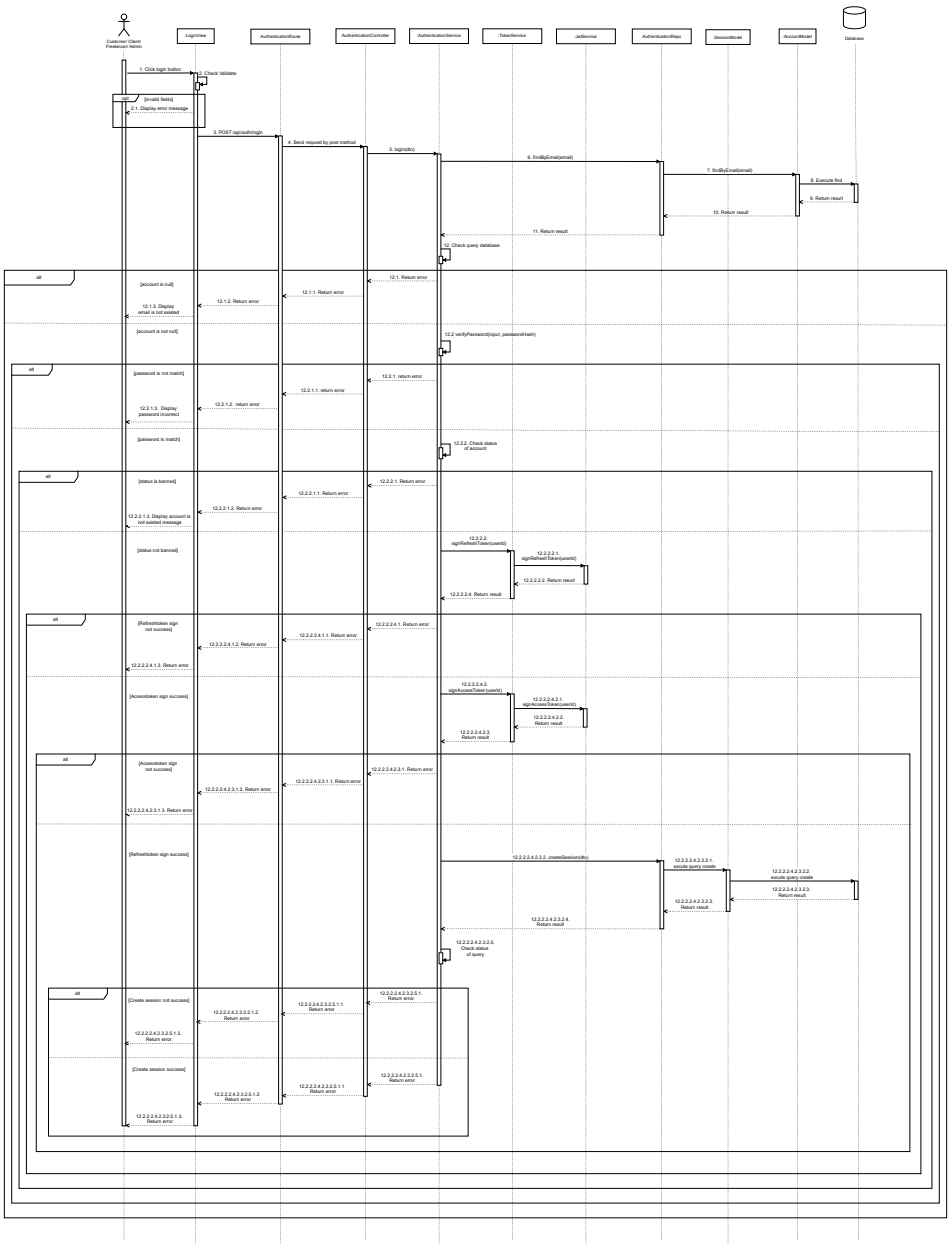

Figure 54: Sequence diagram of UC-02 Login

3.21.3 UC-03 – Logout

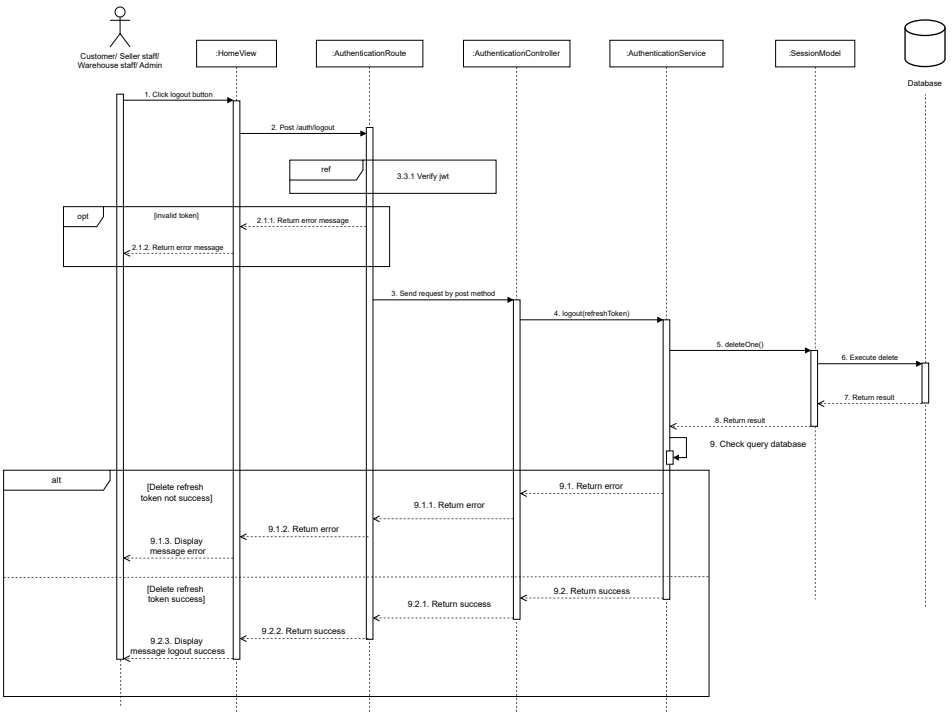

Figure 55: Sequence diagram of UC-03 Logout

3.21.4 UC-04 – Register

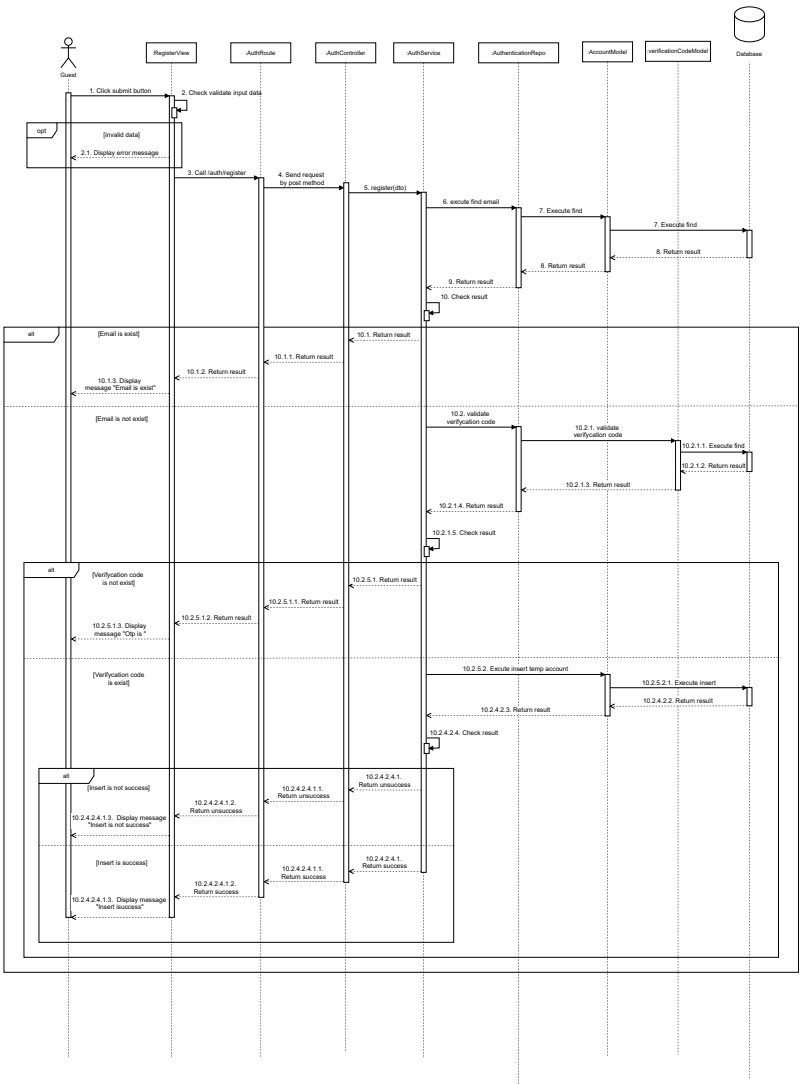

Figure 56: Sequence diagram of UC-04 Register

3.21.5 UC-05 – Forgot Password

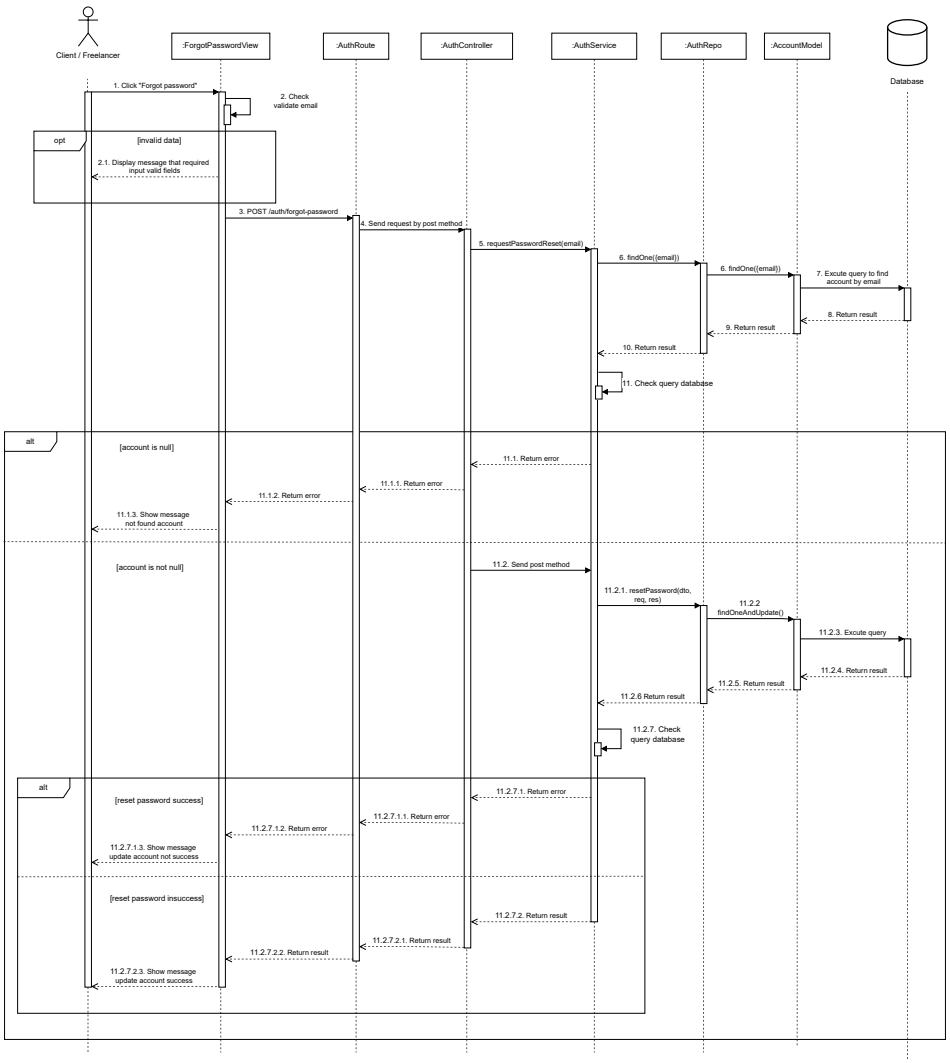

Figure 57: Sequence diagram of UC-05 Forgot Password

3.22 Role Management

Role Management lets an administrator inspect the role catalogue, maintain role metadata, and replace the permission set assigned to a role.

a. Class Diagram

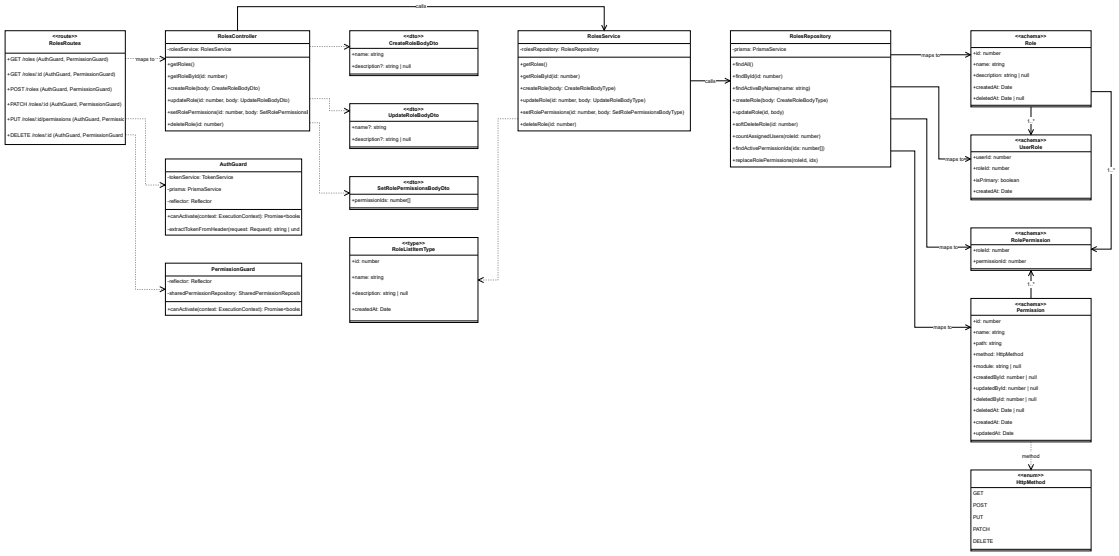

Figure 58: Class diagram of Role Management

b. Sequence Diagram(s)

3.22.1 UC-06.01 – View Role List

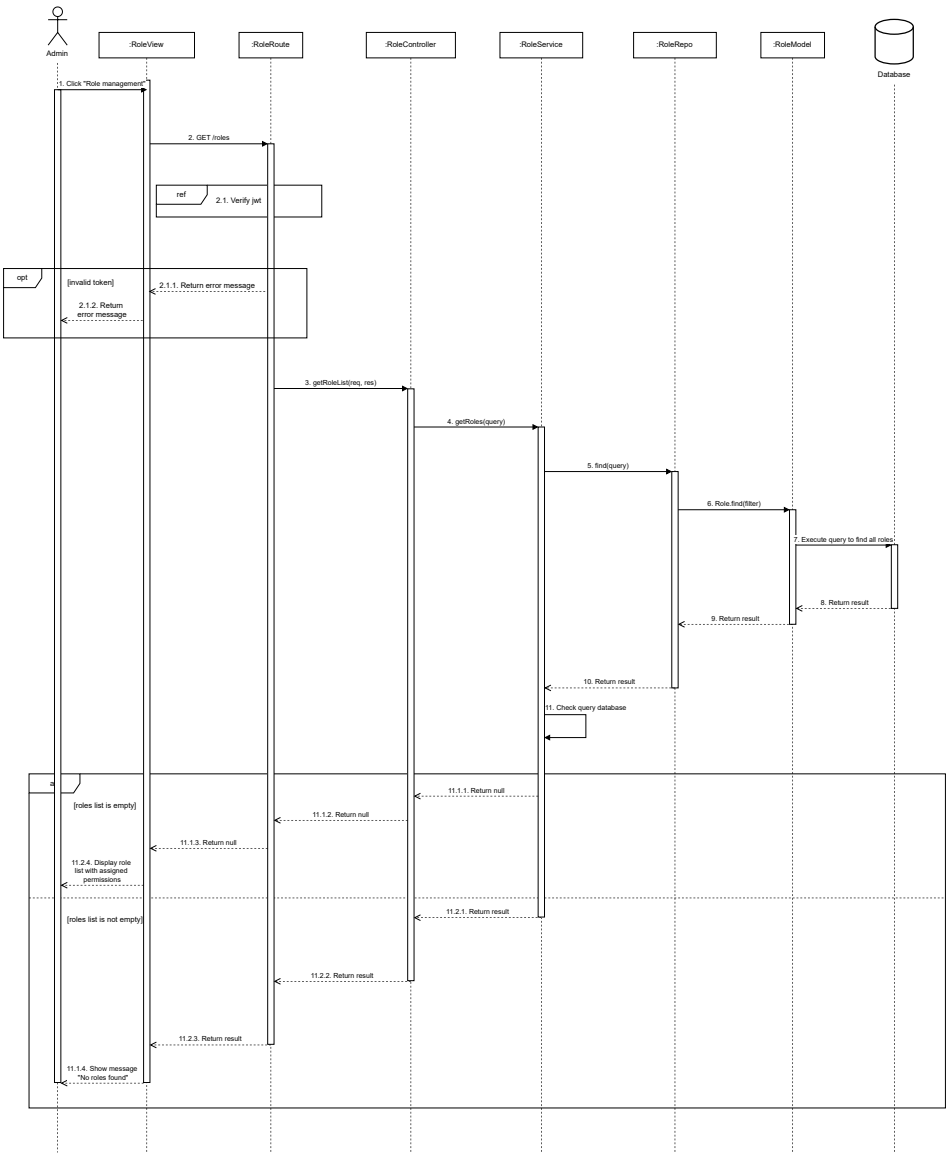

Figure 59: Sequence diagram of UC-06.01 View Role List

3.22.2 UC-06.02 – View Role Detail

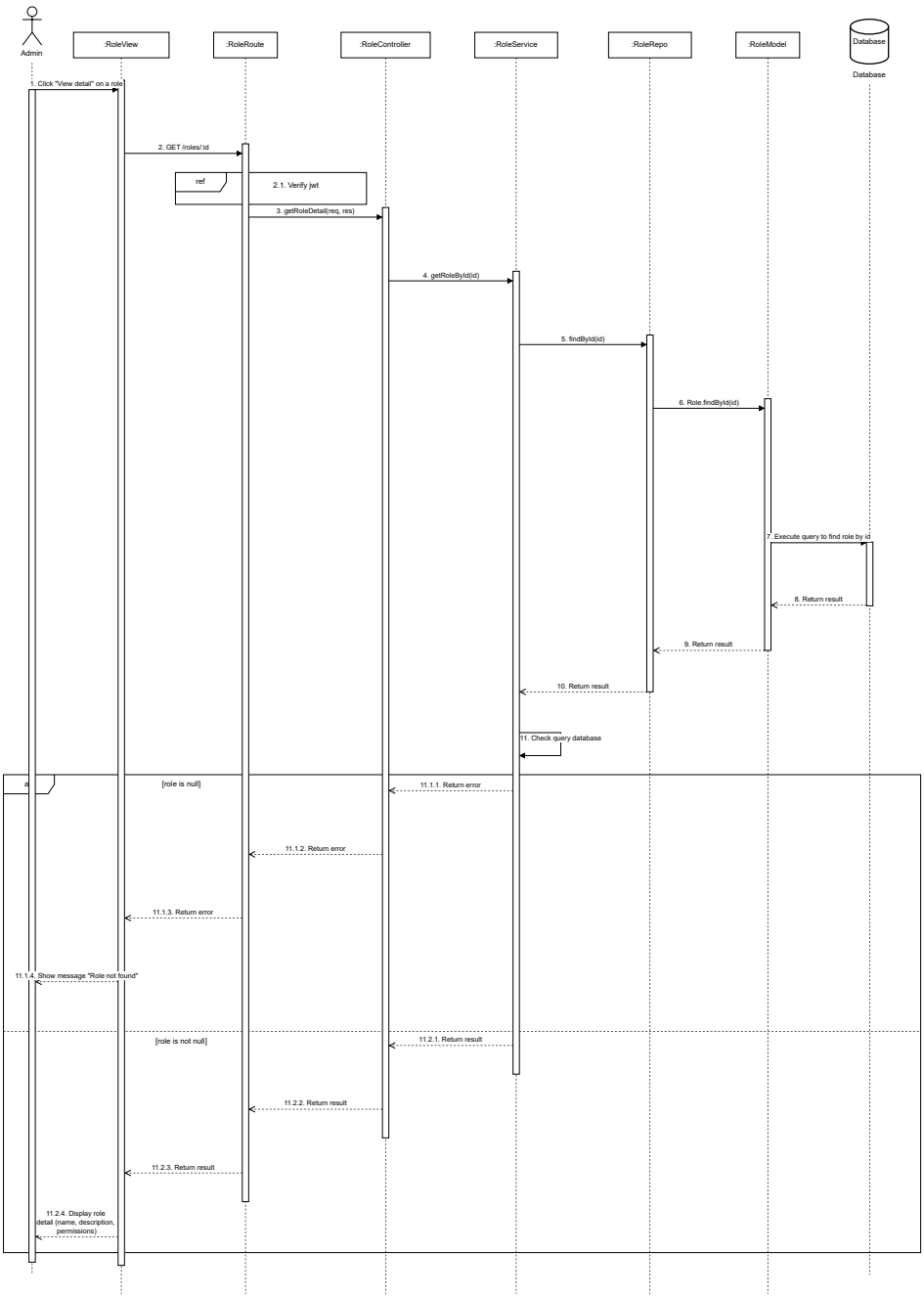

Figure 60: Sequence diagram of UC-06.02 View Role Detail

3.22.3 UC-06.03 – Create Role

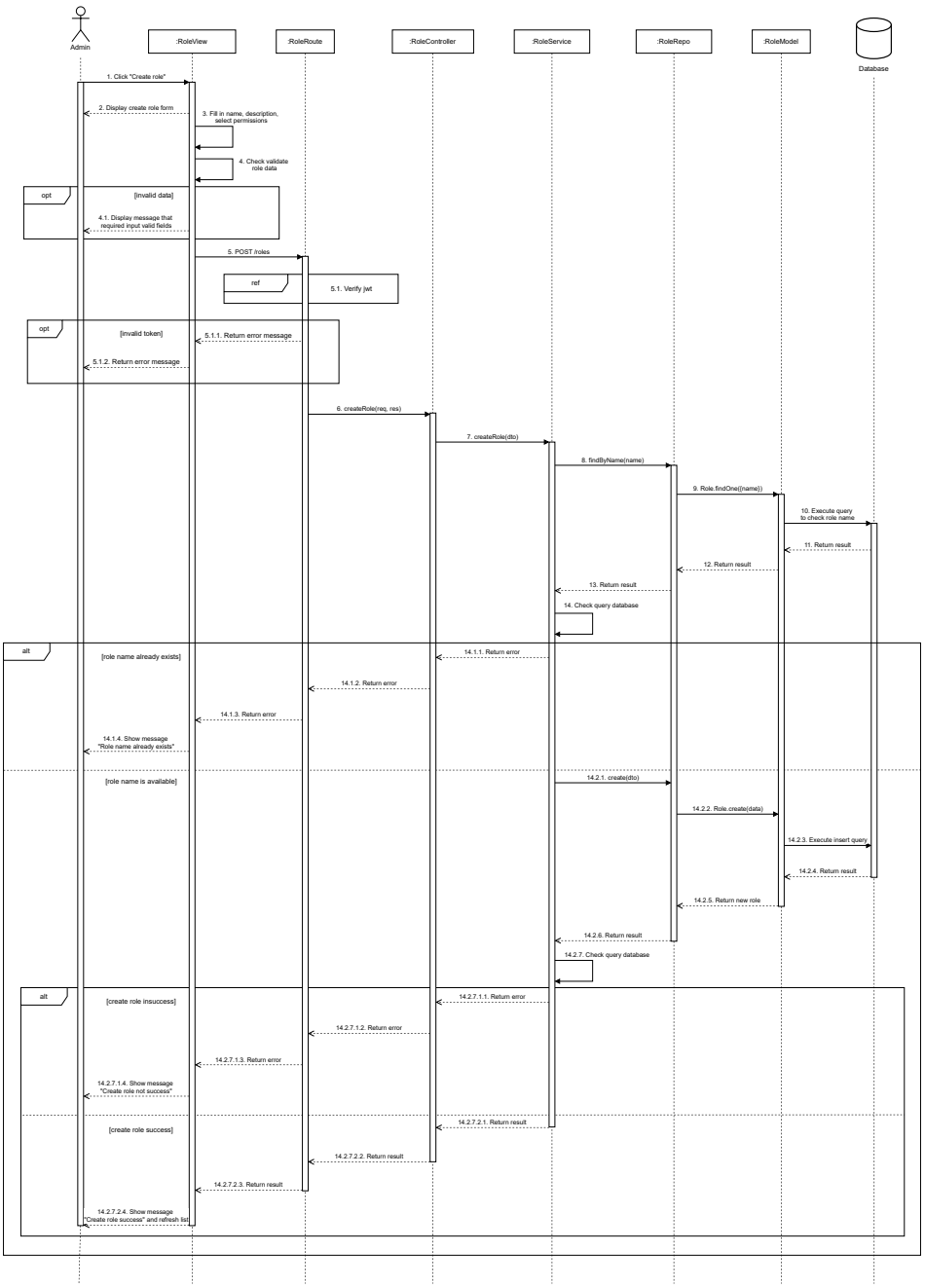

Figure 61: Sequence diagram of UC-06.03 Create Role

3.22.4 UC-06.04 – Update Role

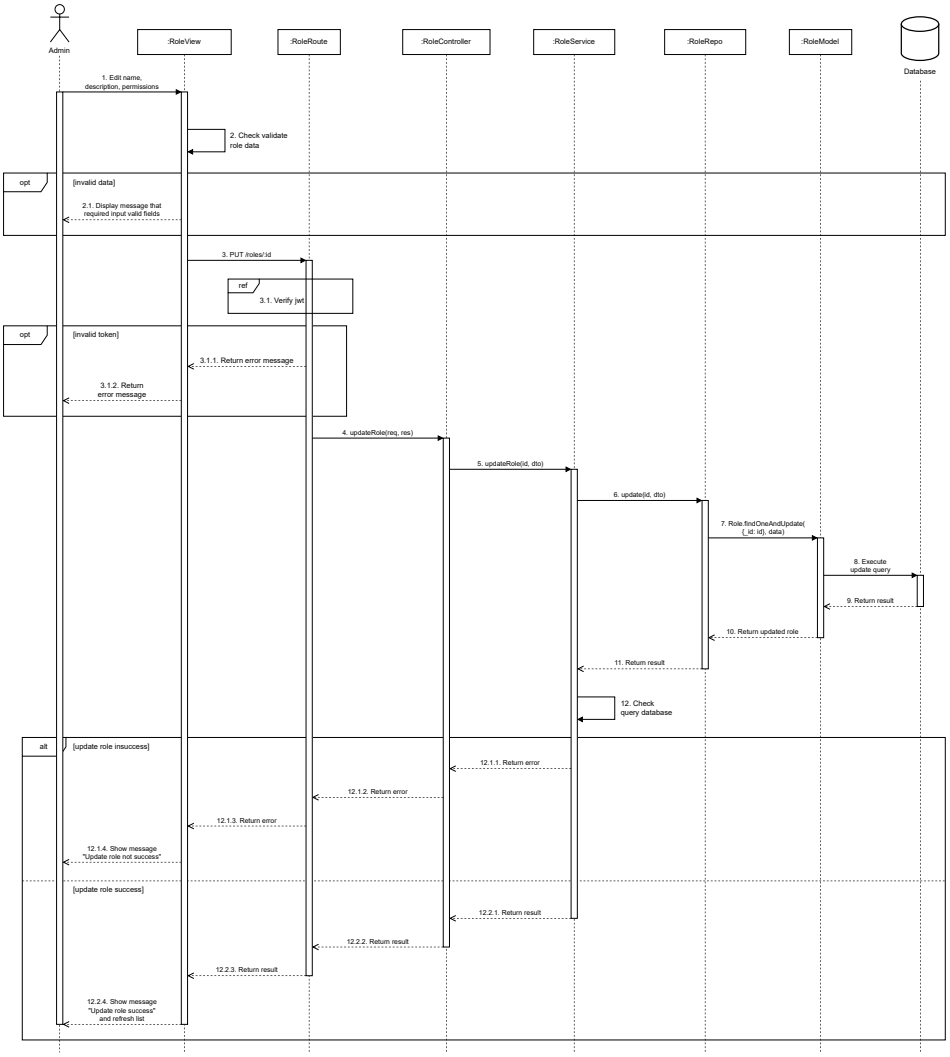

Figure 62: Sequence diagram of UC-06.04 Update Role

3.22.5 UC-06.05 – Delete Role

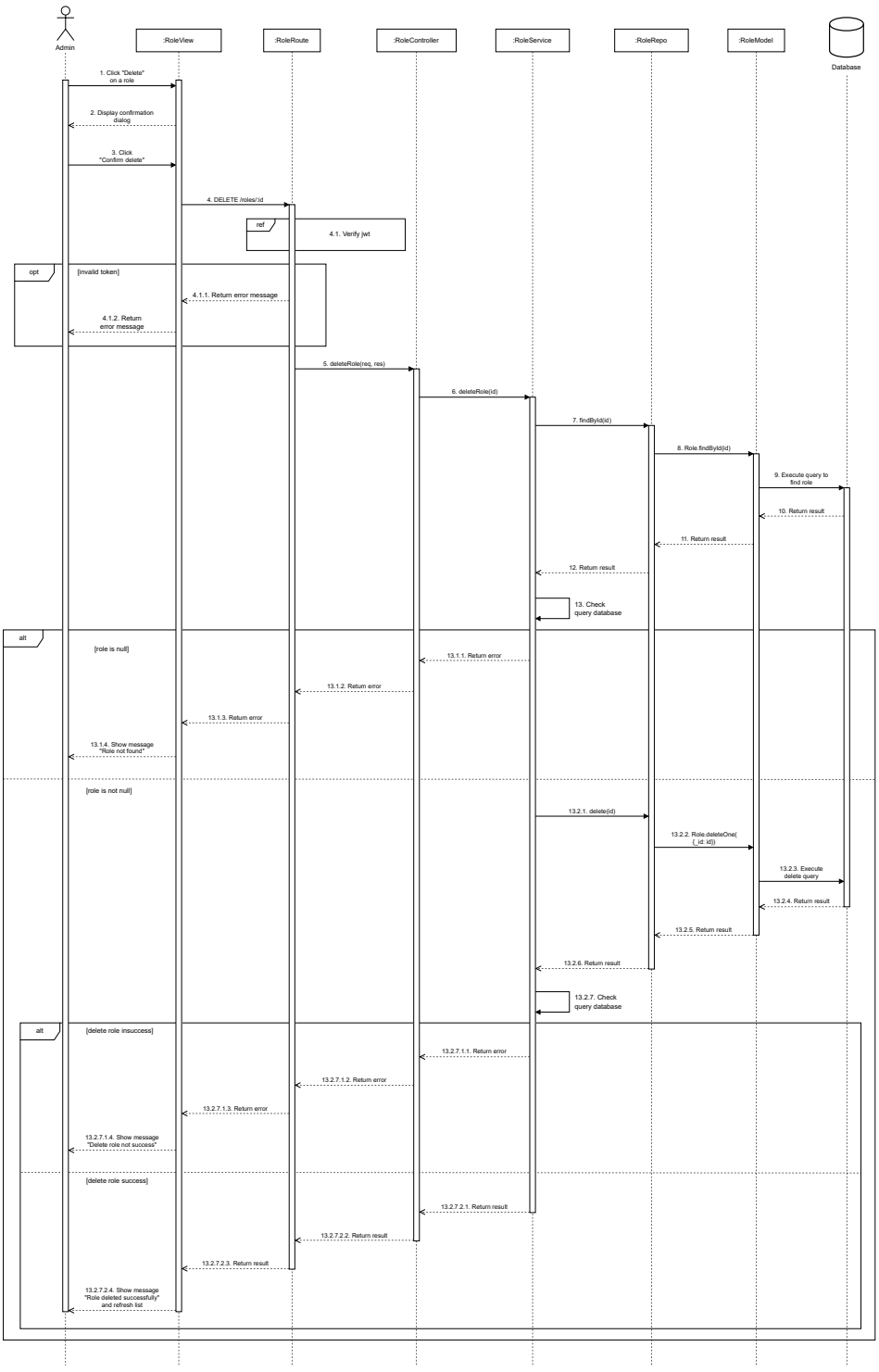

Figure 63: Sequence diagram of UC-06.05 Delete Role

3.22.6 UC-06.06 – Update Role Permissions

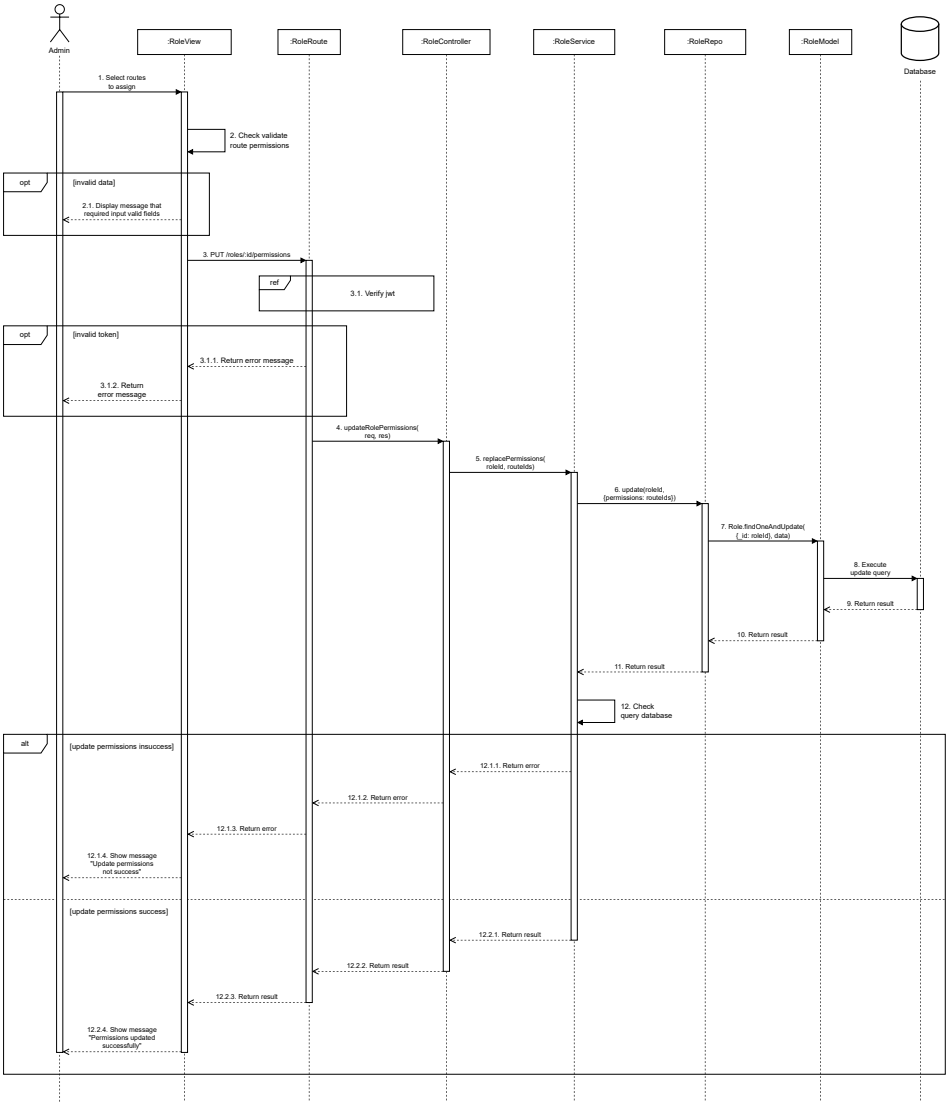

Figure 64: Sequence diagram of UC-06.06 Update Role Permissions

3.23 View Conversation List

a. Class Diagram

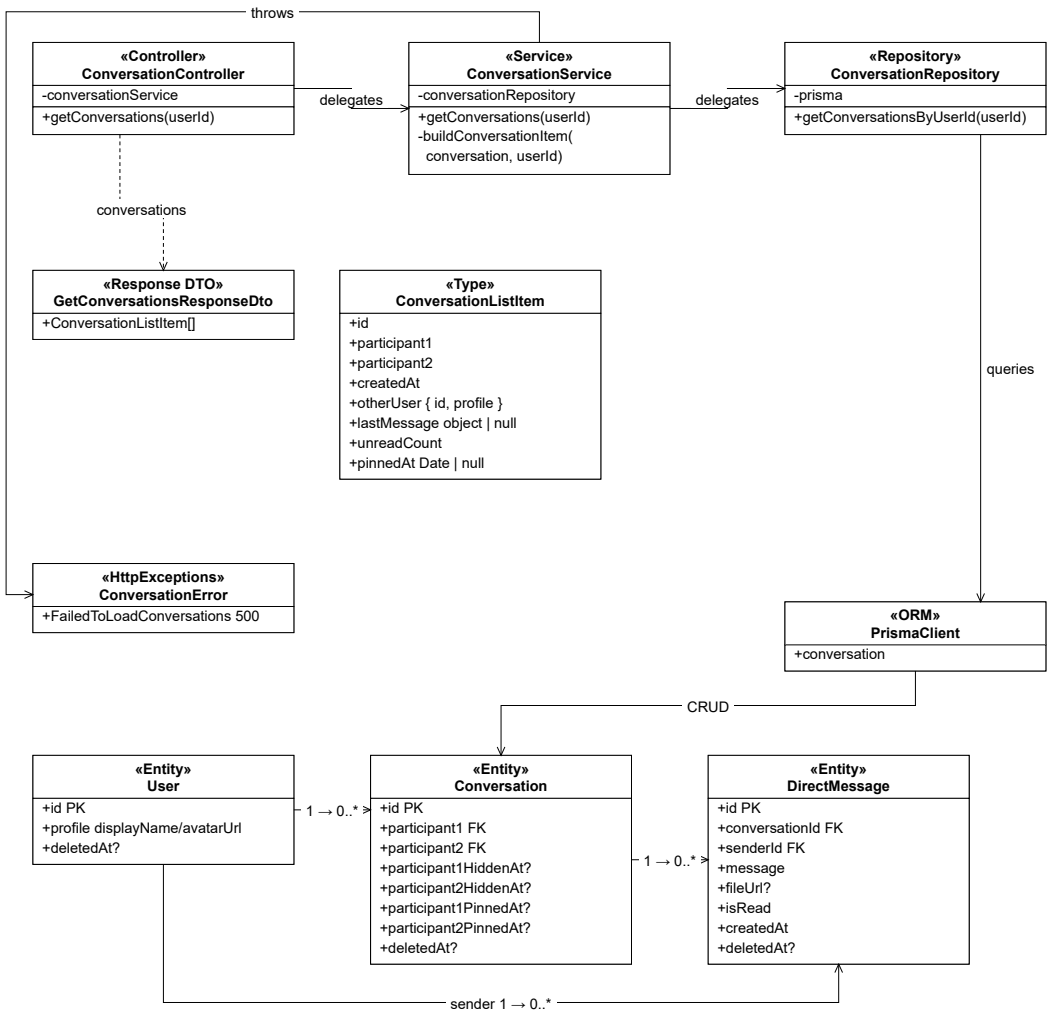

Figure 65: Class diagram of View Conversation List

b. Sequence Diagram(s)

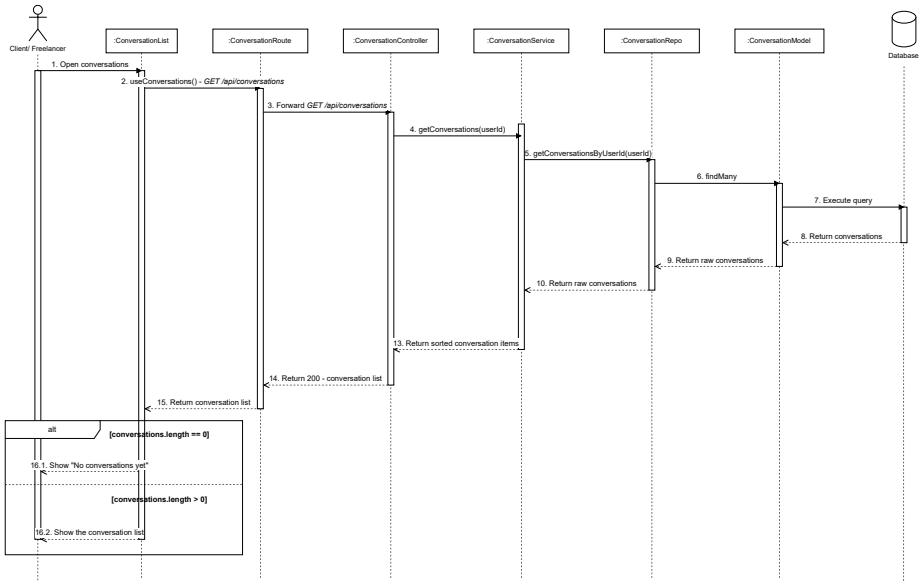

Figure 66: Sequence diagram of View Conversation List

3.24 Start Conversation

a. Class Diagram

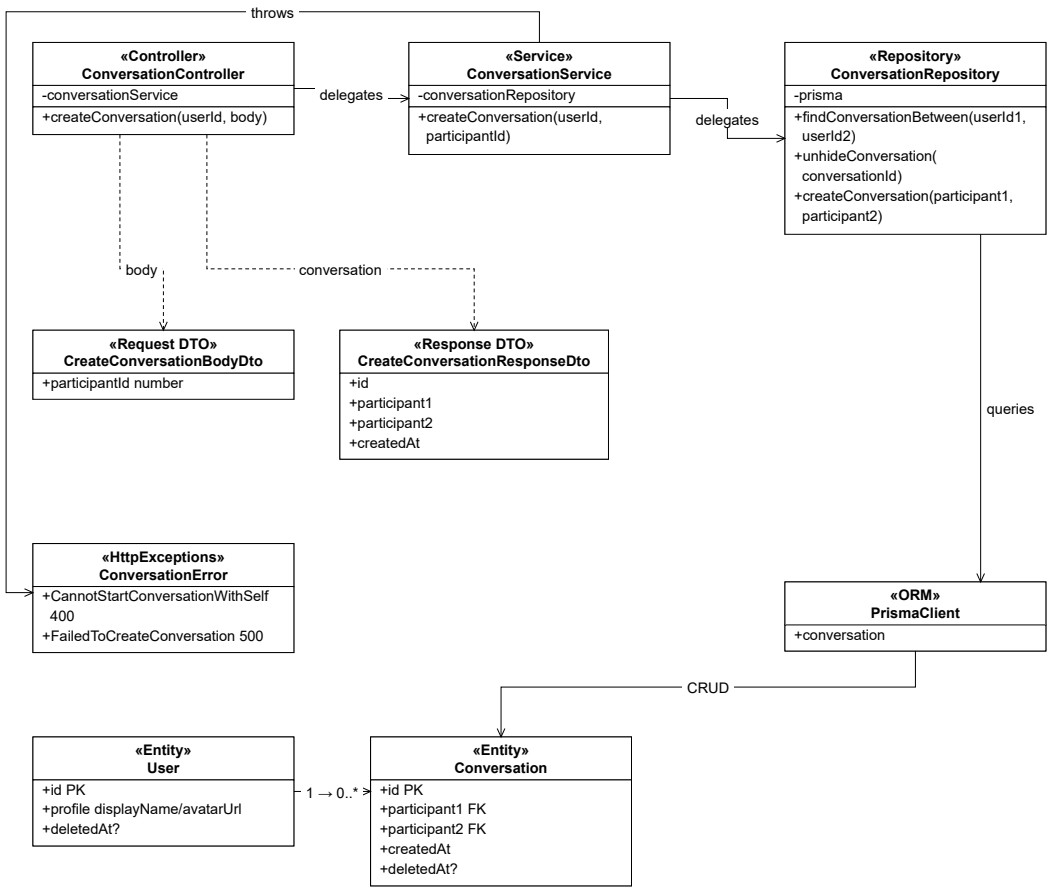

Figure 67: Class diagram of Start Conversation

b. Sequence Diagram(s)

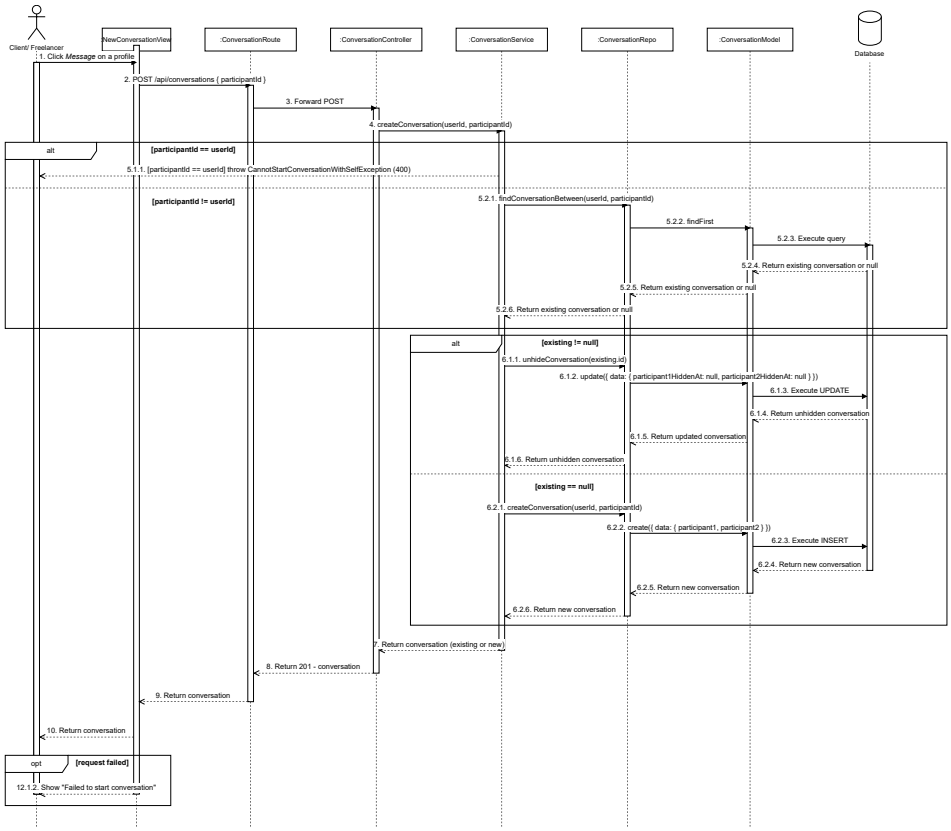

Figure 68: Sequence diagram of Start Conversation

3.25 View Conversation Detail

a. Class Diagram

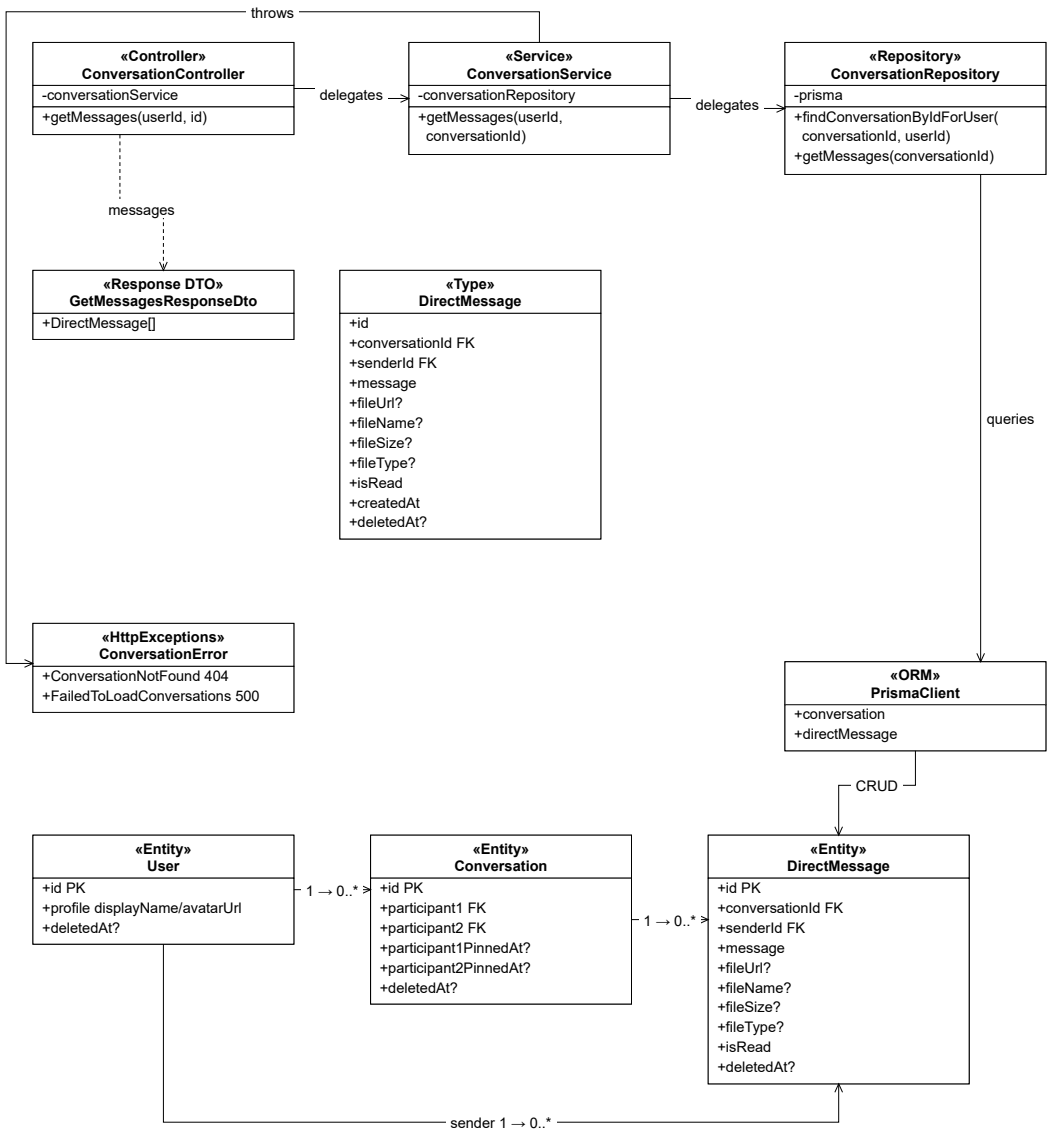

Figure 69: Class diagram of View Conversation Detail

b. Sequence Diagram(s)

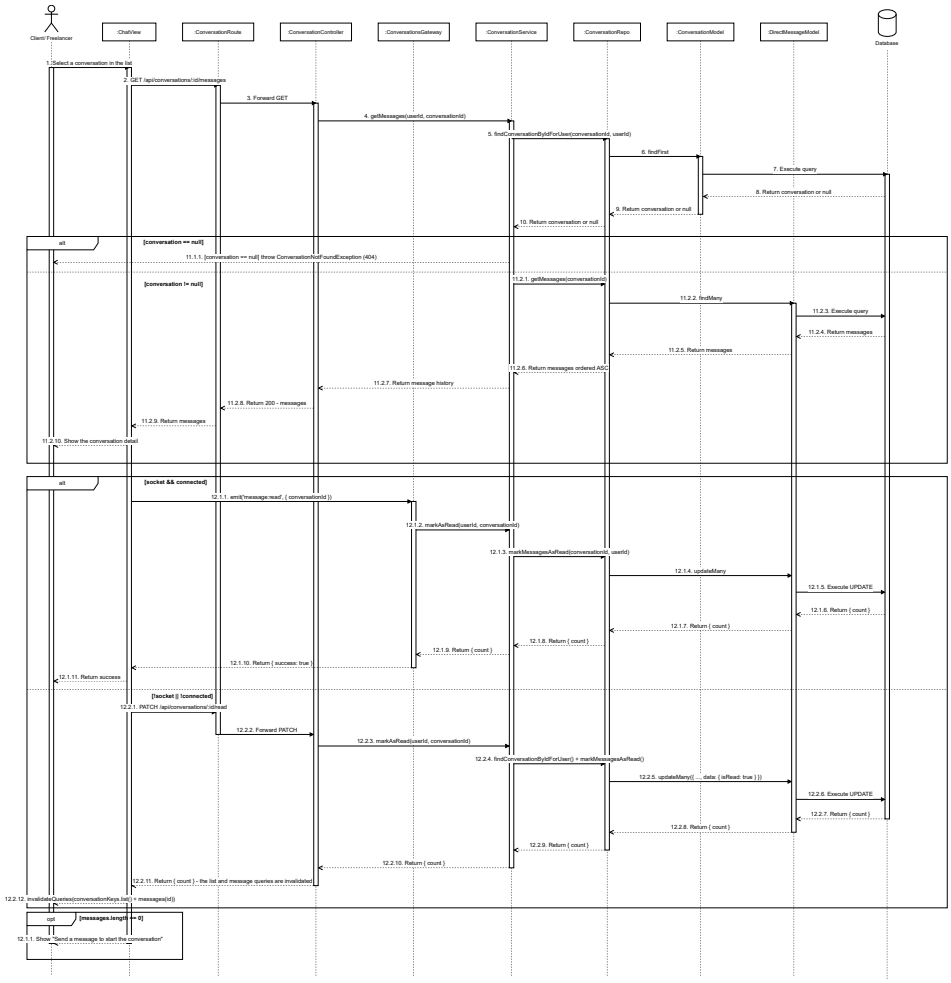

Figure 70: Sequence diagram of View Conversation Detail

3.26 Send Message

a. Class Diagram

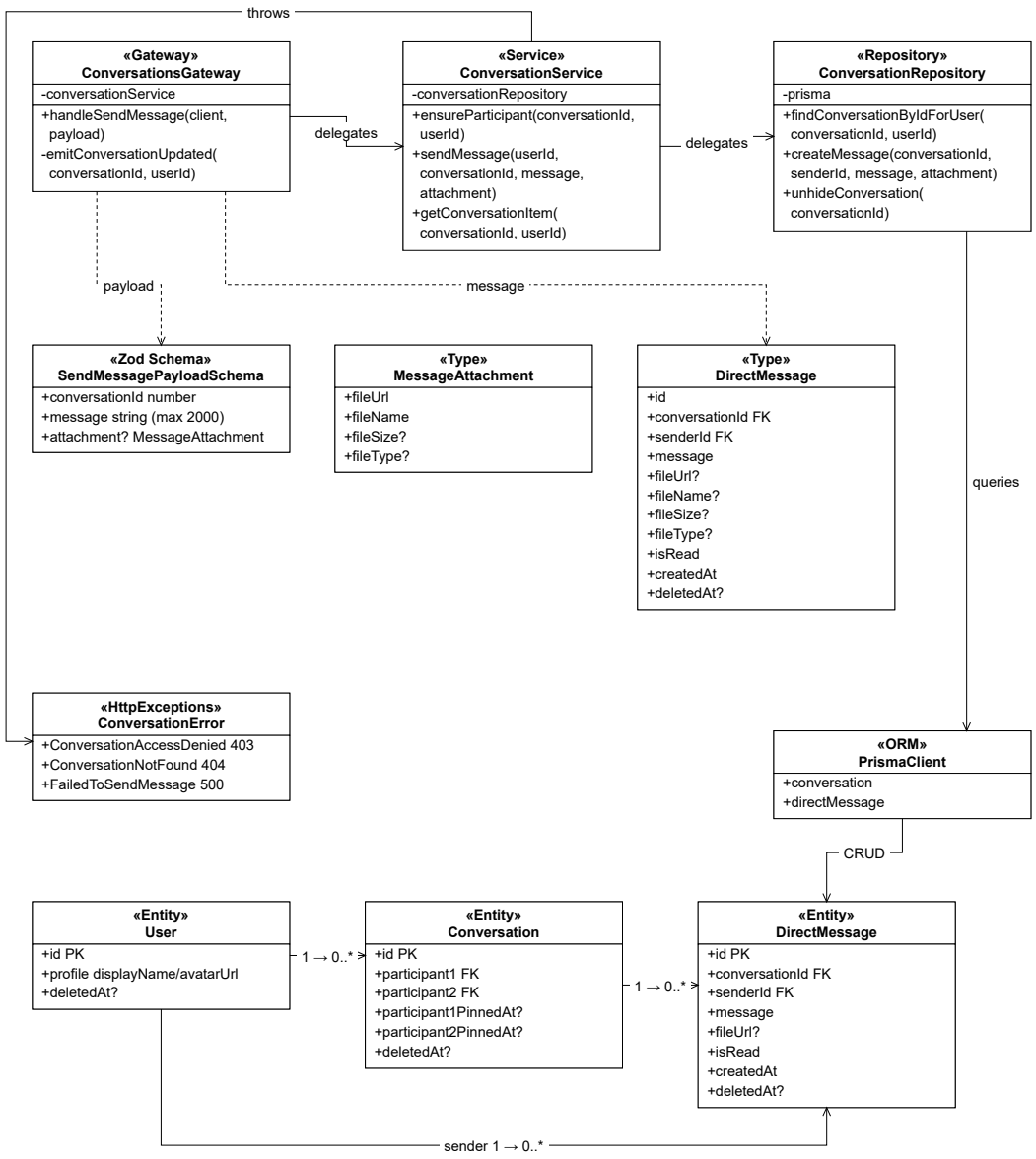

Figure 71: Class diagram of Send Message

b. Sequence Diagram(s)

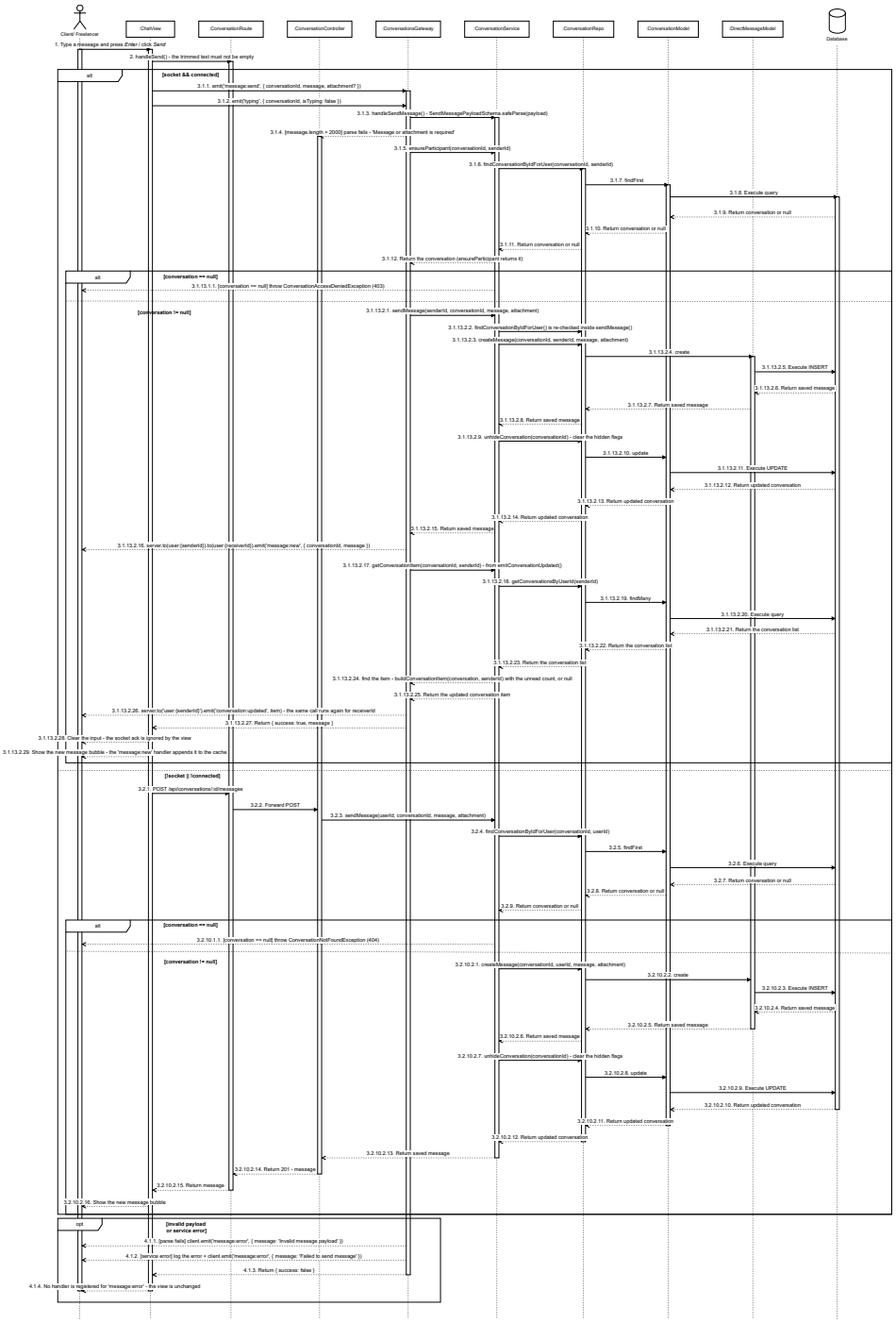

Figure 72: Sequence diagram of Send Message

3.27 Mark Message as Read

a. Class Diagram

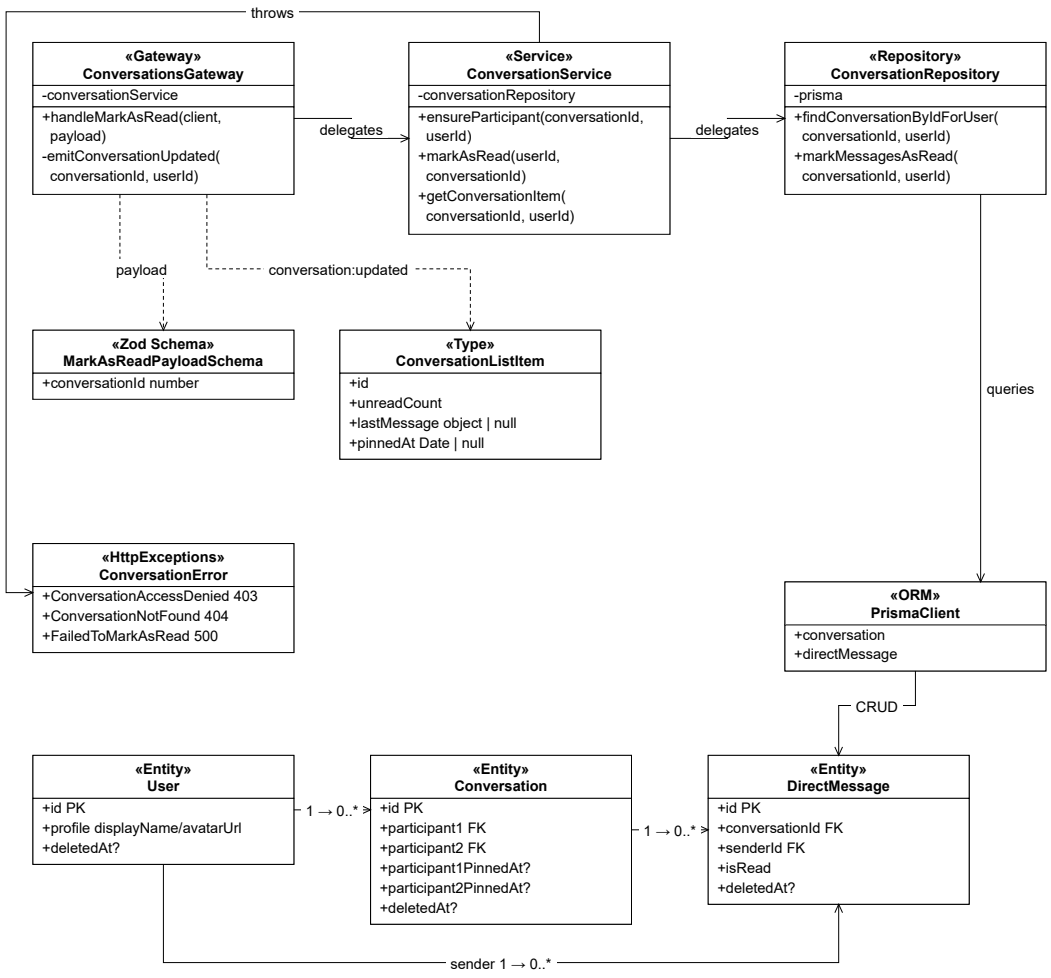

Figure 73: Class diagram of Mark Message as Read

b. Sequence Diagram(s)

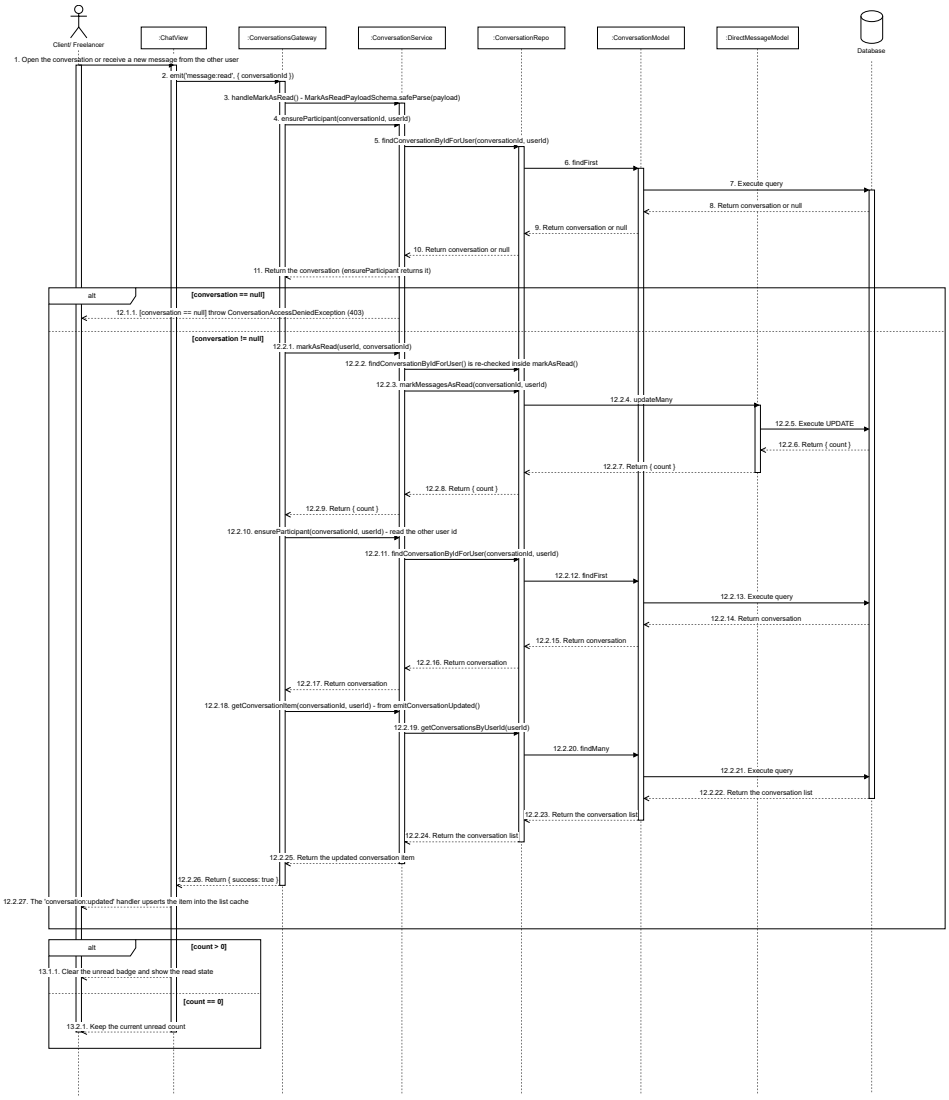

Figure 74: Sequence diagram of Mark Message as Read

3.28 Delete/Recall Message

a. Class Diagram

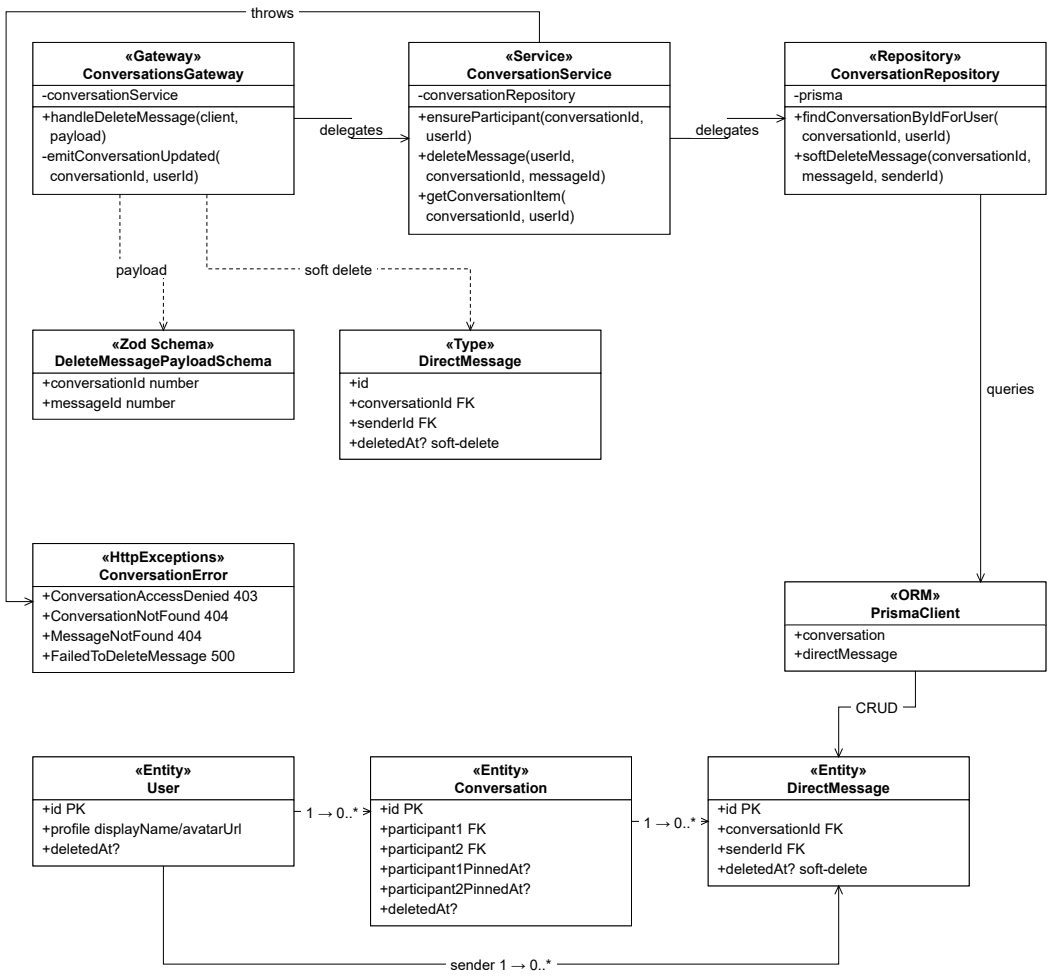

Figure 75: Class diagram of Delete/Recall Message

b. Sequence Diagram(s)

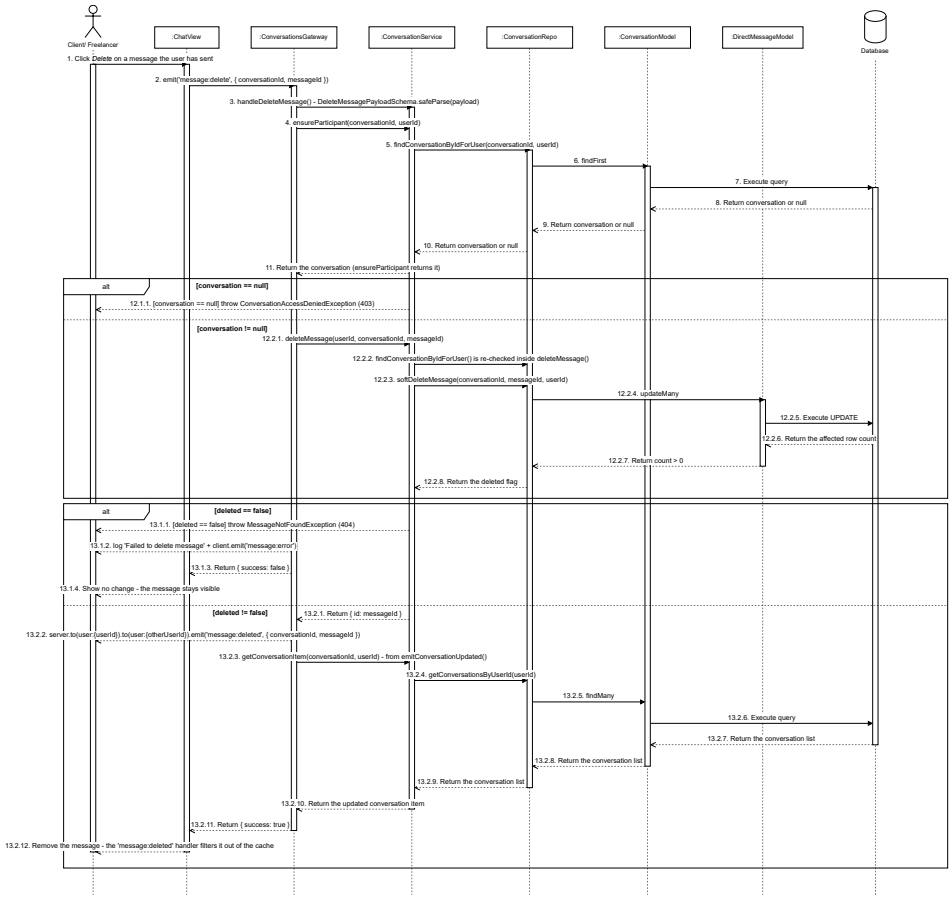

Figure 76: Sequence diagram of Delete/Recall Message

3.29 Pin/Unpin Conversation

a. Class Diagram

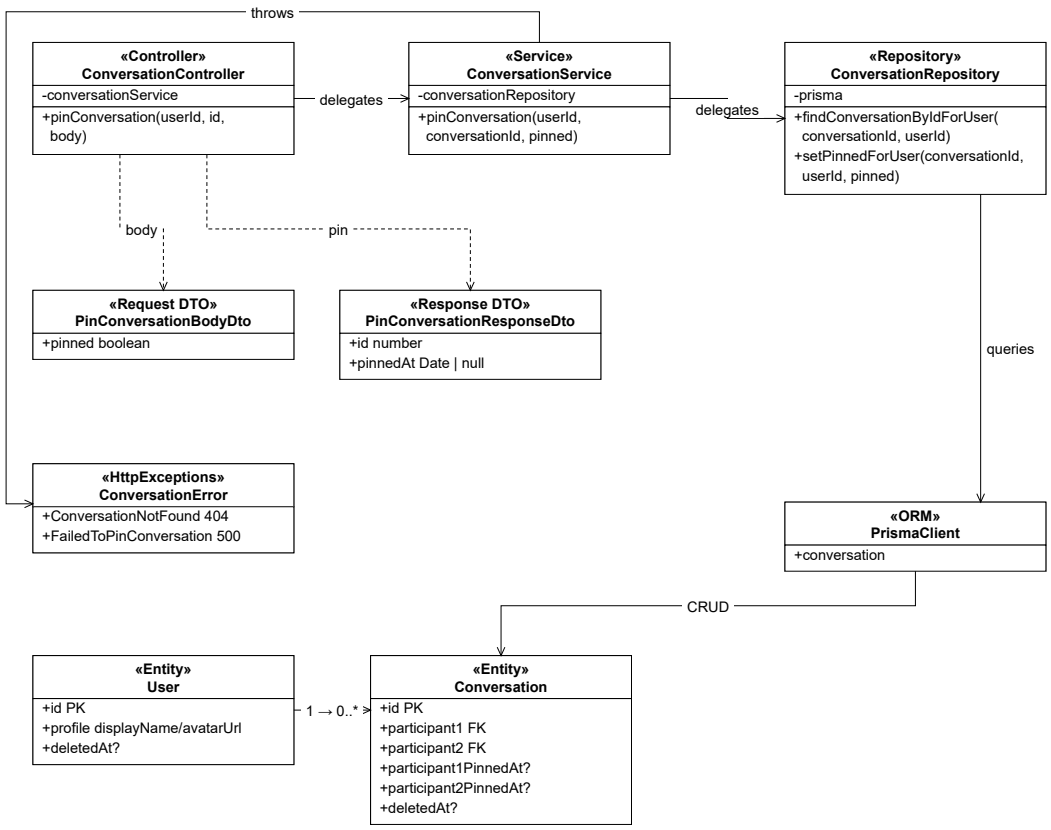

Figure 77: Class diagram of Pin/Unpin Conversation

b. Sequence Diagram(s)

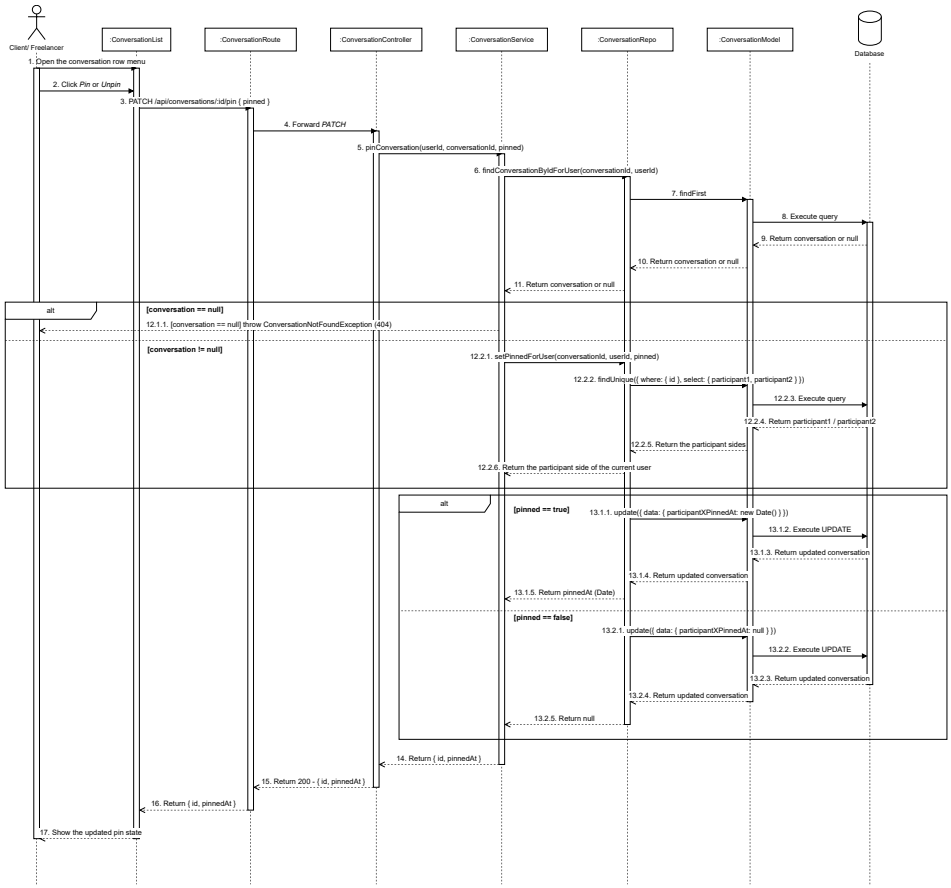

Figure 78: Sequence diagram of Pin/Unpin Conversation

3.30 Send File

a. Class Diagram

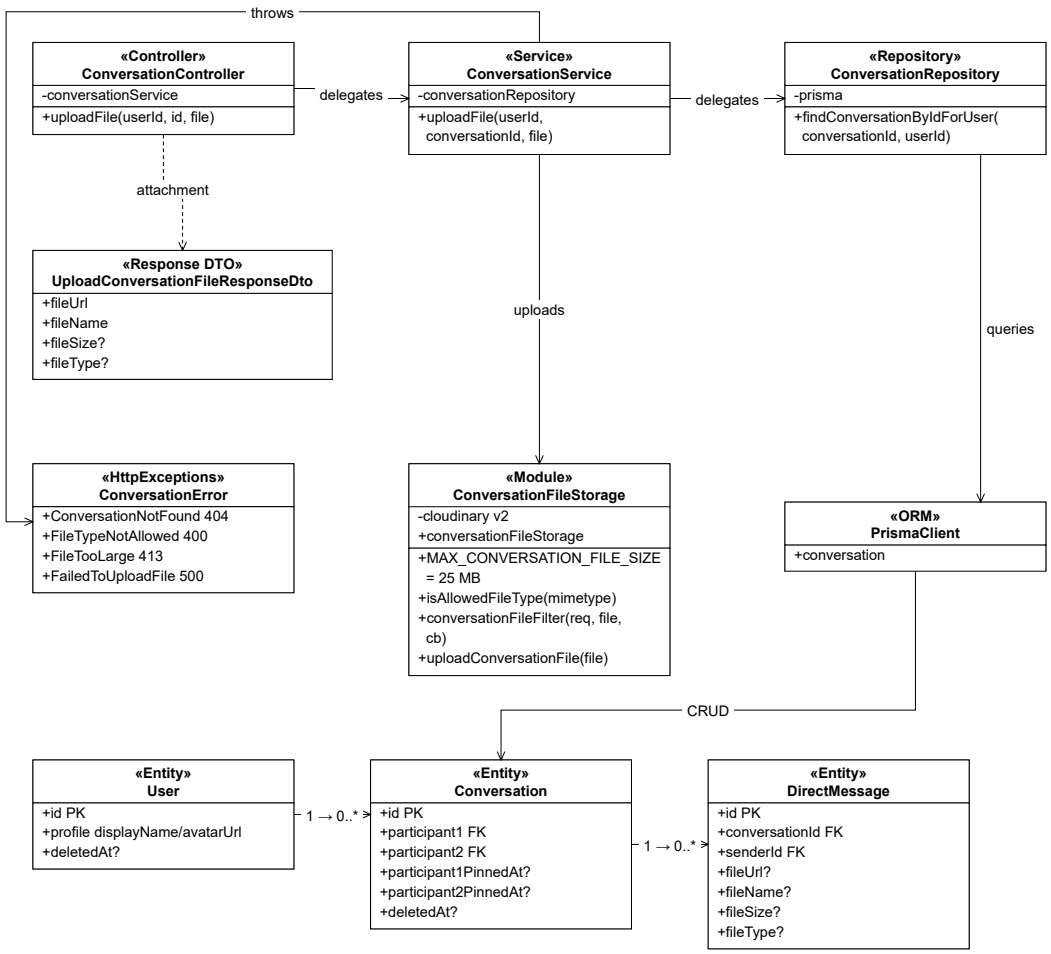

Figure 79: Class diagram of Send File

b. Sequence Diagram(s)

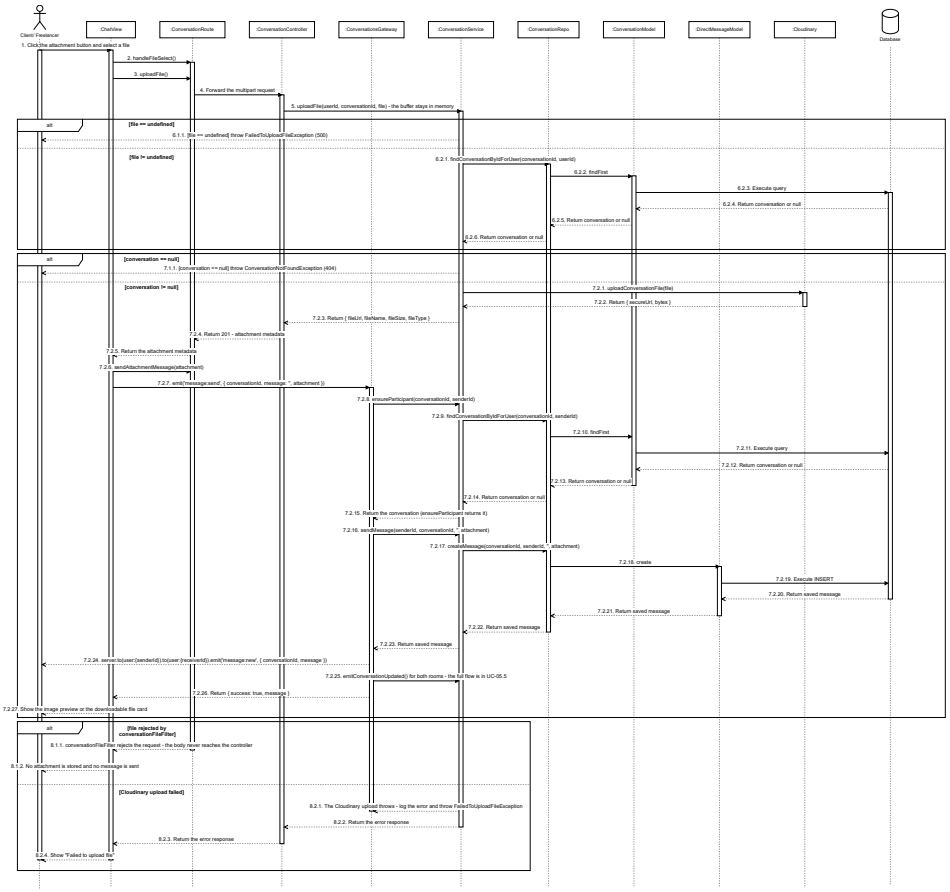

Figure 80: Sequence diagram of Send File

3.31 Identity Verification Management

Freelancers upload identity documents and inspect the latest verification state. The submission workflow includes standard upload and AI verification.

a. Class Diagram

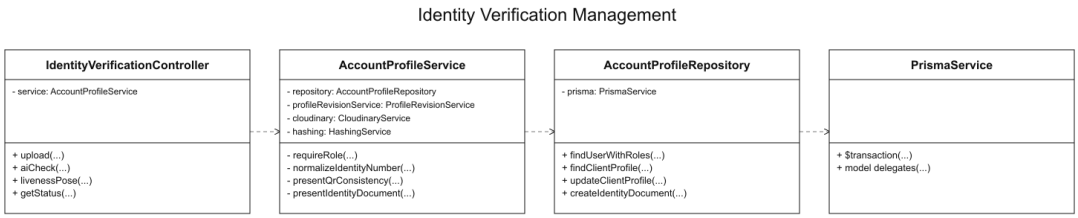

Figure 81: Class diagram of Identity Verification Management

b. Sequence Diagram(s)

3.31.1 UC-18.01 – Upload Identity Documents

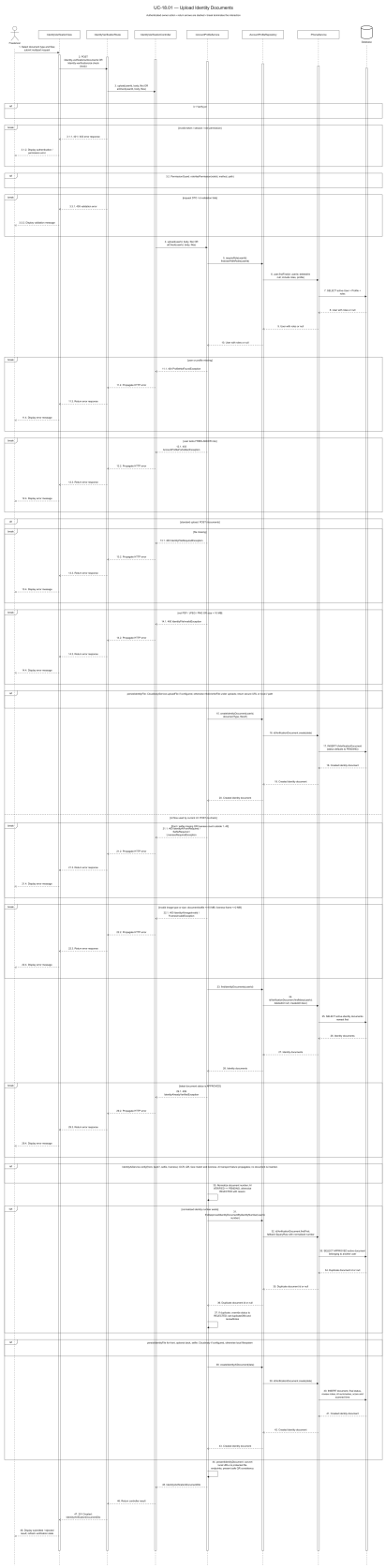

Figure 82: Sequence diagram of UC-18.01 Upload Identity Documents

3.31.2 UC-18.02 – View Verification Status

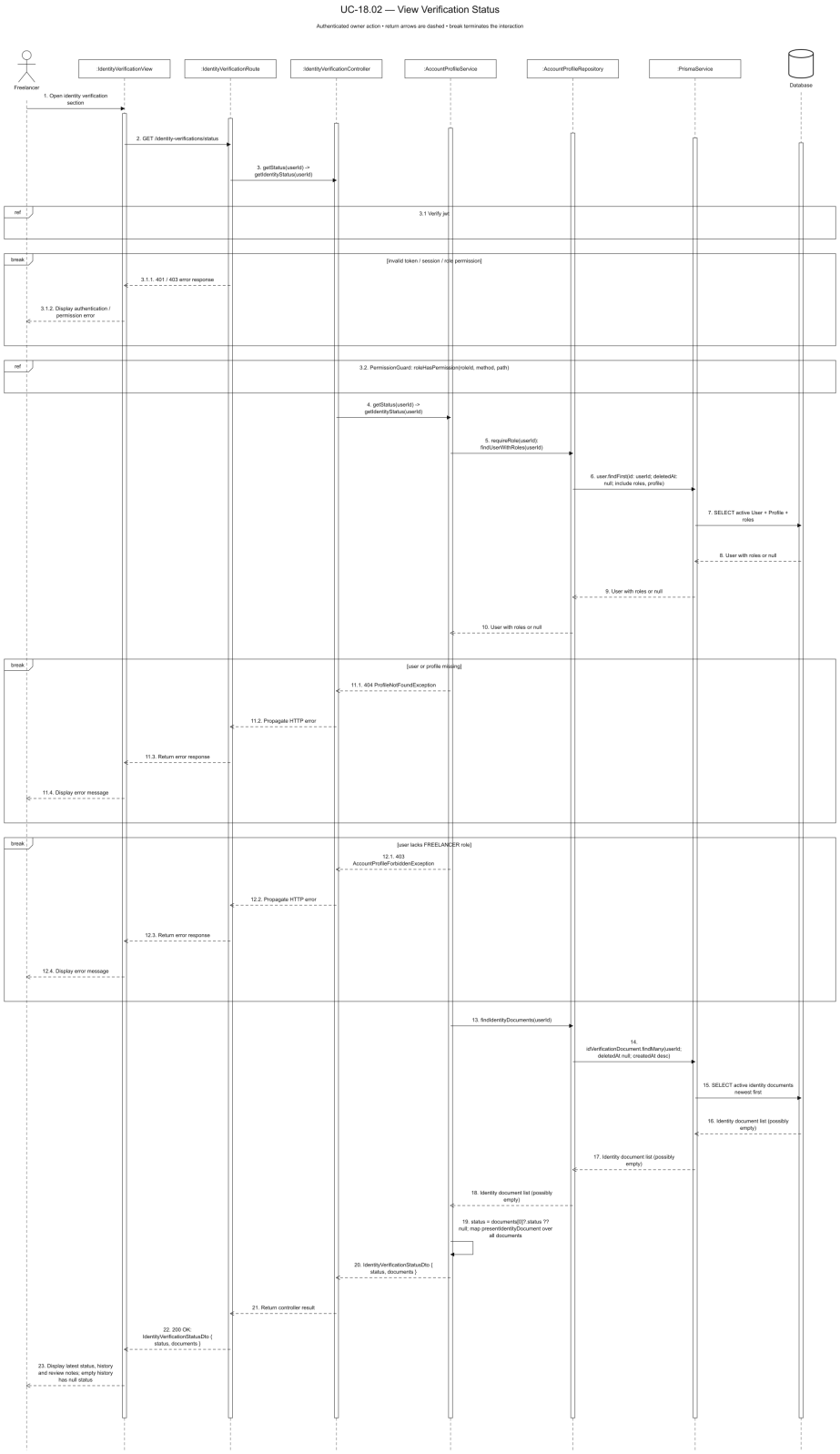

Figure 83: Sequence diagram of UC-18.02 View Verification Status

3.32 Client Profile Management

Public client information is retrieved separately from authenticated company-information updates.

a. Class Diagram

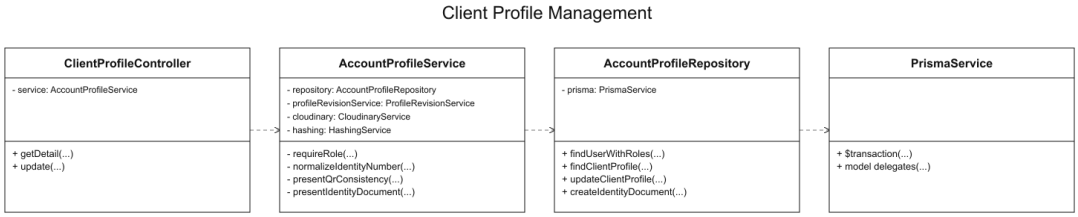

Figure 84: Class diagram of Client Profile Management

b. Sequence Diagram(s)

3.32.1 UC-19.01 – View Client Profile

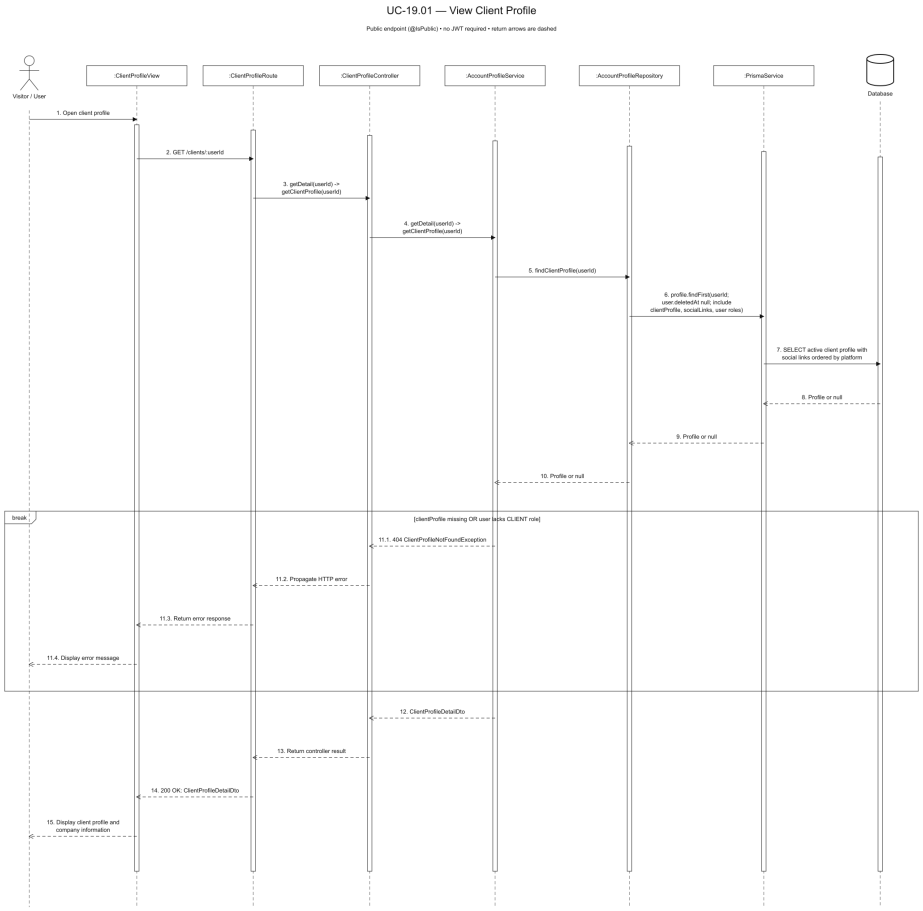

Figure 85: Sequence diagram of UC-19.01 View Client Profile

3.32.2 UC-19.02 – Update Company Information

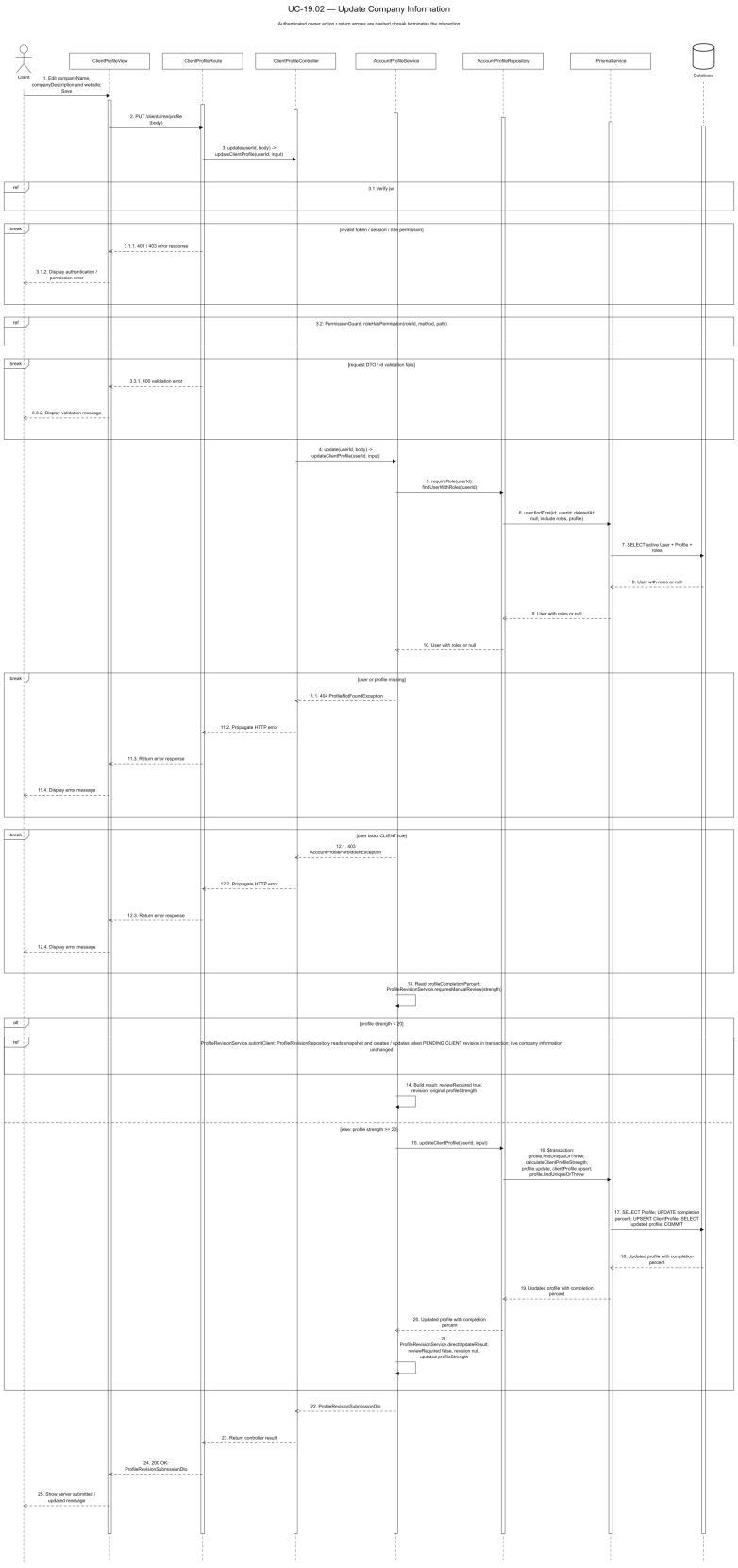

Figure 86: Sequence diagram of UC-19.02 Update Company Information

3.33 Social Link Management

An authenticated user manages links associated with their own profile, with one link per platform.

a. Class Diagram

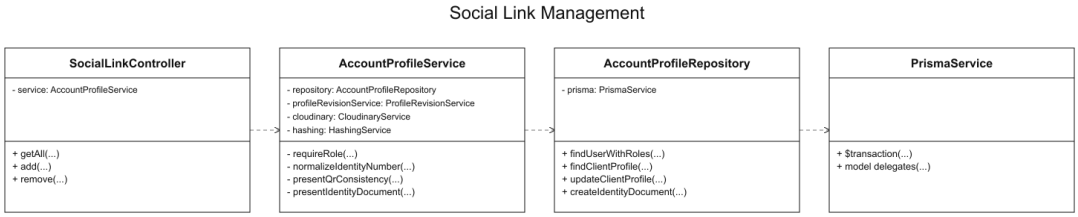

Figure 87: Class diagram of Social Link Management

b. Sequence Diagram(s)

3.33.1 UC-20.01 – View Social Link List

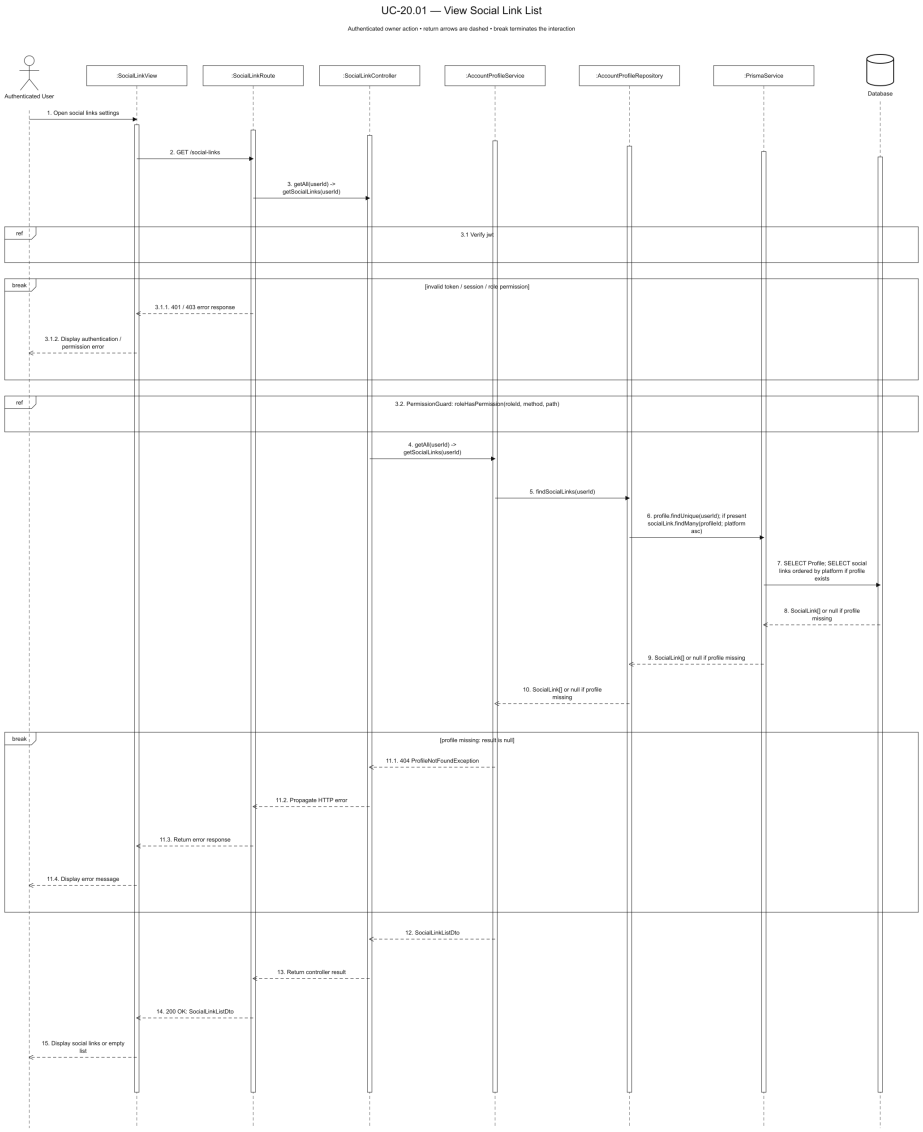

Figure 88: Sequence diagram of UC-20.01 View Social Link List

3.33.2 UC-20.02 – Add Social Link

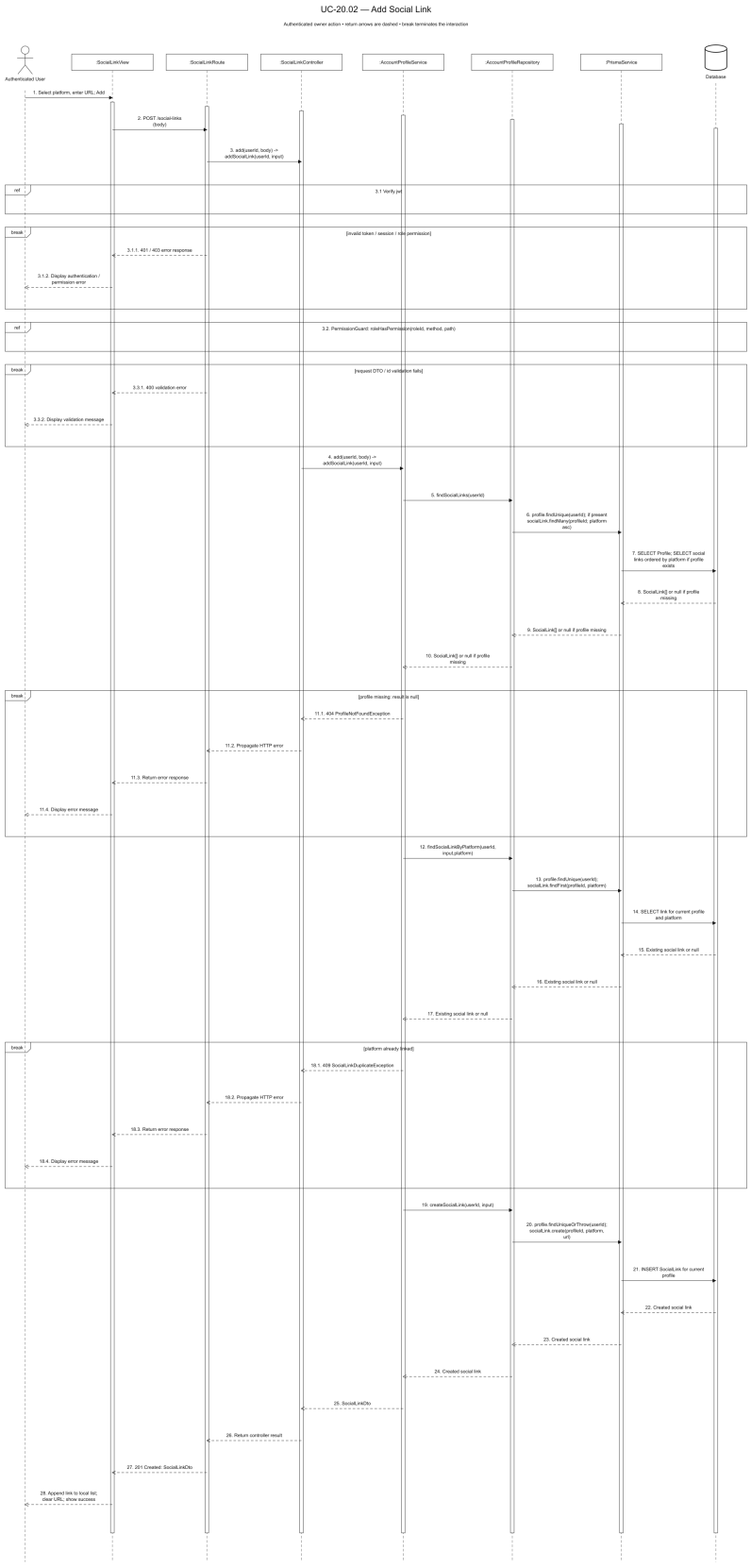

Figure 89: Sequence diagram of UC-20.02 Add Social Link

3.33.3 UC-20.03 – Delete Social Link

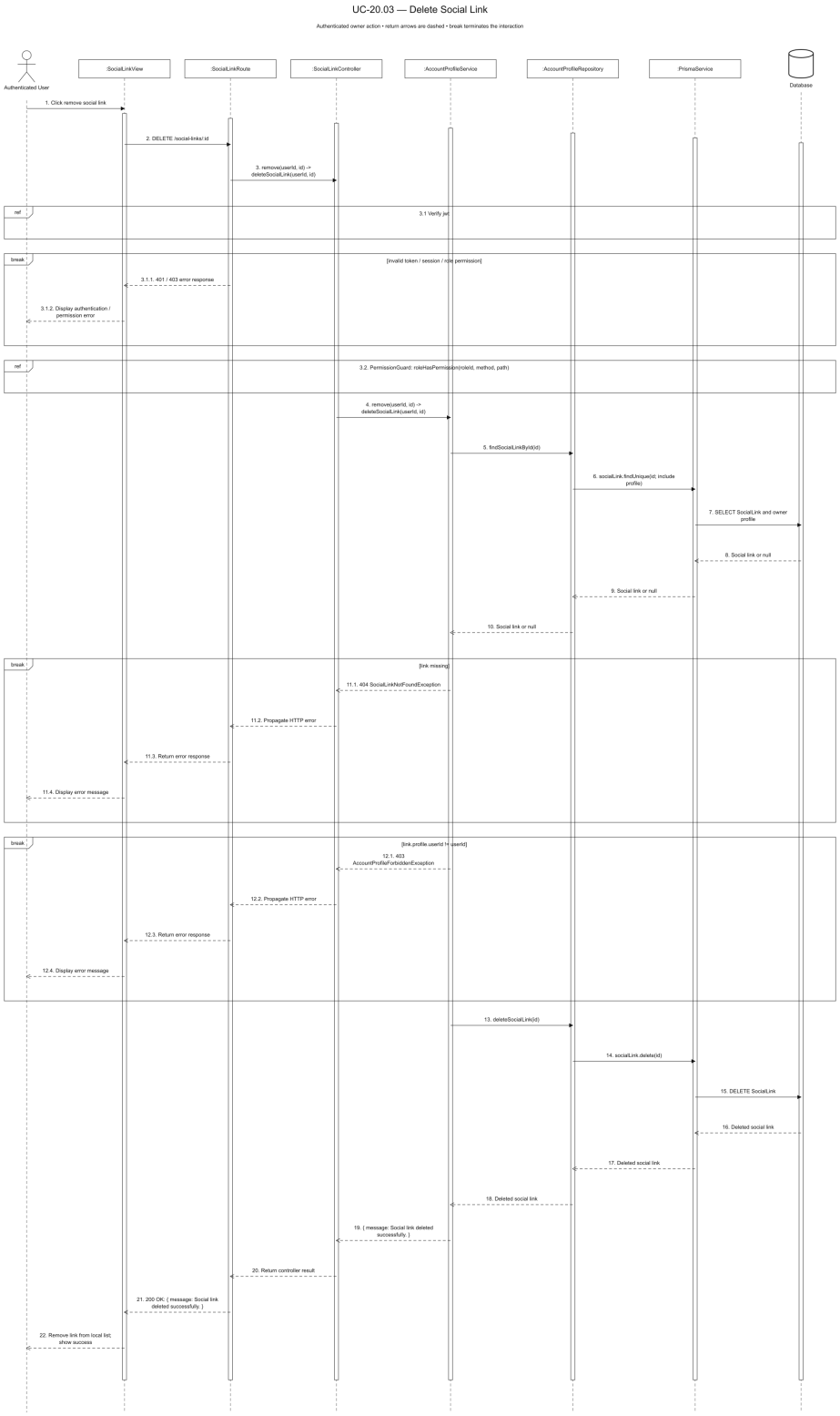

Figure 90: Sequence diagram of UC-20.03 Delete Social Link

3.34 Favorite Freelancer Management

Clients maintain their favorite freelancers without modifying the underlying freelancer accounts.

a. Class Diagram

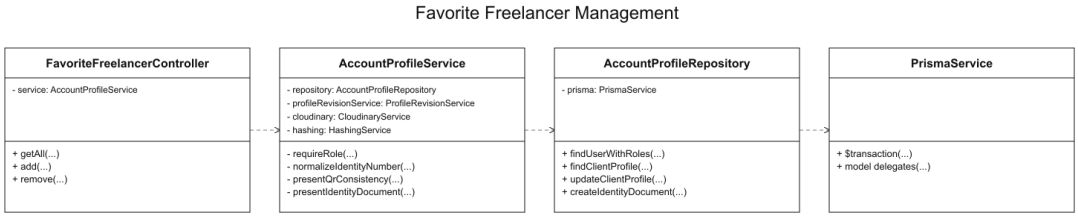

Figure 91: Class diagram of Favorite Freelancer Management

b. Sequence Diagram(s)

3.34.1 UC-21.01 – View Favorite Freelancer List

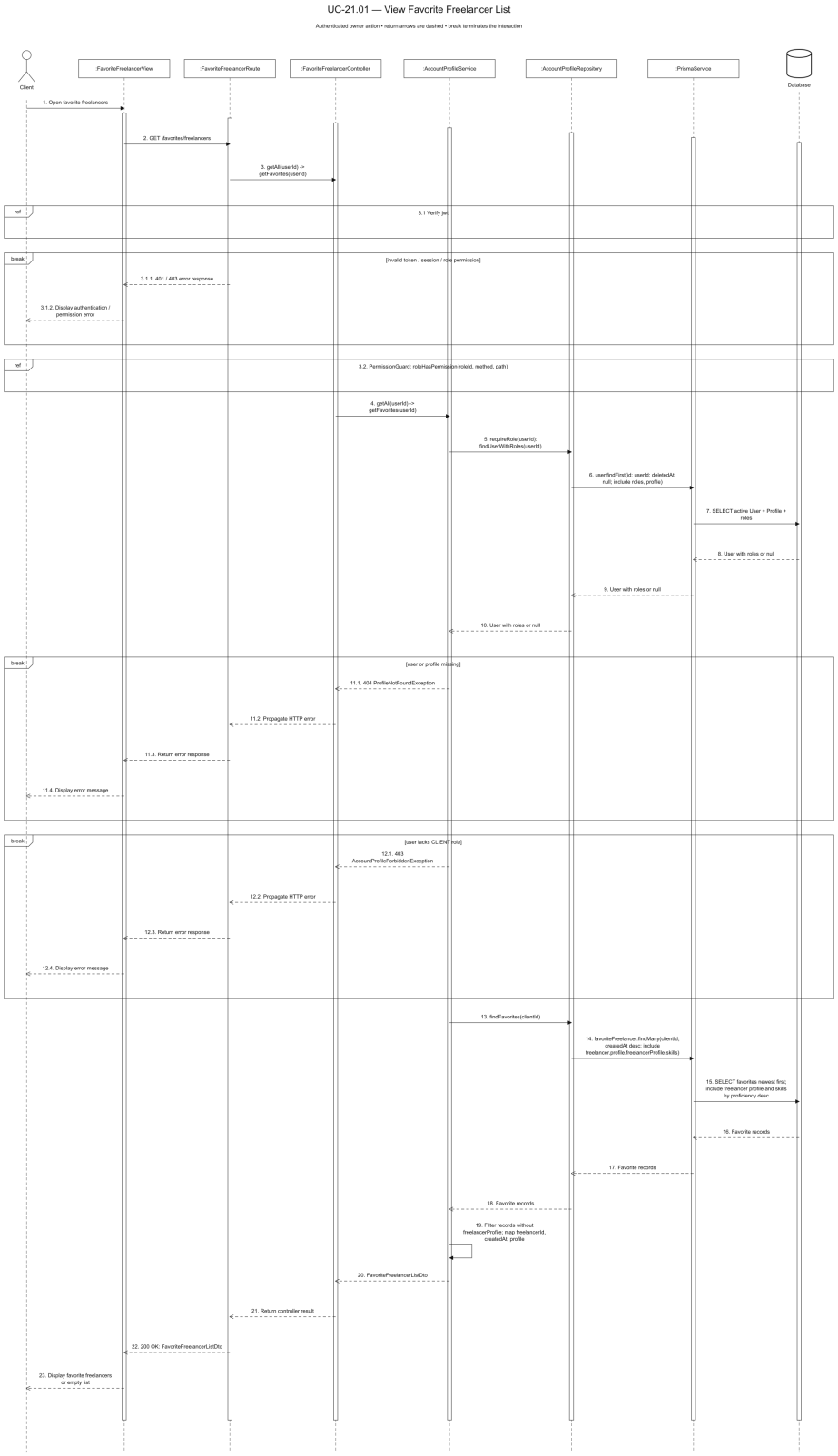

Figure 92: Sequence diagram of UC-21.01 View Favorite Freelancer List

3.34.2 UC-21.02 – Add Freelancer to Favorites

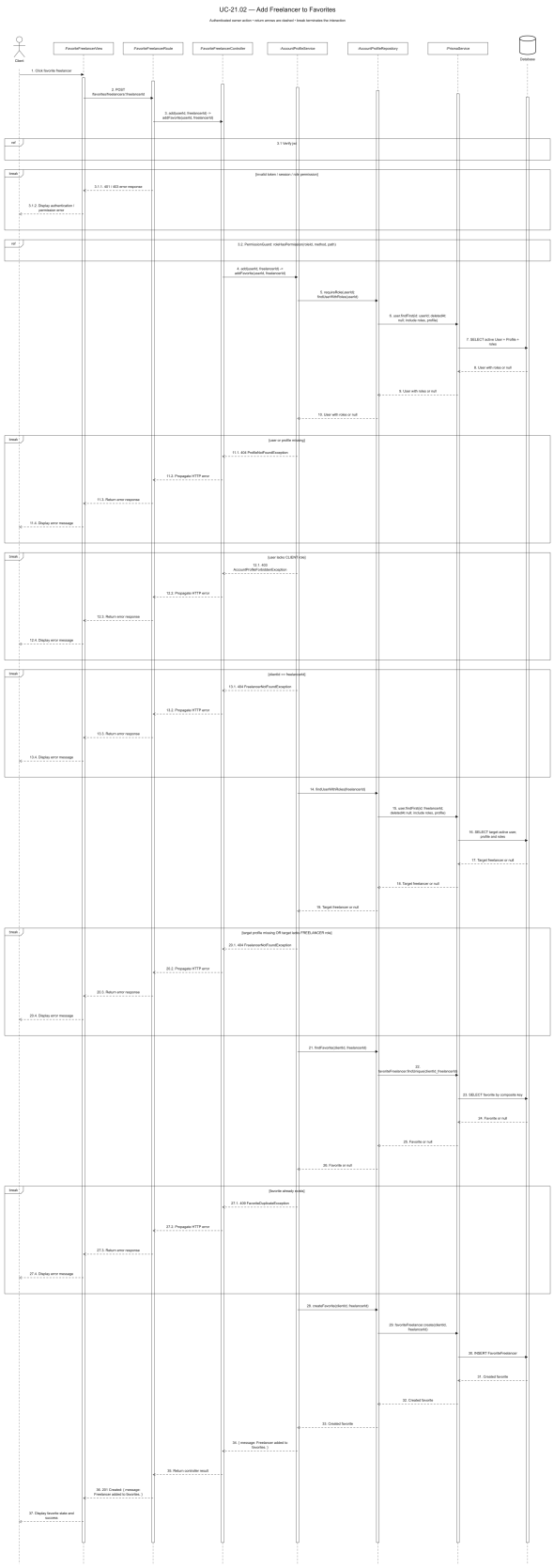

Figure 93: Sequence diagram of UC-21.02 Add Freelancer to Favorites

3.34.3 UC-21.03 – Remove Freelancer from Favorites

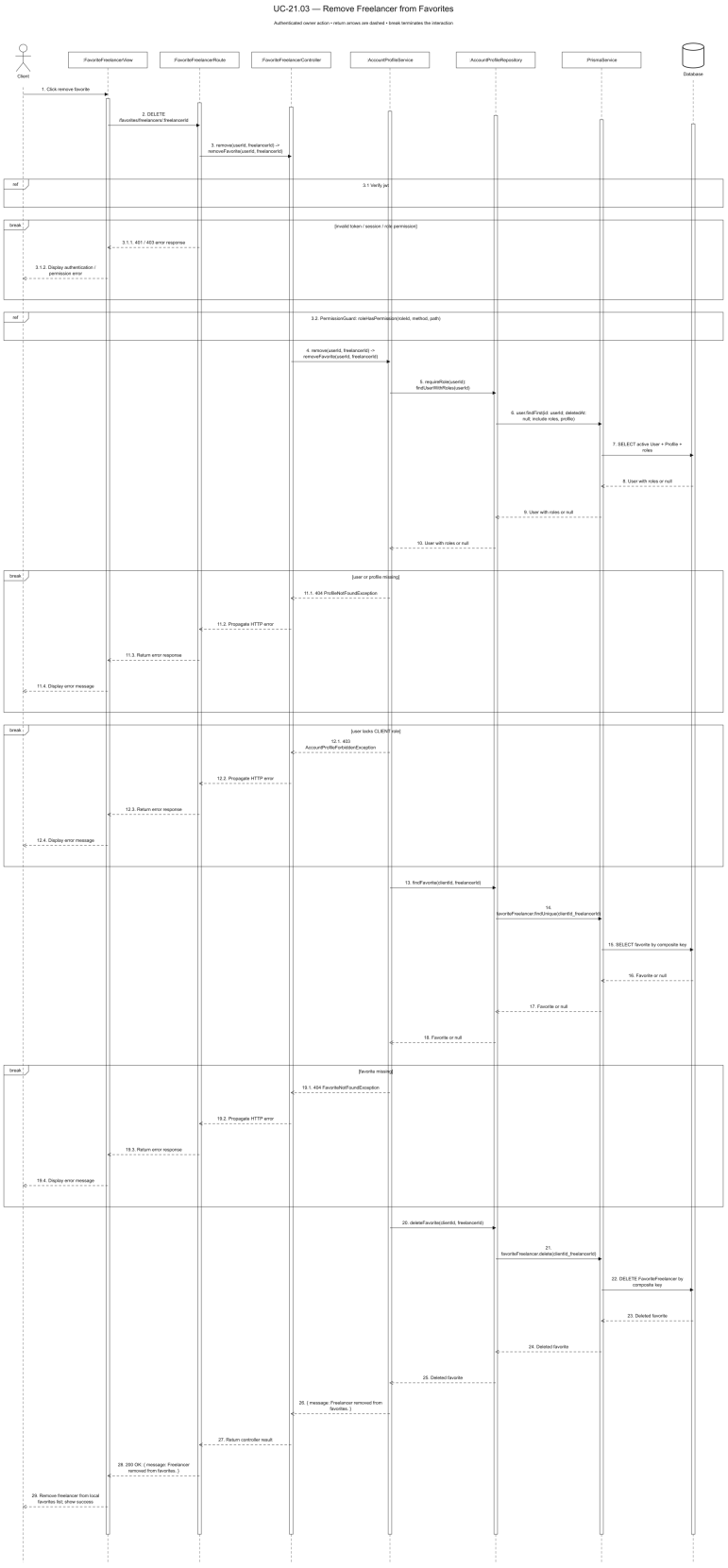

Figure 94: Sequence diagram of UC-21.03 Remove Freelancer from Favorites

3.35 Permission Management

Permission Management exposes the route-level permission catalogue so that an administrator can inspect the permissions available for role assignment.

a. Class Diagram

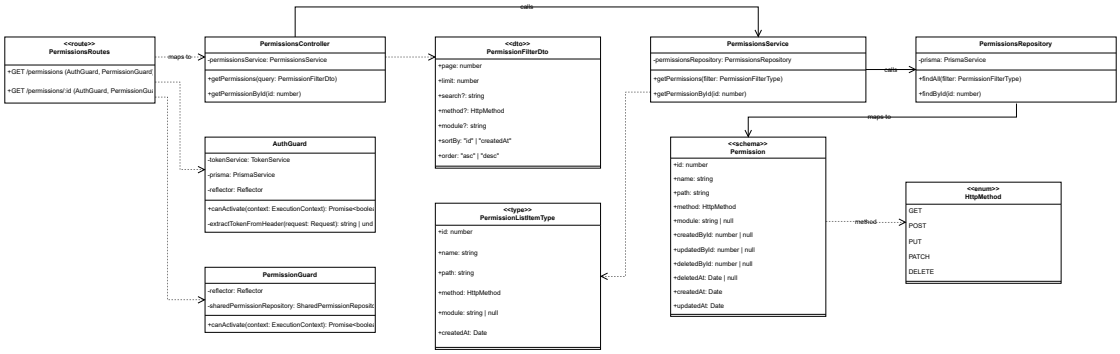

Figure 95: Class diagram of Permission Management

b. Sequence Diagram(s)

3.35.1 UC-15.01 – View Permission List

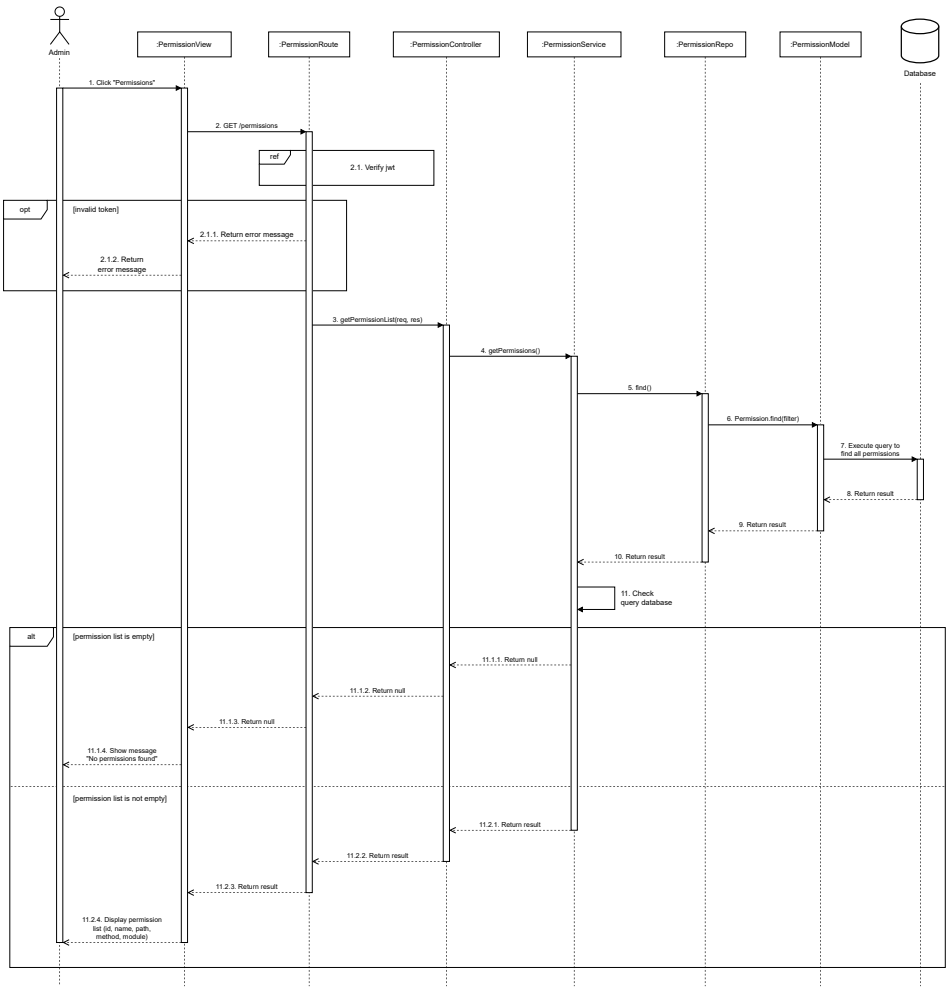

Figure 96: Sequence diagram of UC-15.01 View Permission List

3.35.2 UC-15.02 – View Permission Detail

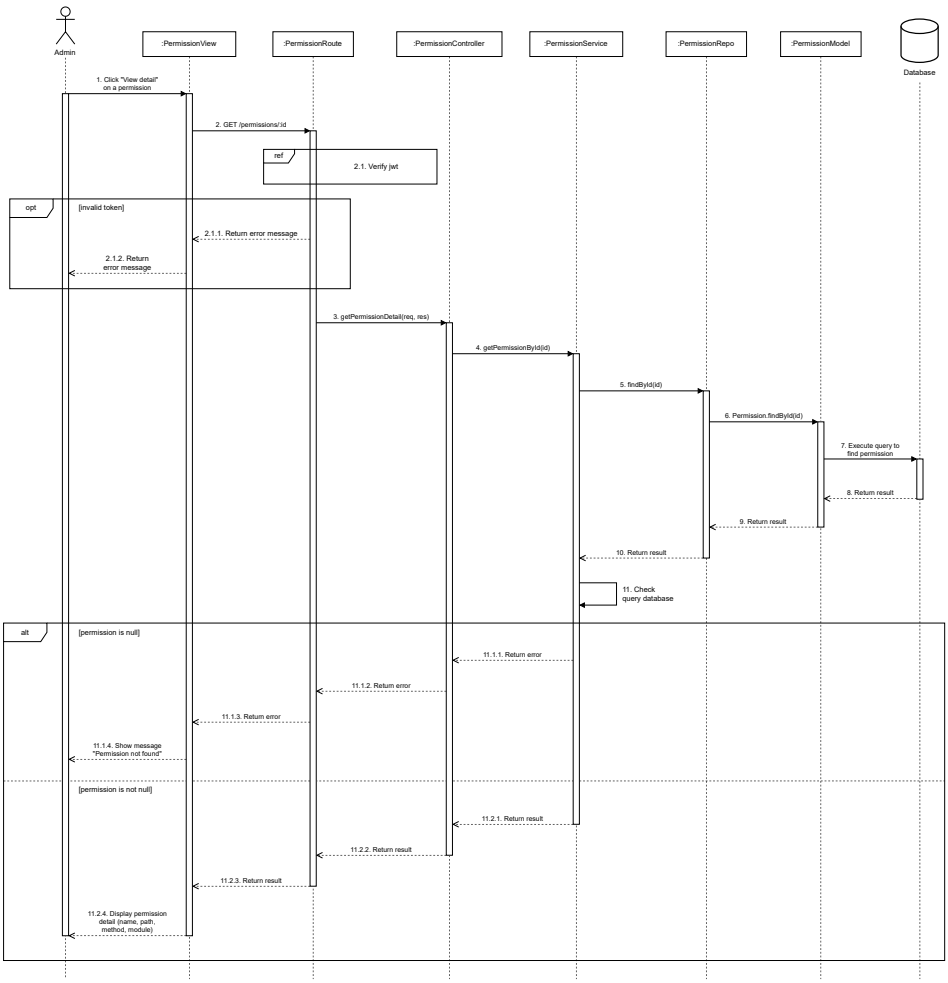

Figure 97: Sequence diagram of UC-15.02 View Permission Detail

3.36 Session Management

Session Management gives an authenticated user visibility over the devices that currently hold a refresh-token session and allows a session to be revoked remotely.

a. Class Diagram

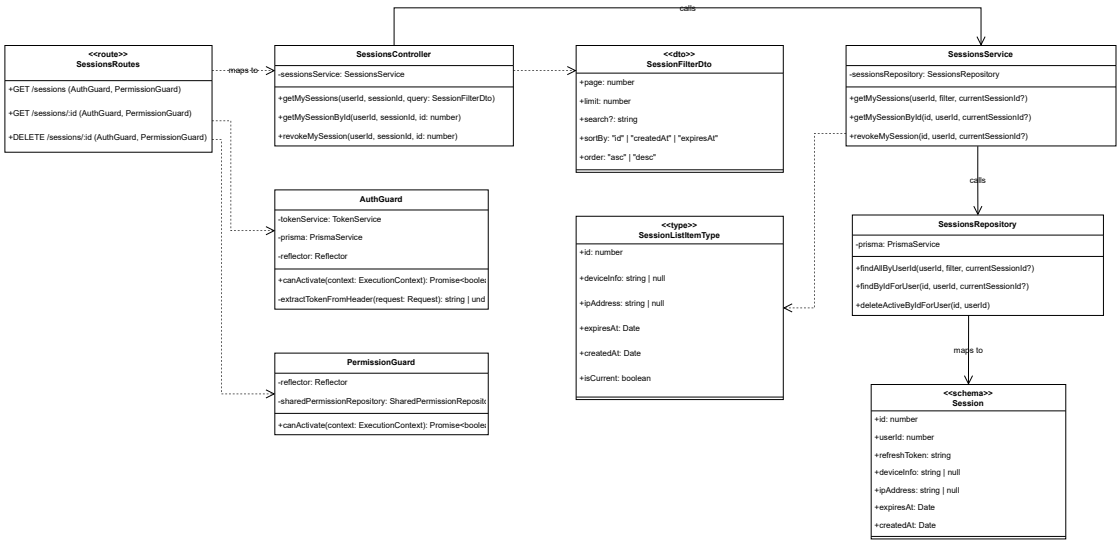

Figure 98: Class diagram of Session Management

b. Sequence Diagram(s)

3.36.1 UC-16.01 – View Session List

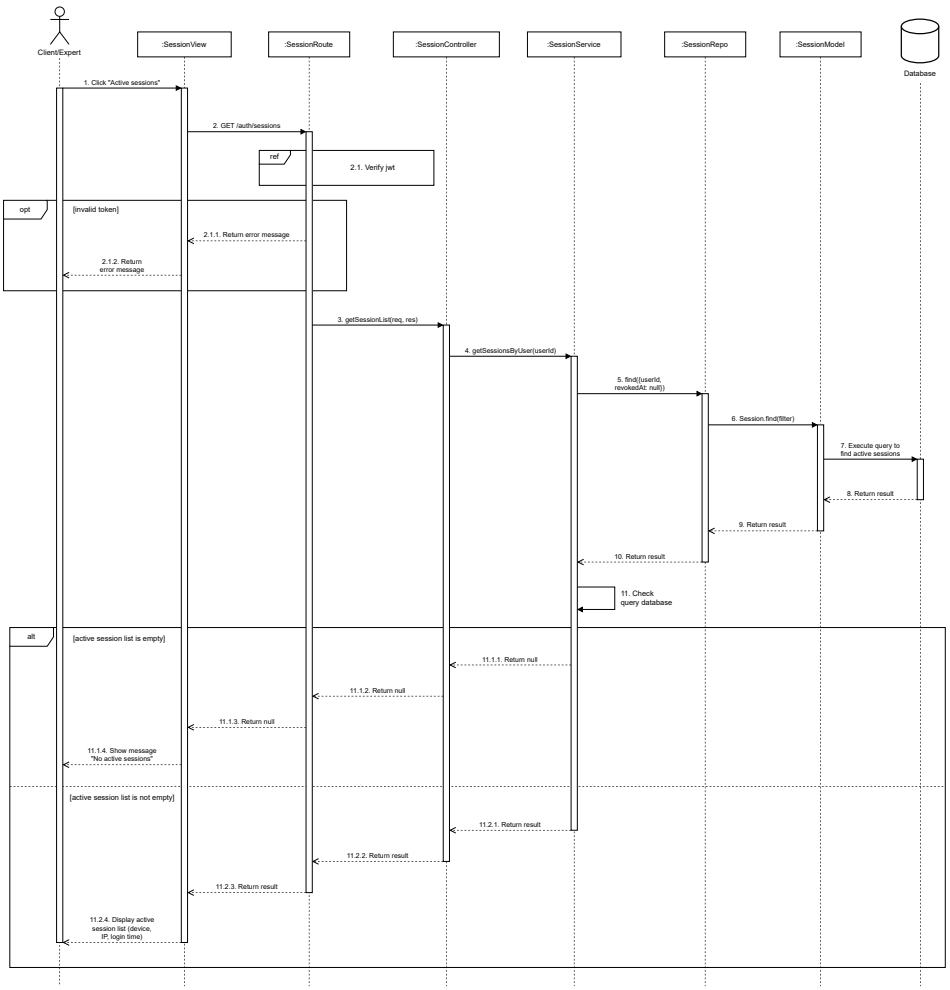

Figure 99: Sequence diagram of UC-16.01 View Session List

3.36.2 UC-16.02 – View Session Detail

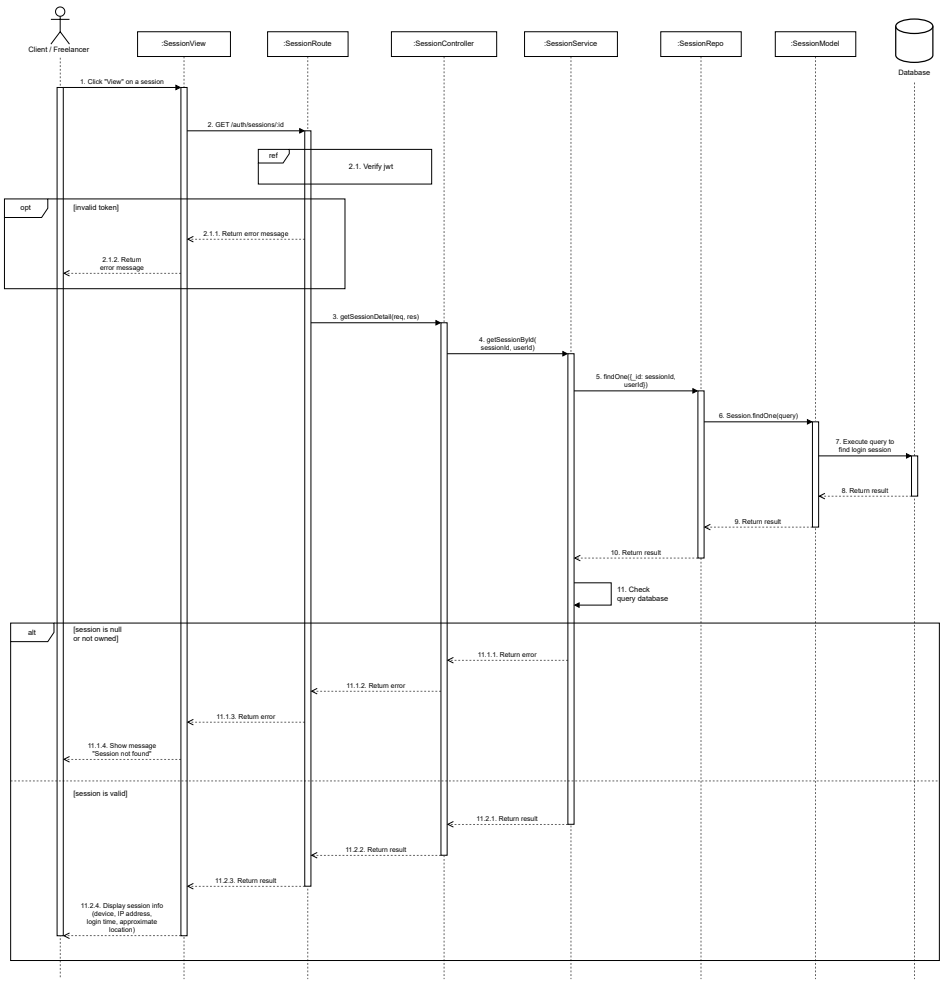

Figure 100: Sequence diagram of UC-16.02 View Session Detail

3.36.3 UC-16.03 – Revoke Session

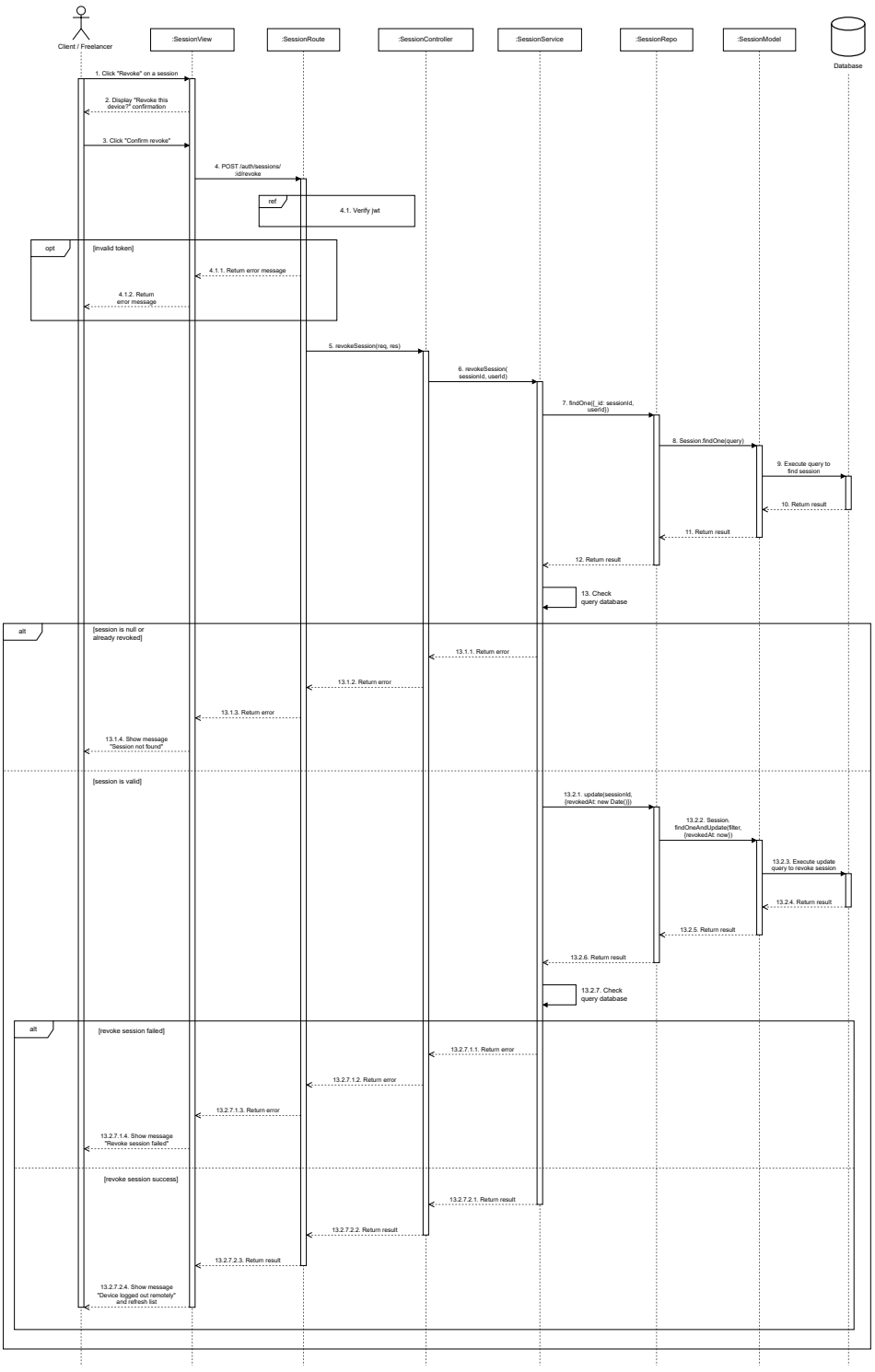

Figure 101: Sequence diagram of UC-16.03 Revoke Session

3.37 Forum Category Management

Forum Category Management covers the administrator-side category catalogue and the public category views used by the forum. Deletion is a soft delete, so a category can be restored later.

a. Class Diagram

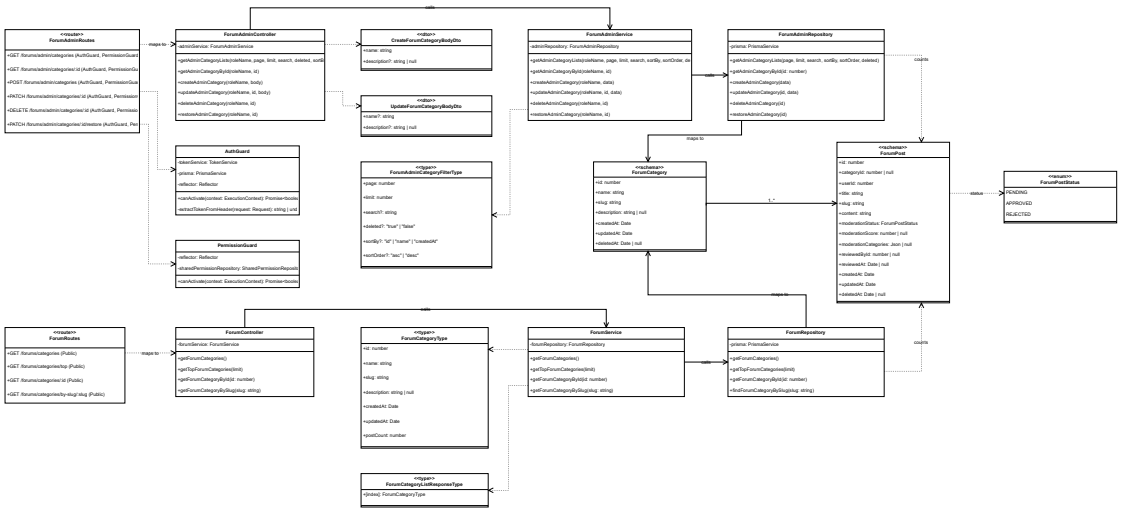

Figure 102: Class diagram of Forum Category Management

b. Sequence Diagram(s)

3.37.1 UC-17.01 – Create Forum Category

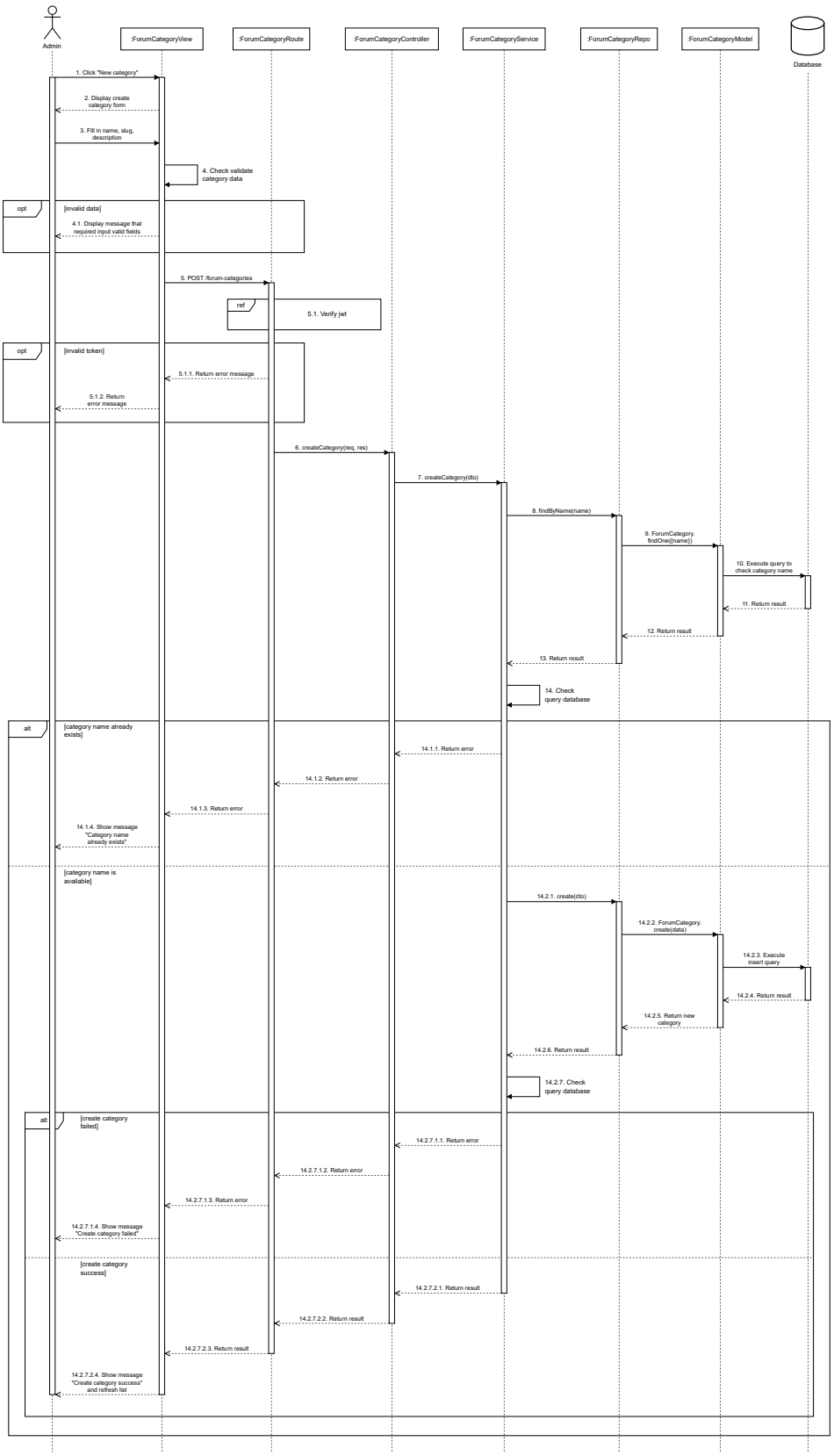

Figure 103: Sequence diagram of UC-17.01 Create Forum Category

3.37.2 UC-17.02 – Update Forum Category

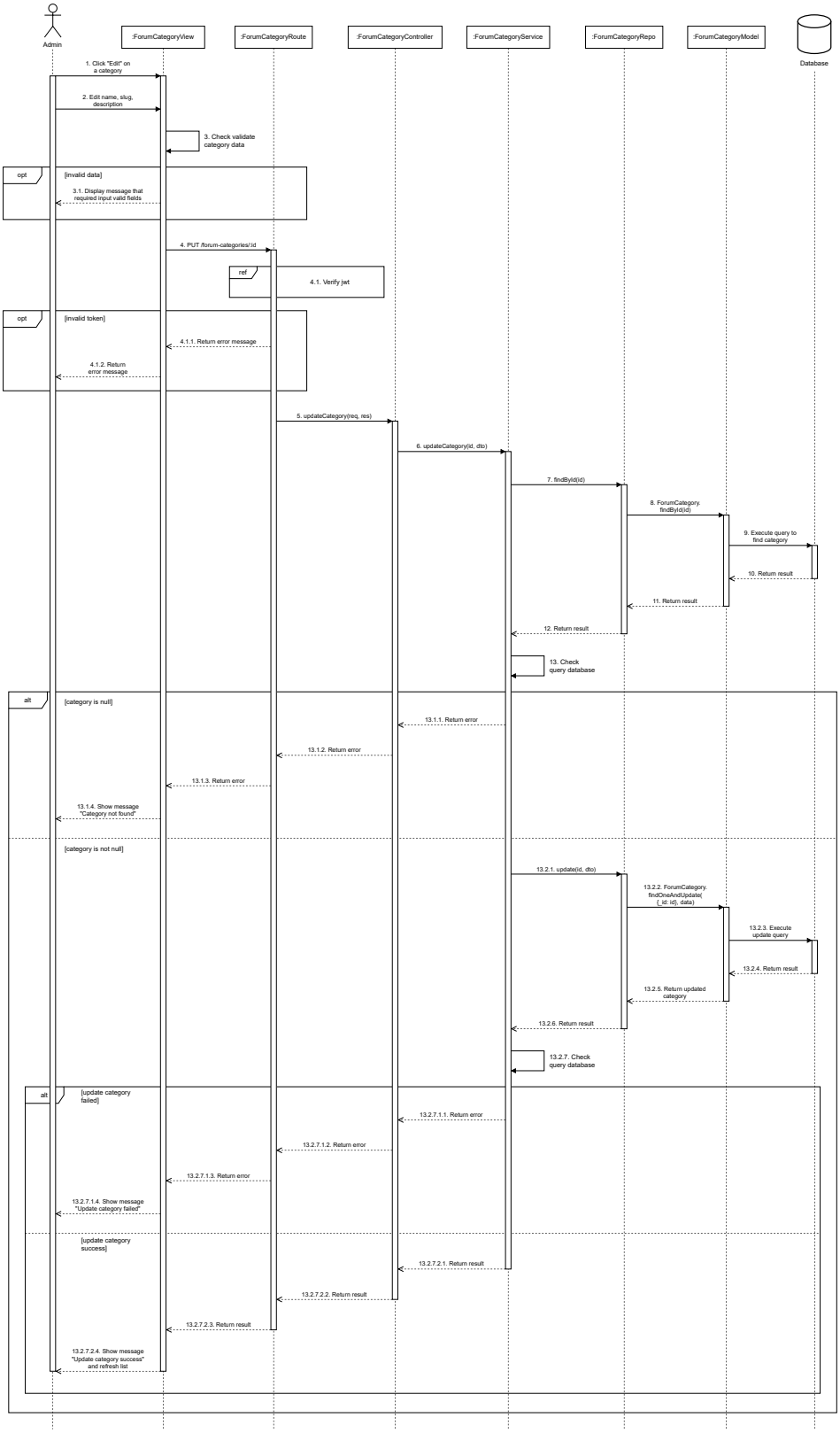

Figure 104: Sequence diagram of UC-17.02 Update Forum Category

3.37.3 UC-17.03 – Delete Forum Category

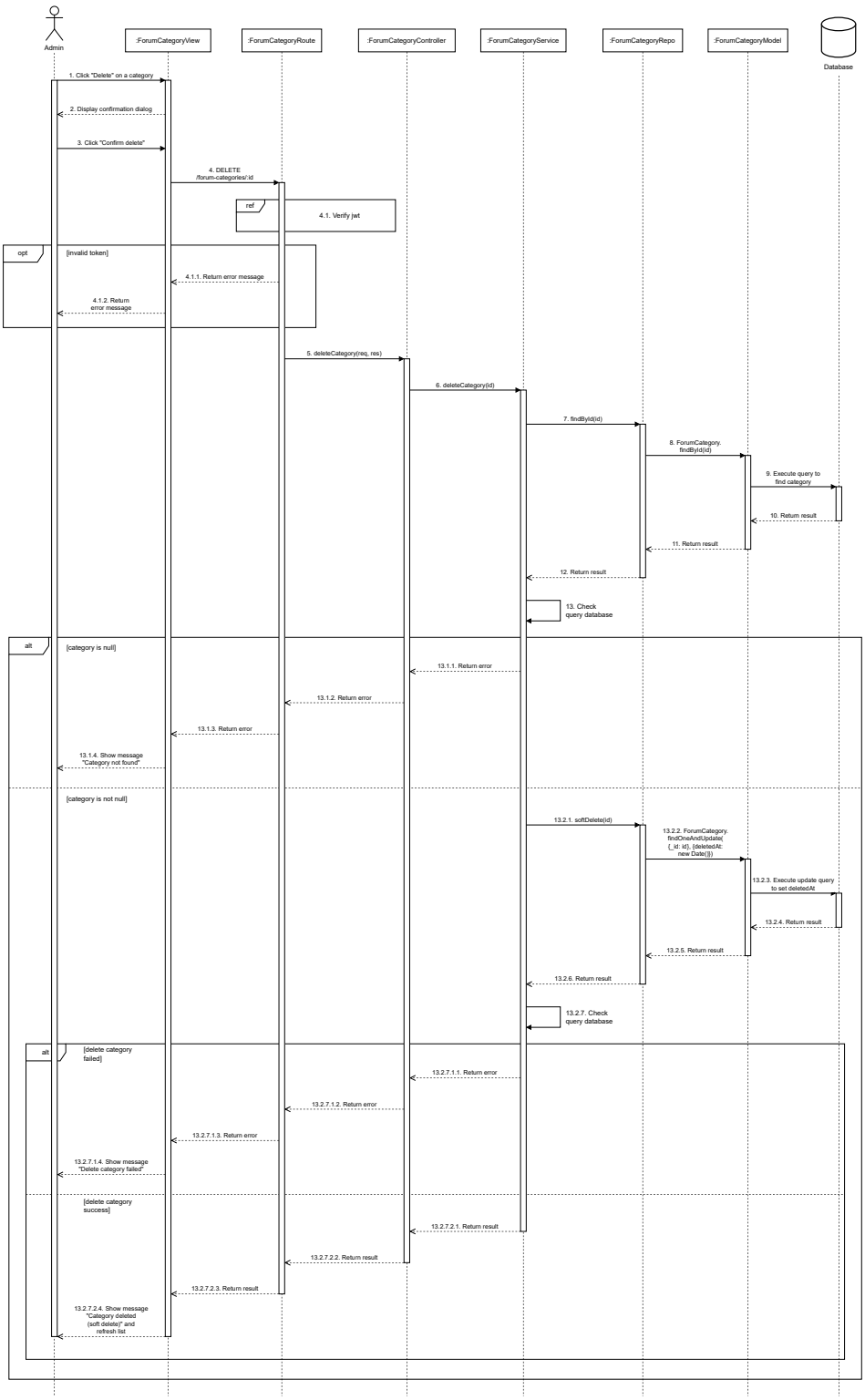

Figure 105: Sequence diagram of UC-17.03 Delete Forum Category

3.37.4 UC-17.06 – Restore Forum Category

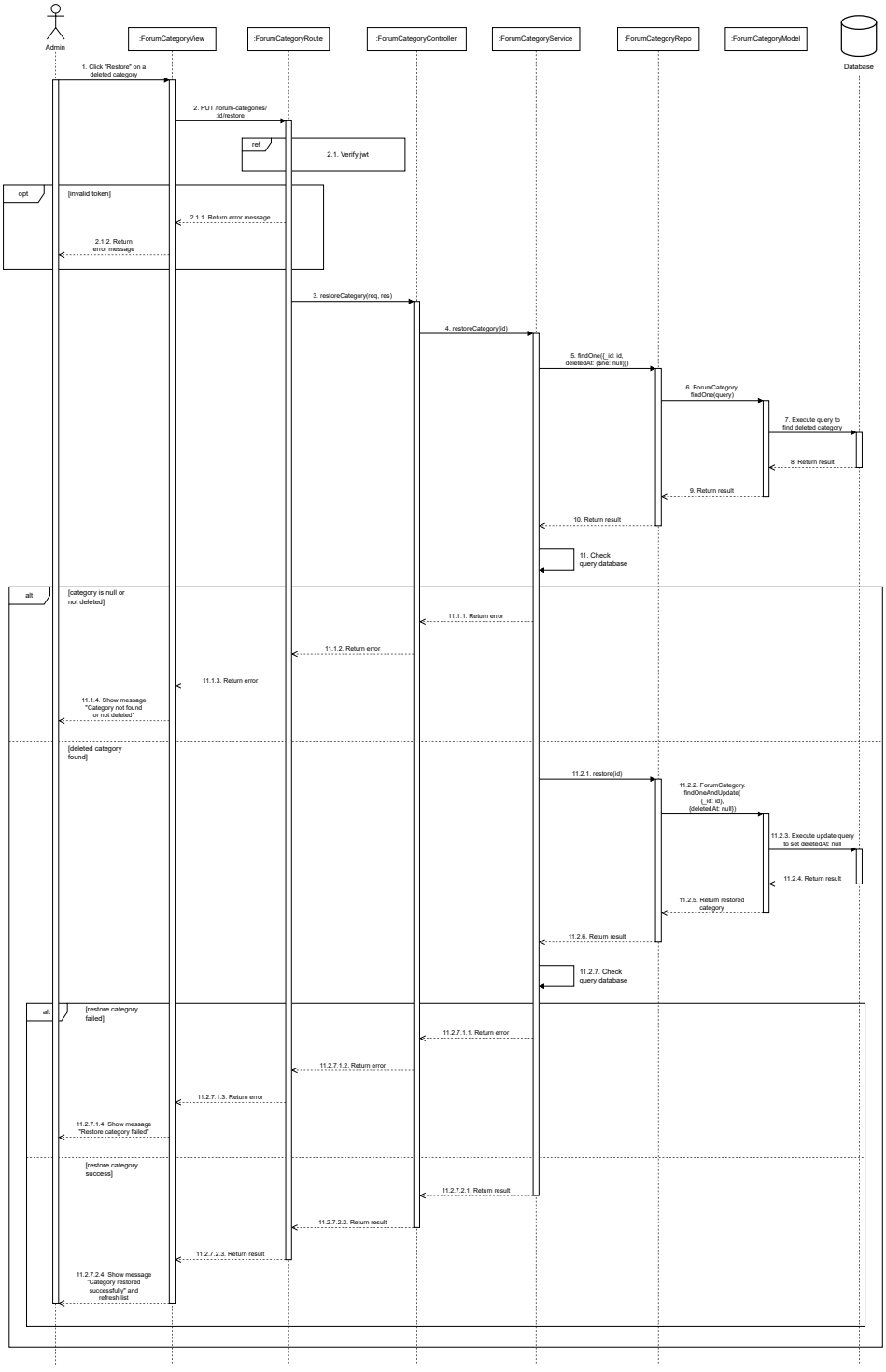

Figure 106: Sequence diagram of UC-17.06 Restore Forum Category

3.37.5 UC-14.01 – View Forum Category List

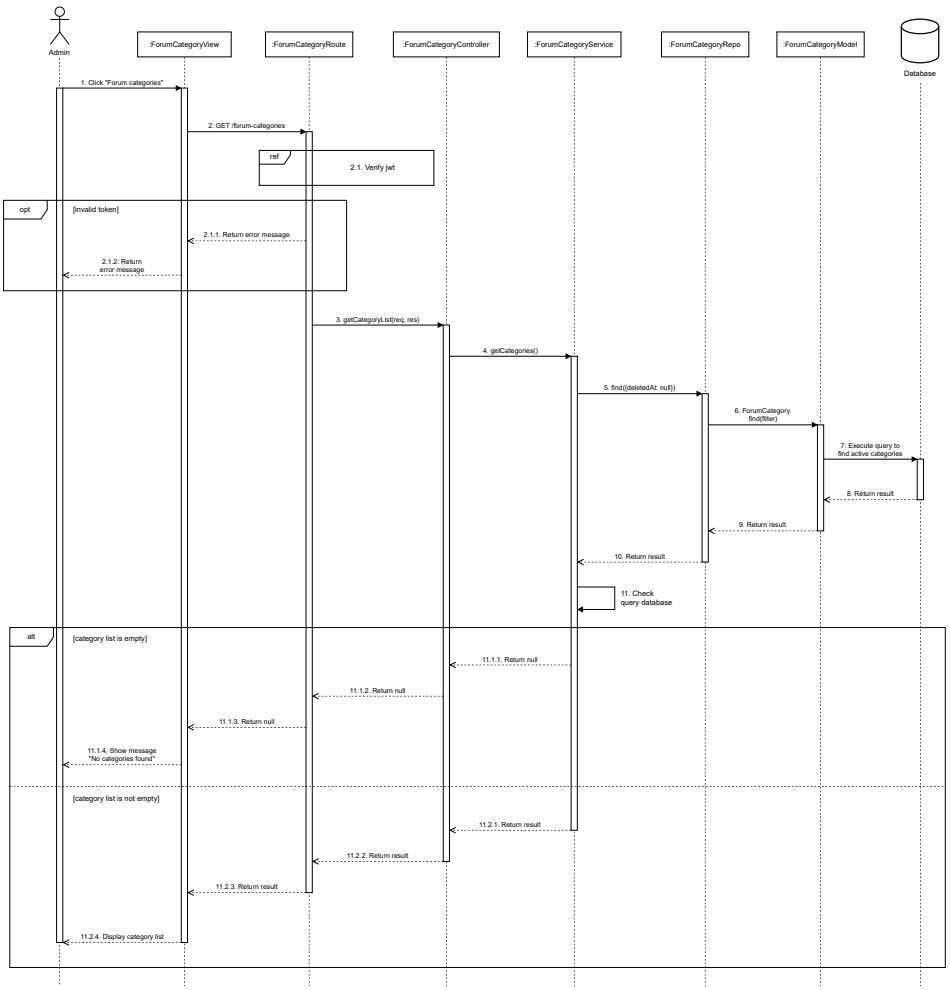

Figure 107: Sequence diagram of UC-14.01 View Forum Category List

3.37.6 UC-14.02 – View Forum Category Detail

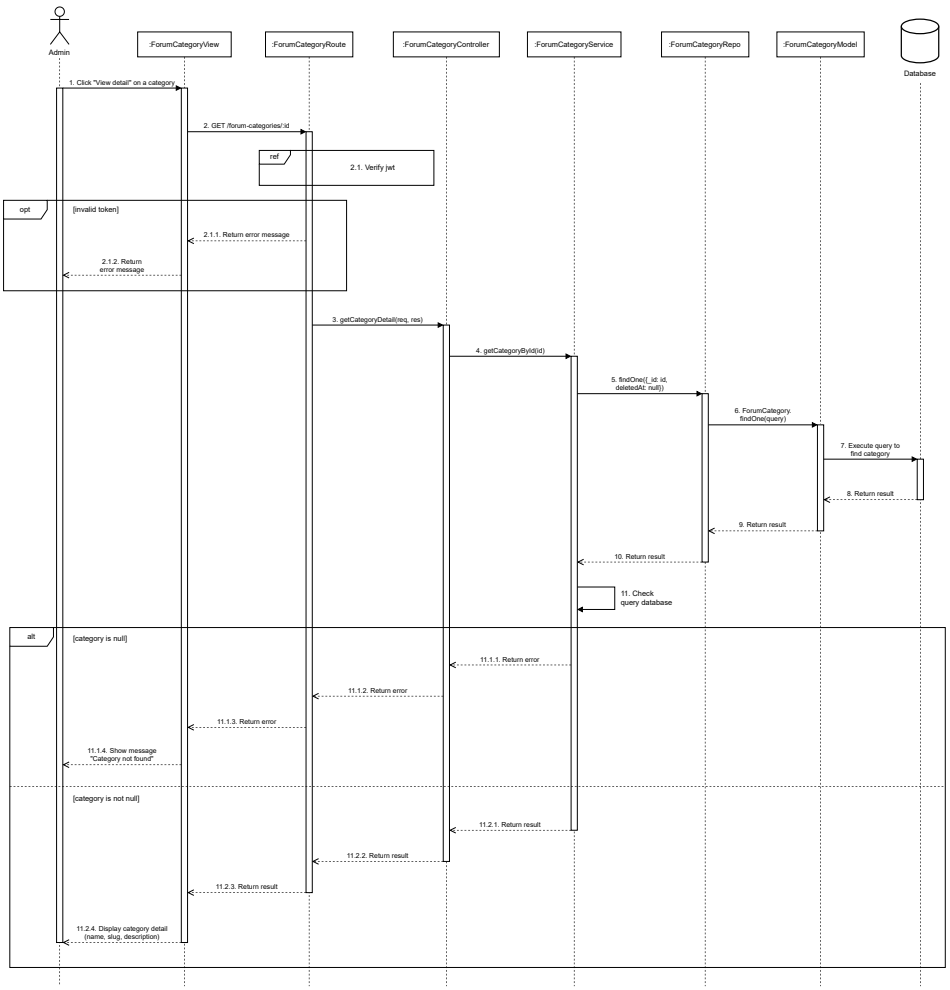

Figure 108: Sequence diagram of UC-14.02 View Forum Category Detail

3.38 View Moderation Queue

a. Class Diagram

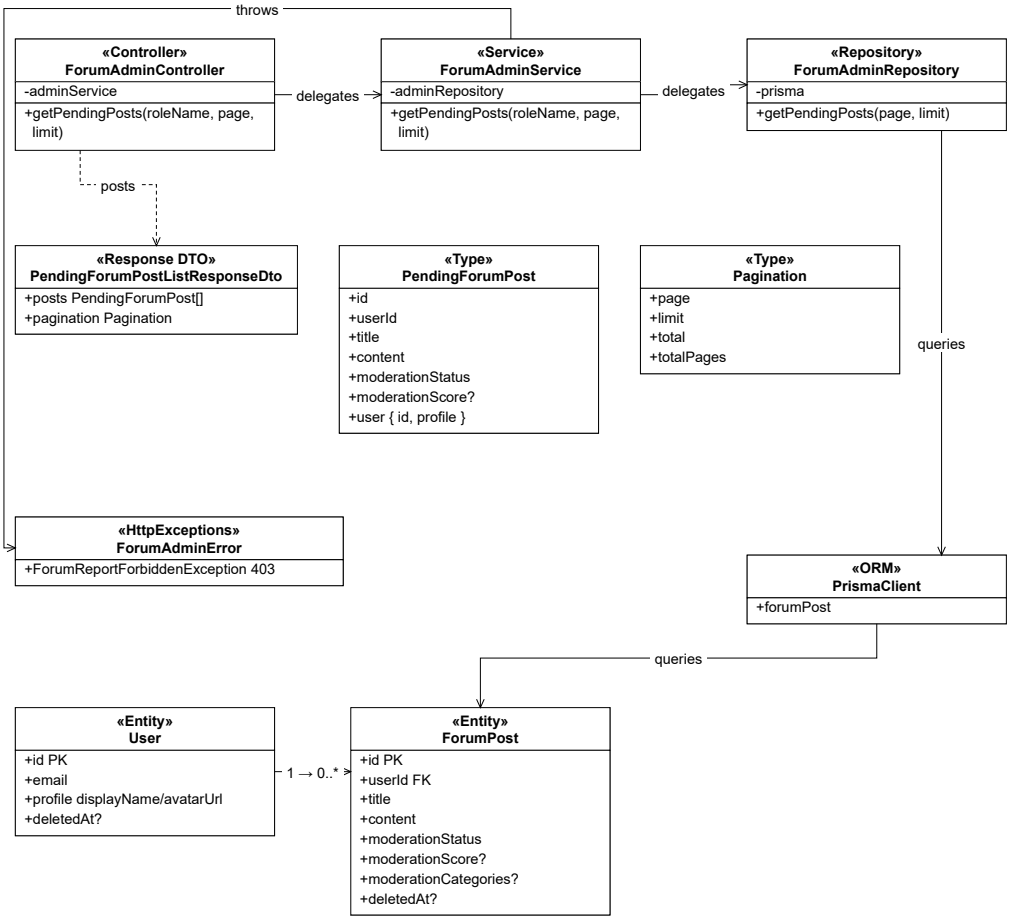

Figure 109: Class diagram of View Moderation Queue

b. Sequence Diagram(s)

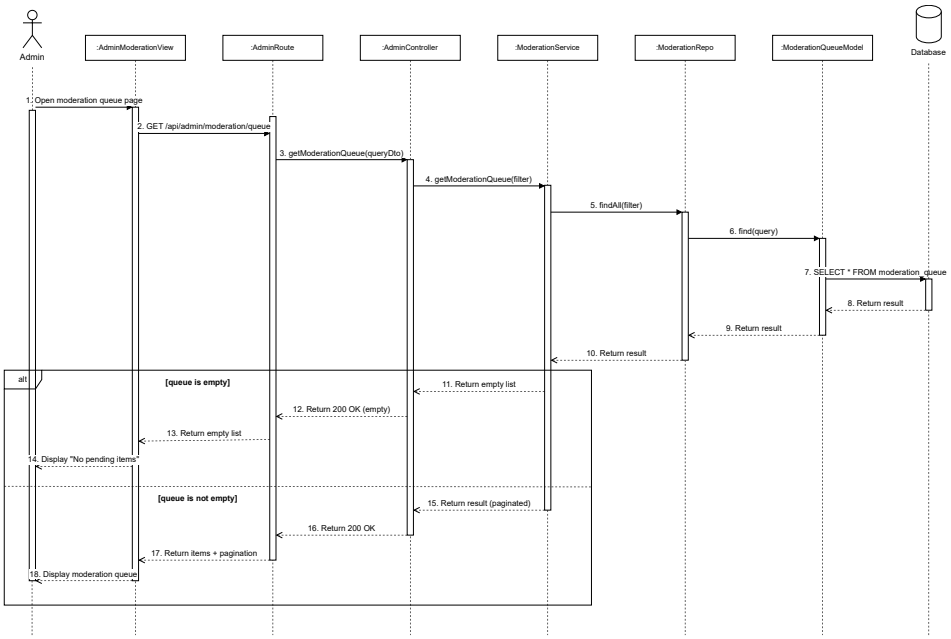

Figure 110: Sequence diagram of View Moderation Queue

3.39 View Moderation Item Detail

a. Class Diagram

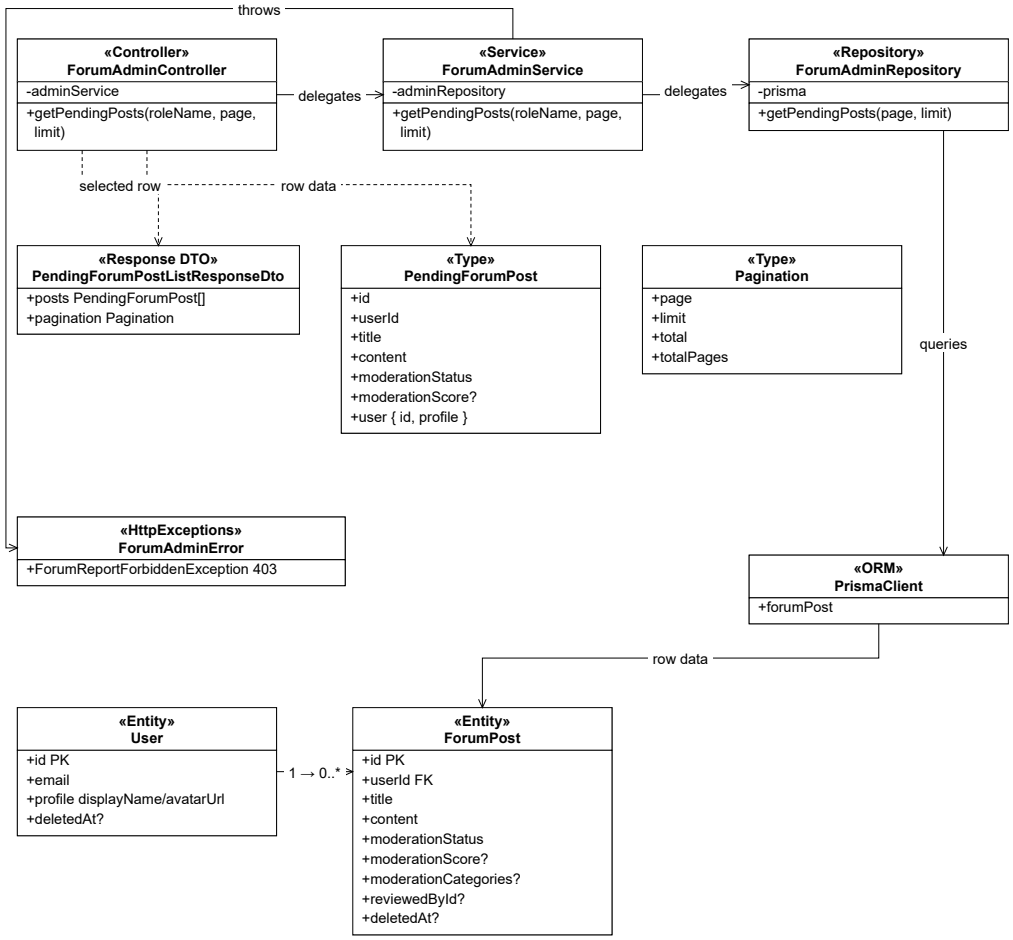

Figure 111: Class diagram of View Moderation Item Detail

b. Sequence Diagram(s)

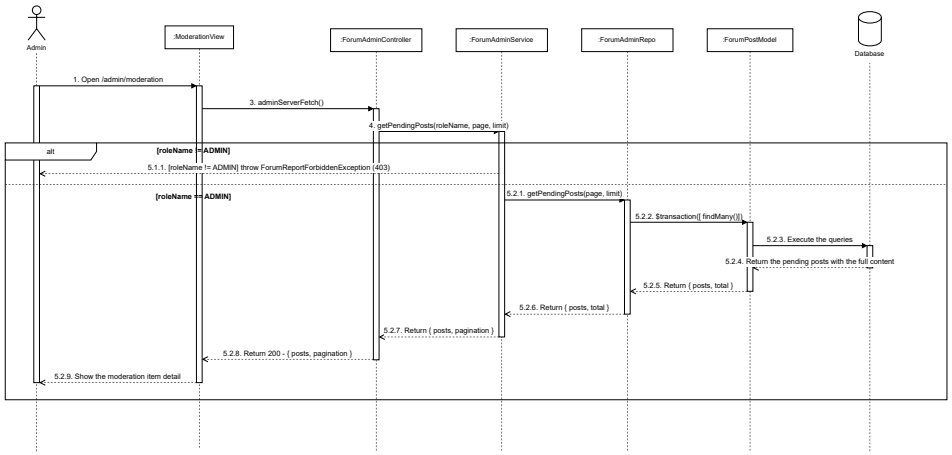

Figure 112: Sequence diagram of View Moderation Item Detail

3.40 Resolve Moderation Item

a. Class Diagram

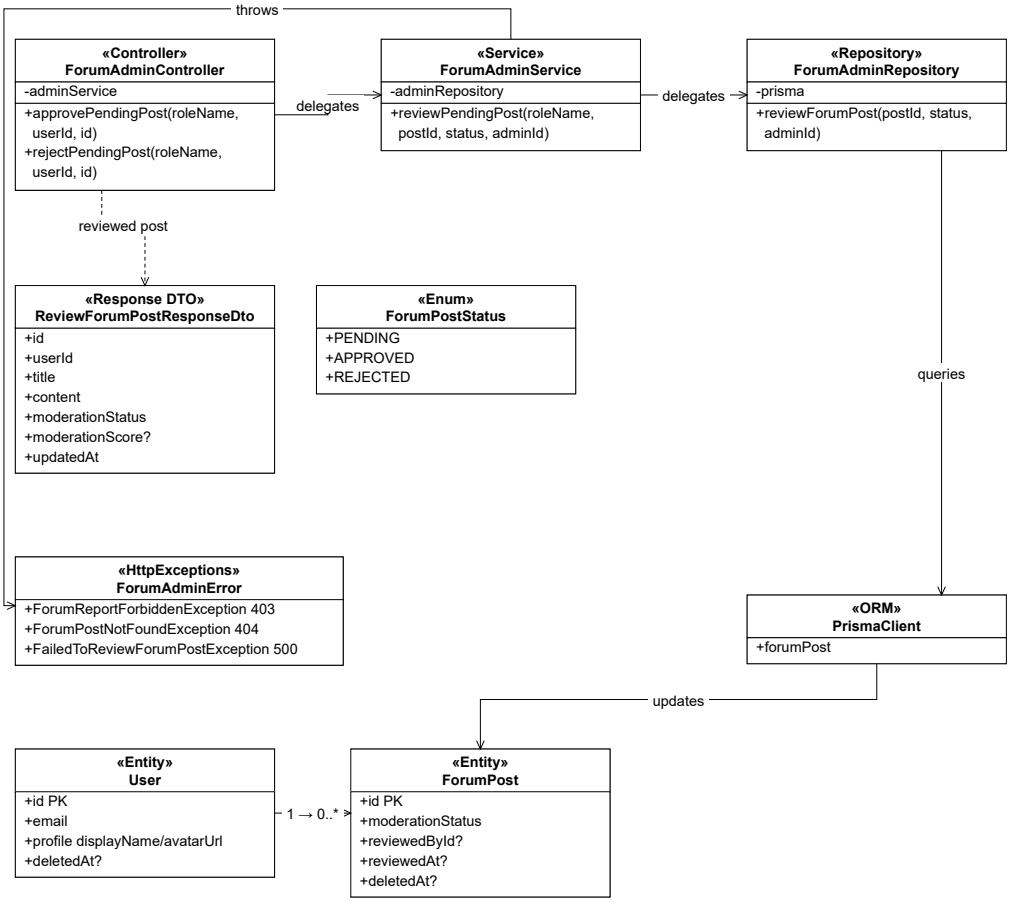

Figure 113: Class diagram of Resolve Moderation Item

b. Sequence Diagram(s)

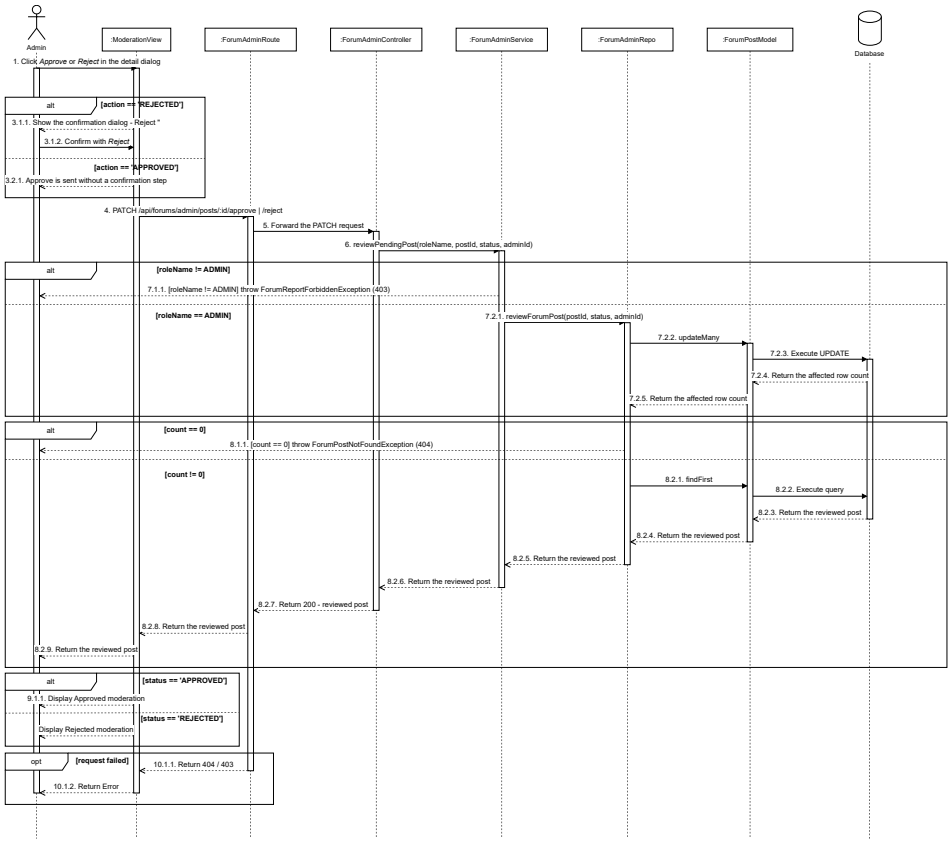

Figure 114: Sequence diagram of Resolve Moderation Item

3.41 View ID Verification Request List

a. Class Diagram

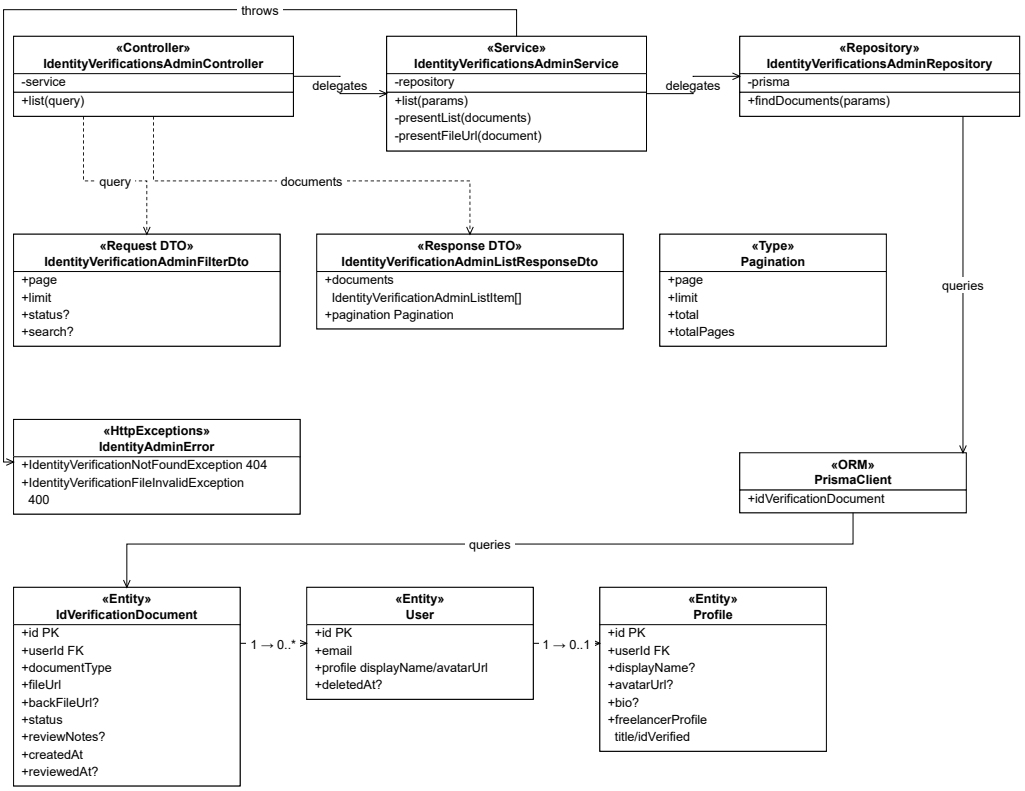

Figure 115: Class diagram of View ID Verification Request List

b. Sequence Diagram(s)

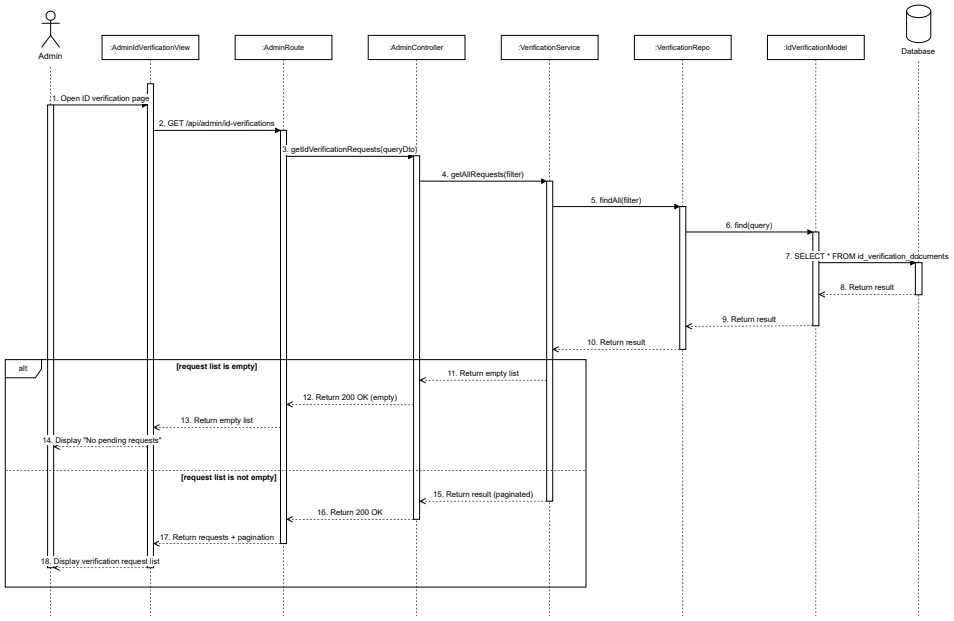

Figure 116: Sequence diagram of View ID Verification Request List

3.42 View ID Verification Request Detail

a. Class Diagram

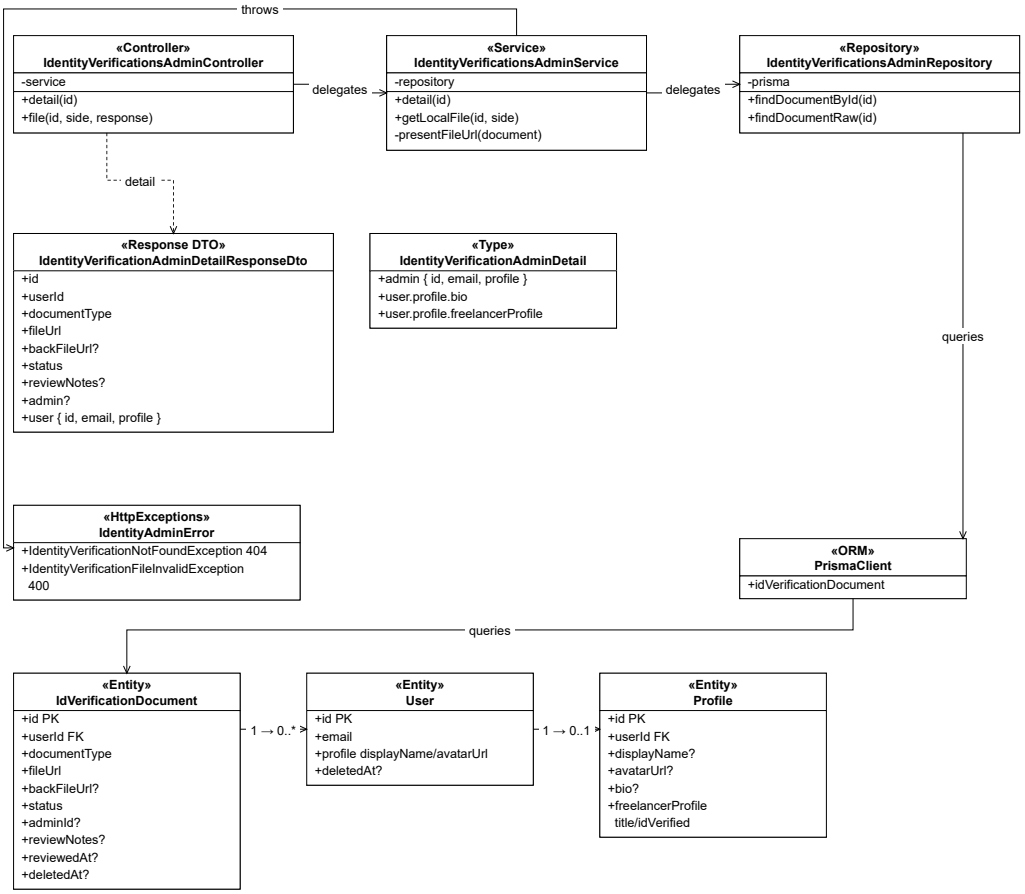

Figure 117: Class diagram of View ID Verification Request Detail

b. Sequence Diagram(s)

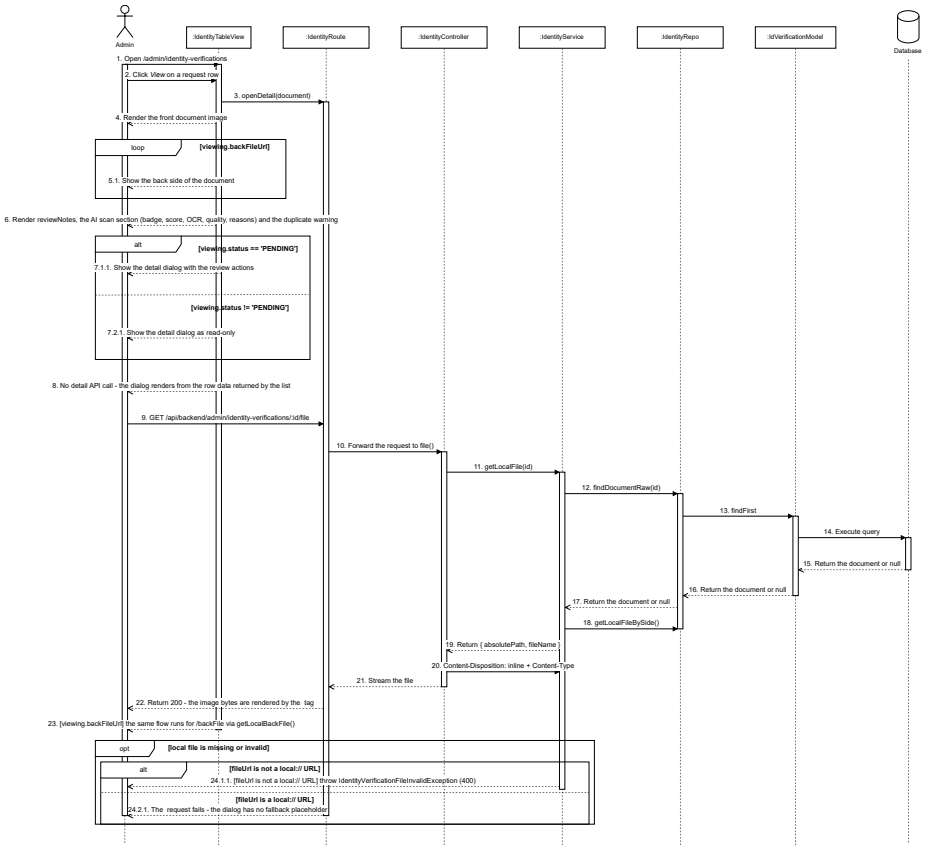

Figure 118: Sequence diagram of View ID Verification Request Detail

3.43 Approve ID Verification

a. Class Diagram

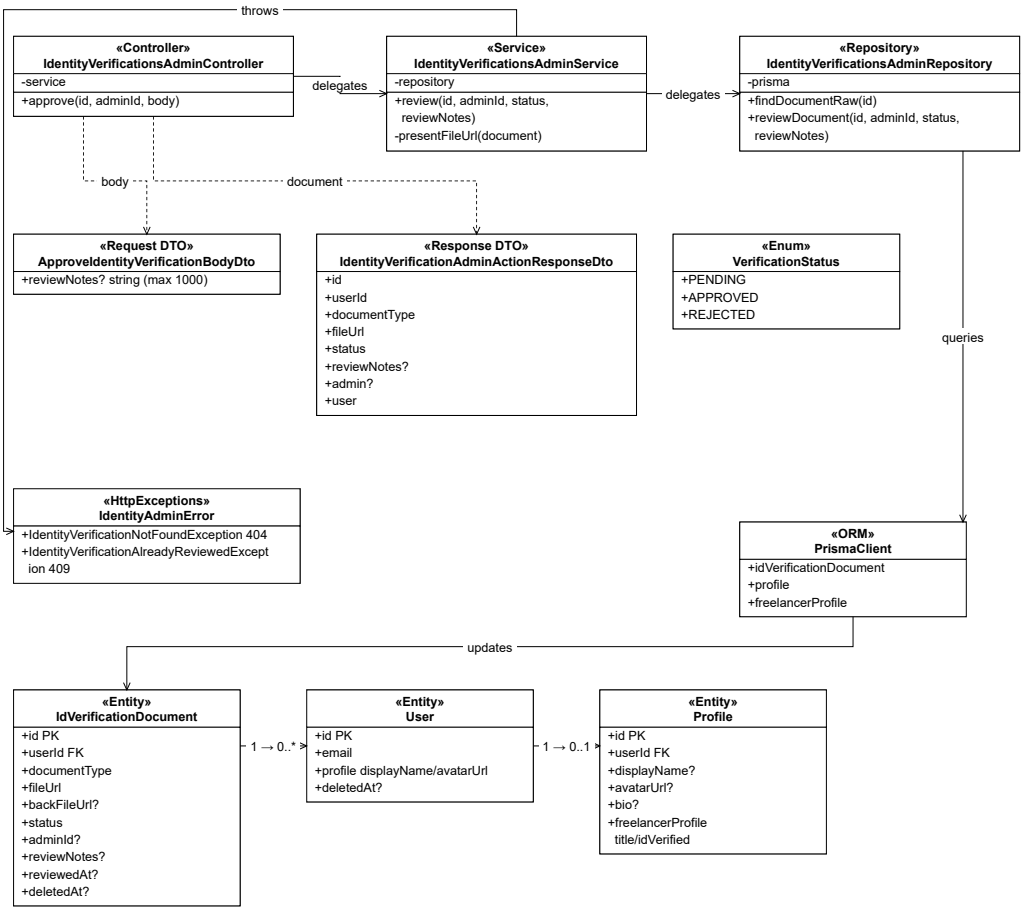

Figure 119: Class diagram of Approve ID Verification

b. Sequence Diagram(s)

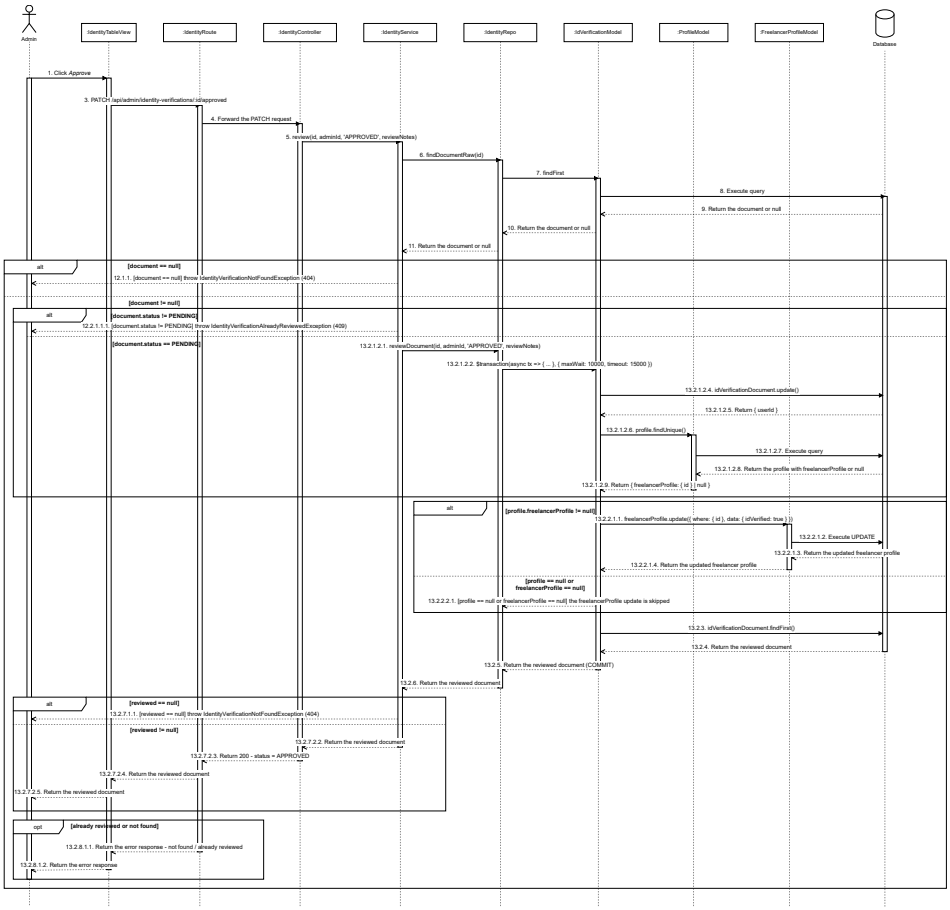

Figure 120: Sequence diagram of Approve ID Verification

3.44 Reject ID Verification

a. Class Diagram

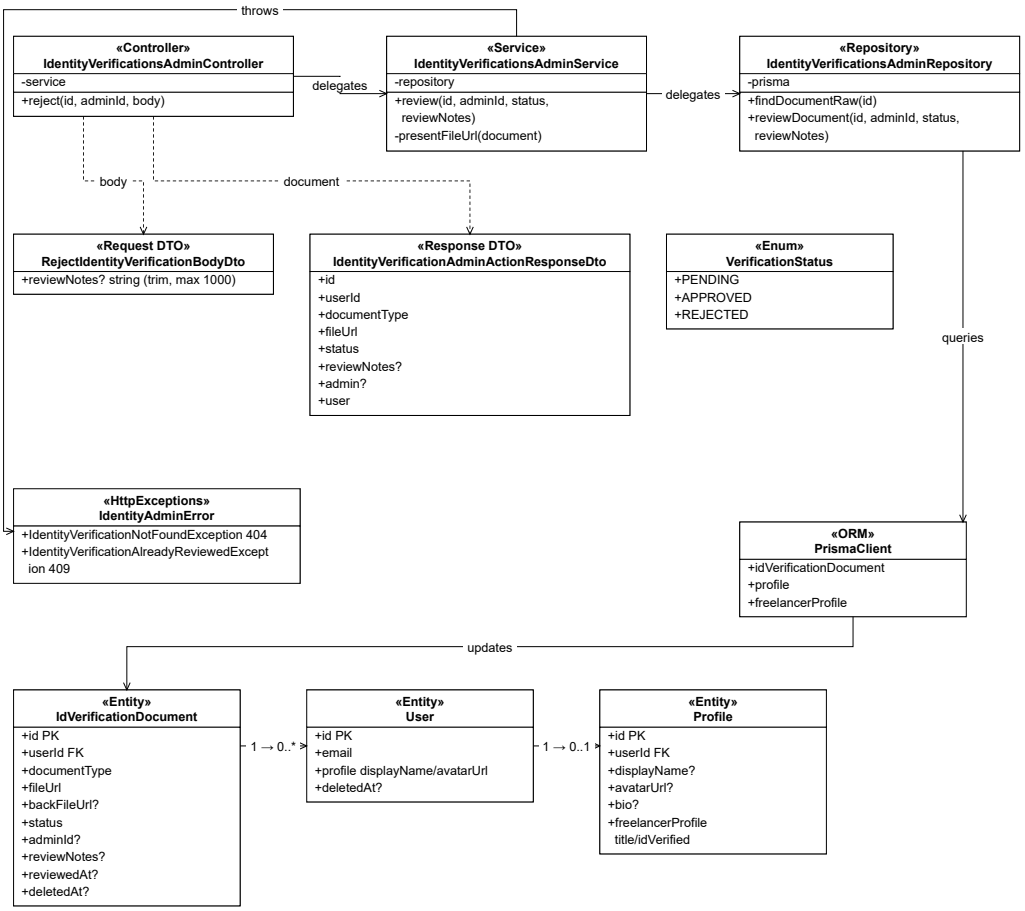

Figure 121: Class diagram of Reject ID Verification

b. Sequence Diagram(s)

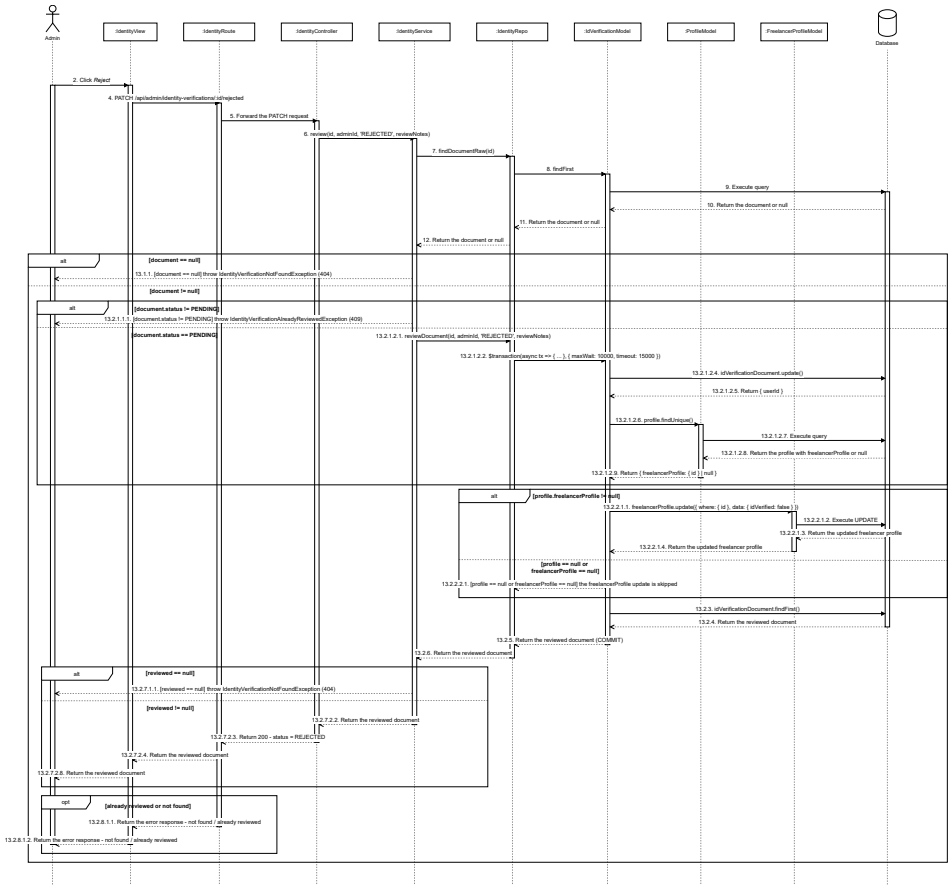

Figure 122: Sequence diagram of Reject ID Verification

3.45 View Deleted Posts/Comments List

a. Class Diagram

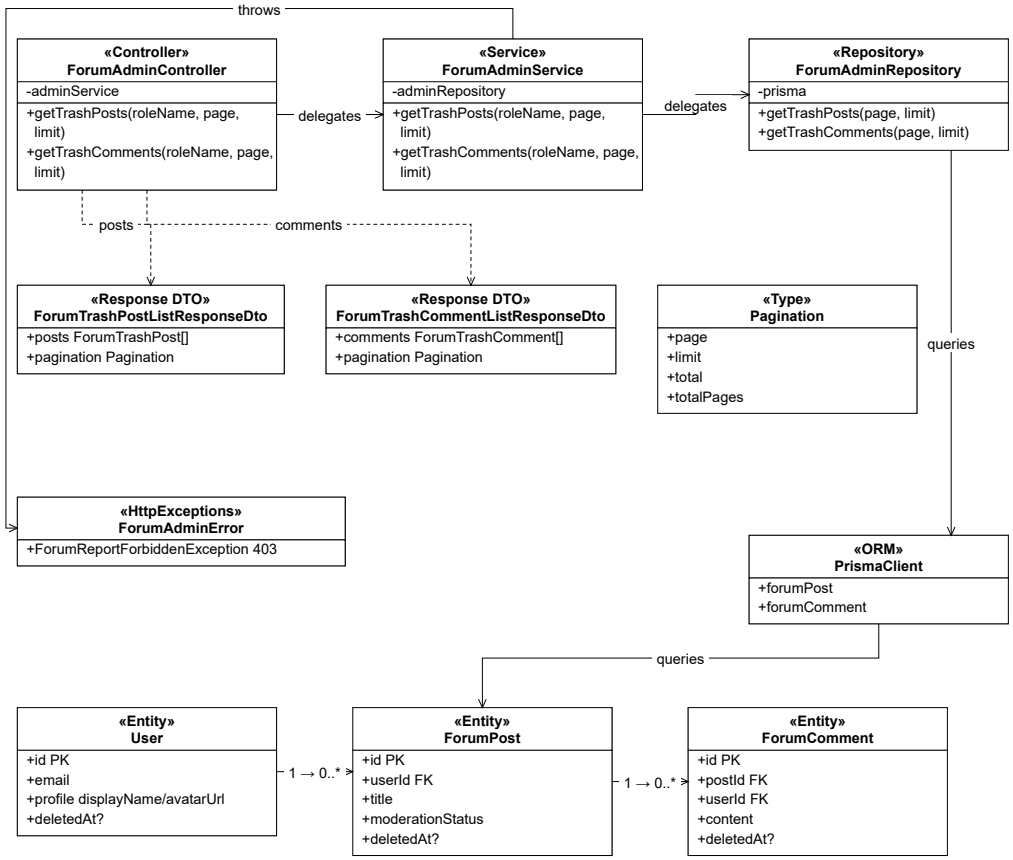

Figure 123: Class diagram of View Deleted Posts/Comments List

b. Sequence Diagram(s)

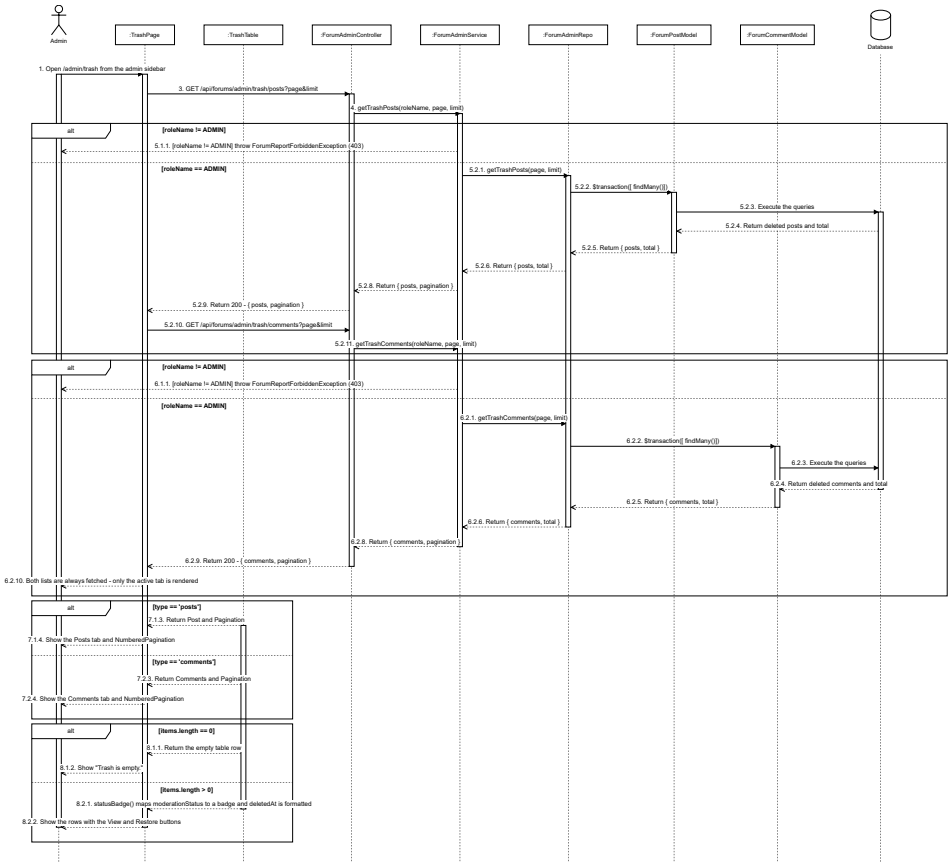

Figure 124: Sequence diagram of View Deleted Posts/Comments List

3.46 View Deleted Posts/Comments Detail

a. Class Diagram

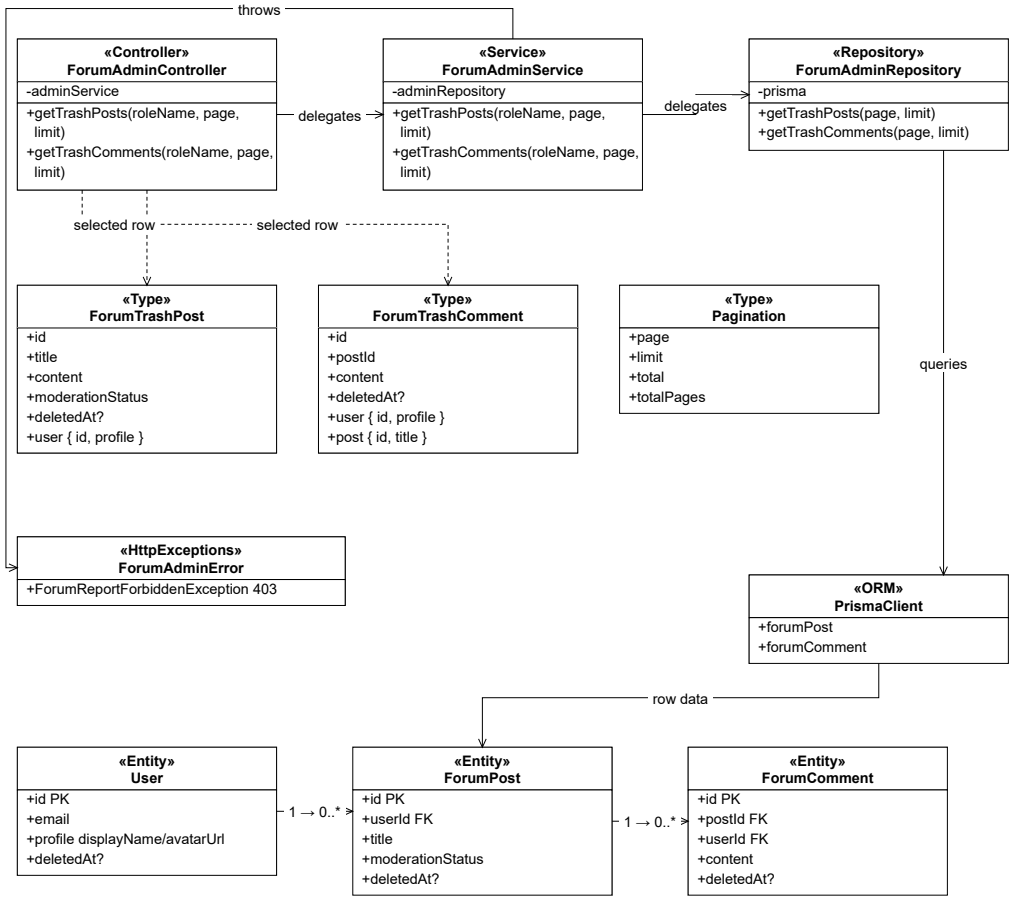

Figure 125: Class diagram of View Deleted Posts/Comments Detail

b. Sequence Diagram(s)

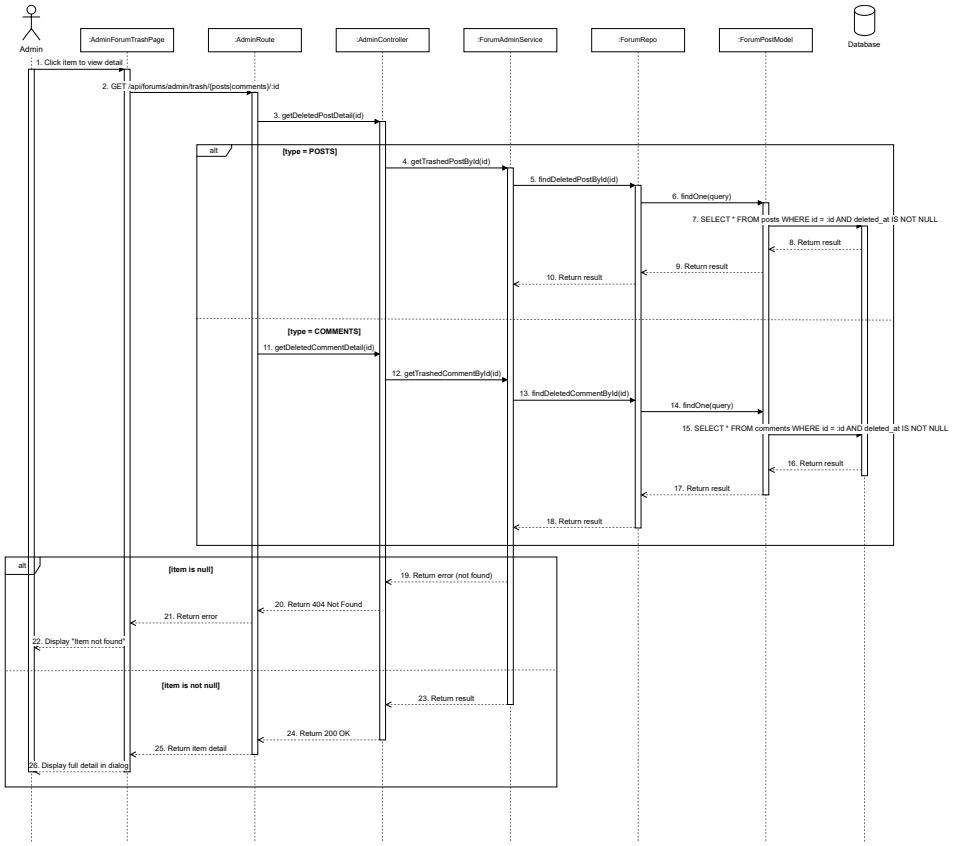

Figure 126: Sequence diagram of View Deleted Posts/Comments Detail

3.47 Restore Deleted Posts/Comments

a. Class Diagram

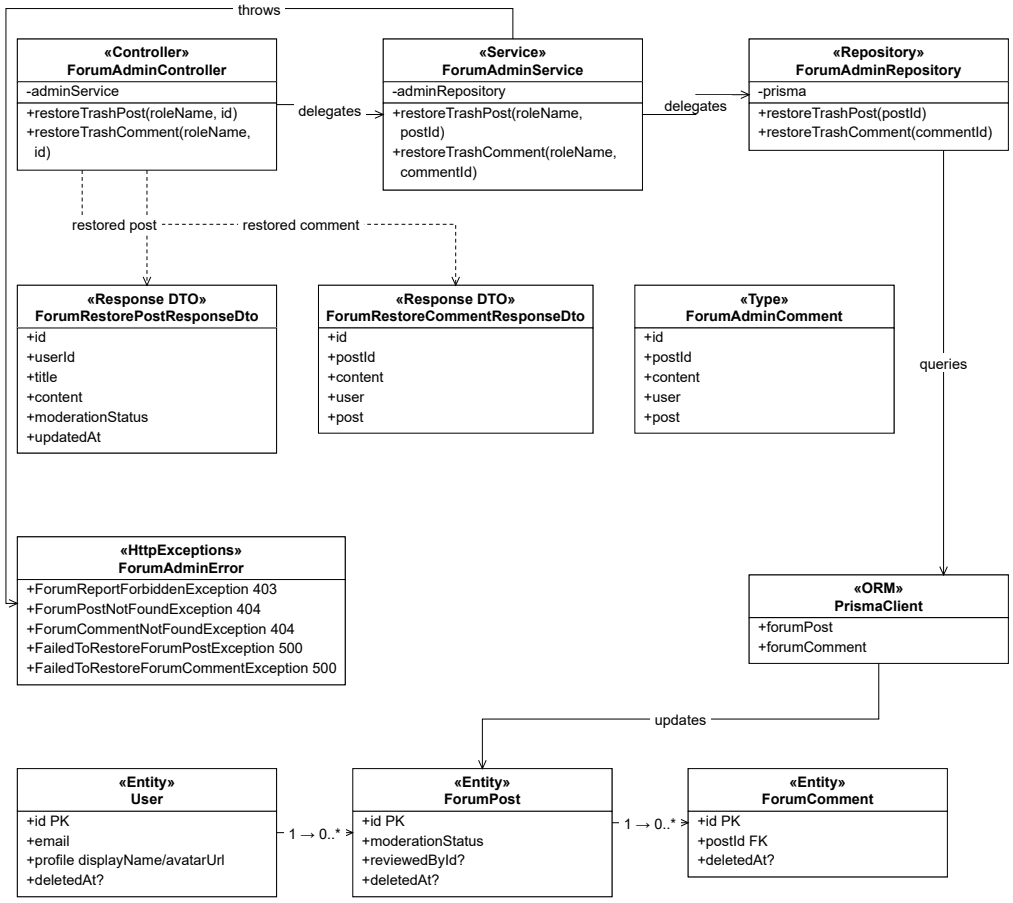

Figure 127: Class diagram of Restore Deleted Posts/Comments

b. Sequence Diagram(s)

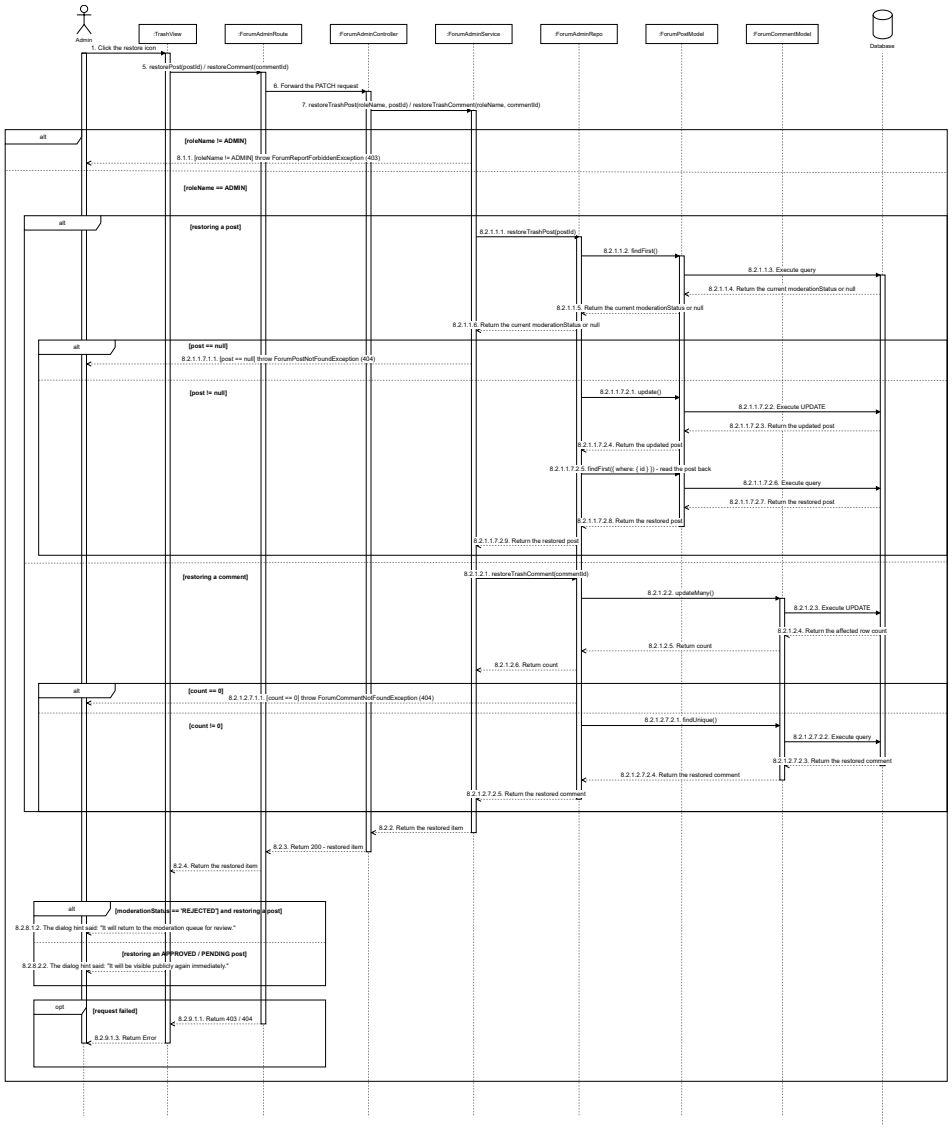

Figure 128: Sequence diagram of Restore Deleted Posts/Comments

3.48 Following Freelancer Management

Clients view followed freelancers, follow a freelancer and remove an existing follow relationship. A new follow creates a notification in the same transaction.

a. Class Diagram

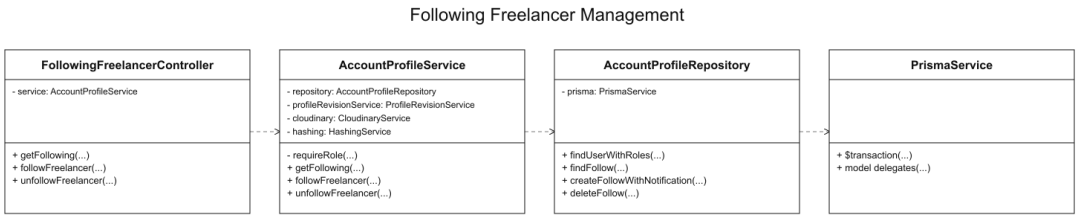

Figure 129: Class diagram of Following Freelancer Management

b. Sequence Diagram(s)

3.48.1 UC-29.01 – View Following List

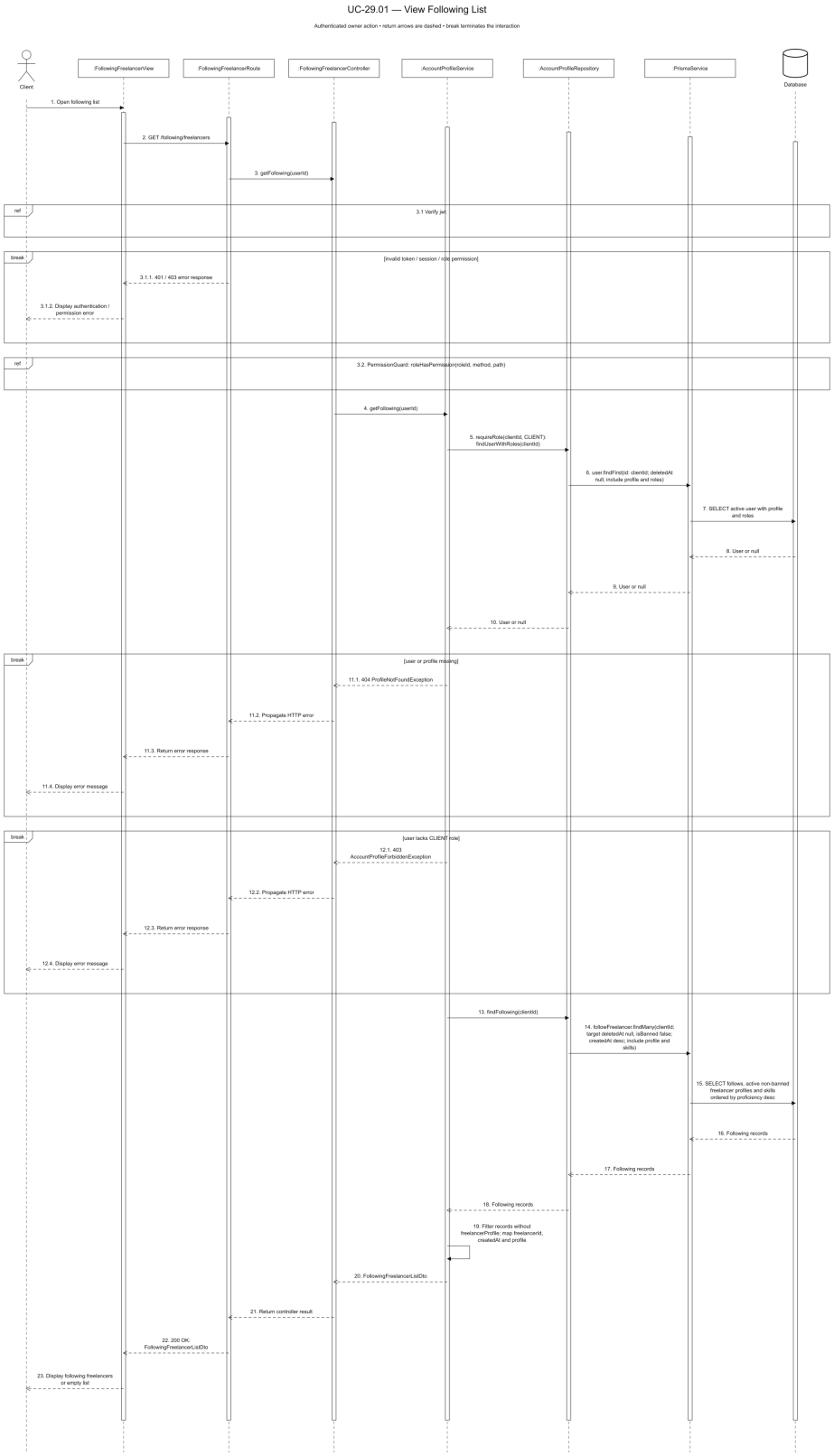

Figure 130: Sequence diagram of UC-29.01 View Following List

3.48.2 UC-29.02 – Follow Freelancer

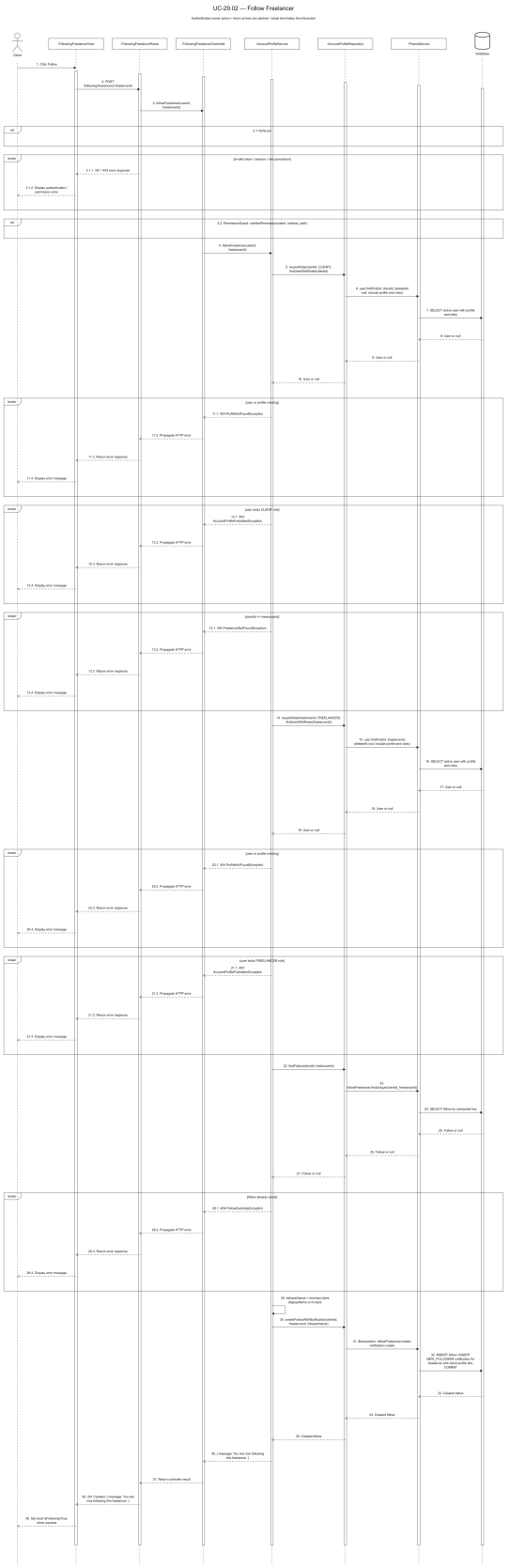

Figure 131: Sequence diagram of UC-29.02 Follow Freelancer

3.48.3 UC-29.03 – Unfollow Freelancer

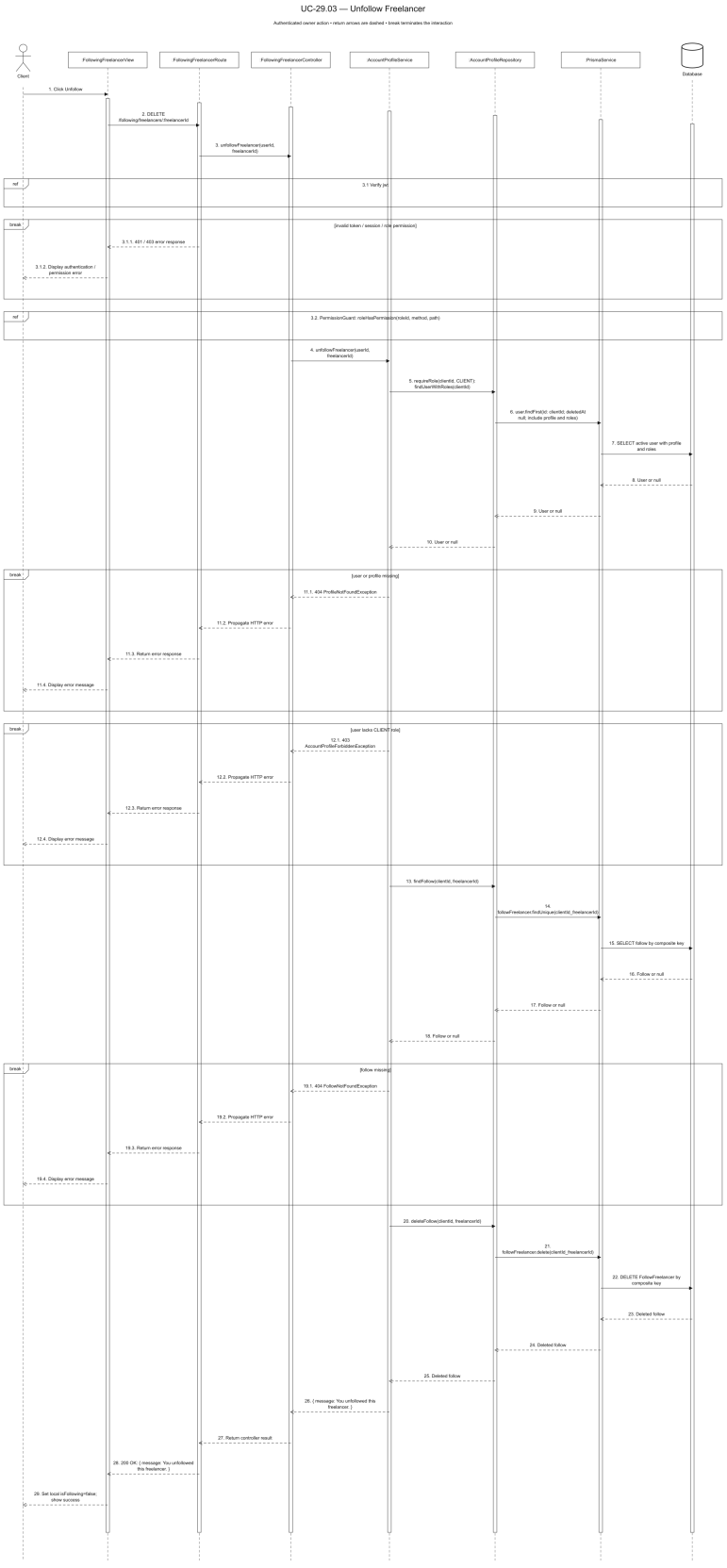

Figure 132: Sequence diagram of UC-29.03 Unfollow Freelancer

3.49 Notification Management

Users view their notifications, mark one or all as read and soft-delete an owned notification.

a. Class Diagram

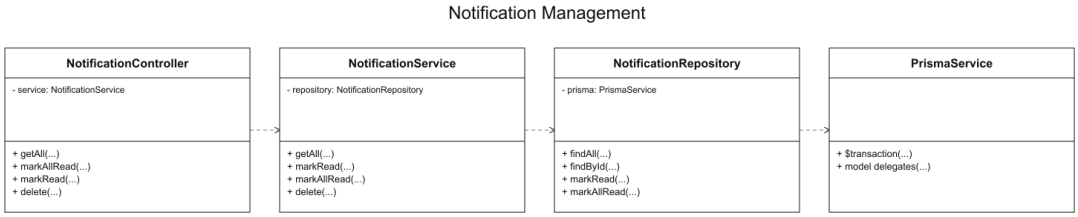

Figure 133: Class diagram of Notification Management

b. Sequence Diagram(s)

3.49.1 UC-30.01 – View Notification List

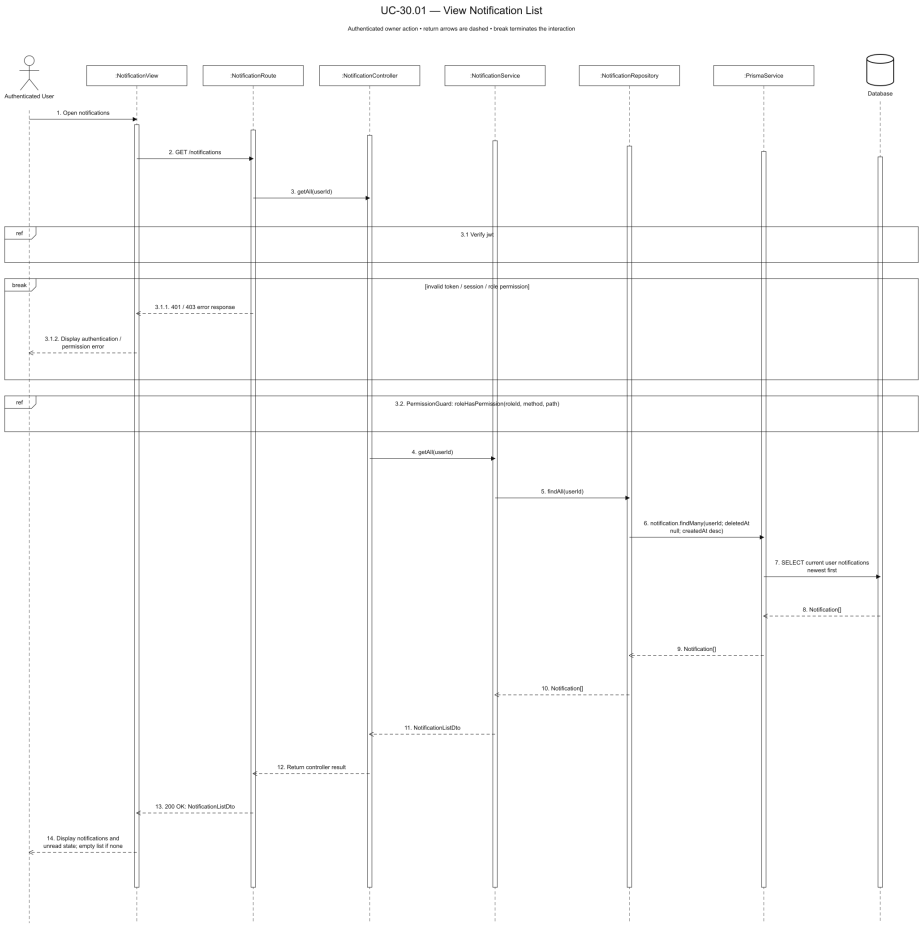

Figure 134: Sequence diagram of UC-30.01 View Notification List

3.49.2 UC-30.02 – Mark Notification as Read

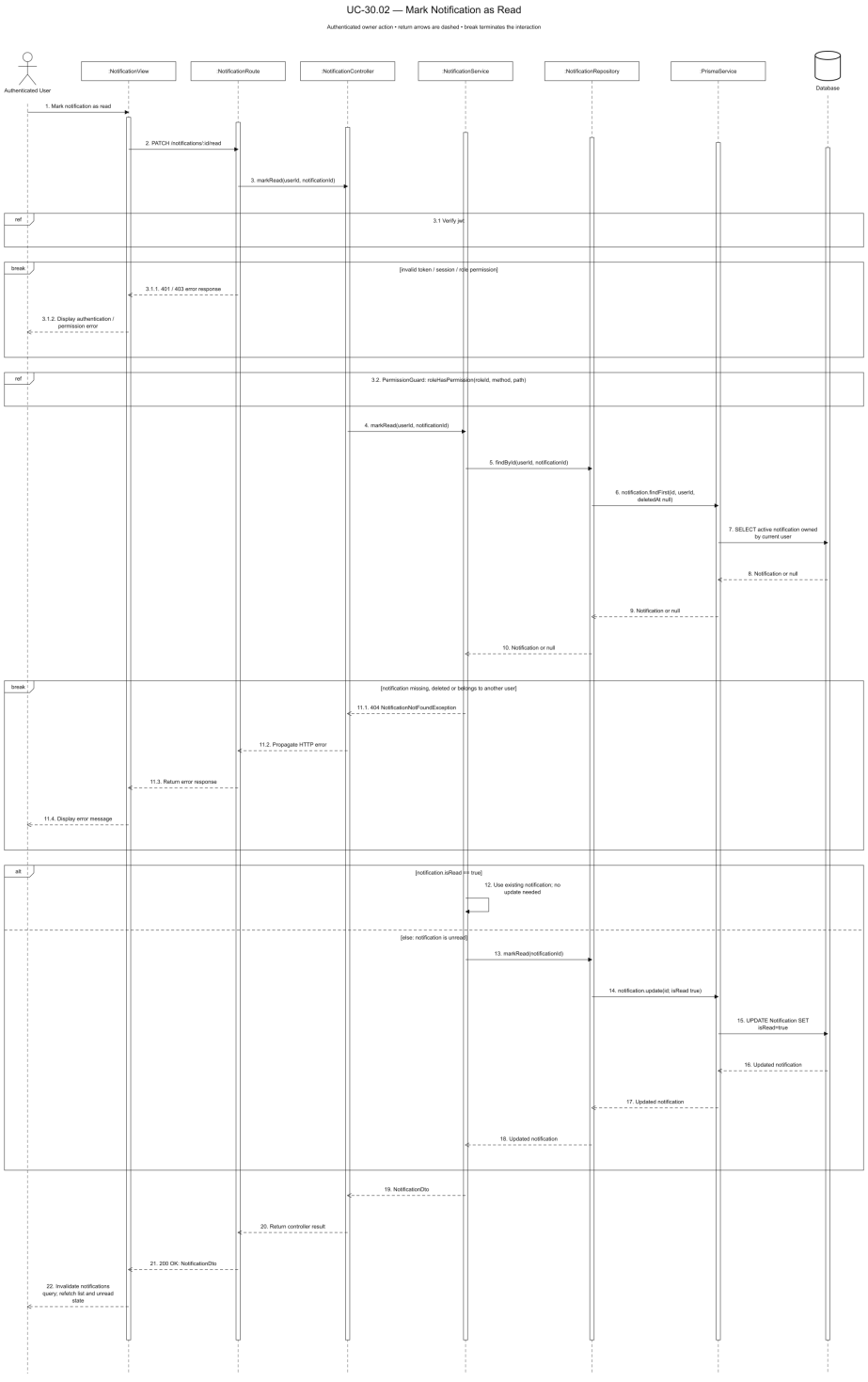

Figure 135: Sequence diagram of UC-30.02 Mark Notification as Read

3.49.3 UC-30.03 – Mark All Notifications as Read

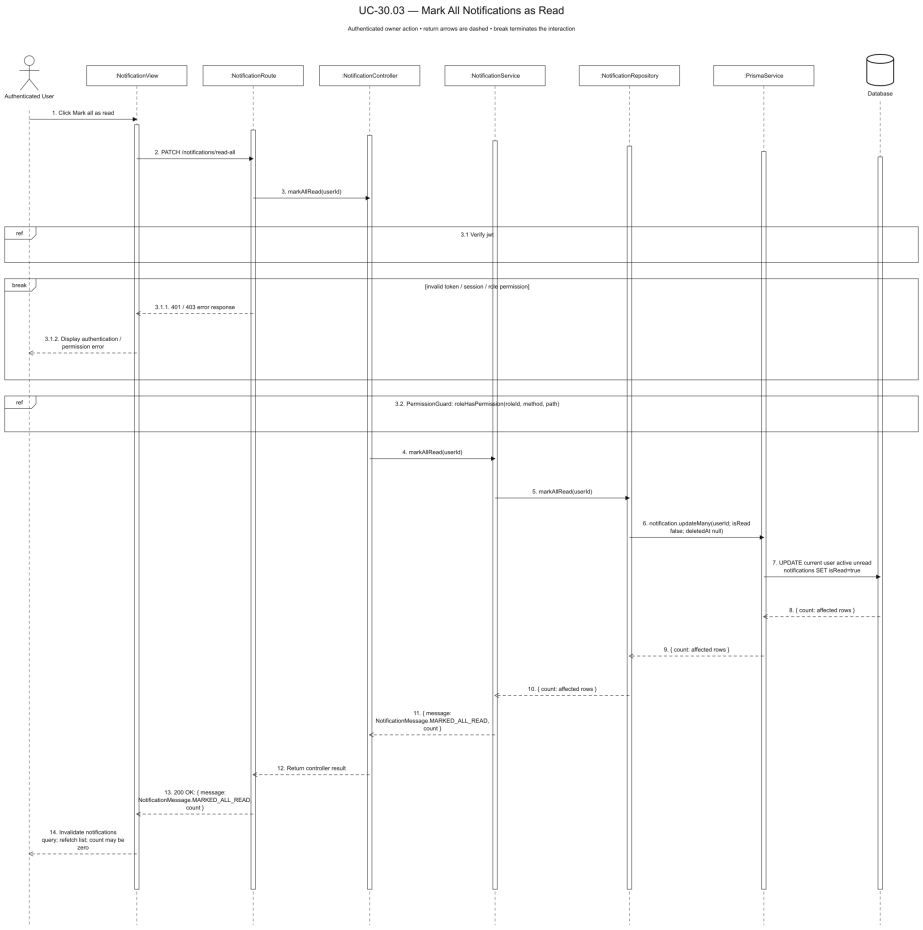

Figure 136: Sequence diagram of UC-30.03 Mark All Notifications as Read

3.49.4 UC-30.04 – Delete Notification

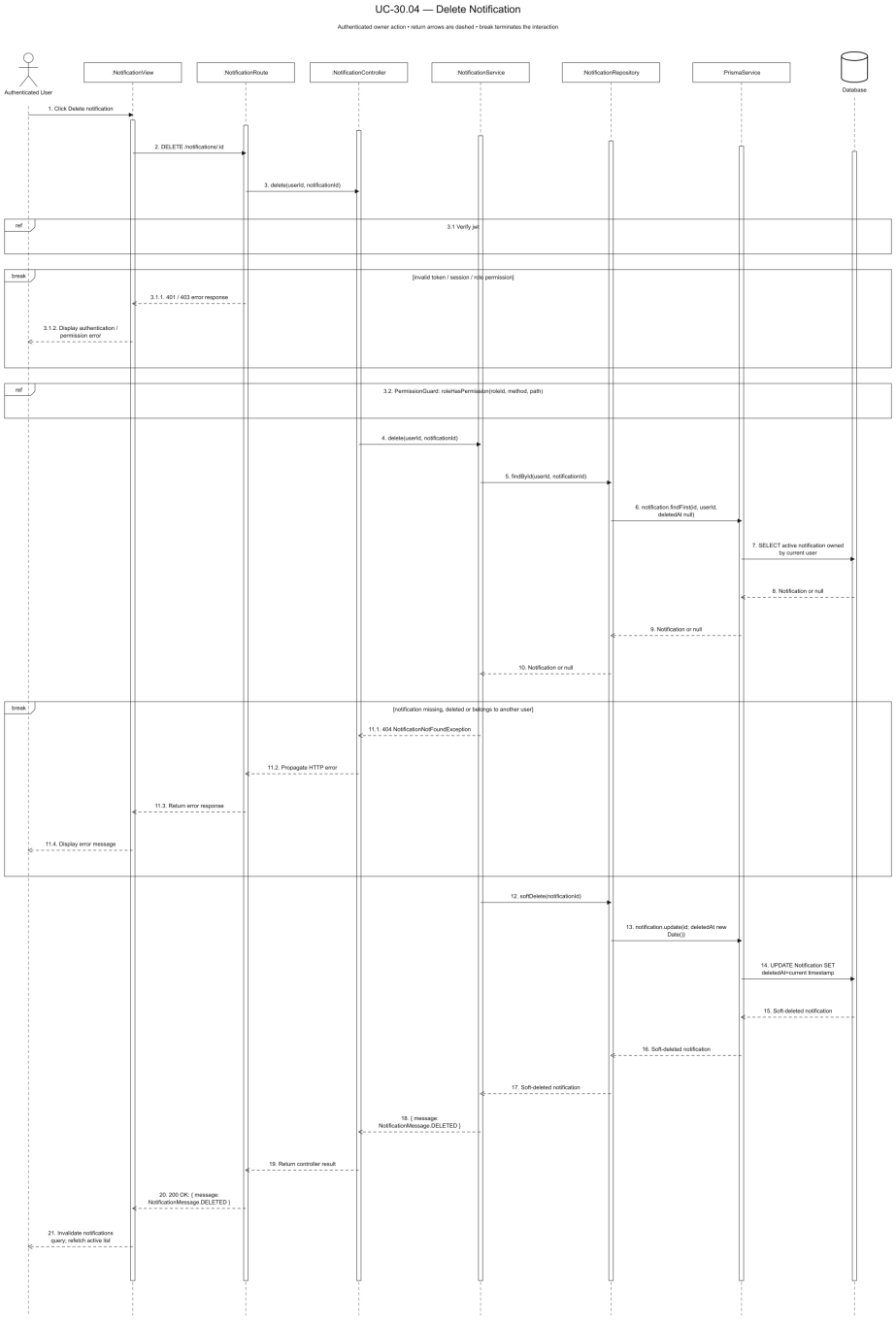

Figure 137: Sequence diagram of UC-30.04 Delete Notification

3.50 Account Role Management

Users activate an assigned role or join a new client/freelancer role. Both operations issue a new access token for the current session.

a. Class Diagram

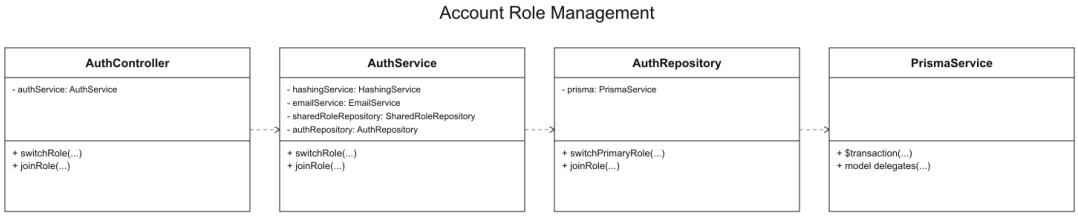

Figure 138: Class diagram of Account Role Management

b. Sequence Diagram(s)

3.50.1 UC-31.01 – Switch Role

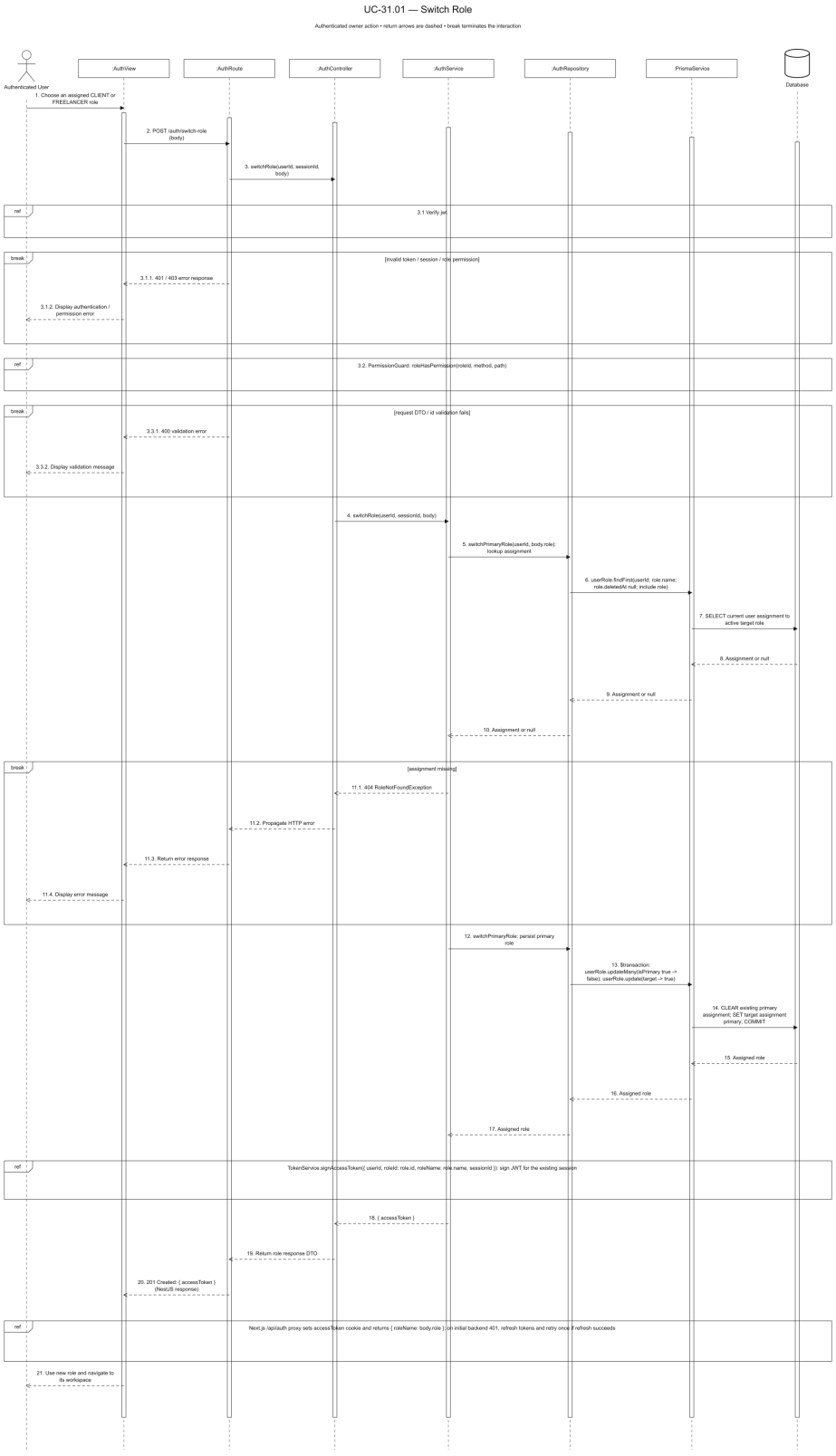

Figure 139: Sequence diagram of UC-31.01 Switch Role

3.50.2 UC-31.02 – Join New Role

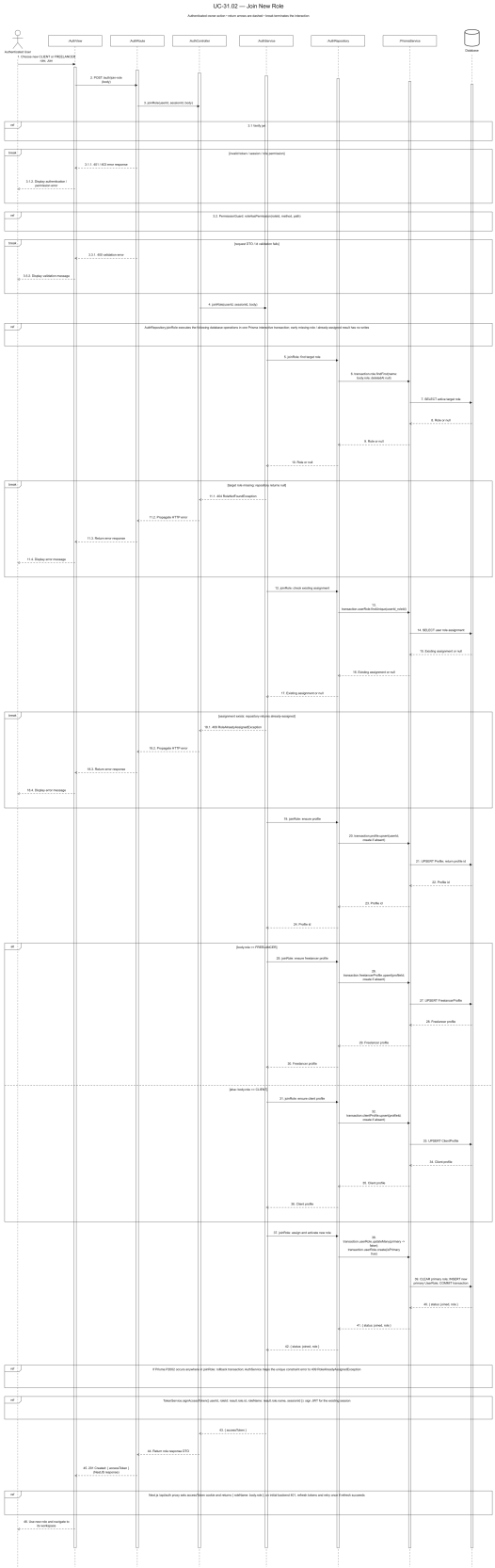

Figure 140: Sequence diagram of UC-31.02 Join New Role

3.51 User Management

User Management provides the administrator with a searchable user catalogue, a consolidated account detail view, and dedicated editors for the Client and Freelancer profiles attached to an account.

a. Class Diagram

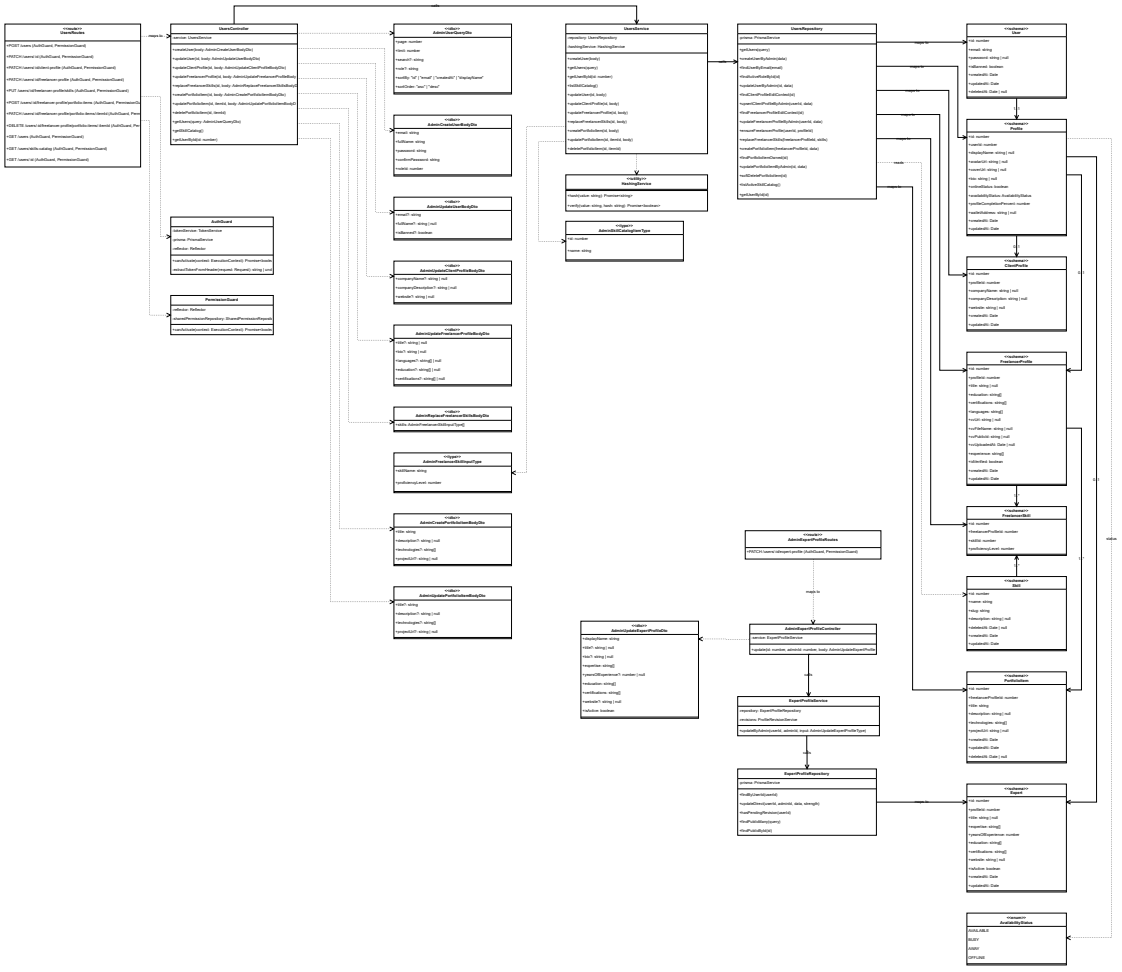

Figure 141: Class diagram of User Management

b. Sequence Diagram(s)

3.51.1 UC-28.01 – View User List

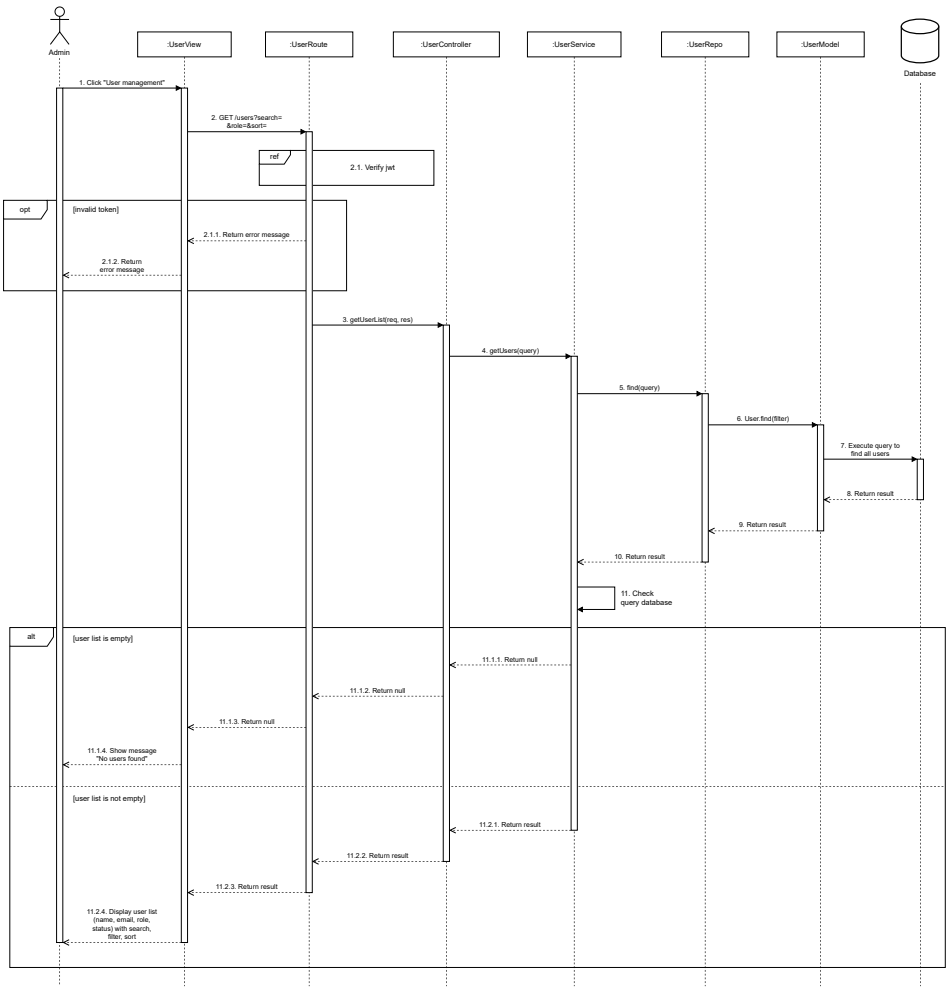

Figure 142: Sequence diagram of UC-28.01 View User List

3.51.2 UC-28.02 – View User Detail

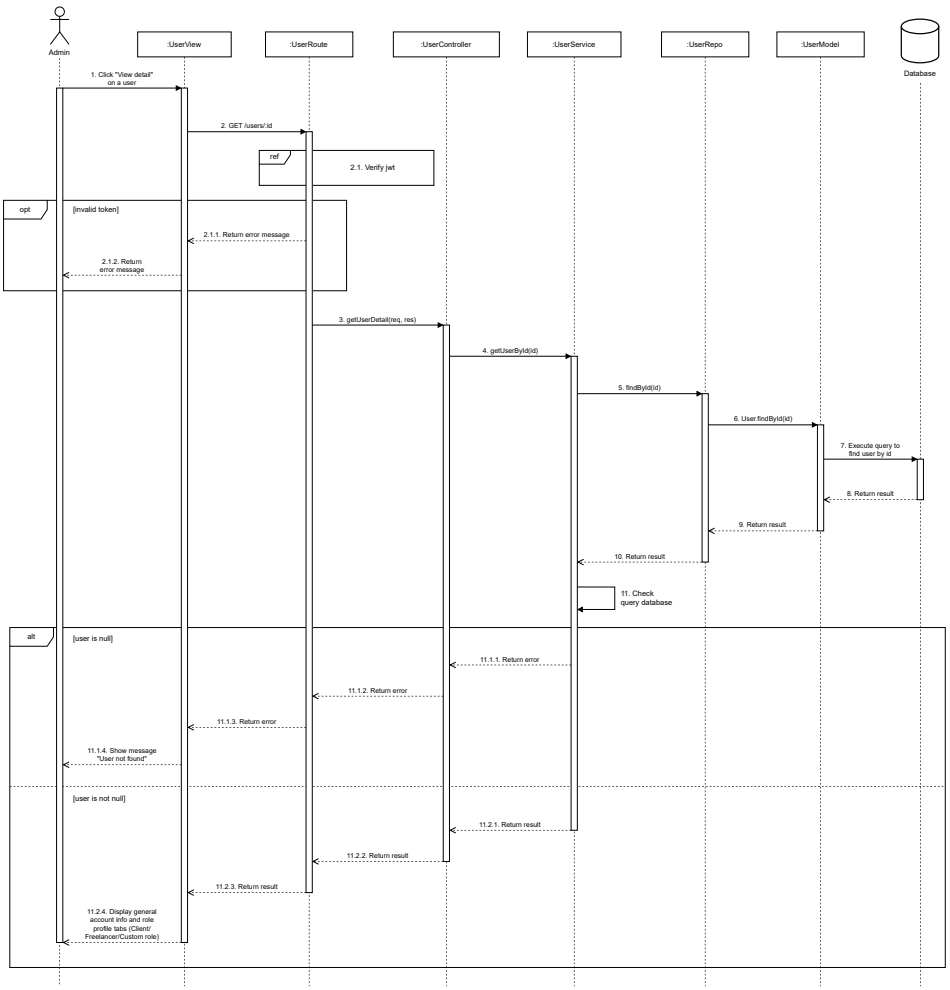

Figure 143: Sequence diagram of UC-28.02 View User Detail

3.51.3 UC-28.03 – Create User

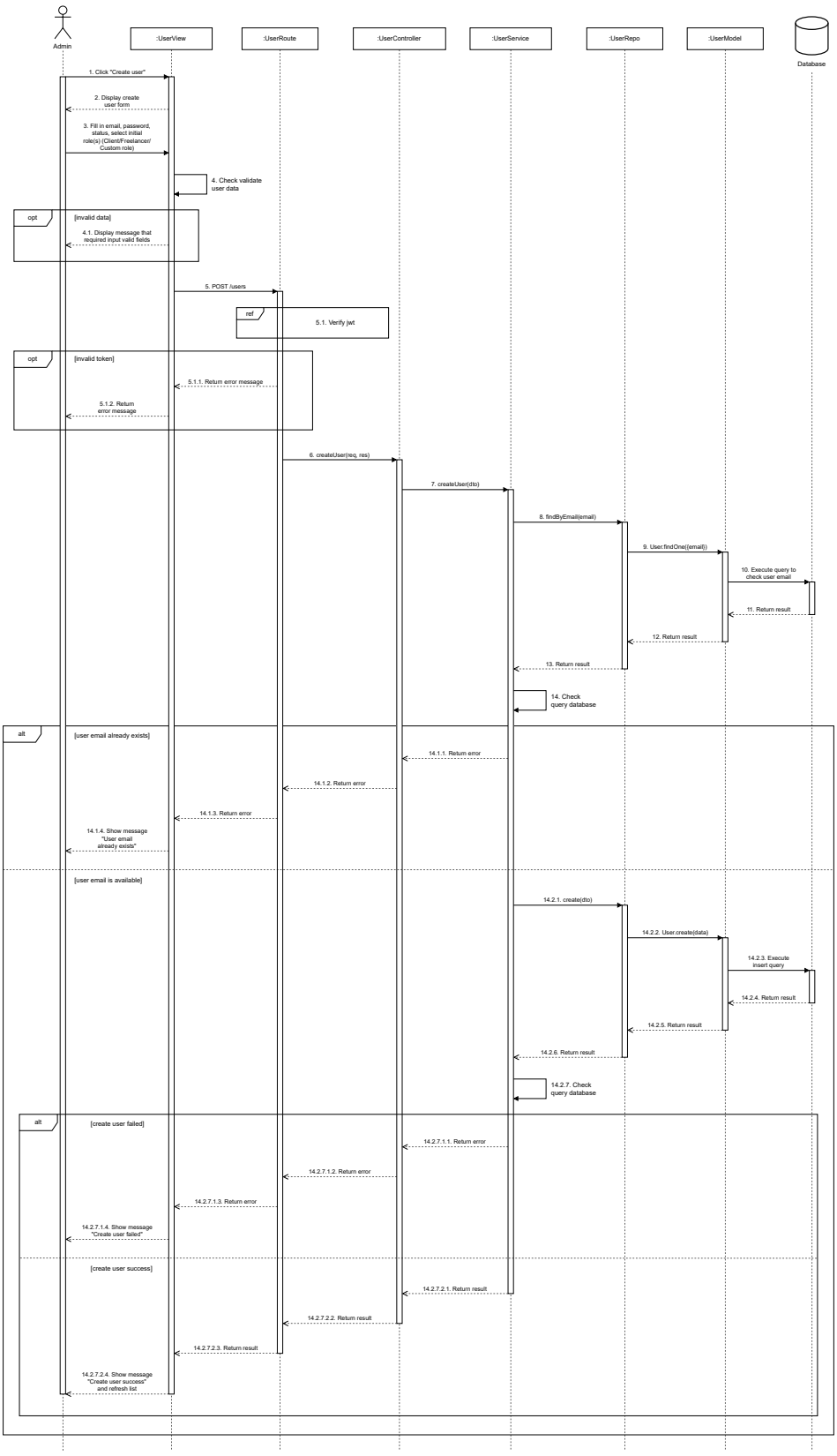

Figure 144: Sequence diagram of UC-28.03 Create User

3.51.4 UC-28.04 – Update User

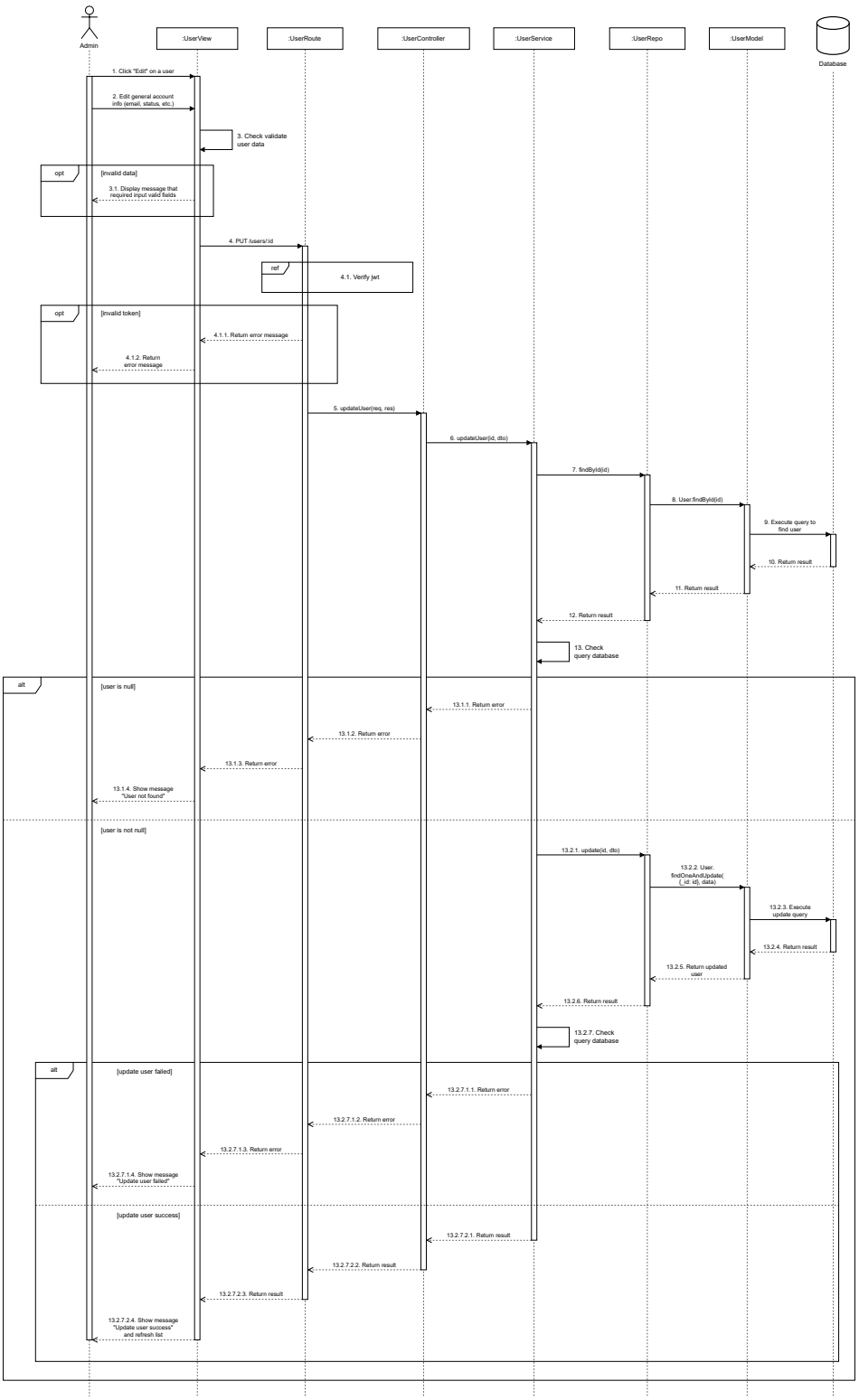

Figure 145: Sequence diagram of UC-28.04 Update User

3.51.5 UC-28.05 – Update Client Profile

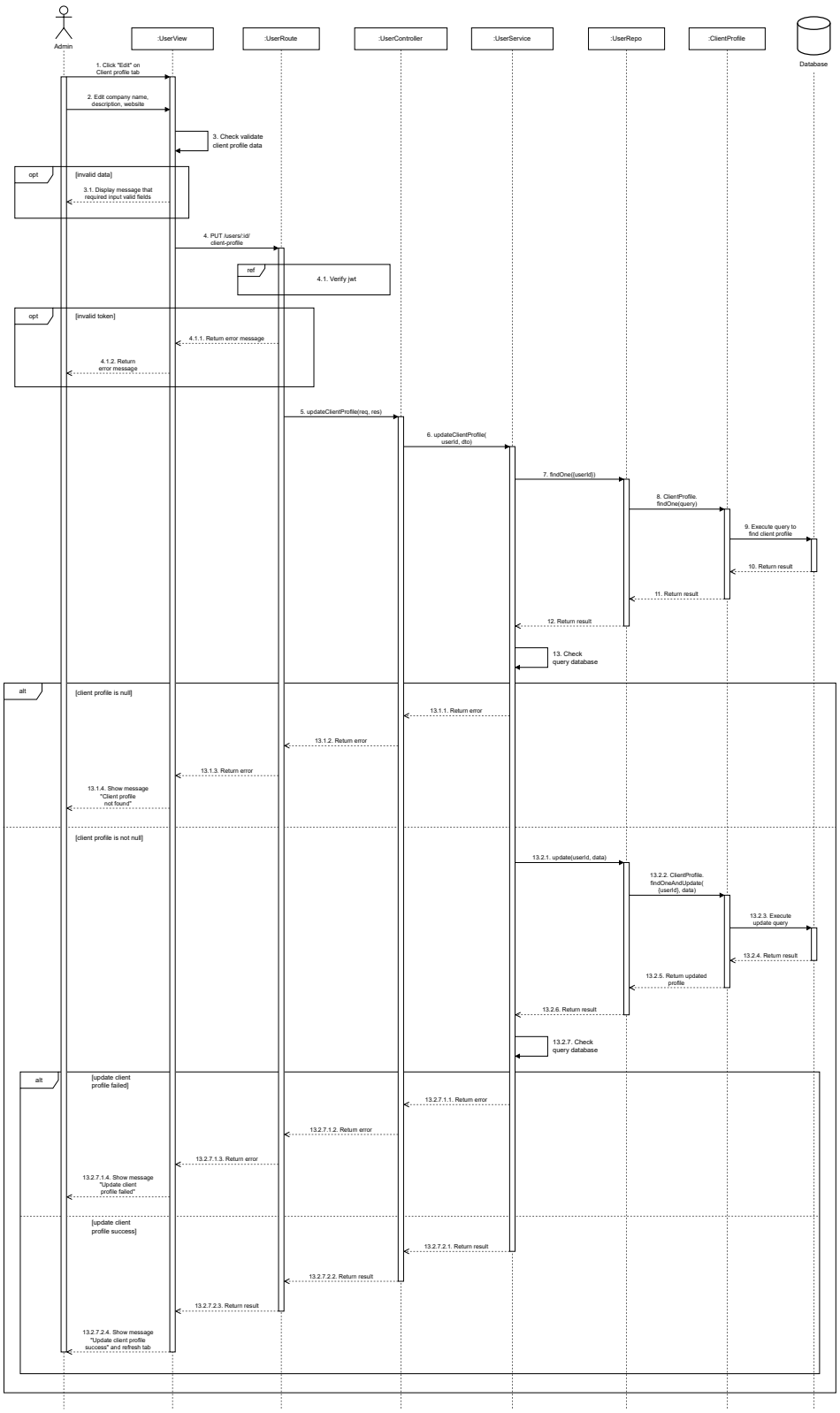

Figure 146: Sequence diagram of UC-28.05 Update Client Profile

3.51.6 UC-28.06 – Update Freelancer Basic Info

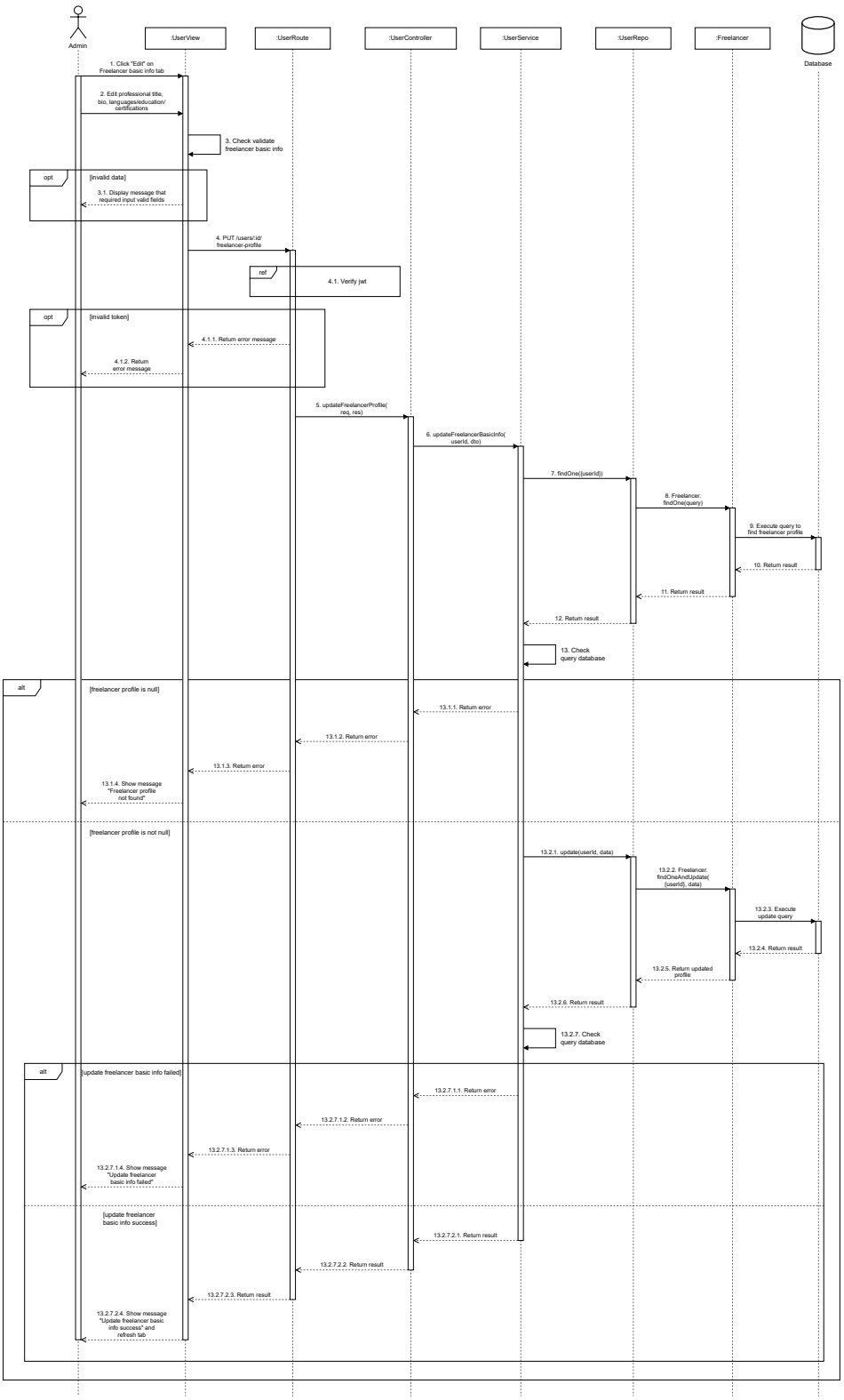

Figure 147: Sequence diagram of UC-28.06 Update Freelancer Basic Info

3.51.7 UC-28.07 – Update Freelancer Skills

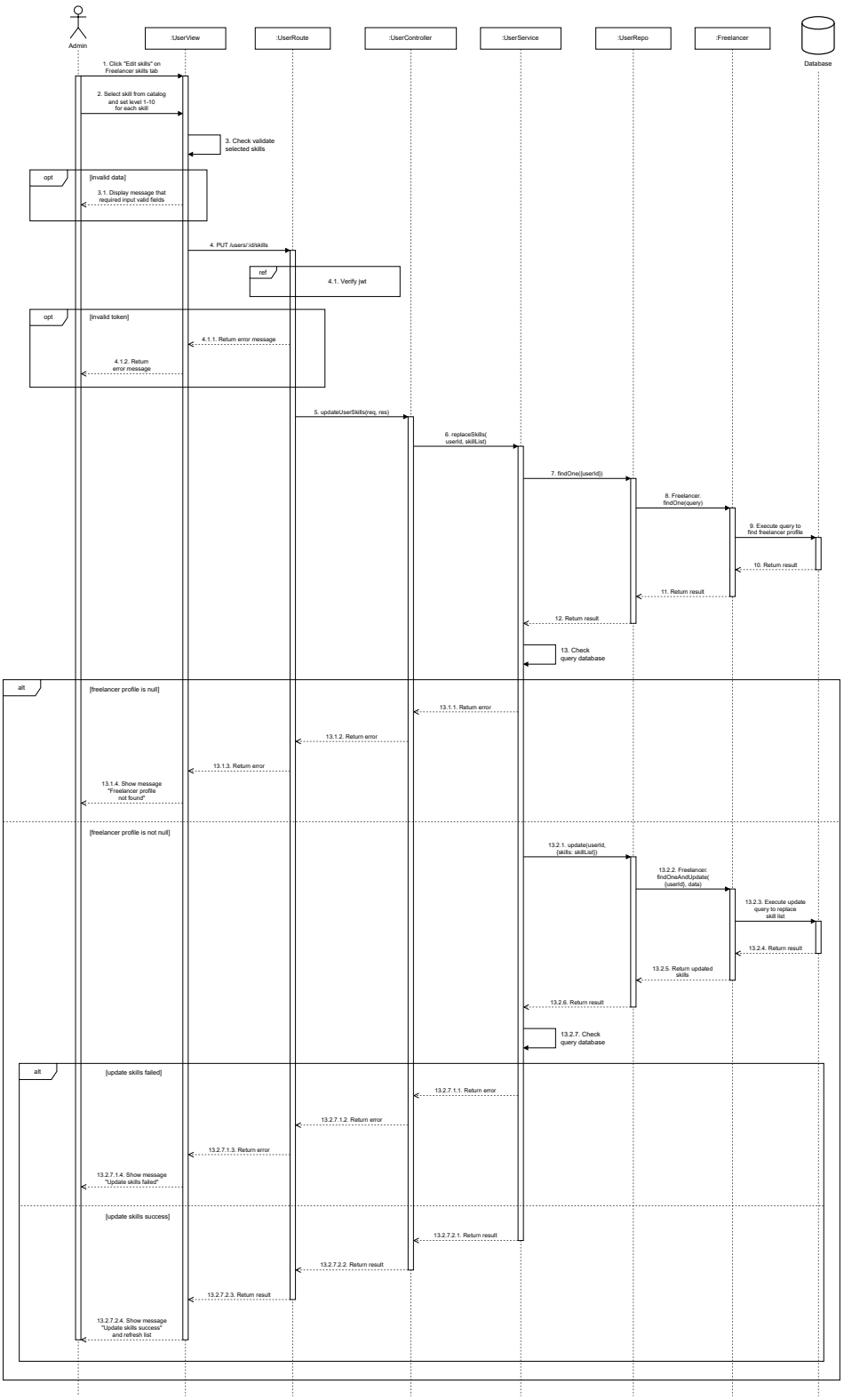

Figure 148: Sequence diagram of UC-28.07 Update Freelancer Skills

3.51.8 UC-28.08 – Update Freelancer Portfolio

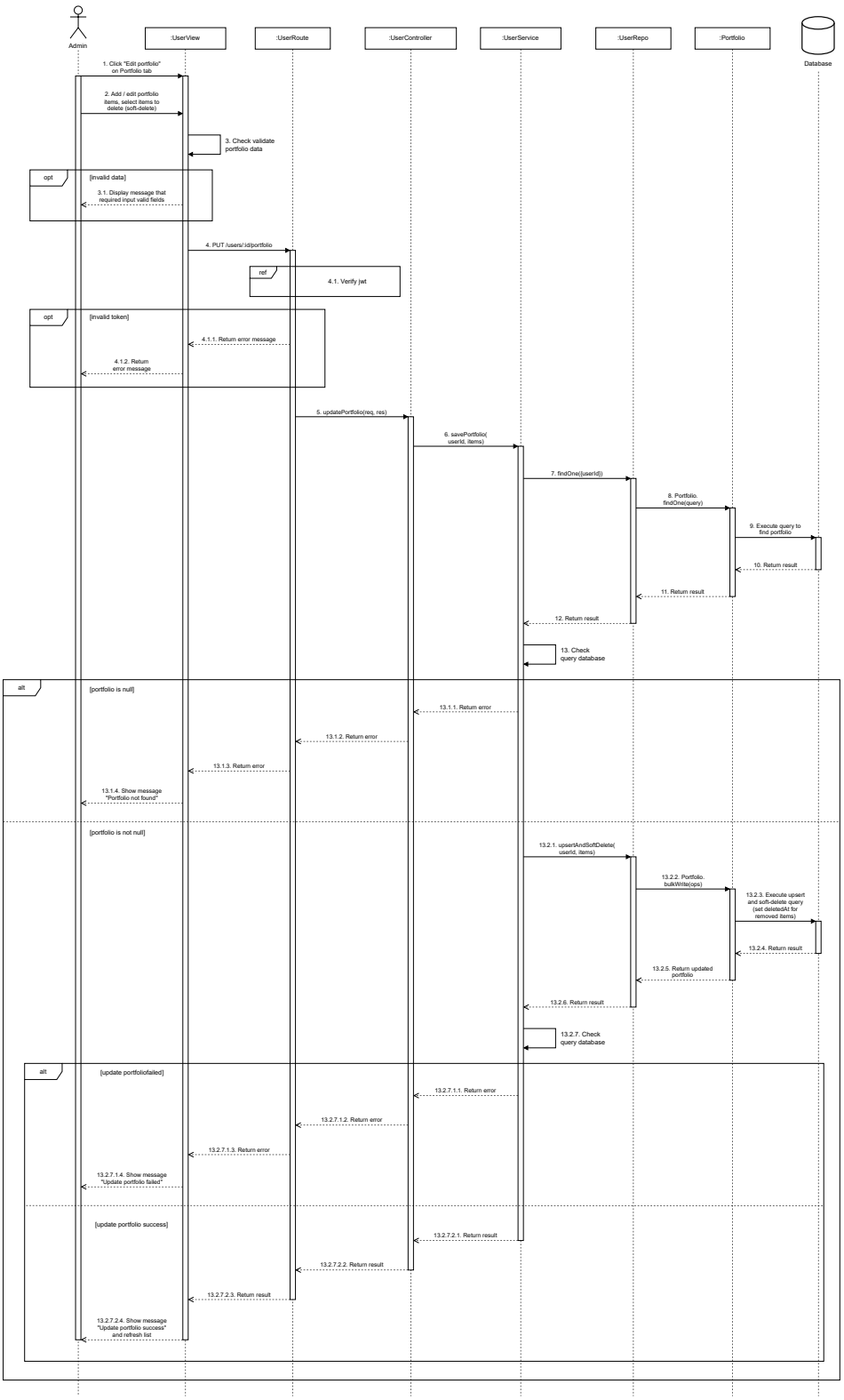

Figure 149: Sequence diagram of UC-28.08 Update Freelancer Portfolio

3.52 Review Management

Contract participants view reviews and write a review after contract completion. Review authors can update or soft-delete their own review.

a. Class Diagram

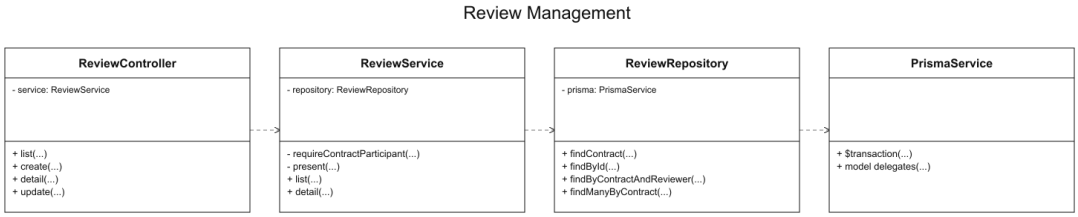

Figure 150: Class diagram of Review Management

b. Sequence Diagram(s)

3.52.1 UC-39.01 – View Review List

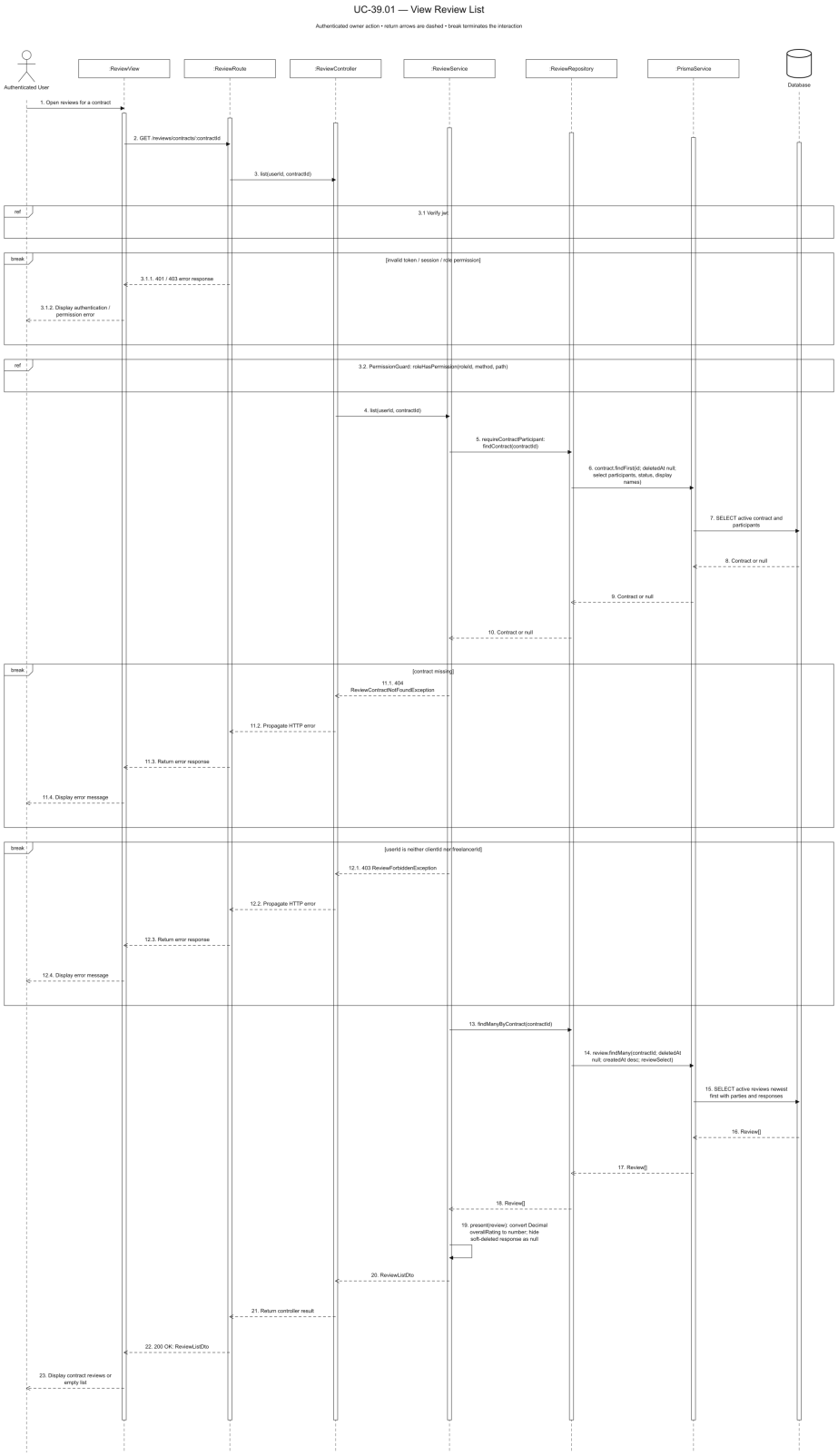

Figure 151: Sequence diagram of UC-39.01 View Review List

3.52.2 UC-39.02 – View Review Detail

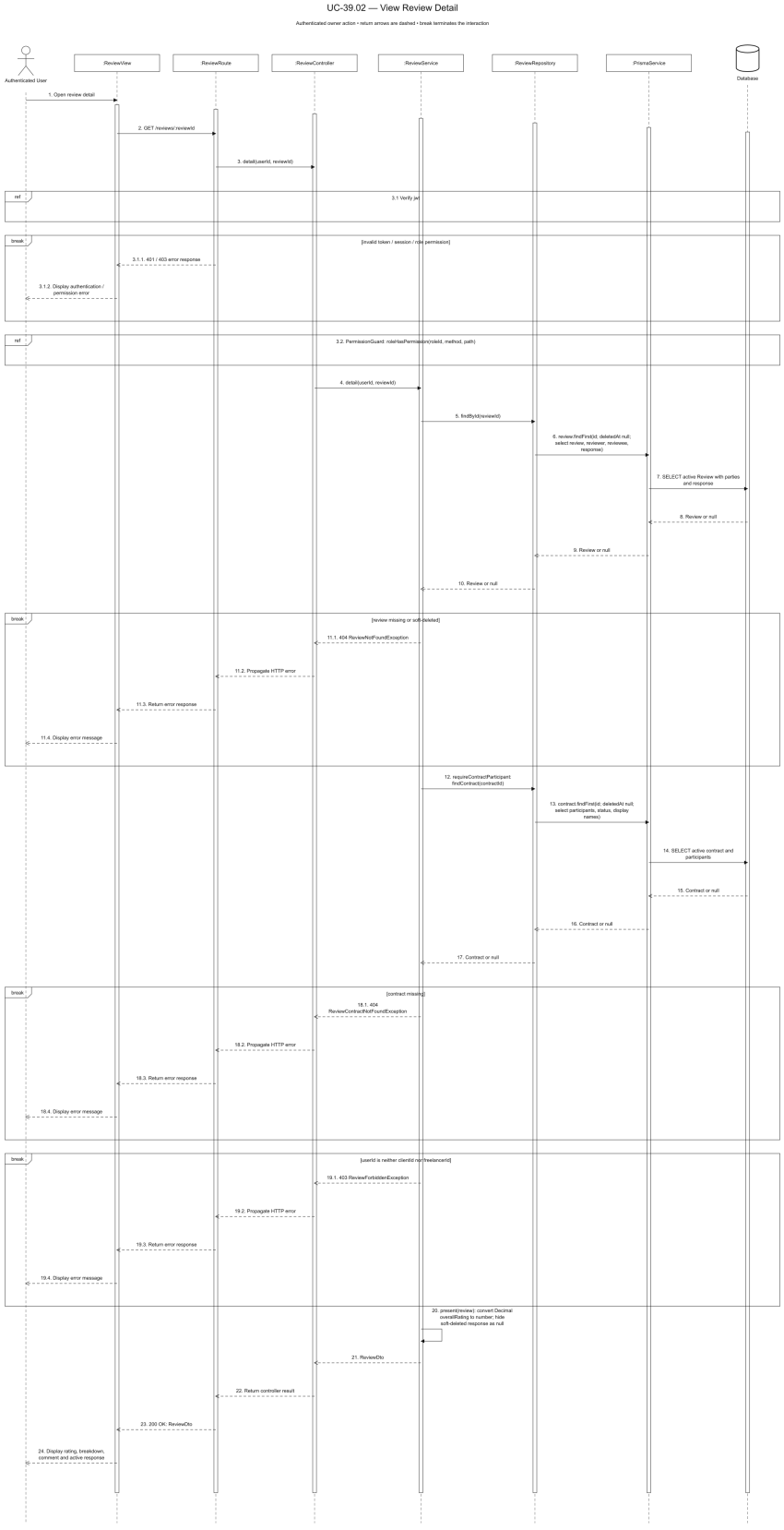

Figure 152: Sequence diagram of UC-39.02 View Review Detail

3.52.3 UC-39.03 – Write Review

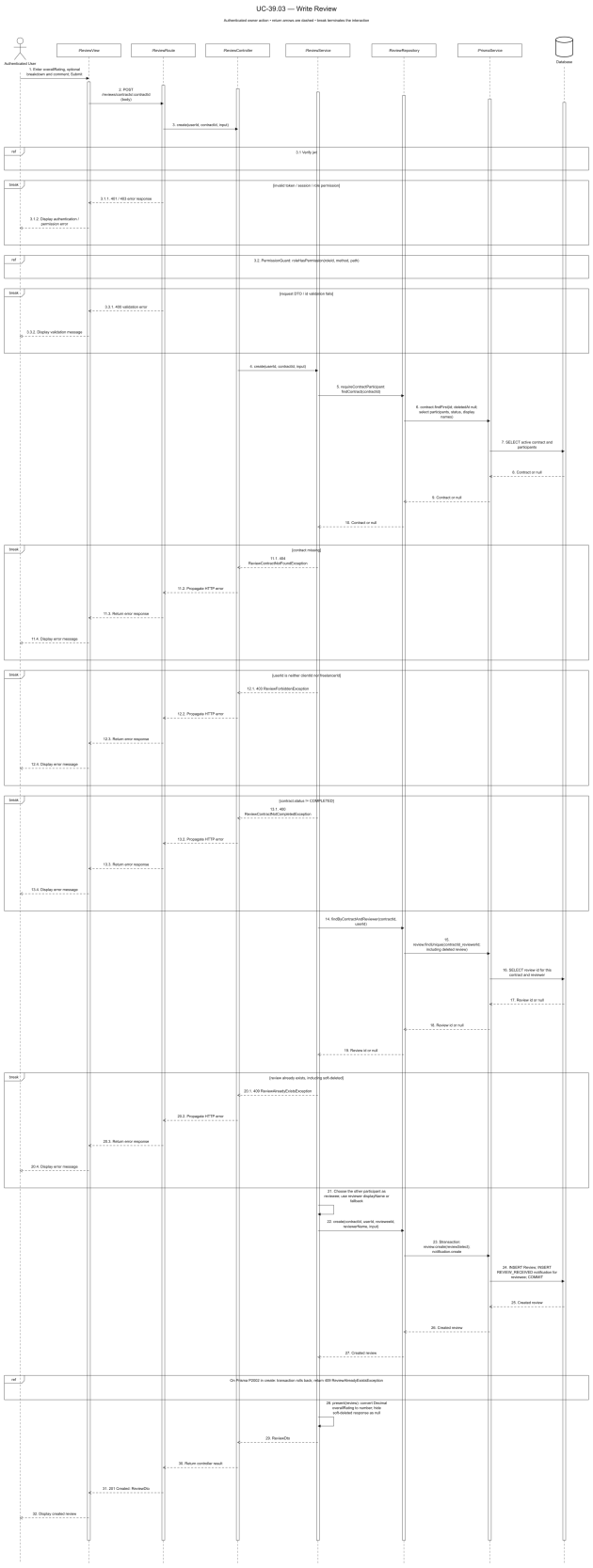

Figure 153: Sequence diagram of UC-39.03 Write Review

3.52.4 UC-39.04 – Update Review

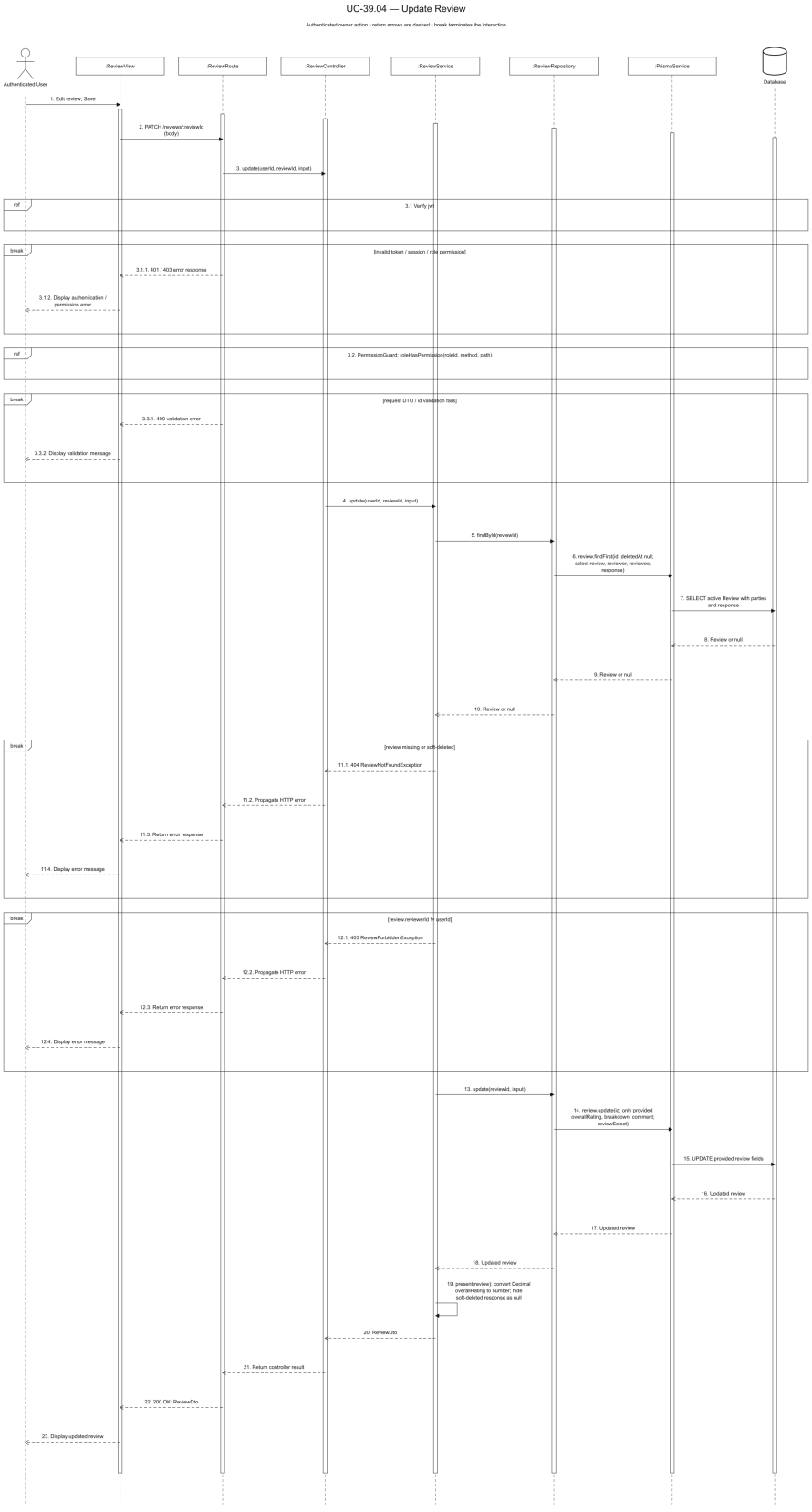

Figure 154: Sequence diagram of UC-39.04 Update Review

3.52.5 UC-39.05 – Delete Review

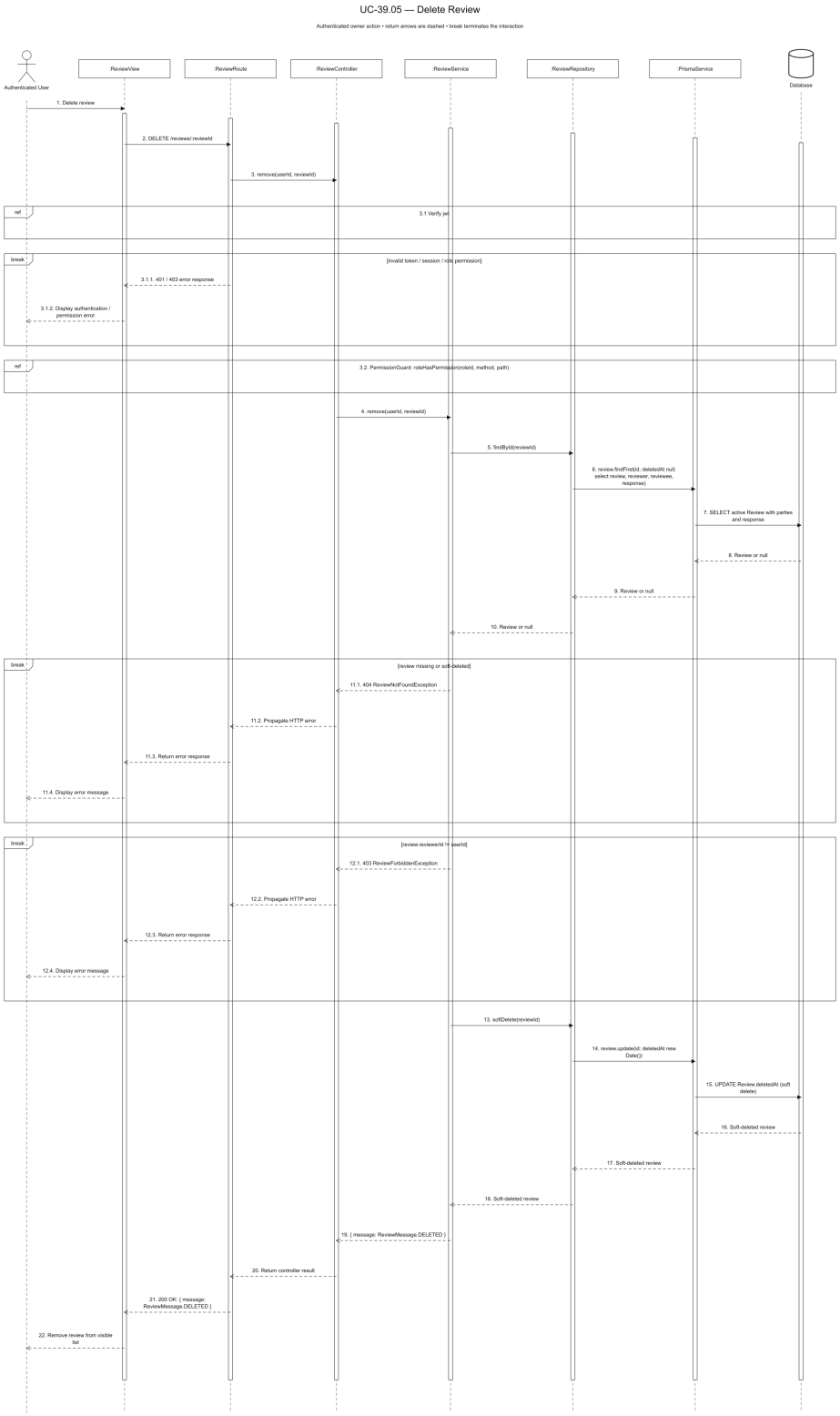

Figure 155: Sequence diagram of UC-39.05 Delete Review

3.53 Review Response Management

The reviewee can create or restore a response. Response authors can update or soft-delete their response.

a. Class Diagram

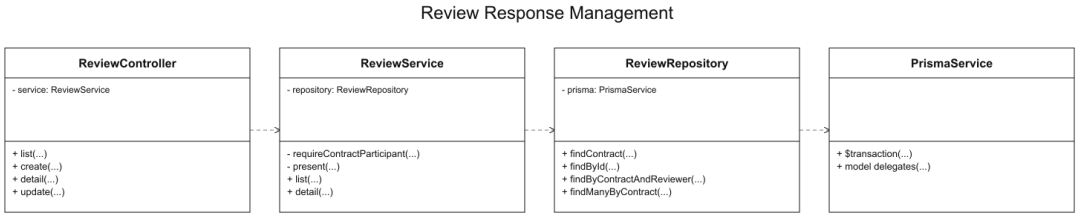

Figure 156: Class diagram of Review Response Management

b. Sequence Diagram(s)

3.53.1 UC-40.01 – Respond to Review

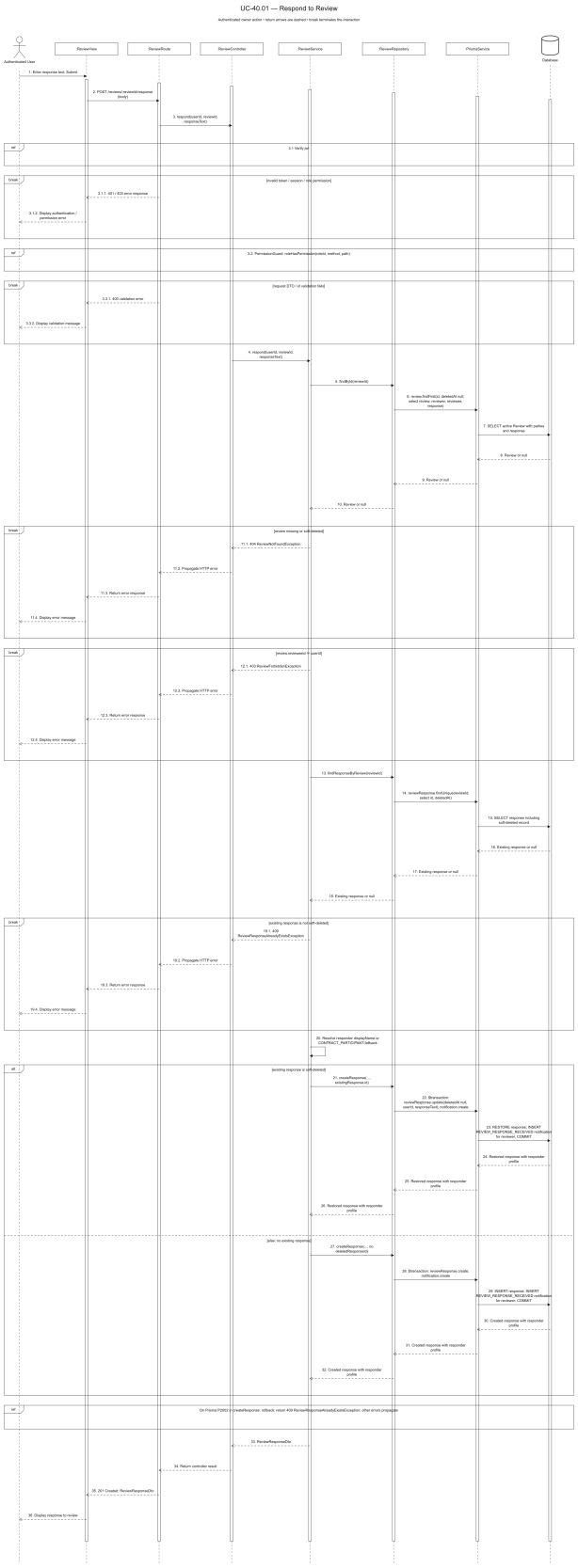

Figure 157: Sequence diagram of UC-40.01 Respond to Review

3.53.2 UC-40.02 – Update Review Response

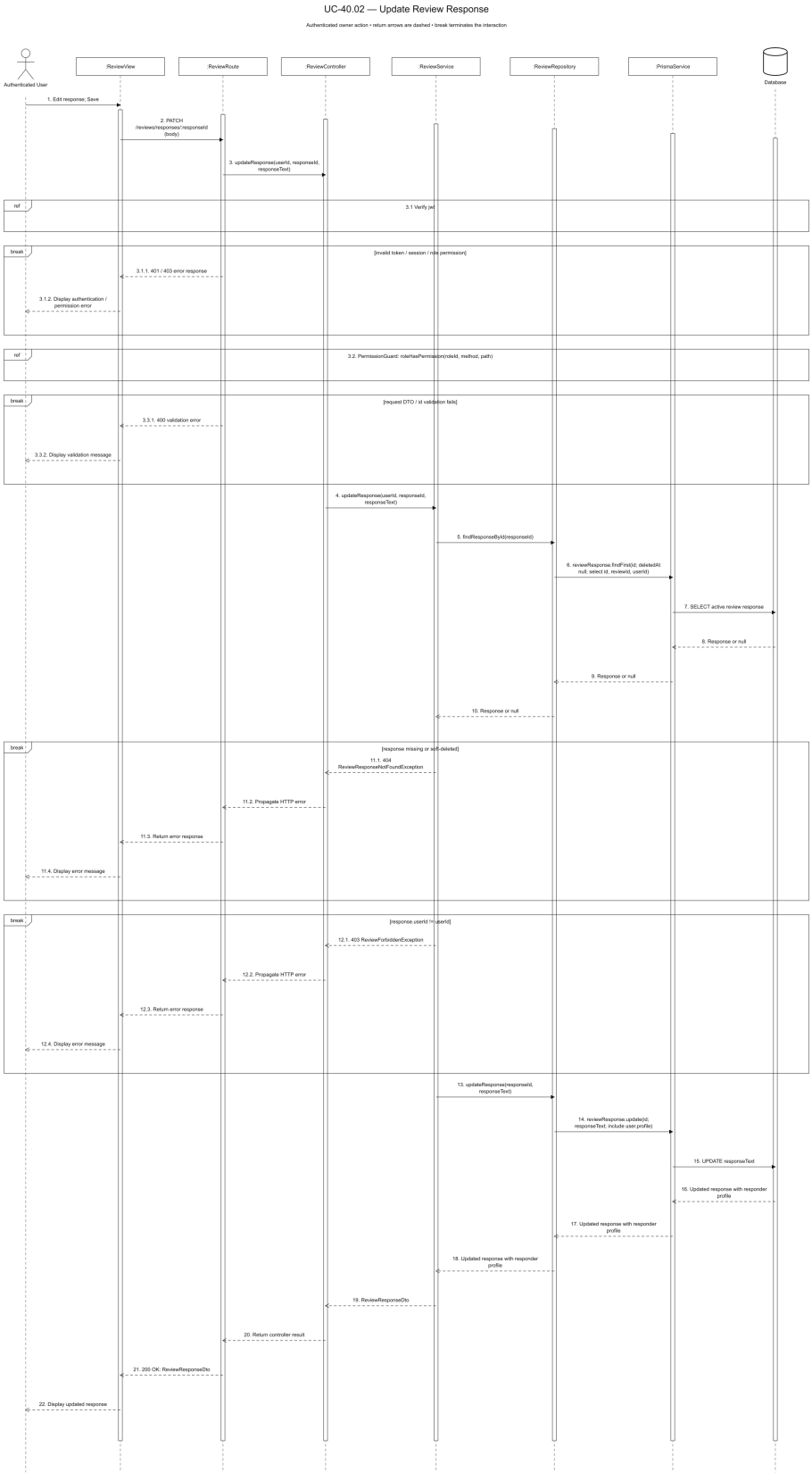

Figure 158: Sequence diagram of UC-40.02 Update Review Response

3.53.3 UC-40.03 – Delete Review Response

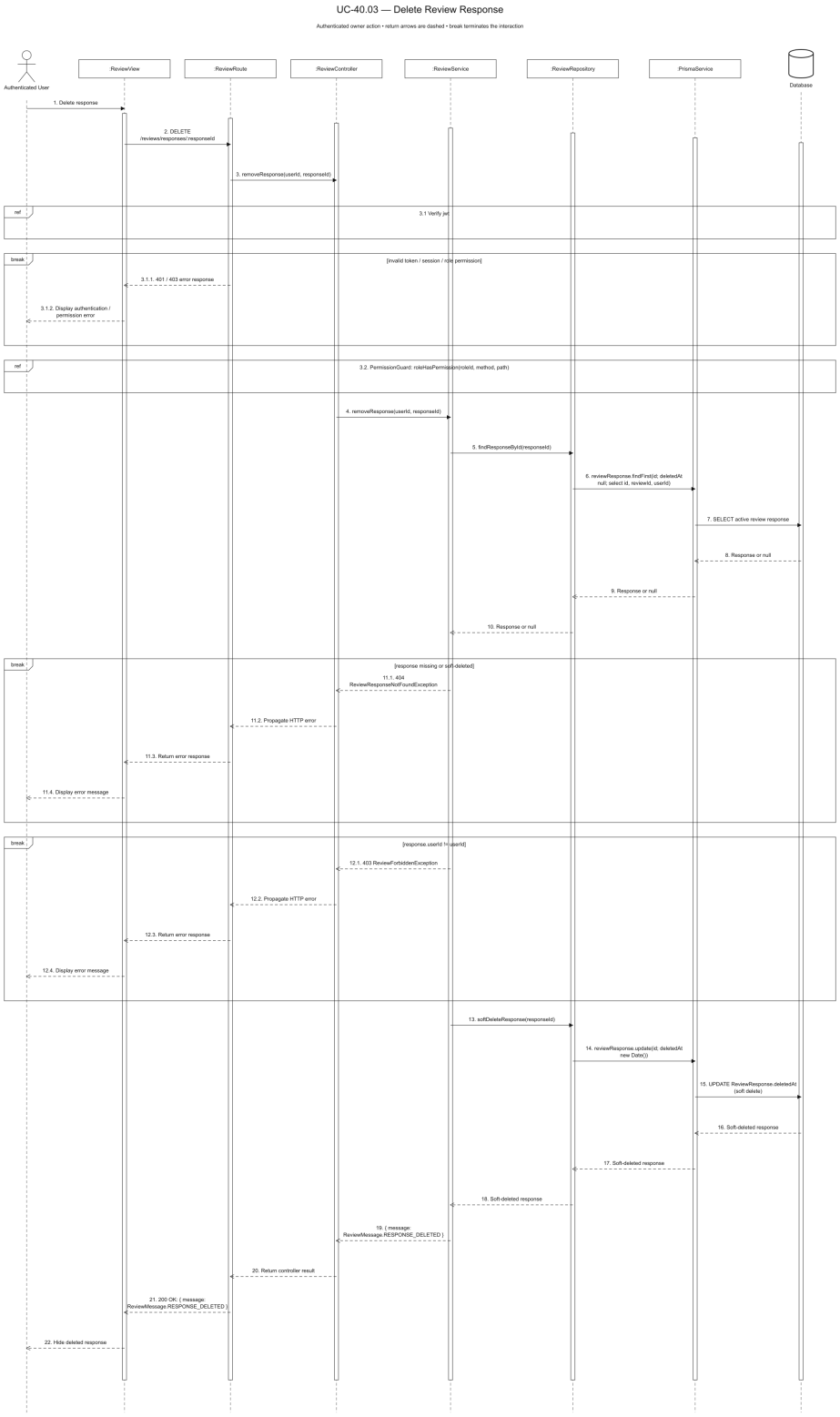

Figure 159: Sequence diagram of UC-40.03 Delete Review Response

3.54 General Account Profile Management

Authenticated users view and update their general profile, change their password and upload an avatar.

a. Class Diagram

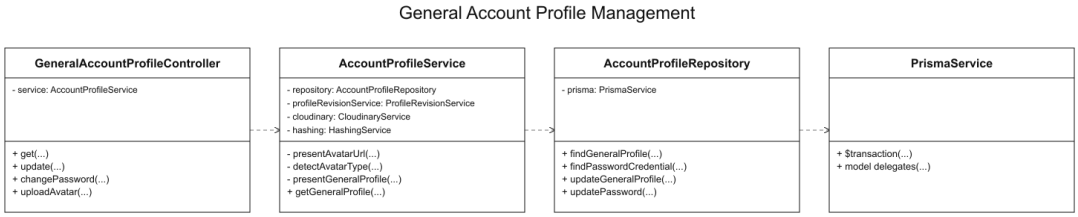

Figure 160: Class diagram of General Account Profile Management

b. Sequence Diagram(s)

3.54.1 UC-41.01 – View Profile

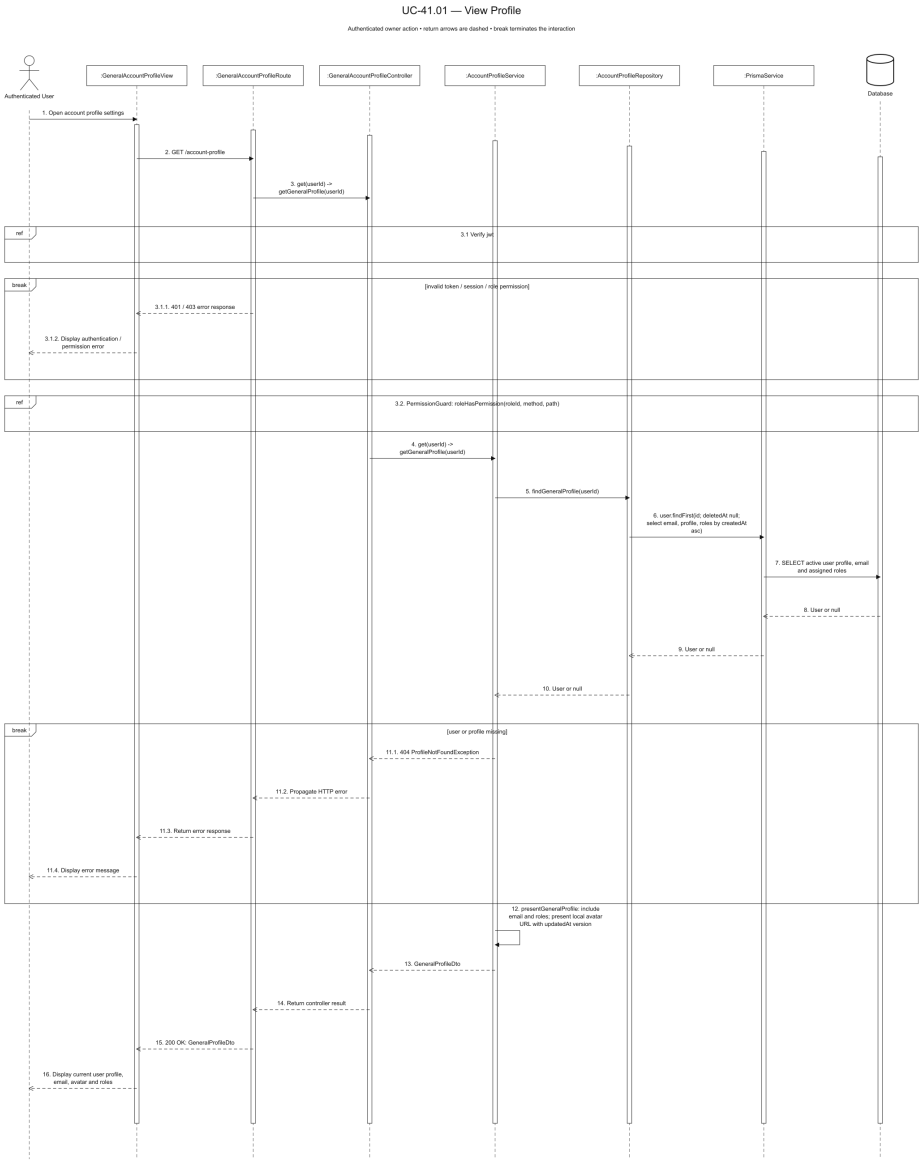

Figure 161: Sequence diagram of UC-41.01 View Profile

3.54.2 UC-41.02 – Update Profile

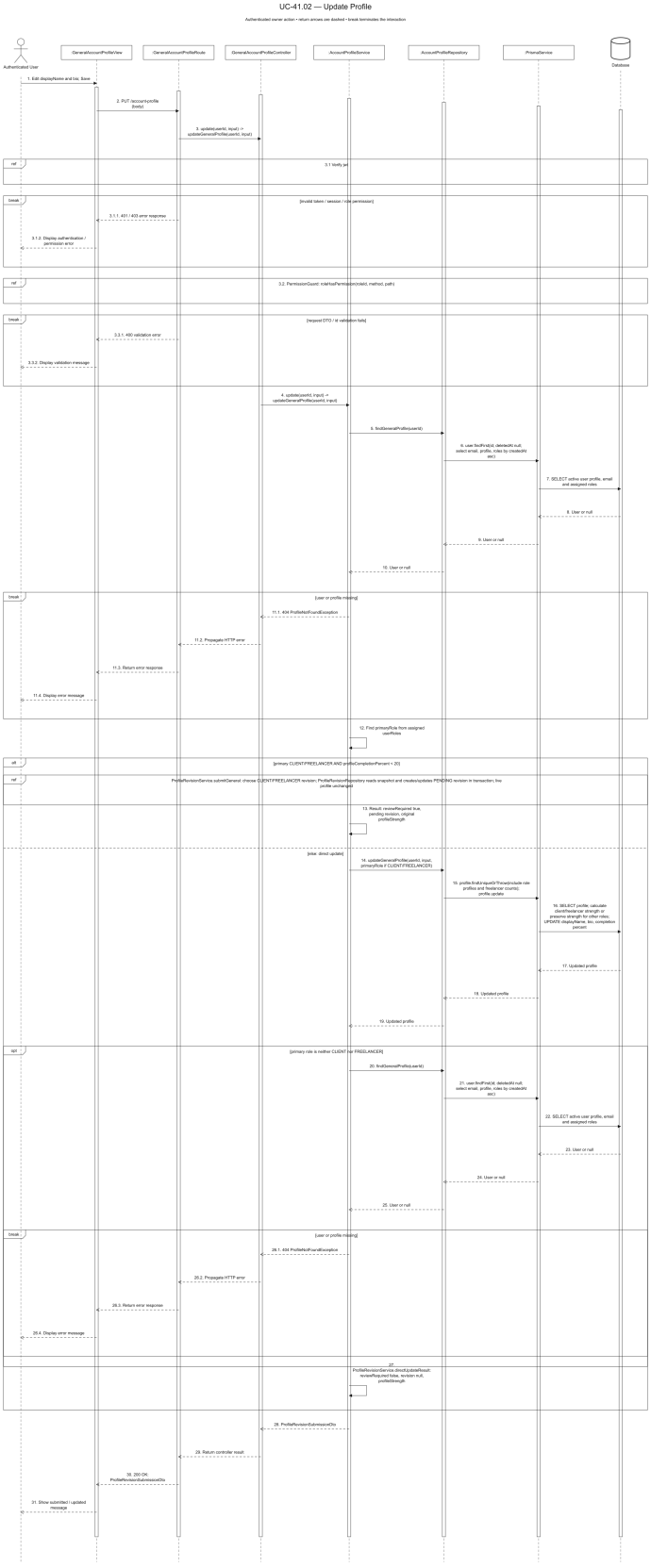

Figure 162: Sequence diagram of UC-41.02 Update Profile

3.54.3 UC-41.03 – Change Password

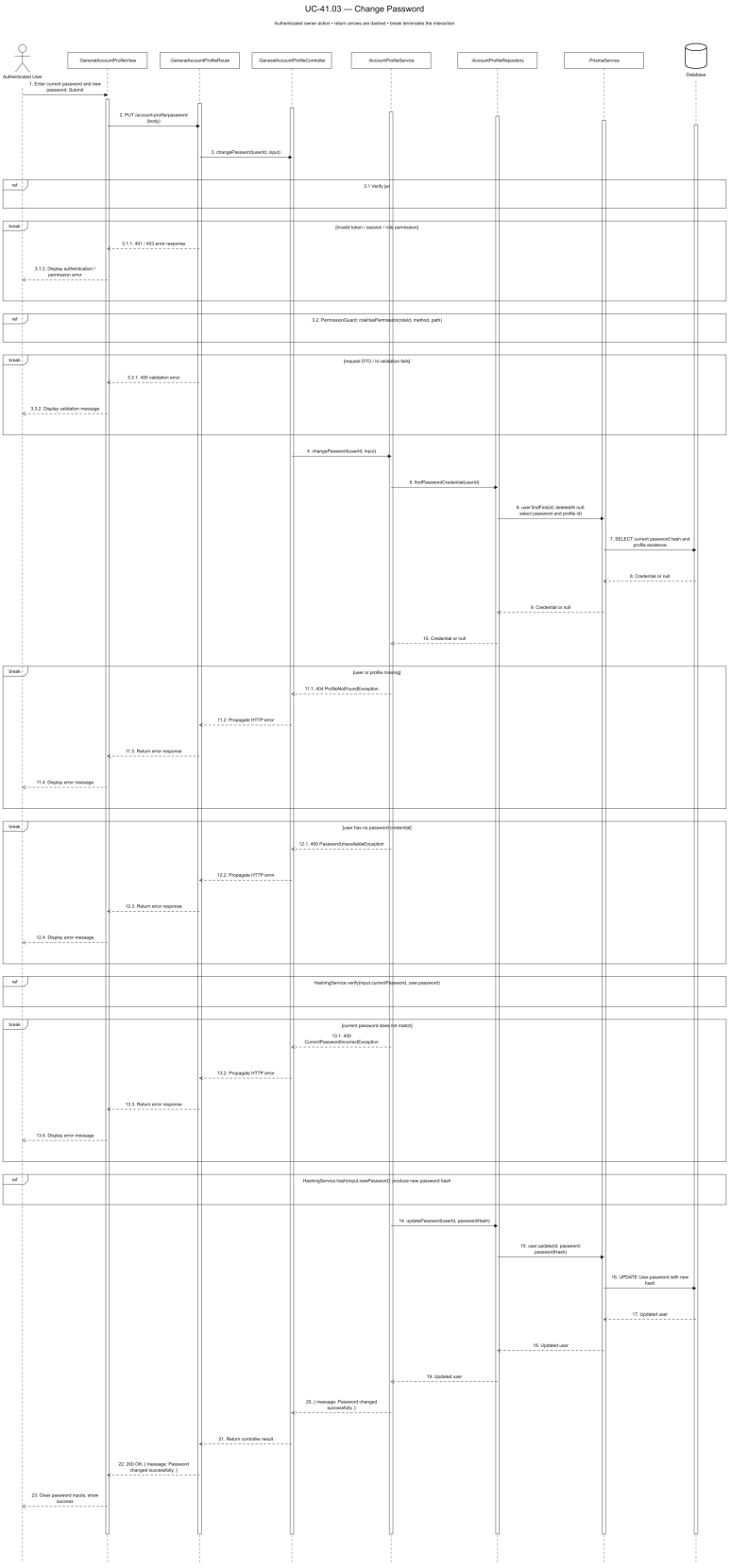

Figure 163: Sequence diagram of UC-41.03 Change Password

3.54.4 UC-41.04 – Upload Avatar

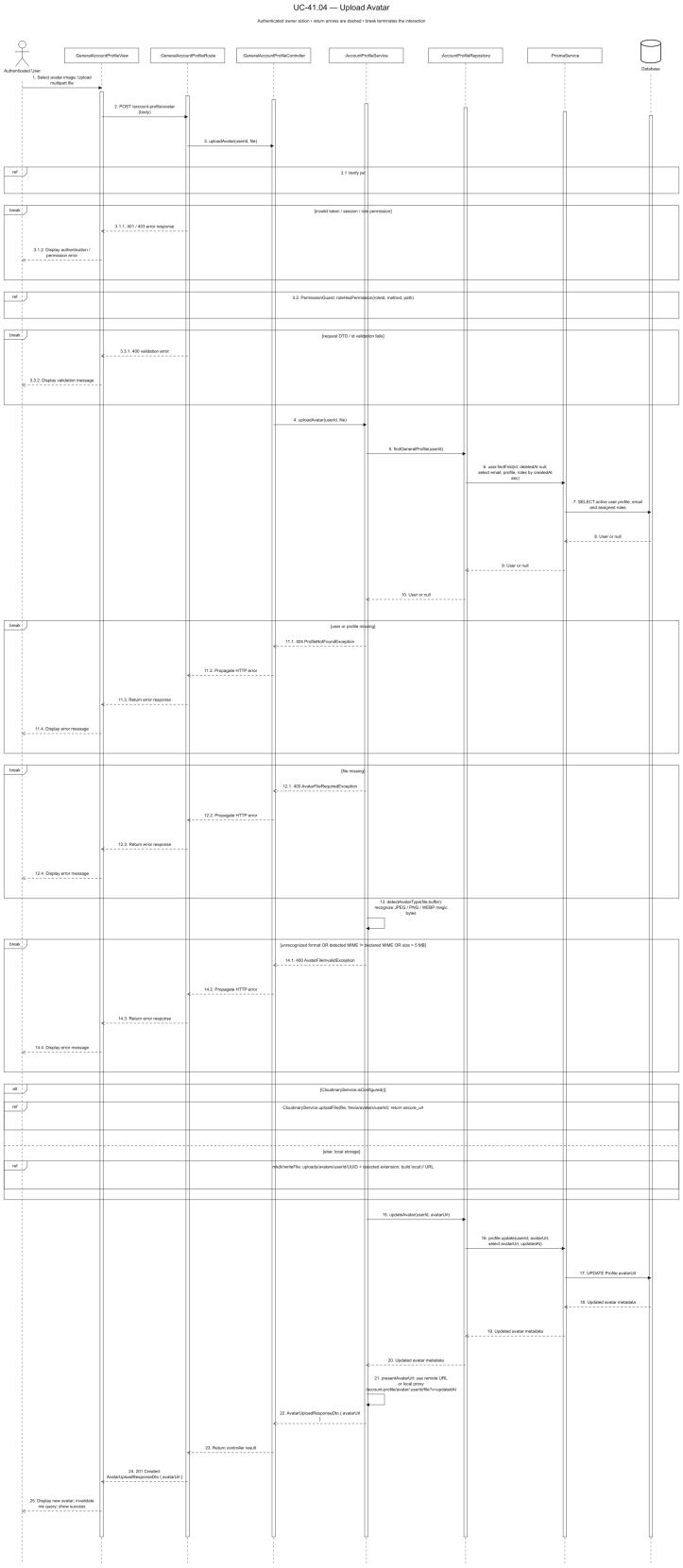

Figure 164: Sequence diagram of UC-41.04 Upload Avatar

3.55 Banner Management

Banner Management maintains the advertisement banners shown in configured page positions. The module supports image upload, lifecycle dates, activation state, and hard deletion.

a. Class Diagram

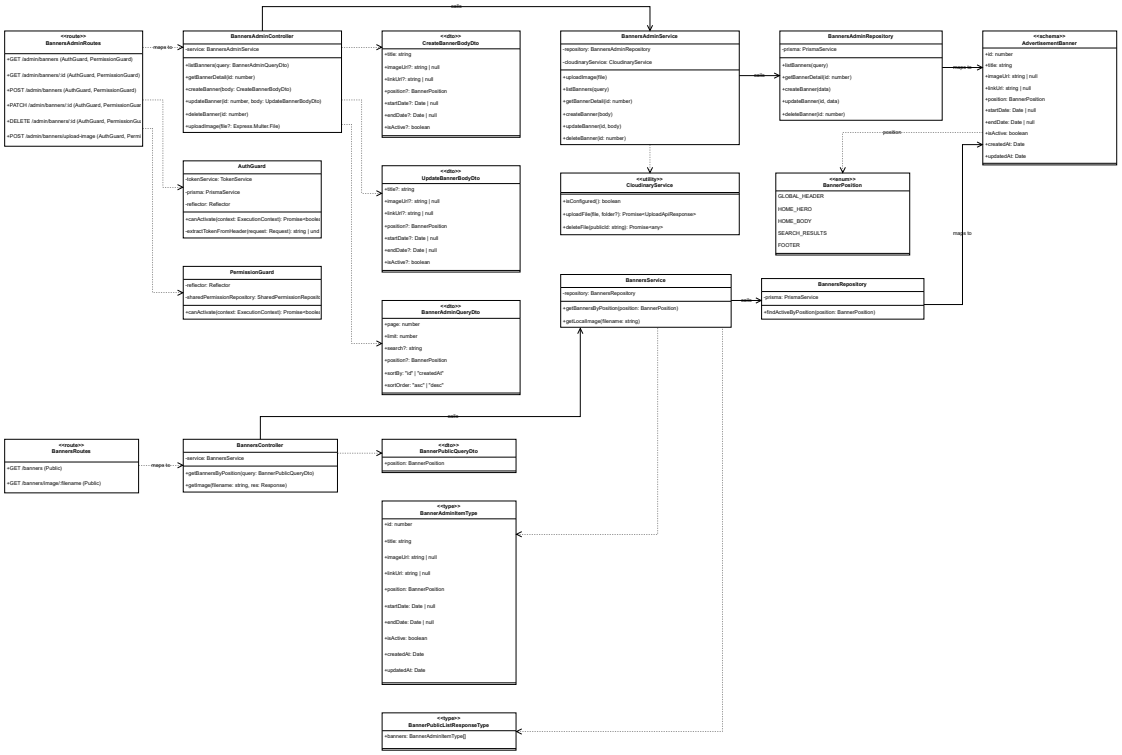

Figure 165: Class diagram of Banner Management

b. Sequence Diagram(s)

3.55.1 UC-37.01 – View Banner List

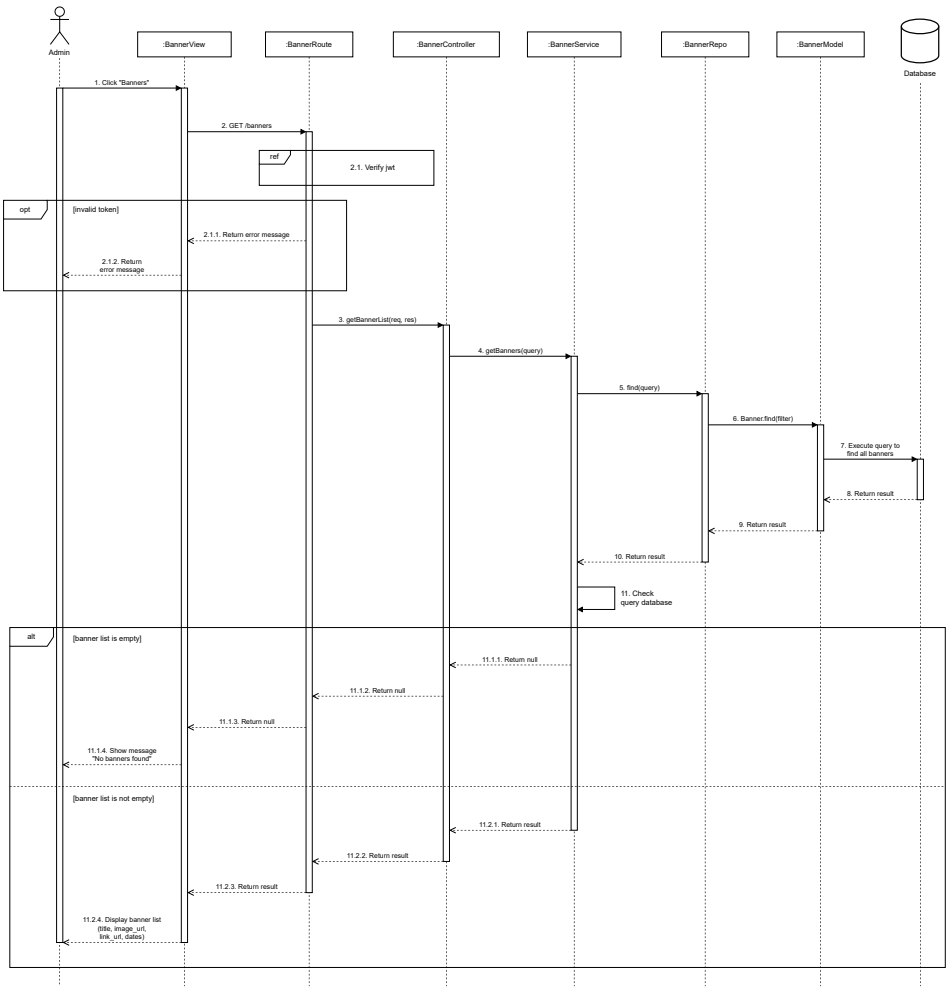

Figure 166: Sequence diagram of UC-37.01 View Banner List

3.55.2 UC-37.02 – View Banner Detail

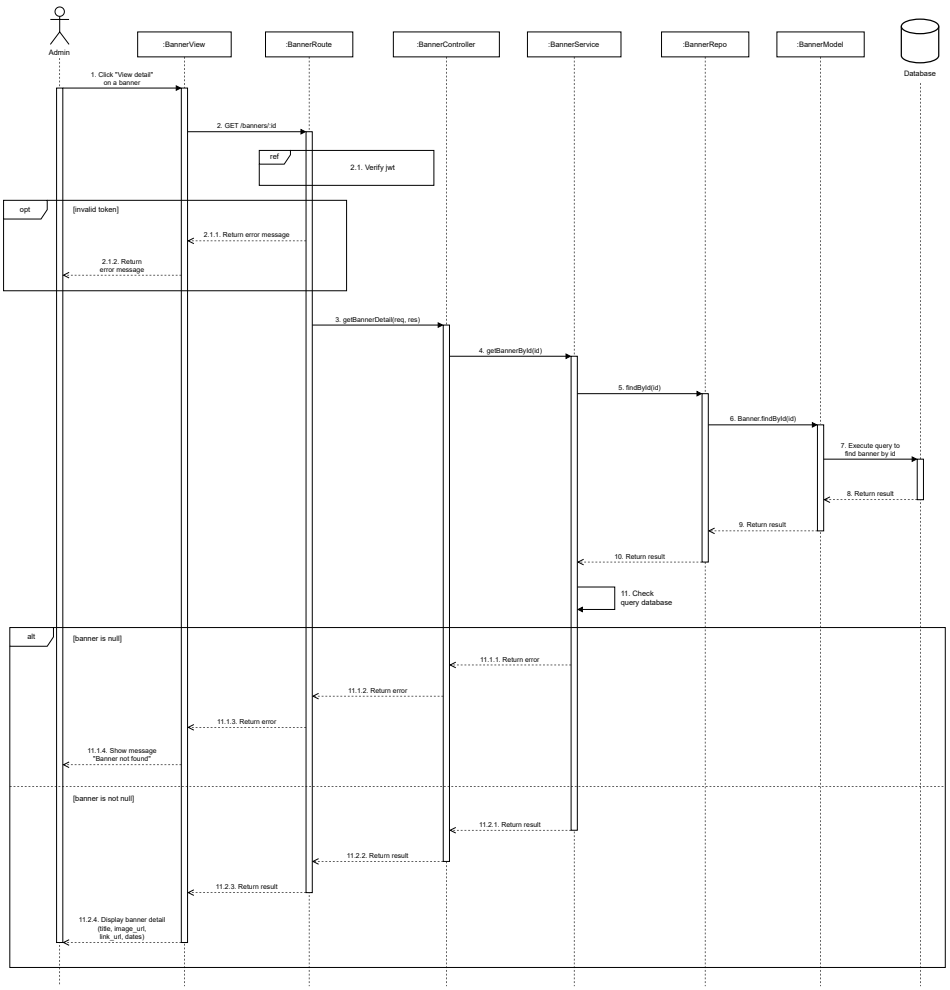

Figure 167: Sequence diagram of UC-37.02 View Banner Detail

3.55.3 UC-37.03 – Create Banner

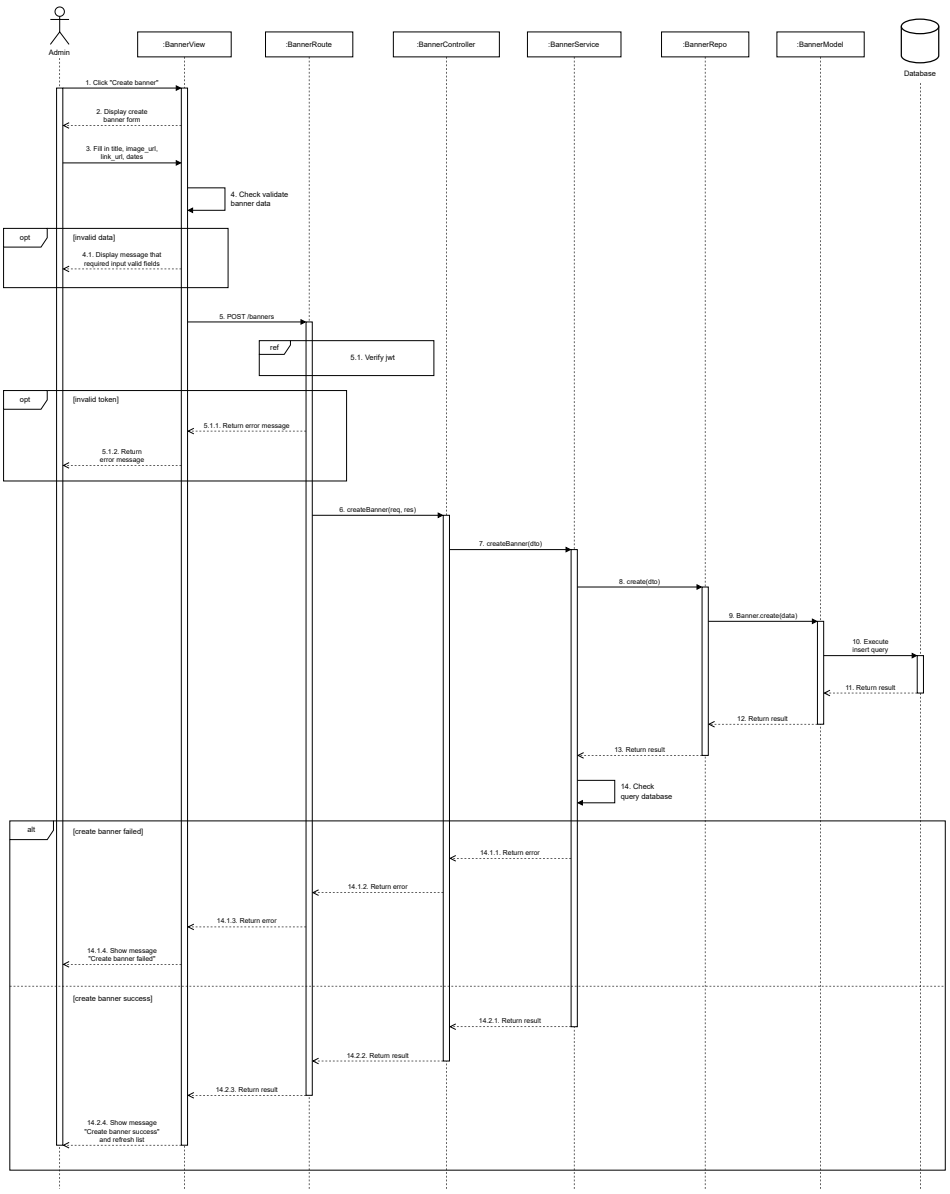

Figure 168: Sequence diagram of UC-37.03 Create Banner

3.55.4 UC-37.04 – Update Banner

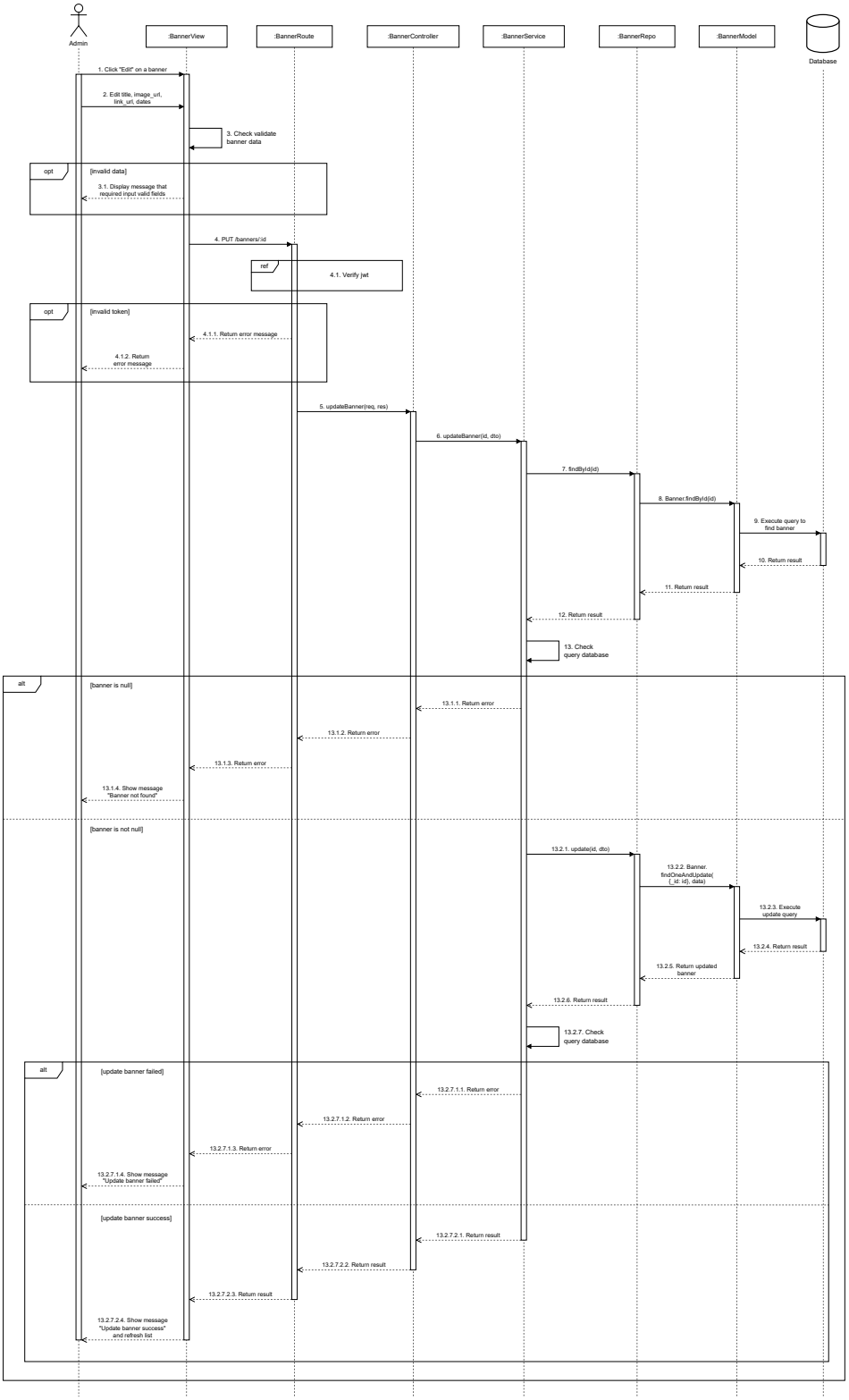

Figure 169: Sequence diagram of UC-37.04 Update Banner

3.55.5 UC-37.05 – Delete Banner

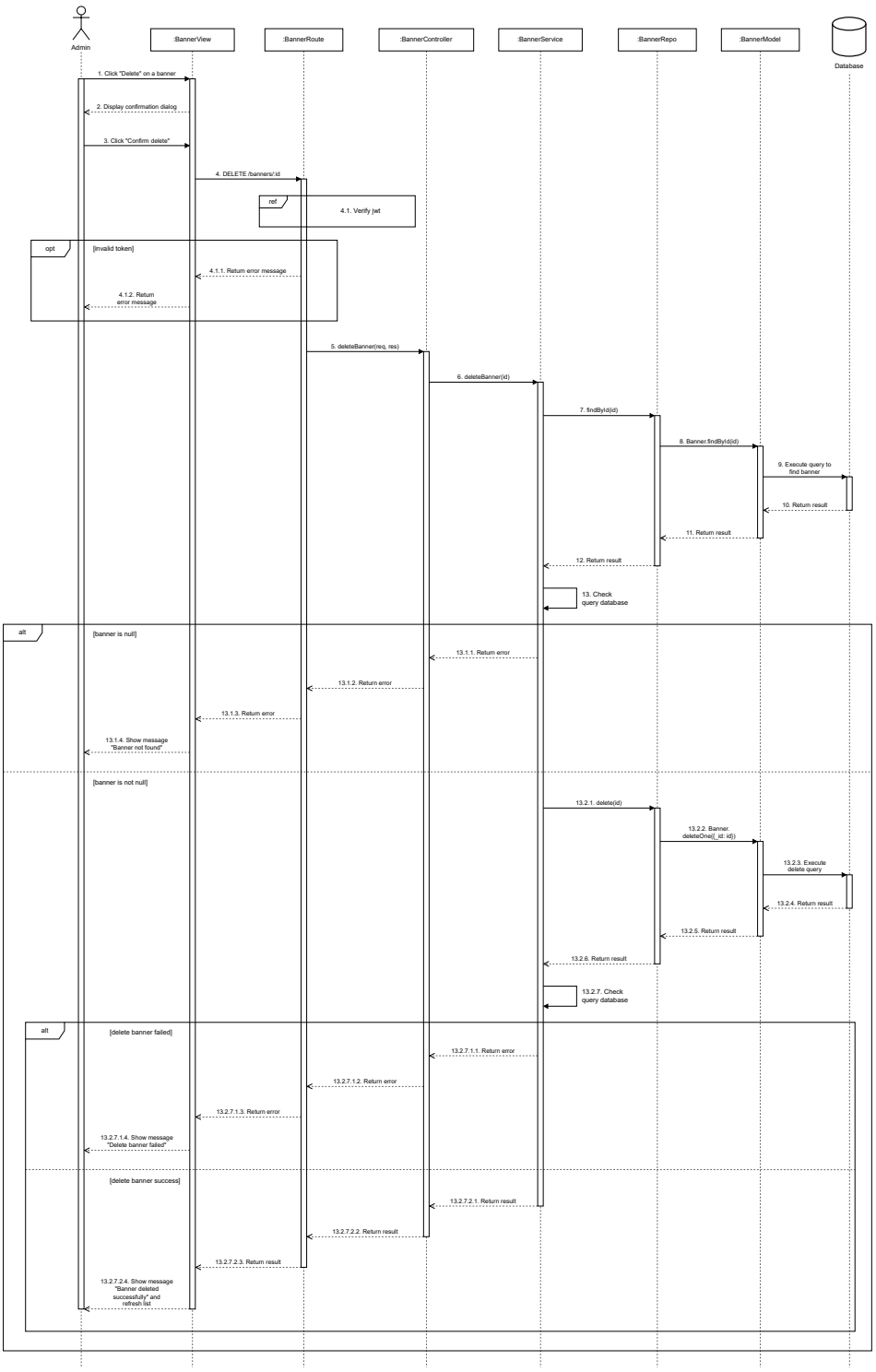

Figure 170: Sequence diagram of UC-37.05 Delete Banner

3.56 Skill Management

Skill Management owns the master skill catalogue used by freelancer profiles and job postings. Skills are soft-deleted so that existing references remain valid, and a deleted skill can be restored.

a. Class Diagram

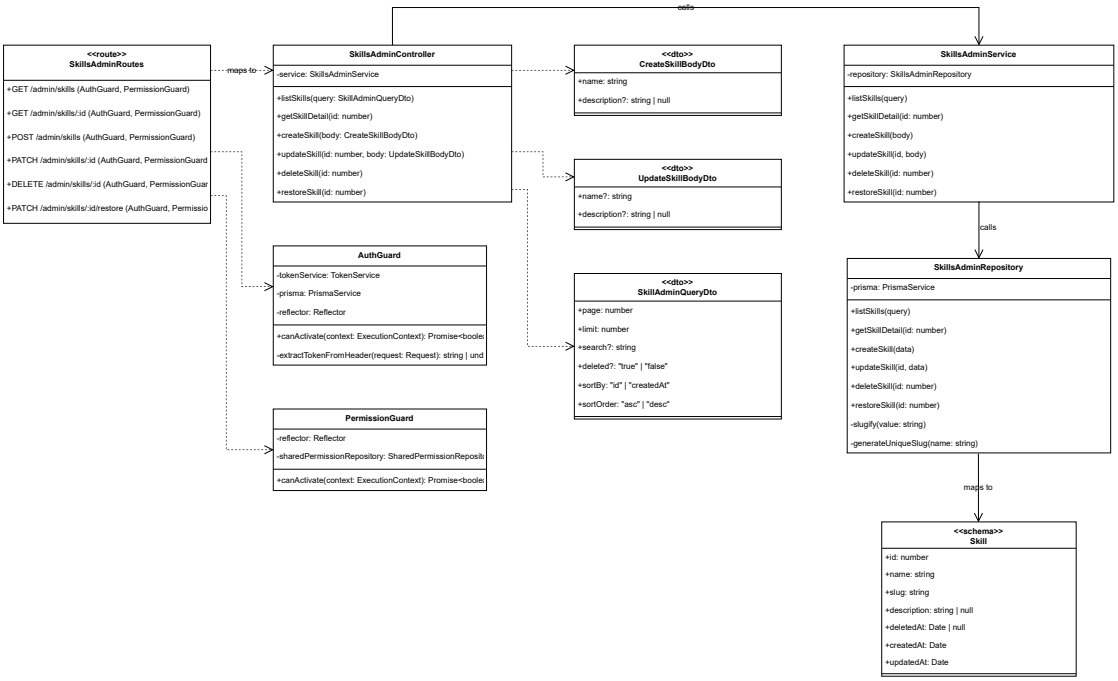

Figure 171: Class diagram of Skill Management

b. Sequence Diagram(s)

3.56.1 UC-38.01 – View Skill List

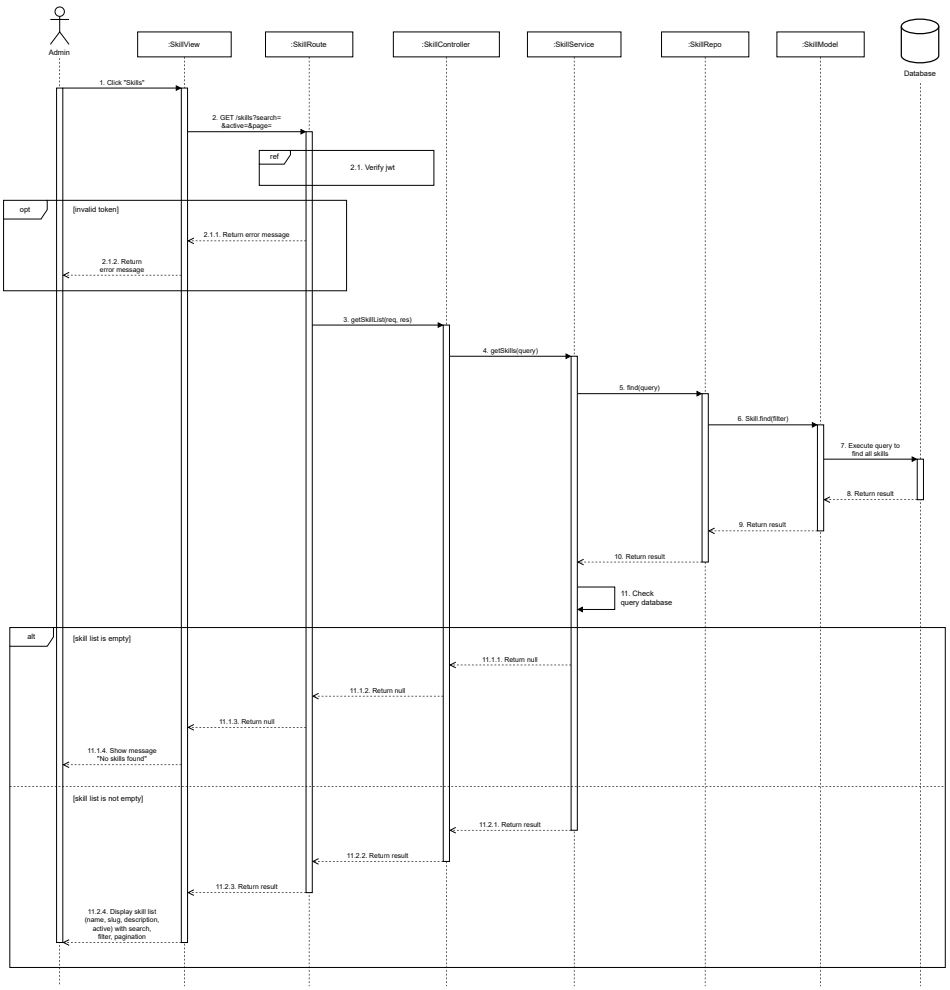

Figure 172: Sequence diagram of UC-38.01 View Skill List

3.56.2 UC-38.02 – View Skill Detail

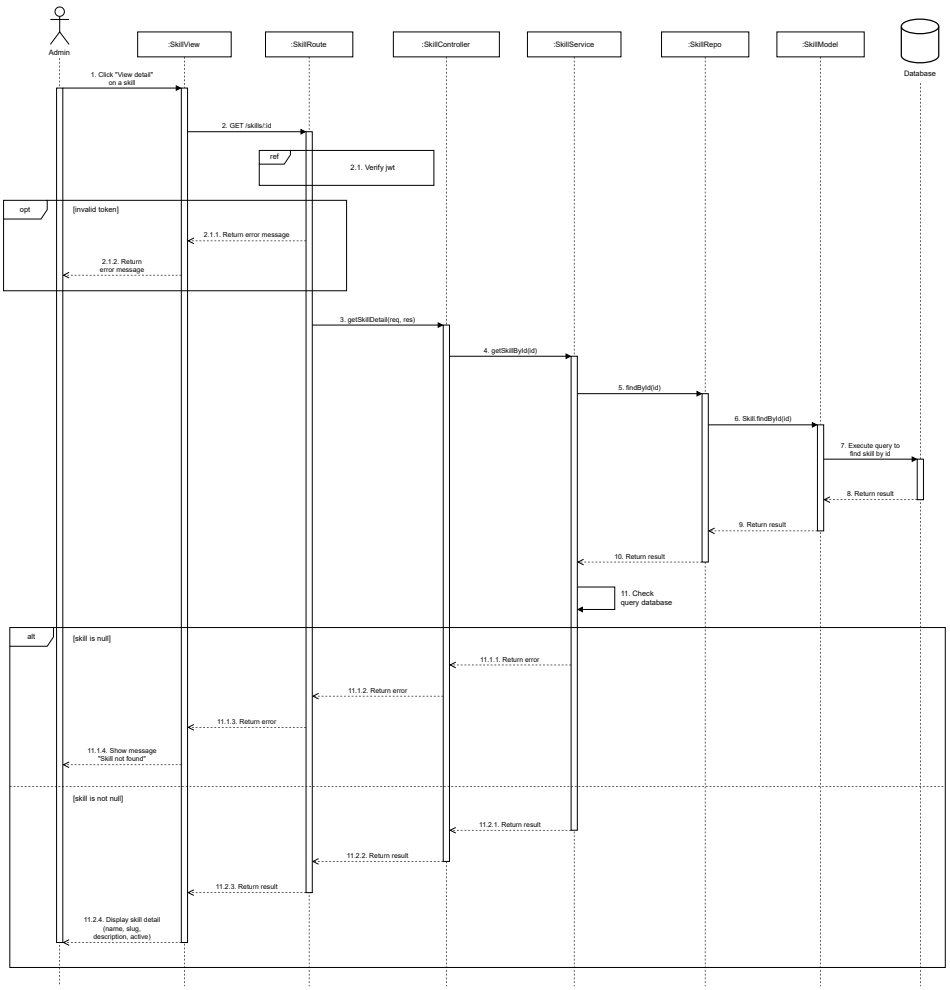

Figure 173: Sequence diagram of UC-38.02 View Skill Detail

3.56.3 UC-38.03 – Create Skill

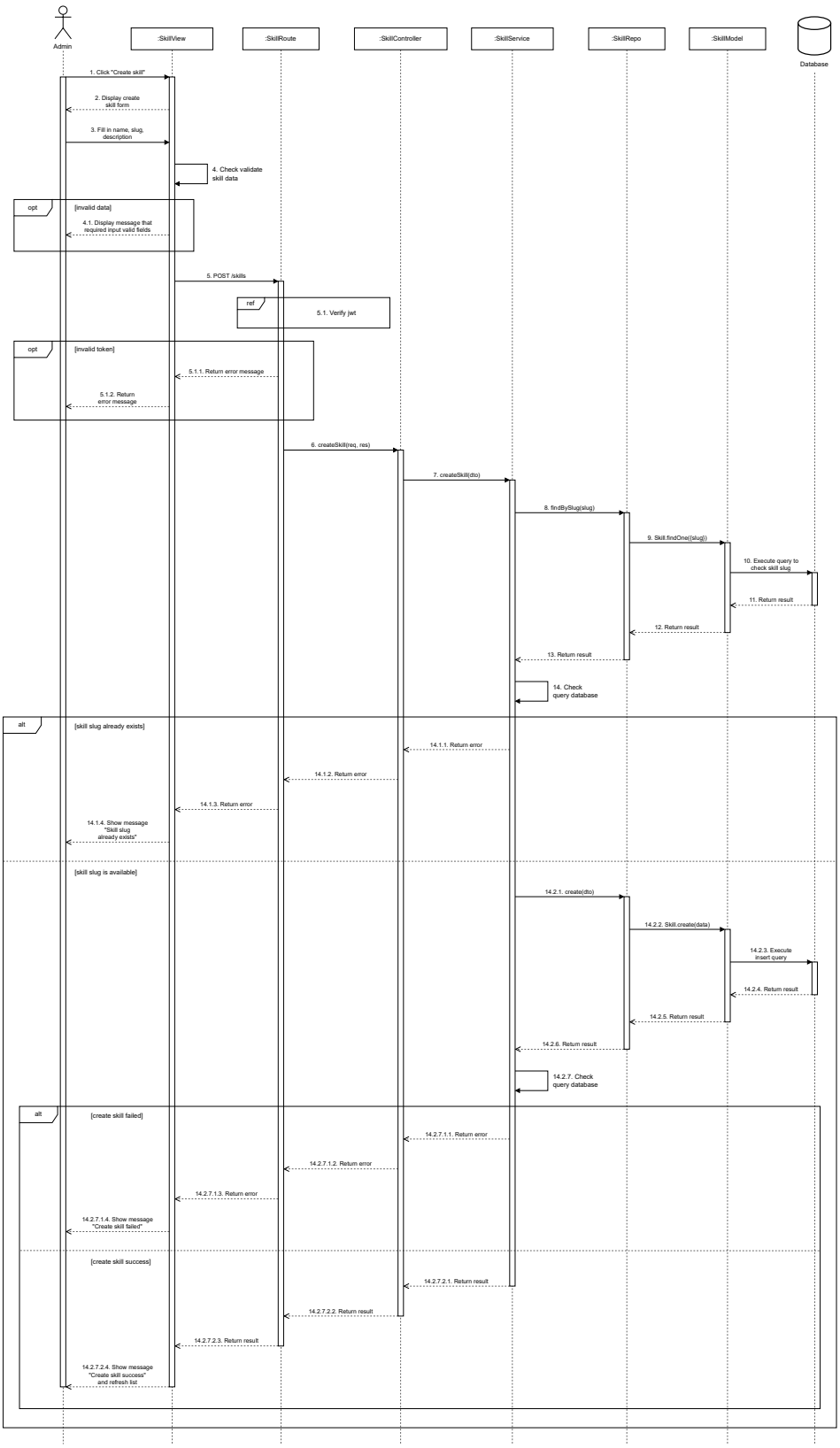

Figure 174: Sequence diagram of UC-38.03 Create Skill

3.56.4 UC-38.04 – Update Skill

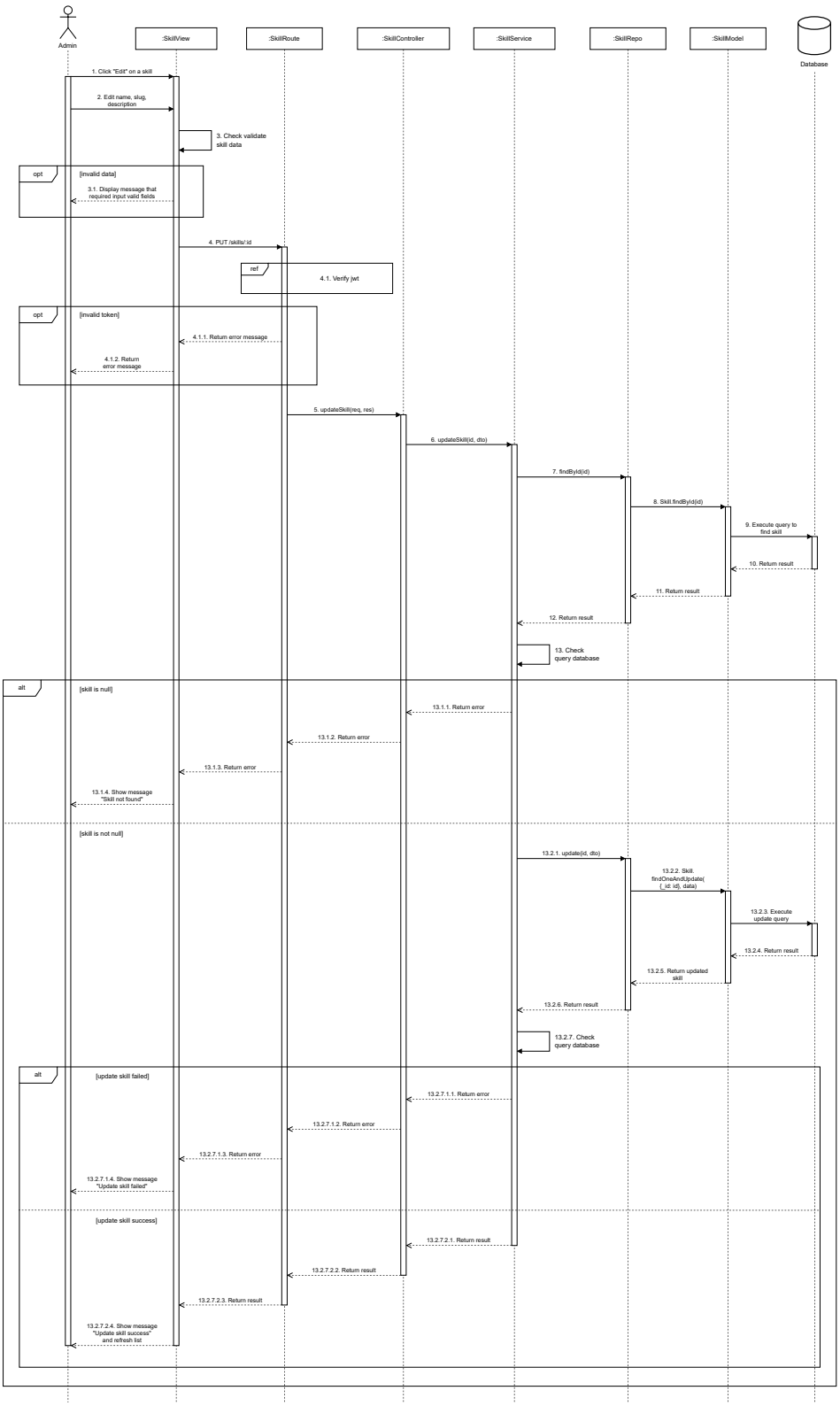

Figure 175: Sequence diagram of UC-38.04 Update Skill

3.56.5 UC-38.05 – Delete Skill

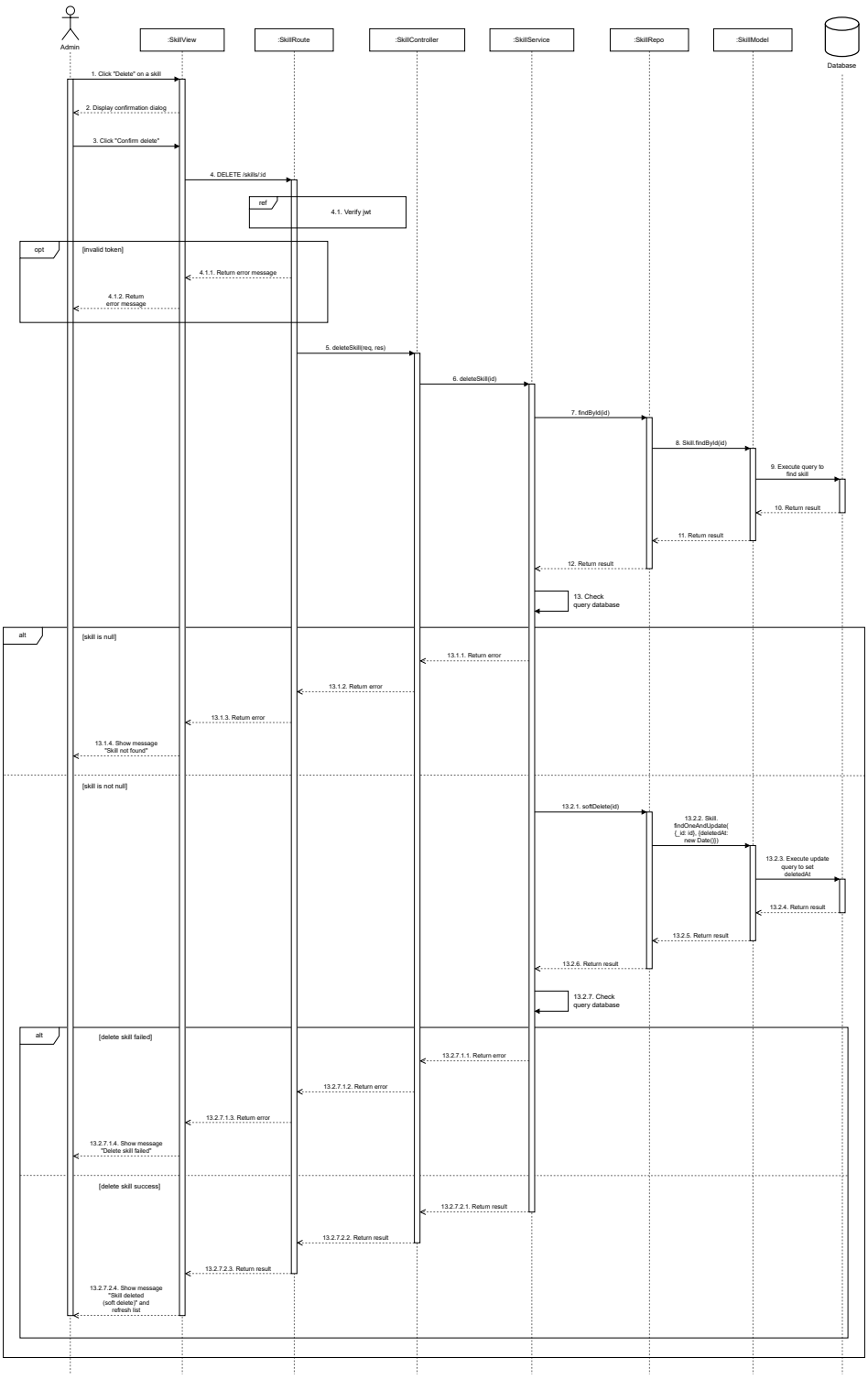

Figure 176: Sequence diagram of UC-38.05 Delete Skill

3.56.6 UC-38.06 – Restore Skill

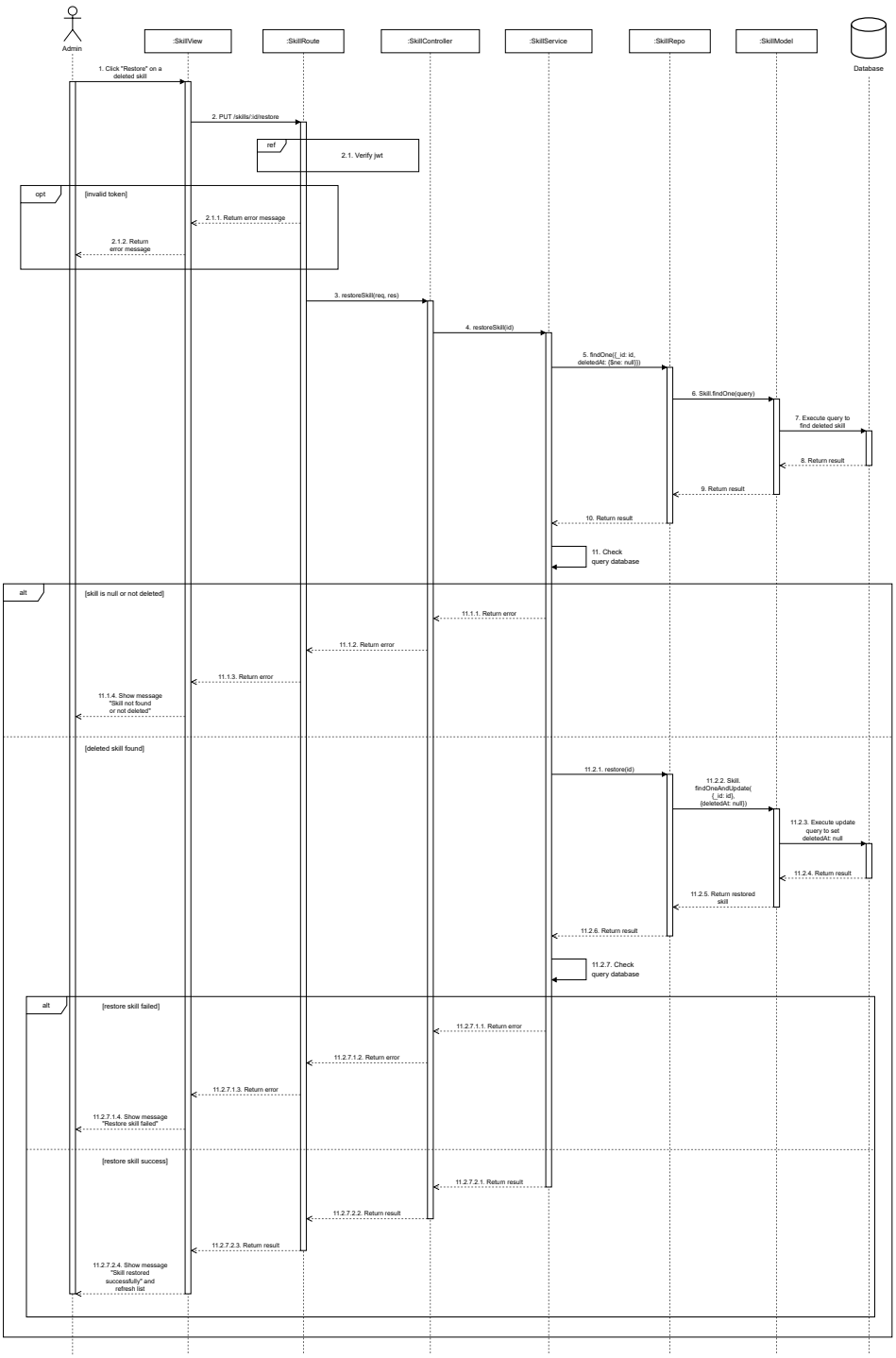

Figure 177: Sequence diagram of UC-38.06 Restore Skill

3.57 Profile Update Request Management

Administrators inspect pending requests, compare proposed changes, approve or reject a pending request and inspect request history.

a. Class Diagram

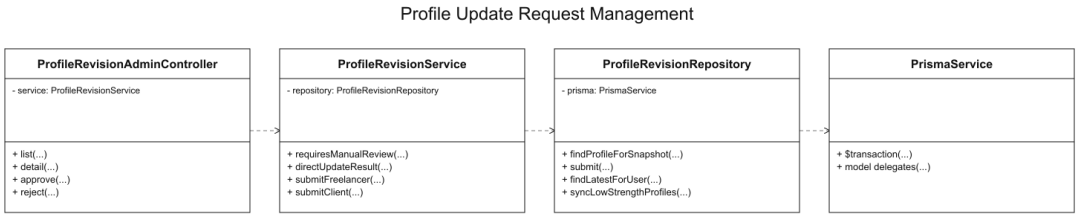

Figure 178: Class diagram of Profile Update Request Management

b. Sequence Diagram(s)

3.57.1 UC-47.01 – View Pending Profile Update Request List

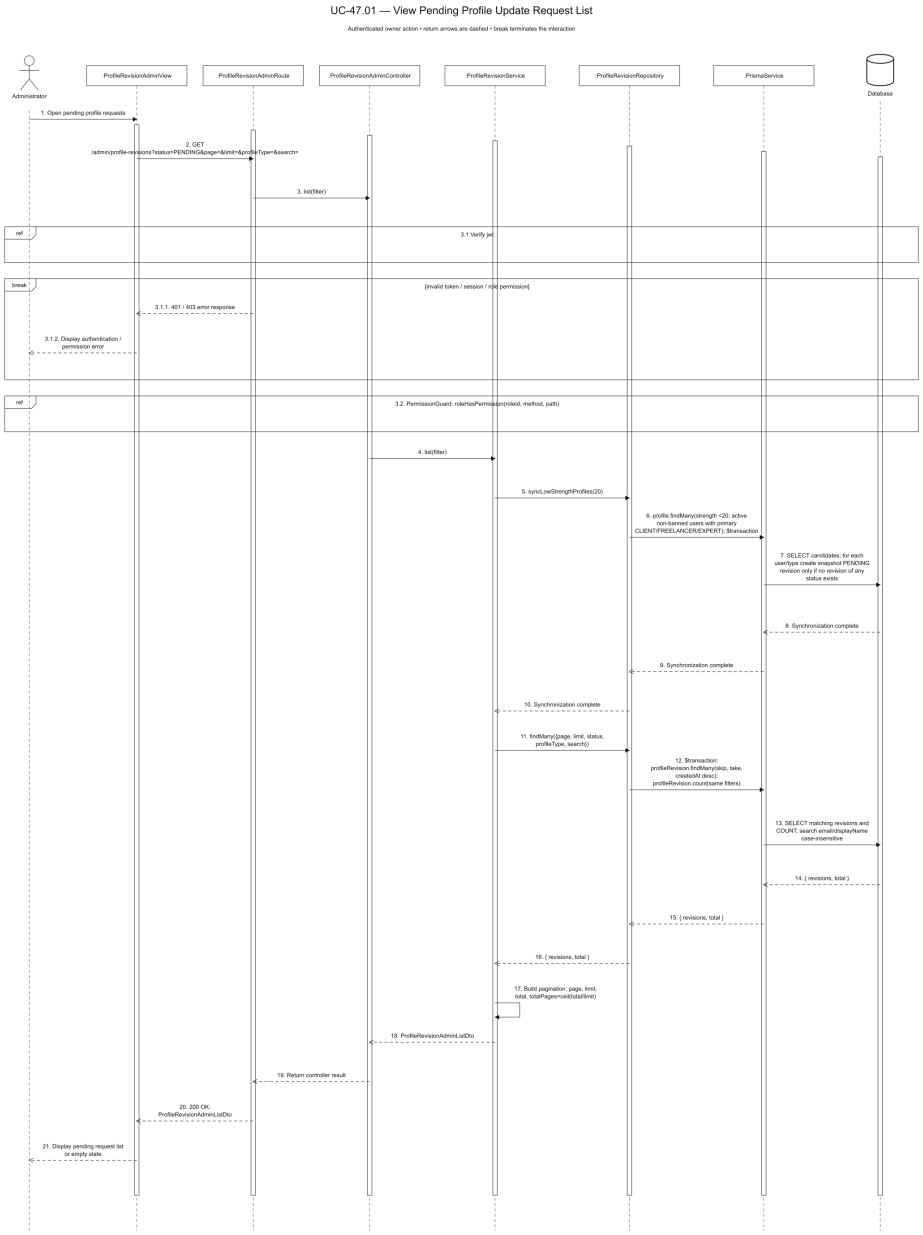

Figure 179: Sequence diagram of UC-47.01 View Pending Profile Update Request List

3.57.2 UC-47.02 – View Profile Update Request Detail

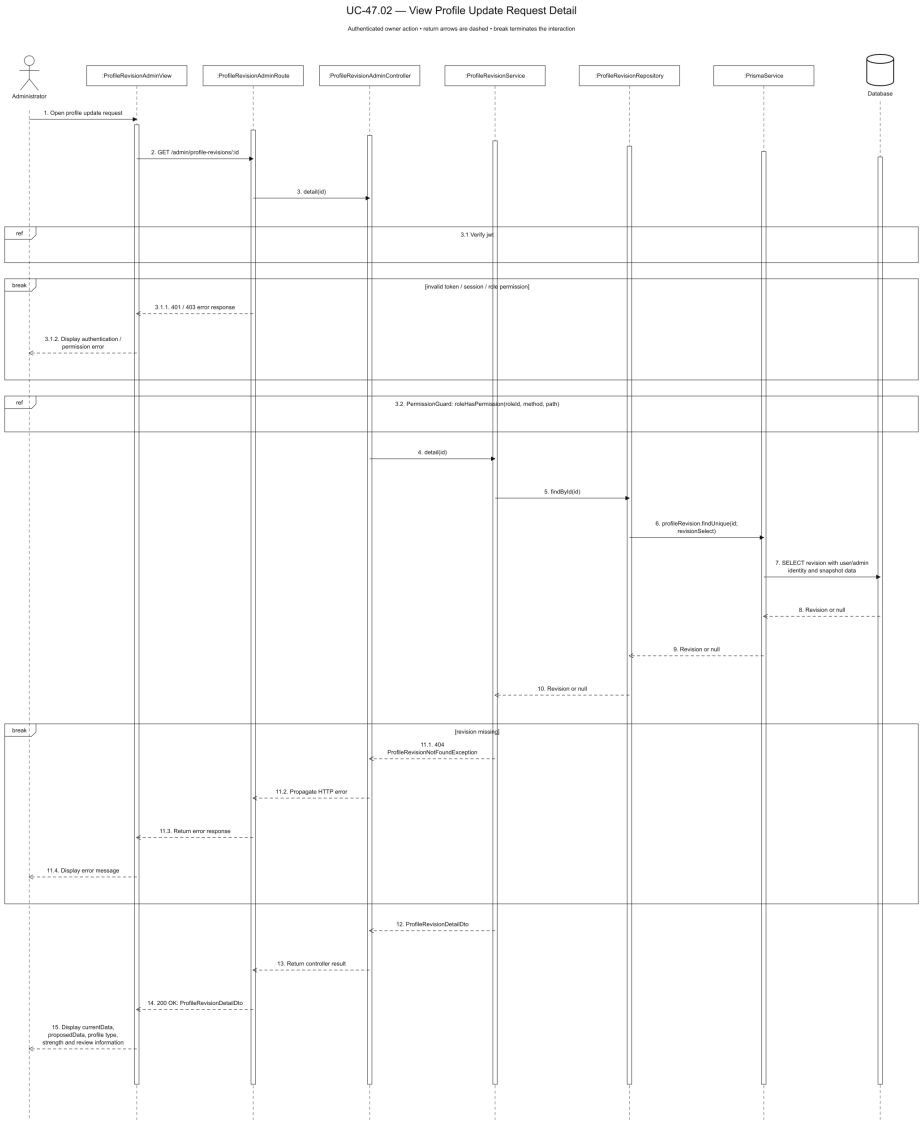

Figure 180: Sequence diagram of UC-47.02 View Profile Update Request Detail

3.57.3 UC-47.03 – Approve Profile Update Request

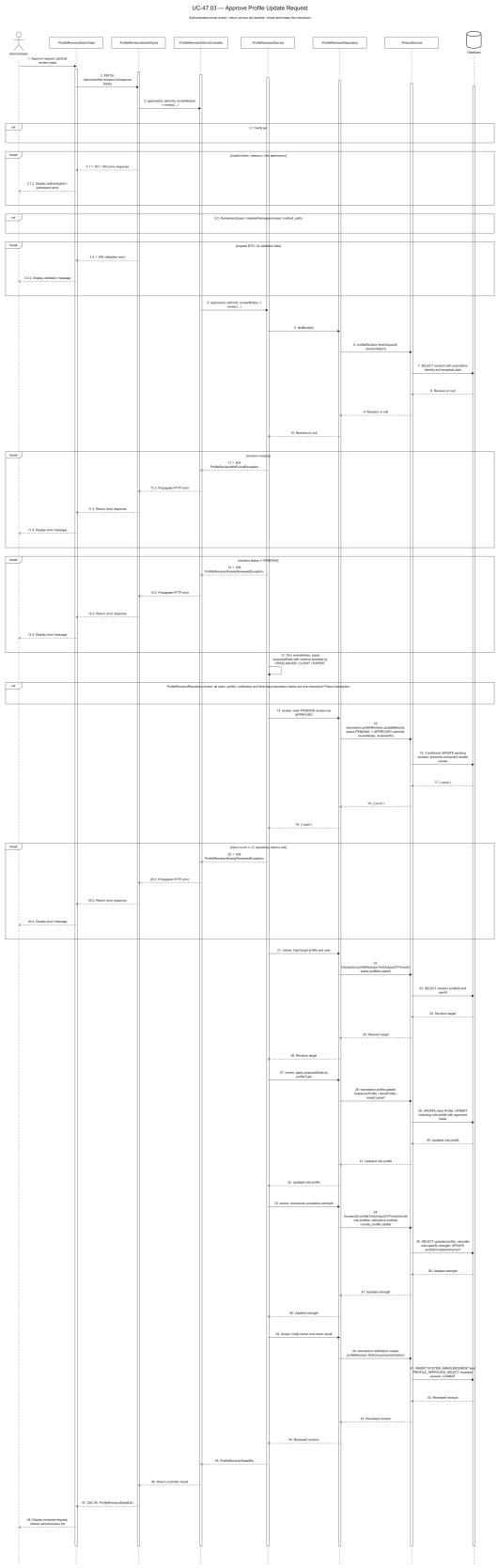

Figure 181: Sequence diagram of UC-47.03 Approve Profile Update Request

3.57.4 UC-47.04 – Reject Profile Update Request

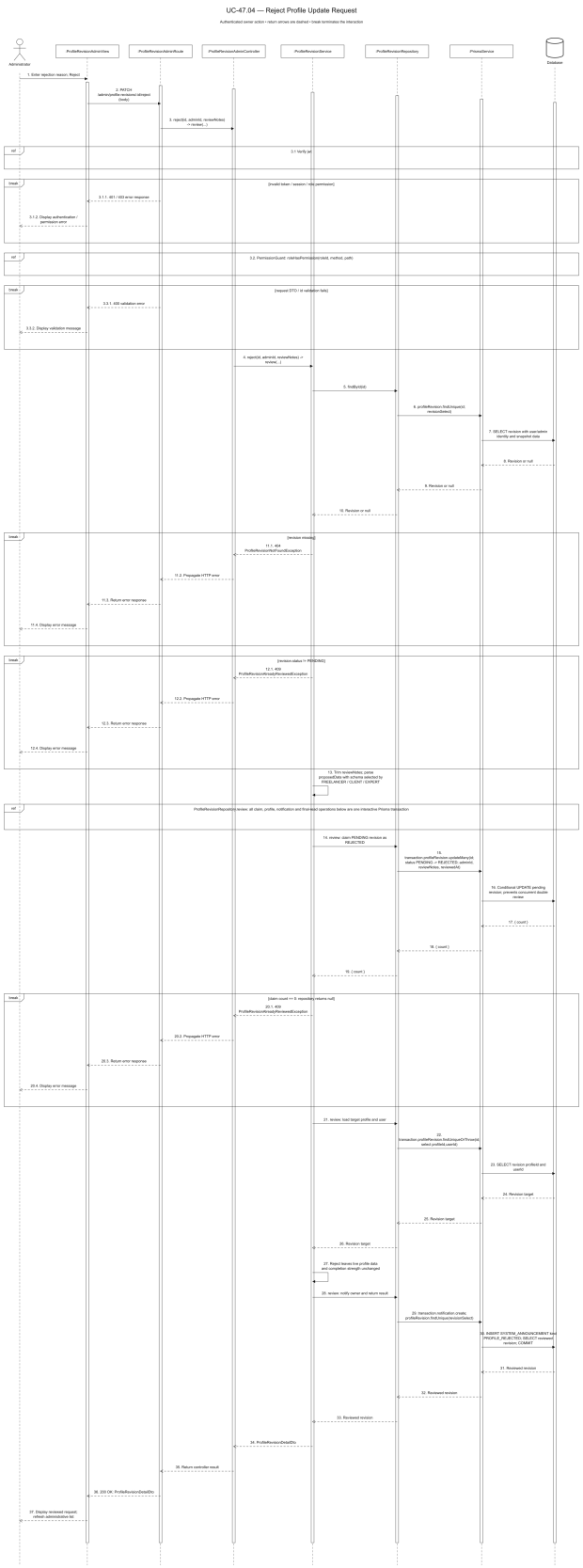

Figure 182: Sequence diagram of UC-47.04 Reject Profile Update Request

3.57.5 UC-47.05 – View Profile Update Request History

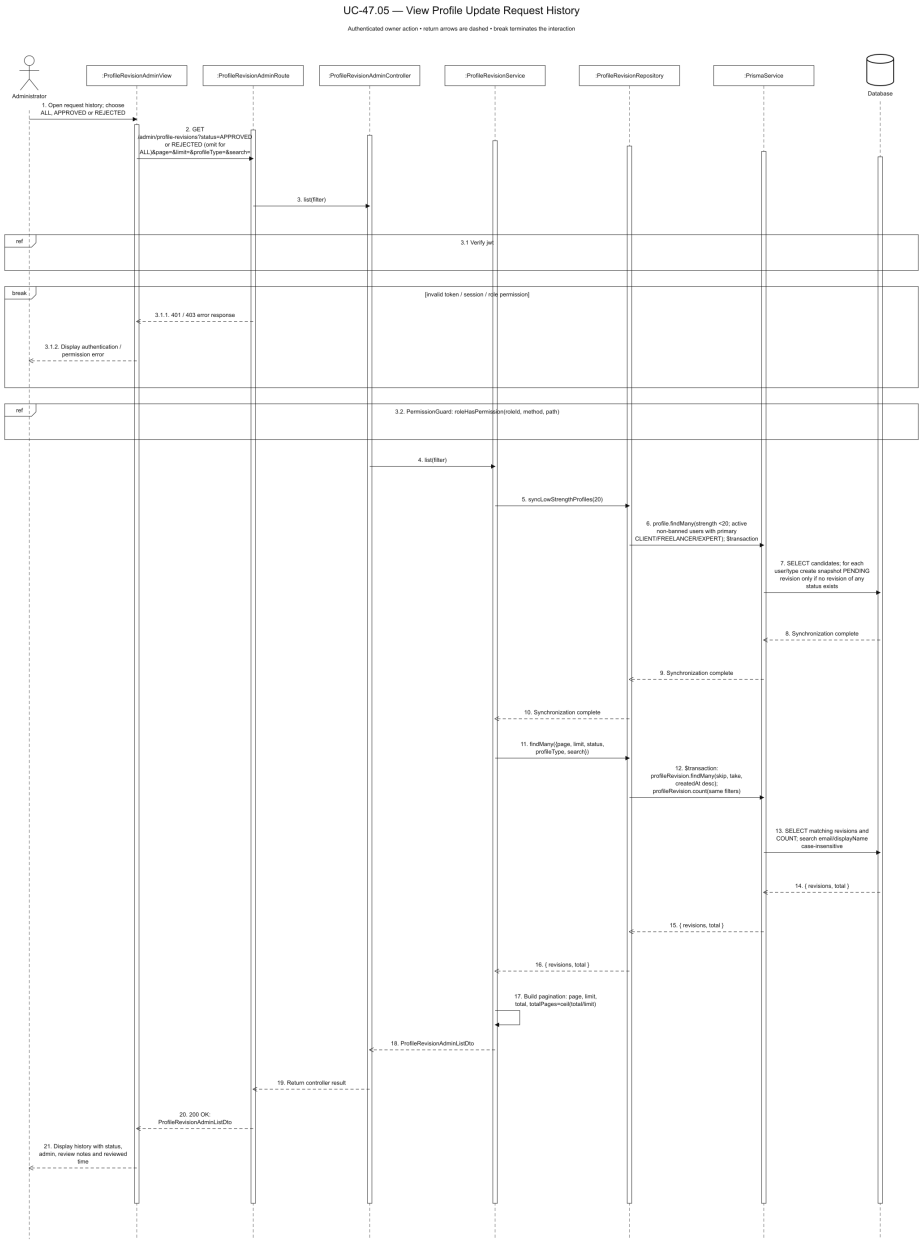

Figure 183: Sequence diagram of UC-47.05 View Profile Update Request History

3.58 Expert Profile Management

Experts submit sanitized profile changes for administrative review. An existing pending expert revision prevents another submission.

a. Class Diagram

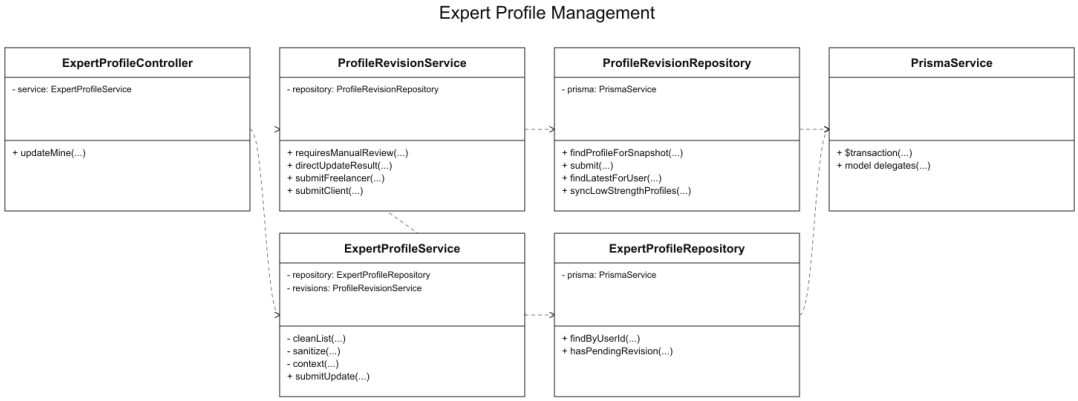

Figure 184: Class diagram of Expert Profile Management

b. Sequence Diagram(s)

3.58.1 UC-47.06 – Update Profile Expert

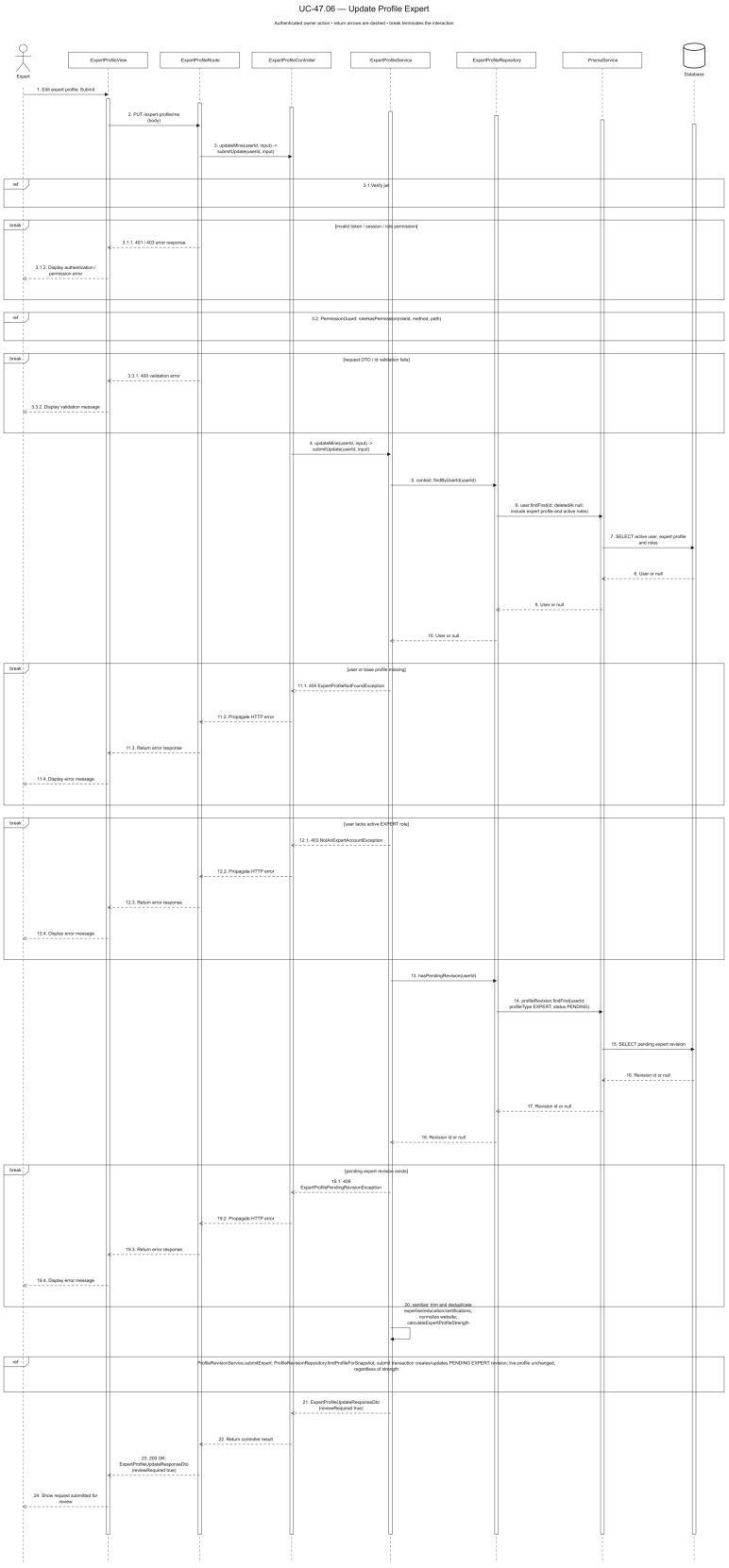

Figure 185: Sequence diagram of UC-47.06 Update Profile Expert

3.59 Freelancer CV Management

Freelancers upload, retrieve, replace and delete their own PDF CV. Upload and replacement use the same endpoint.

a. Class Diagram

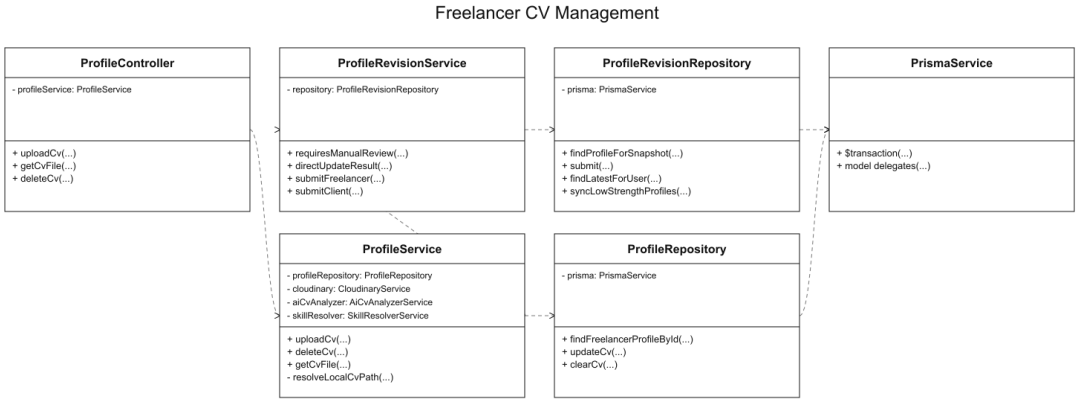

Figure 186: Class diagram of Freelancer CV Management

b. Sequence Diagram(s)

3.59.1 UC-07.03 – Upload CV

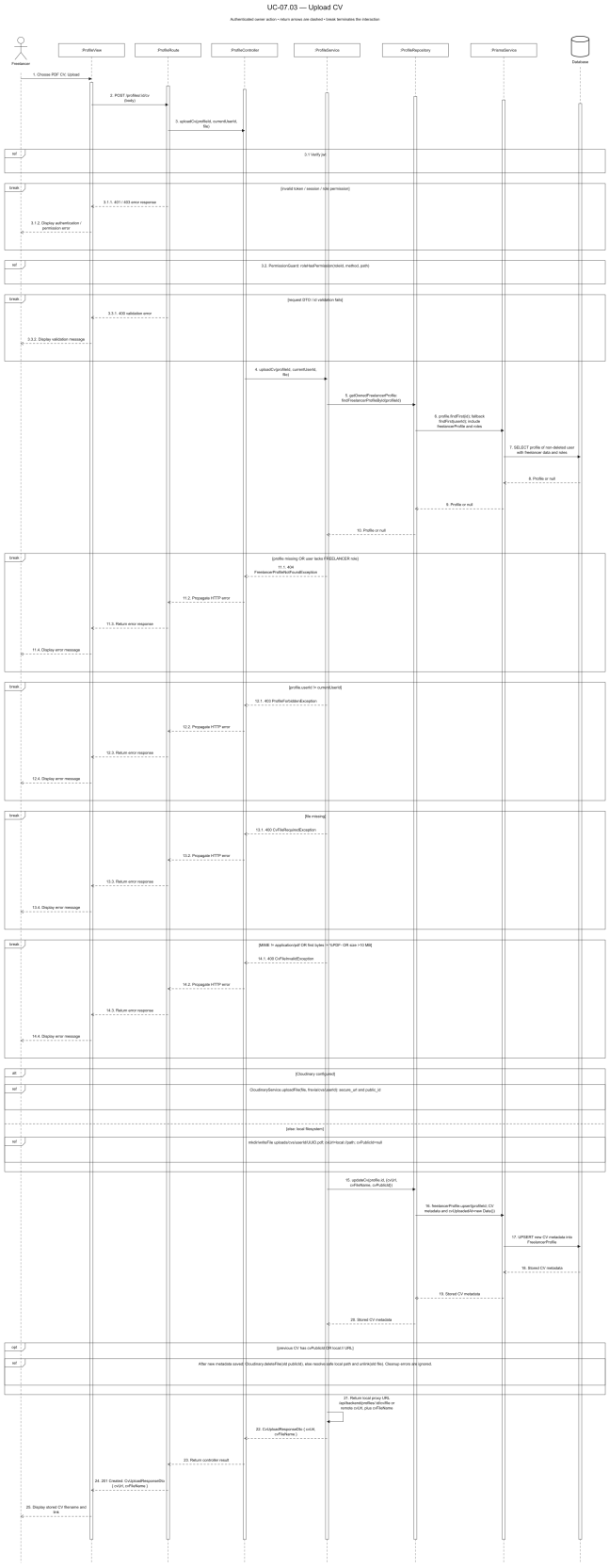

Figure 187: Sequence diagram of UC-07.03 Upload CV

3.59.2 UC-07.04 – View CV

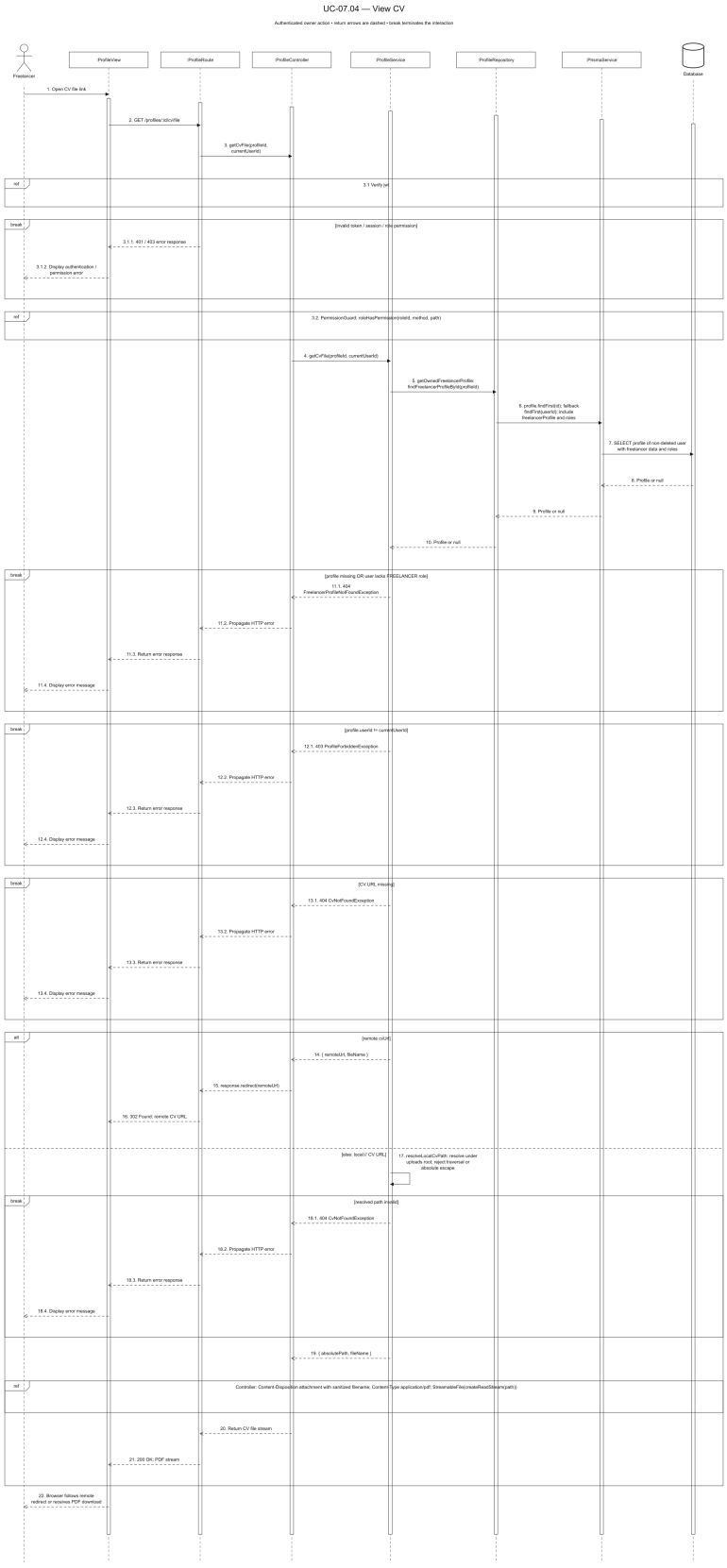

Figure 188: Sequence diagram of UC-07.04 View CV

3.59.3 UC-07.05 – Replace CV

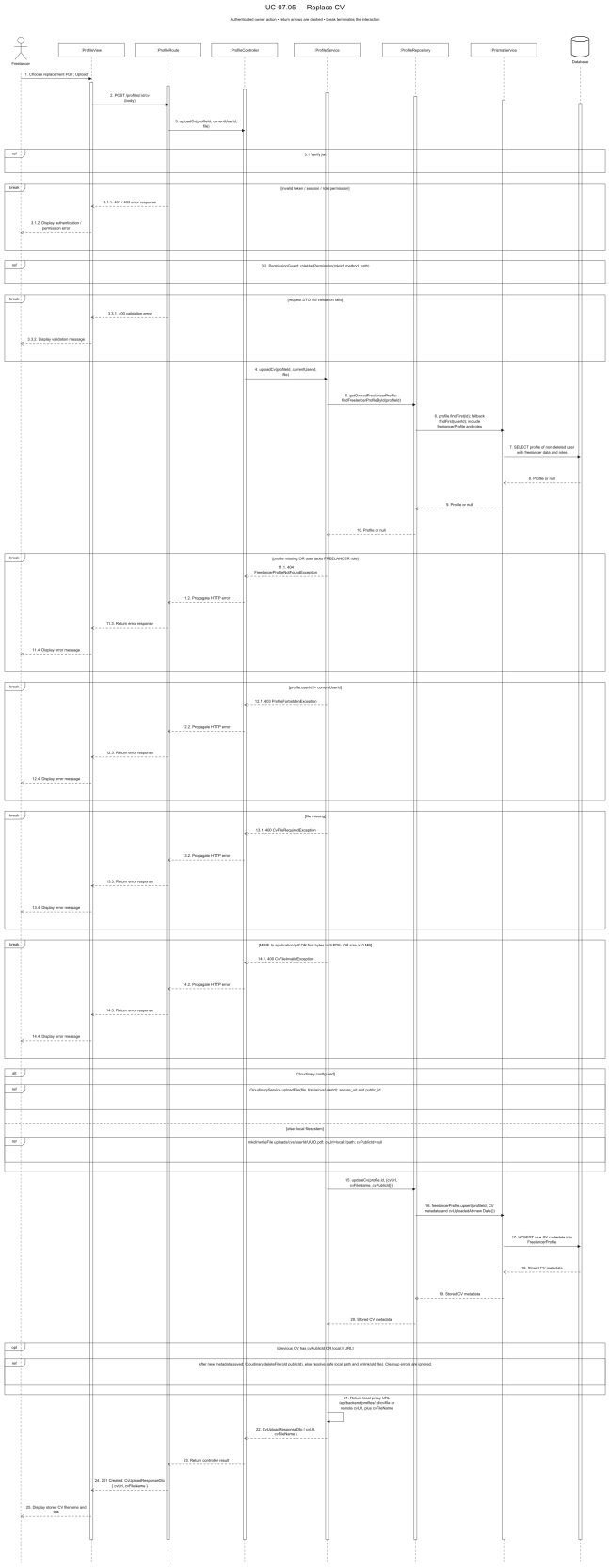

Figure 189: Sequence diagram of UC-07.05 Replace CV

3.59.4 UC-07.06 – Delete CV

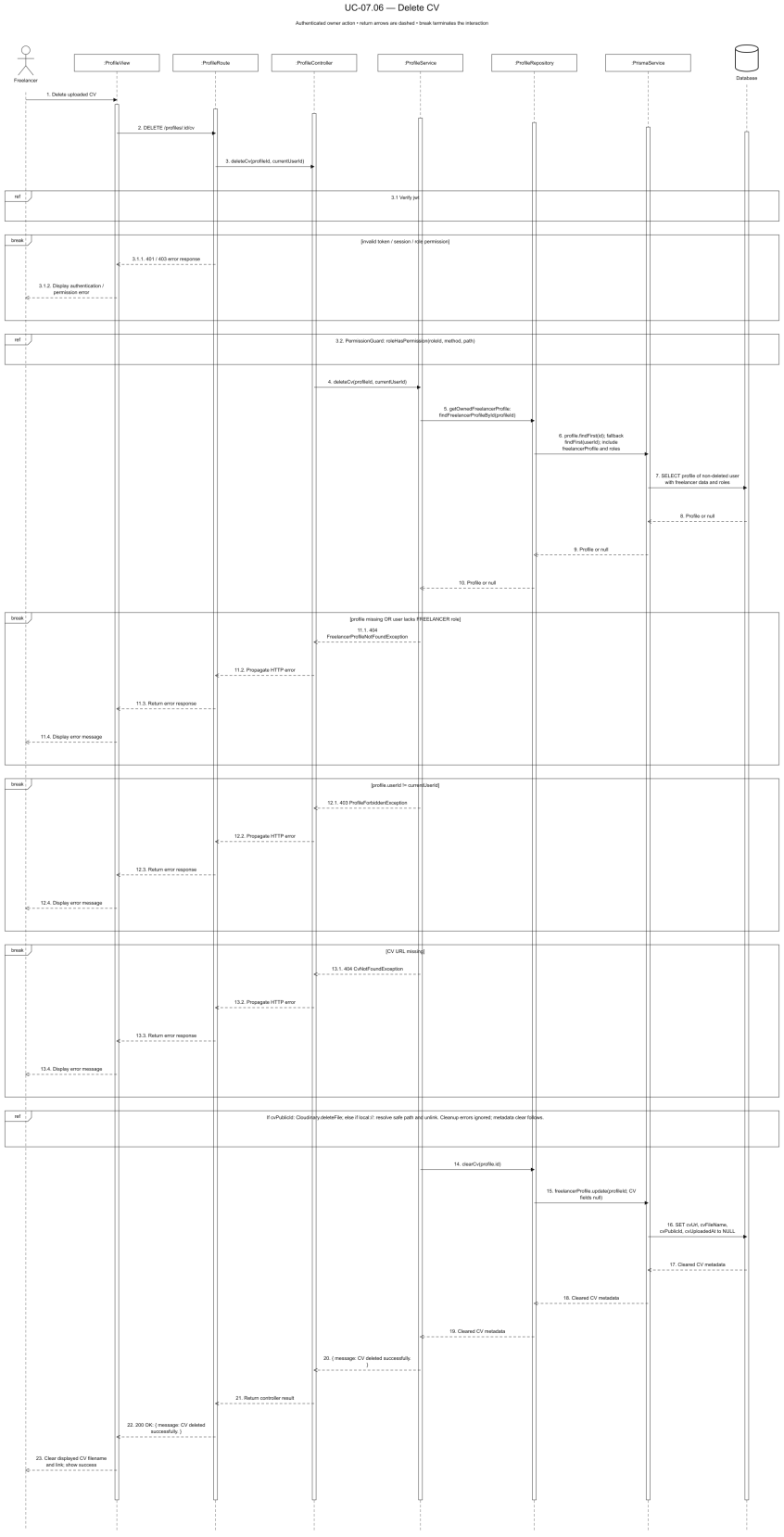

Figure 190: Sequence diagram of UC-07.06 Delete CV

3.60 Expert Profile Management

The administrator-side Expert Profile editor updates an expert account directly. Any pending self-service revision is superseded, and the account owner is notified of the administrative change.

a. Class Diagram

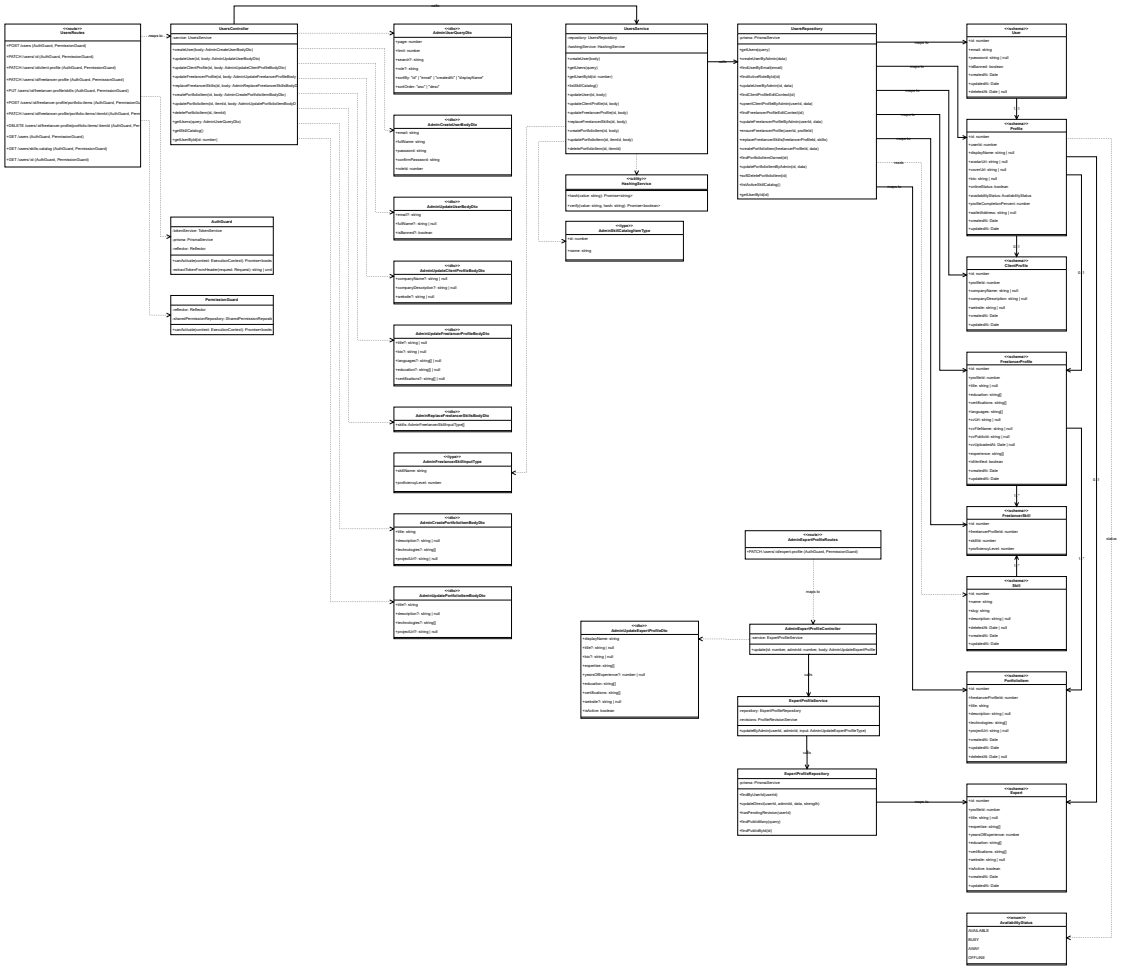

Figure 191: Class diagram of Expert Profile Management

b. Sequence Diagram(s)

3.60.1 UC-28.09 – Update Expert Profile

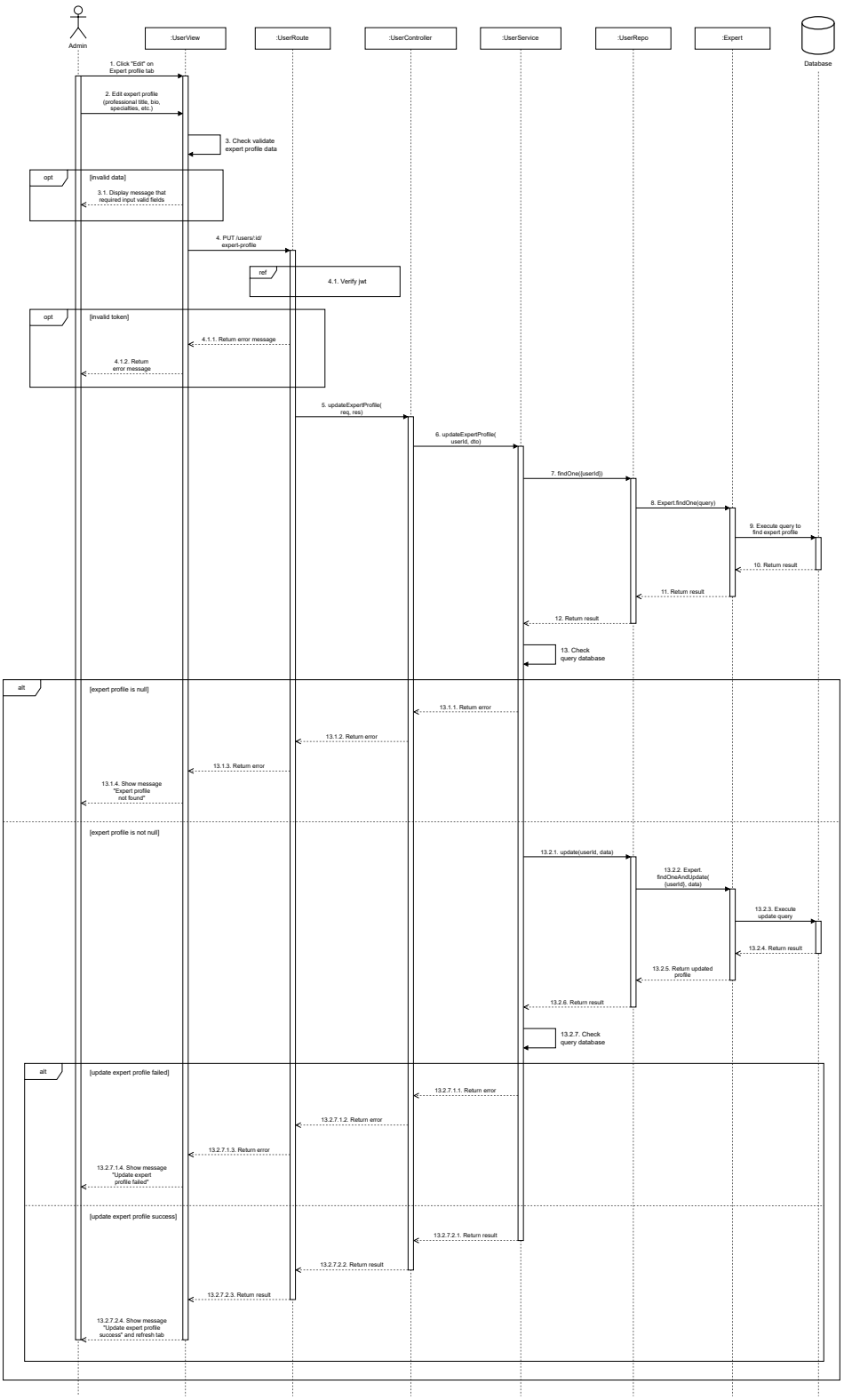

Figure 192: Sequence diagram of UC-28.09 Update Expert Profile

3.61 Job Category Management

Job Category Management maintains the taxonomy used by job postings. A category can be created, updated, soft-deleted, and restored; deletion is blocked while active jobs still reference the category.

a. Class Diagram

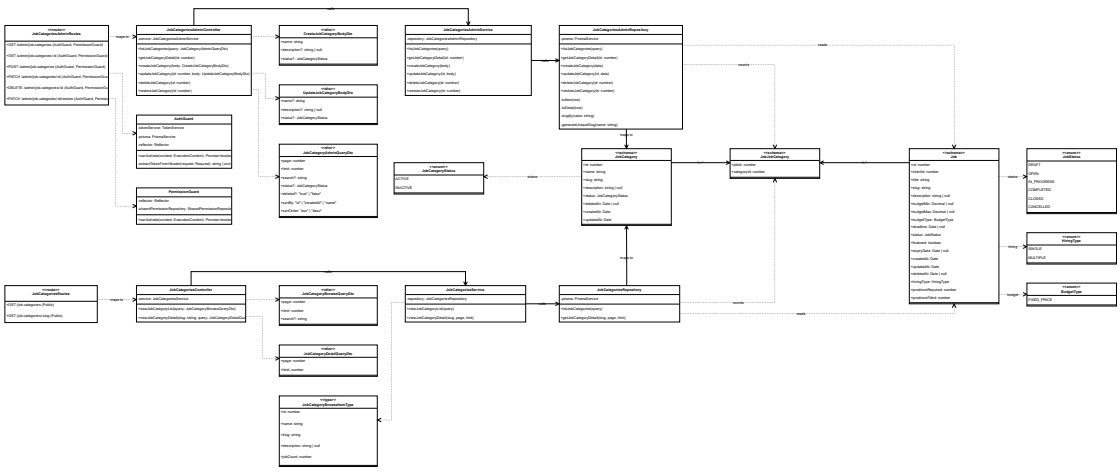

Figure 193: Class diagram of Job Category Management

b. Sequence Diagram(s)

3.61.1 UC-46.01 – View Job Category List

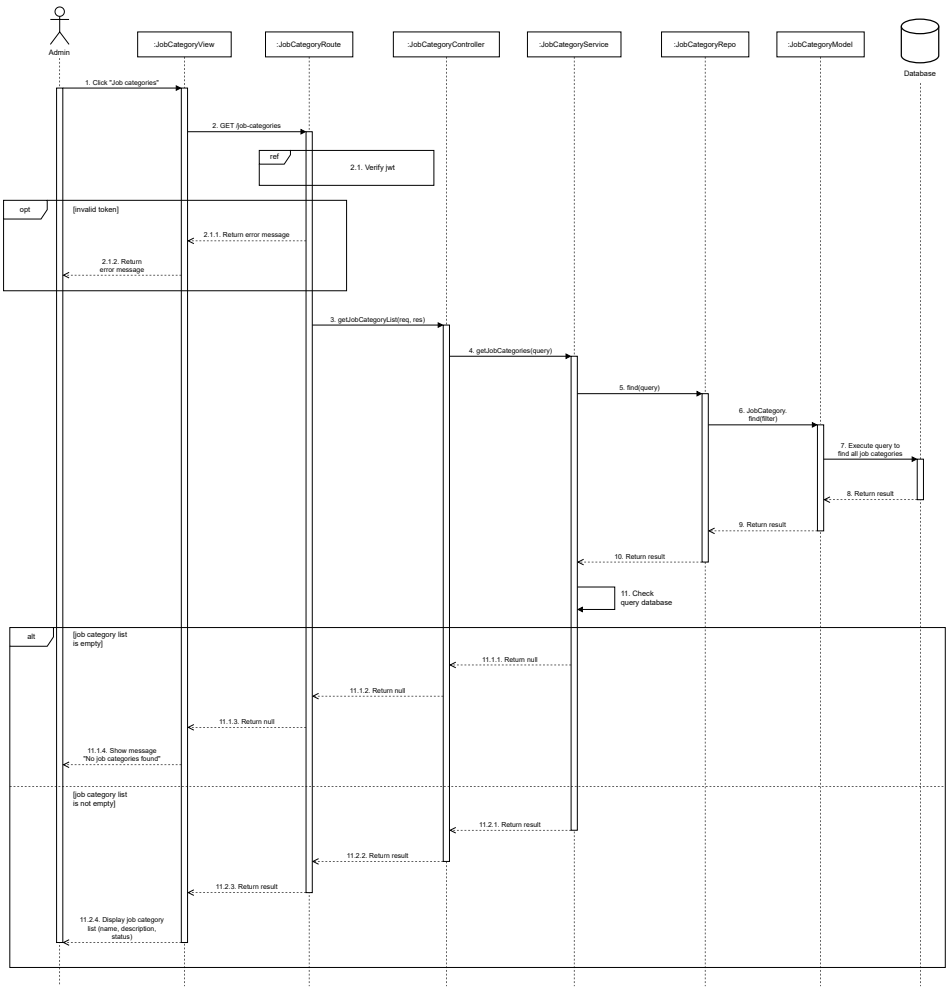

Figure 194: Sequence diagram of UC-46.01 View Job Category List

3.61.2 UC-46.02 – View Job Category Detail

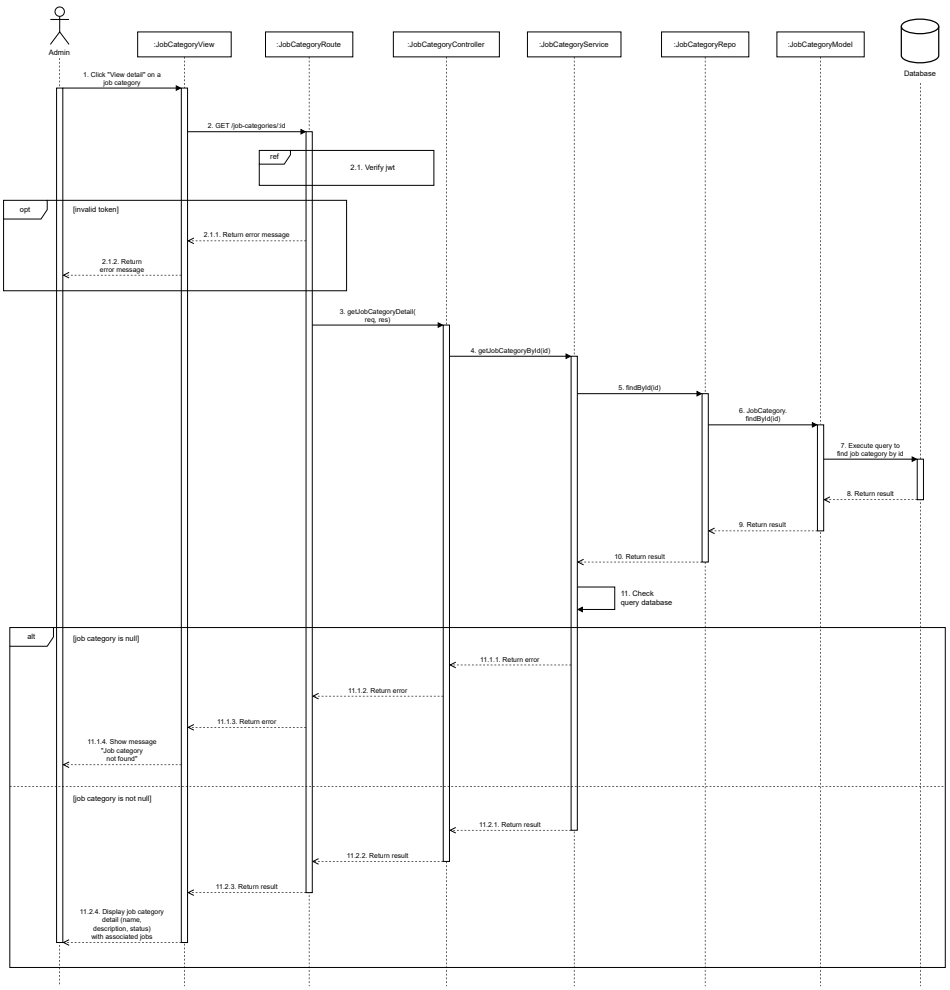

Figure 195: Sequence diagram of UC-46.02 View Job Category Detail

3.61.3 UC-46.03 – Create Job Category

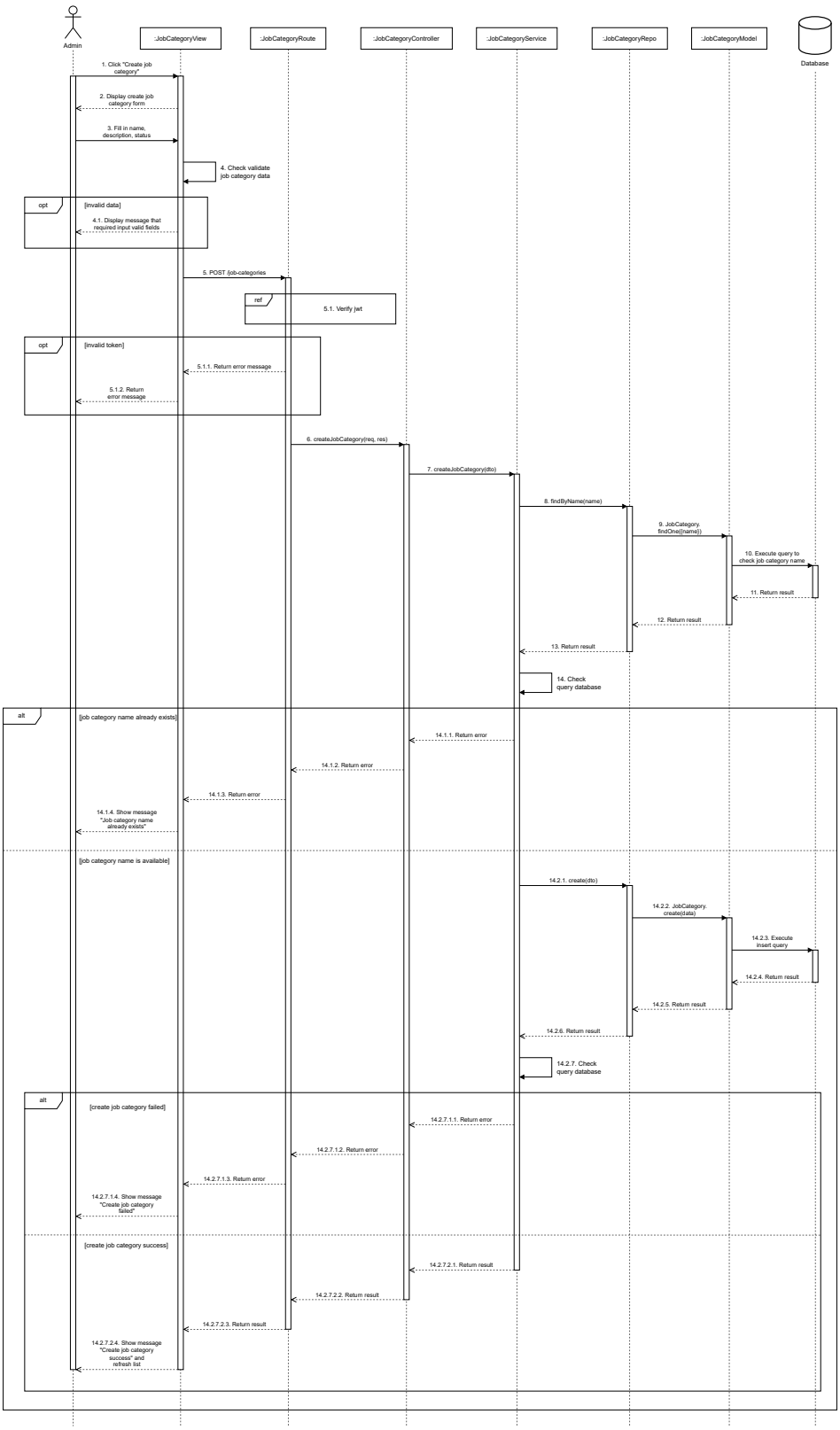

Figure 196: Sequence diagram of UC-46.03 Create Job Category

3.61.4 UC-46.04 – Update Job Category

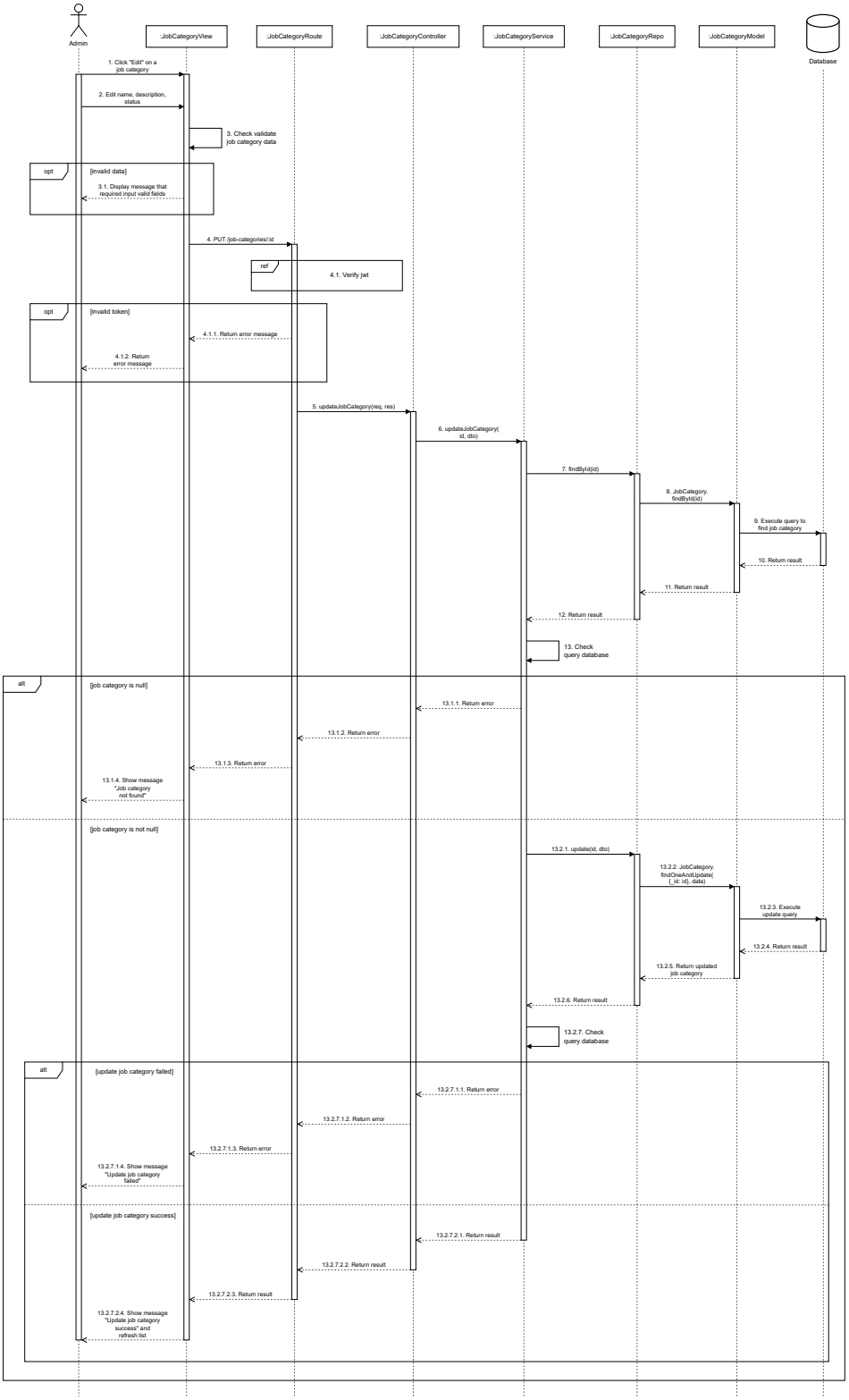

Figure 197: Sequence diagram of UC-46.04 Update Job Category

3.61.5 UC-46.05 – Delete Job Category

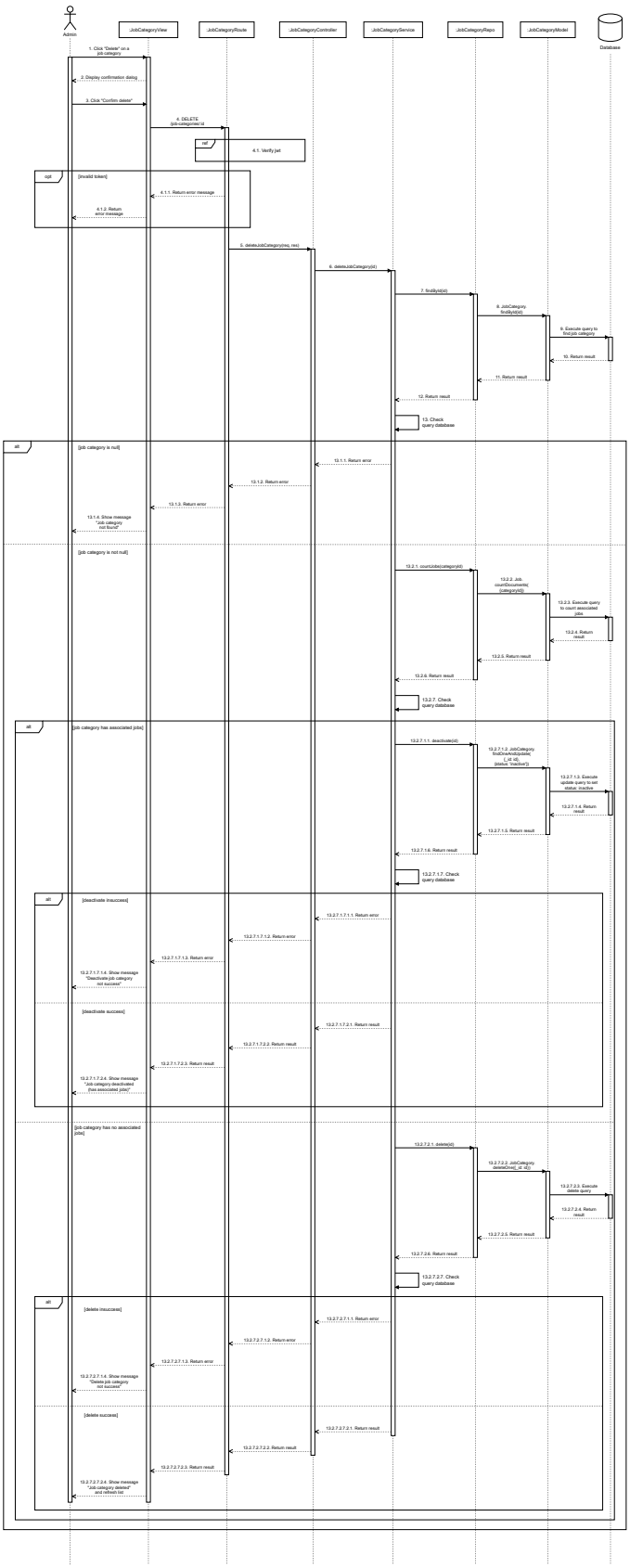

Figure 198: Sequence diagram of UC-46.05 Delete Job Category

4 Class Specification

4.1 ForumController Class

4.1.1 ClassMethod

No	Method	Description
01	`ForumController(forumService: ForumService)`	A method to initialize a new instance of the controller with the forum service. – `forumService`: `ForumService` – the service that owns category and post use cases. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to retrieve all active forum categories. – *Returns:* `Promise<ForumCategoryListResponseType>` – the category list produced by the service. – Route: `GET /forums/categories`; public endpoint.
03	`getTopForumCategories({ query: { limit: number } })`	A method to retrieve the categories with the largest number of public posts. – `query.limit`: `number` – maximum number of categories, validated by `TopLimitSchema` with default 3 and range 1–10. – *Returns:* `Promise<ForumCategoryTopListResponseType>` – ranked category list. – Route: `GET /forums/categories/top`; public endpoint.
04	`getTopActiveUsers({ query: { limit: number } })`	A method to retrieve the users with the most forum activity. – `query.limit`: `number` – maximum number of users, validated by `TopLimitSchema` with default 3 and range 1–10. – *Returns:* `Promise<ForumTopActiveUserListResponseType>` – users with post and comment counts. – Route: `GET /forums/users/top`; public endpoint.

No	Method	Description
05	`getForumCategoryById(id: number)`	A method to retrieve one forum category by its identifier. – `id`: `number` – category identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with post count. – Route: `GET /forums/categories/:id`; public endpoint.
06	`getForumCategoryBySlug(slug: string)`	A method to retrieve one forum category by its slug. – `slug`: `string` – unique category slug. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with post count. – Route: `GET /forums/categories/by-slug/:slug`; public endpoint.
07	`getForumPostLists(filter: ForumPostFilterType)`	A method to retrieve a paginated, filterable list of public posts. – `filter`: `ForumPostFilterType` – page, limit, optional categoryId, optional userId, and optional search text, validated by `ForumPostFilterDto`. – *Returns:* `Promise<ForumPostListResponseType>` – posts plus pagination metadata. – Route: `GET /forums/posts`; public endpoint.
08	`getTopInteractedPosts(query: { limit: number; categoryId?: number })`	A method to retrieve the most interacted posts from the previous seven days. – `query.limit`: `number` – maximum number of posts, default 3. – `query.categoryId`: `number | undefined` – optional category filter. – *Returns:* `Promise<ForumTopPostListResponseType>` – ranked posts with like and comment counts. – Route: `GET /forums/posts/top`; public endpoint.
09	`viewForumPostDetail(id: number)`	A method to retrieve the public detail of one post. – `id`: `number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post, author summary, and category summary. – Route: `GET /forums/posts/:id`; public endpoint.

No	Method	Description
10	`getForumPostBySlug(slug: string)`	A method to retrieve the public detail of one post by slug. – `slug`: `string` – unique post slug. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post detail. – Route: `GET /forums/posts/by-slug/:slug`; public endpoint.
11	`createForumPost( userId: number, body: CreateForumPostType)`	A method to create a forum post for the authenticated user. – `userId`: `number` – author identifier injected by `@UserActive('userId')`. – `body`: `CreateForumPostType` – categoryId, title, content, and optional moderation result, validated by `CreateForumPostDto`. – *Returns:* `Promise<ForumPostType>` – the created post. – Route: `POST /forums/posts`; authenticated endpoint.
12	`deleteForumPost( userId: number, roleName: string, id: number)`	A method to soft-delete a post when the caller is its owner or an administrator. – `userId`: `number` – current user identifier. – `roleName`: `string` – current role name, used for the administrator override. – `id`: `number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post. – Route: `DELETE /forums/posts/:id`; authenticated endpoint.

4.1.2 ClassAttribute

No	Attribute	Type
01	`forumService`	`ForumService` – private readonly service dependency.

4.2 ForumService Class

4.2.1 ClassMethod

No	Method	Description
01	`ForumService(forumRepository: ForumRepository)`	A method to initialize the service with the forum repository. – `forumRepository`: `ForumRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to load all active categories. – *Returns:* `Promise<ForumCategoryListResponseType>` – category list with public post counts. – Failure is mapped to `FailedToLoadForumCategoriesException`.
03	`getTopForumCategories(limit: number = 3)`	A method to load the categories with the most public posts. – `limit`: `number` – maximum number of categories; default 3. – *Returns:* `Promise<ForumCategoryTopListResponseType>` – ranked categories. – Failure is mapped to `FailedToLoadForumCategoriesException`.
04	`getTopActiveUsers(limit: number = 5)`	A method to load the most active users by post and comment counts. – `limit`: `number` – maximum number of users; default 5. – *Returns:* `Promise<ForumTopActiveUserListResponseType>` – user activity summaries. – The current implementation reuses `FailedToLoadForumCategoriesException` for this failure path.
05	`getForumCategoryById(id: number)`	A method to load one category by identifier. – `id`: `number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail. – Prisma known errors are mapped to `FailedToLoadForumCategoriesException`.
06	`getForumCategoryBySlug(slug: string)`	A method to resolve a category slug and return its detail. – `slug`: `string` – category slug. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail. – Throws `ForumCategoryNotFoundException` when the slug does not exist.

No	Method	Description
07	`getForumPostBySlug(slug: string)`	A method to resolve a post slug and return its public detail. – `slug`: `string` – post slug. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post detail. – Throws `ForumPostNotFoundException` when the slug does not exist.
08	`getForumPostLists(filter: ForumPostFilterType)`	A method to build a paginated post list from the repository result. – `filter`: `ForumPostFilterType` – page, limit, category, author, and search filters. – *Returns:* `Promise<ForumPostListResponseType>` – posts and pagination metadata, including `totalPages`. – Prisma known errors are mapped to `FailedToLoadForumPostsException`.
09	`createForumPost(userId: number, body: CreateForumPostType)`	A method to validate the optional category and create a post. – `userId`: `number` – author identifier. – `body`: `CreateForumPostType` – post data and optional AI moderation result. – *Returns:* `Promise<ForumPostType>` – the created post. – Throws `ForumCategoryNotFoundException` for an unknown category and `FailedToCreateForumPostException` for a persistence failure.
10	`viewForumPostDetail(id: number)`	A method to load the public detail of a post. – `id`: `number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post, author, and category summary. – Prisma known errors are mapped to `FailedToViewForumPostException`.

No	Method	Description
11	`updateForumPost( id: number, userId: number, body: UpdateForumPostType)`	A method to update a post after checking existence and ownership. – `id: number` – post identifier. – `userId: number` – current user identifier. – `body: UpdateForumPostType` – updated category, title, and content. – *Returns:* `Promise<ForumPostType>` – the updated post. – Throws `ForumPostNotFoundException` or `ForumPostNotOwnedException`. The controller route is currently commented out.
12	`deleteForumPost( id: number, userId: number, roleName: string)`	A method to authorize and soft-delete a post. – `id: number` – post identifier. – `userId: number` – current user identifier. – `roleName: string` – current role name; `ADMIN` bypasses the owner check. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post. – Throws `ForumPostNotFoundException`, `ForumPostNotOwnedException`, or `FailedToDeleteForumPostException`.
13	`getTopInteractedPosts( limit: number = 3, categoryId?: number)`	A method to return the most interacted posts from the last seven days. – `limit: number` – maximum number of posts; default 3. – `categoryId: number | undefined` – optional category restriction. – *Returns:* `Promise<ForumTopPostListResponseType>` – ranked posts. – Failure is mapped to `FailedToLoadForumPostsException`.

4.2.2 ClassAttribute

No	Attribute	Type
01	`forumRepository`	`ForumRepository` – private readonly persistence dependency.

4.3 ForumRepository Class

4.3.1 ClassMethod

No	Method	Description
01	`ForumRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – pooled database client. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to query active categories ordered by name. – *Returns:* `Promise<Array<Pick<ForumCategory Type, 'id' | 'name' | 'slug' | 'description' | 'createdAt' | 'updatedAt' | 'postCount'>>>` – category rows with public post counts. – Only non-deleted posts with moderation status `APPROVE D` are counted.
03	`getTopForumCategories( limit: number = 3)`	A method to query active categories ordered by public post count. – `limit`: `number` – maximum number of rows; default 3. – *Returns:* the same category projection as `getForum Categories()`, ranked by `posts._count` descending.
04	`getTopActiveUsers(limit: number = 5)`	A method to query non-deleted, non-banned users ordered by post count. – `limit`: `number` – maximum number of rows; default 5. – *Returns:* `Promise<ForumTopActiveUserType[]>` – user id, display name, avatar, post count, and comment count. – Comment counts include only non-deleted comments.
05	`findForumCategoryById(id: number)`	A method to check whether an active category exists. – `id`: `number` – category identifier. – *Returns:* `Promise<Pick<ForumCategoryType, 'id'> | null>` – the id when found, otherwise `null`.
06	`getForumCategoryById(id: number)`	A method to load one active category with its post count. – `id`: `number` – category identifier. – *Returns:* category projection with `postCount`. – Throws `ForumCategoryNotFoundException` when no active row exists.

No	Method	Description
07	`findForumCategoryBySlug( slug: string)`	A method to resolve an active category by slug. – `slug`: `string` – category slug. – *Returns:* `Promise<Pick<ForumCategoryType, 'id'> | null>` – the id when found, otherwise `null`.
08	`getForumPostLists(filter: ForumPostFilterType)`	A method to query a page of public posts and the matching total in one transaction. – `filter`: `ForumPostFilterType` – pagination, category, author, and case-insensitive title search. – *Returns:* `Promise<{ posts: ForumPostWithUserType[]; total: number }>` – mapped post rows and total count. – Visibility requires `deletedAt = null` and moderation status `APPROVED`.
09	`createForumPost( categoryId: number | null, userId: number, title: string, content: string, moderation?: ForumModerationType)`	A method to insert a post and generate its slug. – `categoryId`: `number | null` – optional category. – `userId`: `number` – author identifier. – `title`, `content`: `string` – post text. – `moderation`: `ForumModerationType | undefined` – optional AI result; status defaults to `PENDING`. – *Returns:* `Promise<ForumPostType>` – the inserted post.
10	`viewForumPostDetail(id: number)`	A method to query one public post with author and category summaries. – `id`: `number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post detail. – Throws `ForumPostNotFoundException` when the post is missing, deleted, or not approved.
11	`findForumPostById(id: number)`	A method to load a non-deleted post for ownership checks. – `id`: `number` – post identifier. – *Returns:* `Promise<ForumPostType | null>` – the post or `null`.
12	`findForumPostBySlug(slug: string)`	A method to resolve a public post slug. – `slug`: `string` – post slug. – *Returns:* `Promise<Pick<ForumPostType, 'id'> | null>` – the id when found, otherwise `null`.

No	Method	Description
13	`updateForumPost( id: number, data: UpdateForumPostType)`	A method to update a non-deleted post and regenerate its slug from the title. – `id`: `number` – post identifier. – `data`: `UpdateForumPostType` – category, title, and content. – *Returns:* `Promise<ForumPostType>` – the updated post.
14	`softDeleteForumPost(id: number)`	A method to set `deletedAt` on a non-deleted post. – `id`: `number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post.
15	`getTopInteractedPosts( limit: number = 3, categoryId?: number)`	A method to query posts created during the previous seven days and rank them by interactions. – `limit`: `number` – maximum number of rows; default 3. – `categoryId`: `number | undefined` – optional category restriction. – *Returns:* `Promise<ForumTopPostType[]>` – posts with like count, comment count, and `interactionScore`. – Ordering is by likes, then comments, then creation time, all descending.

4.3.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for category and post management

Class / Element	Attributes or Signature	Responsibility
`ForumCategoryListResponseDto`	Array of `ForumCategoryType`: `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, `postCount`.	Response of `GET /forums/categories`.

Class / Element	Attributes or Signature	Responsibility
ForumCategoryDetail ResponseDto	id: number, name: string, slug: string, description: string | null, createdAt: Date, updatedAt: Date, postCount: number.	Detail response for category id and slug routes.
ForumCategoryTop ListResponseDto	Array of ForumCategoryType.	Response of the top-category route.
ForumTopActiveUser ListResponseDto	Array of ForumTopActiveUserType: id, displayName, avatarUrl, postCount, commentCount.	Response of the top-active-user route.
ForumPostList ResponseDto	posts: ForumPostWithUserType[], pagination: PaginationType.	Paginated post-list response.
ForumPostFilterDto	page: number, limit: number, categoryId?: number, userId?: number, search?: string.	Query validation for the post list.
CreateForumPostDto	categoryId?: number, title: string, content: string, moderation?: ForumModerationType.	Request body for post creation.
CreateForumPost ResponseDto	ForumPostType: id, categoryId, userId, title, slug, content, moderationStatus, moderationScore, moderation Categories, createdAt, updatedAt.	Response of post creation.
ViewForumPostDetail ResponseDto	ForumPostType fields plus category: { id, name, slug } | null and user: { id, profile }.	Public post-detail response.
UpdateForumPostDto	categoryId?: number, title: string, content: string.	Request body for the implemented update-post method.
UpdateForumPost ResponseDto	ForumPostType.	Response of the implemented update-post method.
ForumTopPostList ResponseDto	Array of ForumTopPostType: post fields plus likeCount, commentCount, interaction Score, user, and category.	Response of the top-post route.

4.4 ForumLikeController Class

4.4.1 ClassMethod

No	Method	Description
01	`ForumLikeController(forumLikeService: ForumLikeService)`	A method to initialize the controller with the post-like service. – `forumLikeService`: `ForumLikeService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`toggleLikePost(userId: number, id: number)`	A method to toggle the current user's like on a post. – `userId`: `number` – current user identifier. – `id`: `number` – post identifier. – *Returns:* `Promise<ToggleLikeResponseType>` – `liked: true` after a like, `liked: false` after an unlike. – Route: `POST /forums/posts/:id/like`; authenticated endpoint.
03	`viewForumLikeDetail(postId: number)`	A method to list the users who liked a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – like rows with user summaries. – Route: `GET /forums/posts/:id/like`; public endpoint.

4.4.2 ClassAttribute

No	Attribute	Type
01	`forumLikeService`	`ForumLikeService` – private readonly service dependency.

4.5 ForumLikeService Class

4.5.1 ClassMethod

No	Method	Description
01	`ForumLikeService(forumLikeRepository: ForumLikeRepository)`	A method to initialize the service with the post-like repository. – `forumLikeRepository`: `ForumLikeRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`toggleLikePost(userId: number, postId: number)`	A method to decide whether a like must be created or removed. – `userId`: `number` – current user identifier. – `postId`: `number` – post identifier. – *Returns:* `Promise<ToggleLikeResponseType>` – the new like state. – Throws `ForumPostNotFoundException` when the post is missing; other failures become `FailedToLikeForumPostException`.
03	`viewForumLikeDetail(postId: number)`	A method to load the like rows of a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – like rows with user summaries. – Failure is mapped to `FailedToViewForumLikeDetailException`.

4.5.2 ClassAttribute

No	Attribute	Type
01	`forumLikeRepository`	`ForumLikeRepository` – private readonly persistence dependency.

4.6 ForumLikeRepository Class

4.6.1 ClassMethod

No	Method	Description
01	`ForumLikeRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findLikeByUserAndPost( userId: number, postId: number)`	A method to find a user's reaction row for a post. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number; commentId: number | null } | null>` – reaction id and optional comment link.
04	`findCommentOnlyLike(user Id: number)`	A method to find a reaction row that currently links only to a comment. – `userId`: `number` – user identifier. – *Returns:* `Promise<{ id: number } | null>` – the reusable reaction row, otherwise `null`.
05	`createPostLike(userId: number, postId: number)`	A method to create a post reaction, reusing a comment-only row when possible. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* the created or updated `ForumLike` row.
06	`deletePostLike(userId: number, postId: number)`	A method to remove a post reaction while preserving a comment reaction on the same row. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* the updated or deleted `ForumLike` row, or `undefined` when no row exists.
07	`viewForumLikeDetail(post Id: number)`	A method to query all reaction rows for a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponse Type>` – reaction rows with user id, profile display name, and avatar.

4.6.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for post reactions

Class / Element	Attributes or Signature	Responsibility
`ToggleLikeResponseDto`	`liked: boolean`.	Response of the toggle-like route.
`ViewForumLikeDetail ResponseDto`	Array of `ForumLikeType` with `user`: `id`, `user Id`, `postId`, `commentId`, `reactionType`, `createdAt`, and user profile summary.	Response of the post-like detail route.

4.7 ForumCommentController Class

4.7.1 ClassMethod

No	Method	Description
01	`ForumCommentController( forumCommentService: ForumCommentService)`	A method to initialize the controller with the comment service. – `forumCommentService`: `ForumCommentService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`getForumCommentLists( postId: number, filter: ForumComment FilterType, userId: number)`	A method to list the comments of a post with pagination. – `postId`: `number` – parent post identifier. – `filter`: `ForumCommentFilterType` – page and limit. – `userId`: `number` – current user, used to compute `liked ByMe`. – *Returns:* `Promise<ForumCommentListResponse Type>` – comments and pagination. – Route: `GET /forums/posts/:postId/comments`; authenticated endpoint.

No	Method	Description
03	`createForumComment( userId: number, postId: number, body: CreateForumCommentType)`	A method to create a comment on a post. – `userId: number` – author identifier. – `postId: number` – parent post identifier. – `body: CreateForumCommentType` – comment content. – *Returns:* `Promise<ForumCommentType>` – the created comment. – Route: `POST /forums/posts/:postId/comments`; authenticated endpoint.
04	`updateForumComment( userId: number, postId: number, id: number, body: EditForumCommentType)`	A method to edit a comment owned by the current user. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – `body: EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment. – Route: `PATCH /forums/posts/:postId/comments/:id`; authenticated endpoint.
05	`deleteForumComment( userId: number, roleName: string, postId: number, id: number)`	A method to soft-delete a comment owned by the user or by an administrator. – `userId: number` – current user identifier. – `roleName: string` – current role name. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – *Returns:* `Promise<ForumCommentType>` – the soft-deleted comment. – Route: `DELETE /forums/posts/:postId/comments/:id`; authenticated endpoint.
06	`toggleLikeComment( userId: number, postId: number, commentId: number)`	A method to toggle the current user's like on a comment. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<ToggleLikeCommentType>` – the new like state. – Route: `POST /forums/posts/:postId/comments/:commentId/like`; authenticated endpoint.

4.7.2 ClassAttribute

No	Attribute	Type
01	`forumCommentService`	`ForumCommentService` – private readonly service dependency.

4.8 ForumCommentService Class

4.8.1 ClassMethod

No	Method	Description
01	`ForumCommentService( forumCommentRepository: ForumCommentRepository)`	A method to initialize the service with the comment repository. – `forumCommentRepository`: `ForumComment Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`createForumComment( postId: number, userId: number, body: CreateForumComment Type)`	A method to verify the parent post and create a comment. – `postId`: `number` – parent post identifier. – `userId`: `number` – author identifier. – `body`: `CreateForumCommentType` – comment content. – *Returns:* `Promise<ForumCommentType>` – the created comment. – Throws `ForumPostNotFoundException` or `FailedTo CreateForumCommentException`.
03	`updateForumComment( postId: number, id: number, userId: number, body: EditForumCommentType)`	A method to verify comment existence, parent post, and ownership before editing. – `postId`: `number` – parent post identifier. – `id`: `number` – comment identifier. – `userId`: `number` – current user identifier. – `body`: `EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment. – Throws `ForumCommentNotFoundException` or `Forum CommentNotOwnedException`.

No	Method	Description
04	deleteForumComment(postId: number, id: number, userId: number, roleName: string)	A method to authorize and soft-delete a comment. – postId: number – parent post identifier. – id: number – comment identifier. – userId: number – current user identifier. – roleName: string – ADMIN bypasses the owner check. – *Returns:* Promise<ForumCommentType> – the soft-deleted comment. – Throws ForumCommentNotFoundException, ForumCommentNotOwnedException, or FailedToDeleteForumCommentException.
05	toggleLikeComment(userId: number, postId: number, commentId: number)	A method to create or remove a comment reaction. – userId: number – current user identifier. – postId: number – parent post identifier. – commentId: number – comment identifier. – *Returns:* Promise<ToggleLikeCommentType> – the new like state. – Throws ForumCommentNotFoundException or FailedToLikeForumCommentException.
06	getForumCommentLists(filter: ForumCommentFilterType)	A method to build a paginated comment list. – filter: ForumCommentFilterType – page, limit, postId, and optional userId. – *Returns:* Promise<ForumCommentListResponseType> – comments and pagination metadata. – Prisma known errors are mapped to FailedToLoadForumCommentsException.

4.8.2 ClassAttribute

No	Attribute	Type
01	forumCommentRepository	ForumCommentRepository – private readonly persistence dependency.

4.9 ForumCommentRepository Class

4.9.1 ClassMethod

No	Method	Description
01	`ForumCommentRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findCommentById(id: number)`	A method to load a non-deleted comment with its parent post and author summary. – `id`: `number` – comment identifier. – *Returns:* the comment projection or `null`.
04	`createForumComment( postId: number, userId: number, content: string)`	A method to insert a comment and return the response shape. – `postId`: `number` – parent post identifier. – `userId`: `number` – author identifier. – `content`: `string` – comment text. – *Returns:* `Promise<ForumCommentType>` – the comment with `likeCount = 0` and `likedByMe = false`.
05	`updateForumComment( id: number, data: EditForumCommentType)`	A method to update the content of a non-deleted comment. – `id`: `number` – comment identifier. – `data`: `EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment.
06	`softDeleteForumComment( id: number)`	A method to set `deletedAt` on a non-deleted comment. – `id`: `number` – comment identifier. – *Returns:* `Promise<ForumCommentType>` – the soft-deleted comment.
07	`getForumCommentLists( filter: ForumCommentFilterType)`	A method to query a page of comments and the matching total in one transaction. – `filter`: `ForumCommentFilterType` – page, limit, postId, and optional current user. – *Returns:* `Promise<{ comments: ForumCommentType[]; total: number }>` – mapped comments and total. – `likedByMe` is computed from the optional user reaction.

No	Method	Description
08	`findLikeByUserAndComment( userId: number, commentId: number)`	A method to find a user's comment reaction. – `userId: number` – user identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<{ id: number; postId: number | null } | null>` – reaction id and optional post link.
09	`findLikeByUserAndPost( userId: number, postId: number)`	A method to find a user's post reaction for reuse by the comment-like flow. – `userId: number` – user identifier. – `postId: number` – post identifier. – *Returns:* `Promise<{ id: number; commentId: number | null } | null>` – reaction id and optional comment link.
10	`createCommentLike( userId: number, postId: number, commentId: number)`	A method to create a comment reaction, reusing a post-only row when possible. – `userId: number` – user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* the created or updated `ForumLike` row.
11	`deleteCommentLike( userId: number, commentId: number)`	A method to remove a comment reaction while preserving a post reaction on the same row. – `userId: number` – user identifier. – `commentId: number` – comment identifier. – *Returns:* the updated or deleted `ForumLike` row, or `undefined` when no row exists.

4.9.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for comments

Class / Element	Attributes or Signature	Responsibility
ForumCommentList ResponseDto	comments: ForumCommentType[], pagination: PaginationType.	Paginated comment-list response.
ForumCommentFilter Dto	page: number, limit: number.	Query validation for comment listing.
CreateForumComment Dto	content: string.	Comment-creation body.
CreateForumComment ResponseDto	ForumCommentType: id, postId, user, content, createdAt, updatedAt, like Count, likedByMe.	Comment-creation response.
EditForumCommentDto	content: string.	Comment-update body.
EditForumComment ResponseDto	ForumCommentType.	Comment-update response.
DeleteForumComment ResponseDto	ForumCommentType.	Soft-delete response.
ToggleLikeComment ResponseDto	liked: boolean.	Comment-like toggle response.

4.10 ForumReportController Class

4.10.1 ClassMethod

No	Method	Description
01	ForumReportController(forumReportService: ForumReportService)	A method to initialize the controller with the report service. – forumReportService: ForumReportService – service dependency. – *Returns:* void – constructor, no return value.

No	Method	Description
02	`createPostReport( userId: number, postId: number, body: CreateForumReportType)`	A method to report a post. – `userId: number` – reporter identifier. – `postId: number` – reported post identifier. – `body: CreateForumReportType` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Route: `POST /forums/posts/:postId/reports`; authenticated endpoint.
03	`createCommentReport( userId: number, postId: number, commentId: number, body: CreateForumReportType)`	A method to report a comment. – `userId: number` – reporter identifier. – `postId: number` – parent post identifier. – `commentId: number` – reported comment identifier. – `body: CreateForumReportType` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Route: `POST /forums/posts/:postId/comments/:commentId/reports`; authenticated endpoint.
04	`checkPostReported( userId: number, postId: number)`	A method to check whether the current user has reported a post. – `userId: number` – current user identifier. – `postId: number` – post identifier. – *Returns:* `Promise<{ reported: boolean }>` – true when a matching report exists. – Route: `GET /forums/posts/:postId/is-reported`; authenticated endpoint.
05	`checkCommentReported( userId: number, postId: number, commentId: number)`	A method to check whether the current user has reported a comment. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<{ reported: boolean }>` – true when a matching report exists. – Route: `GET /forums/posts/:postId/comments/:commentId/is-reported`; authenticated endpoint.

No	Method	Description
06	`getForumReportLists( userId: number, roleName: string, page: number, limit: number, status: string, search: string)`	A method to list the report queue for administrators. – `userId`: `number` – current user identifier. – `roleName`: `string` – role checked by the service. – `page`, `limit`: `number` – pagination, defaulted to 1 and 10. – `status`, `search`: `string` – optional status and reason filters. – *Returns:* `Promise<ForumReportListResponseType>` – reports and pagination. – Route: `GET /forums/reports`; administrator endpoint.
07	`updateReportStatus( userId: number, roleName: string, reportId: number, body: UpdateReportStatusType)`	A method to change the status of a report. – `userId`: `number` – administrator identifier recorded on the report. – `roleName`: `string` – role checked by the service. – `reportId`: `number` – report identifier. – `body`: `UpdateReportStatusType` – target status. – *Returns:* `Promise<UpdateReportStatusResponseType>` – the updated report. – Route: `PATCH /forums/reports/:reportId/status`; administrator endpoint.

4.10.2 ClassAttribute

No	Attribute	Type
01	`forumReportService`	`ForumReportService` – private readonly service dependency.

4.11 ForumReportService Class

4.11.1 ClassMethod

No	Method	Description
01	`ForumReportService( forumReportRepository: ForumReportRepository)`	A method to initialize the service with the report repository. – `forumReportRepository`: `ForumReportRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`createPostReport( reporterId: number, postId: number, reason: string)`	A method to validate a post and create its first report by the user. – `reporterId`: `number` – reporter identifier. – `postId`: `number` – reported post identifier. – `reason`: `string` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Throws `ForumPostNotFoundException`, `ForumReportAlreadyExistsException`, or `FailedToCreateForumReportException`.
03	`createCommentReport( reporterId: number, postId: number, commentId: number, reason: string)`	A method to validate a comment and create its first report by the user. – `reporterId`: `number` – reporter identifier. – `postId`: `number` – parent post identifier. – `commentId`: `number` – reported comment identifier. – `reason`: `string` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Throws `ForumCommentNotFoundException`, `ForumReportAlreadyExistsException`, or `FailedToCreateForumReportException`.
04	`checkReportedByUser( reporterId: number, postId: number | null, commentId: number | null)`	A method to find an existing report for the current user and target. – `reporterId`: `number` – reporter identifier. – `postId`: `number | null` – post target, or `null`. – `commentId`: `number | null` – comment target, or `null`. – *Returns:* `Promise<{ id: number } | null>` – the report id when one exists.

No	Method	Description
05	`getForumReportLists( roleName: string, page: number, limit: number, status?: string, search?: string)`	A method to authorize an administrator and return the report queue. – `roleName: string` – must equal `ADMIN`. – `page, limit: number` – pagination. – `status, search: string | undefined` – optional filters. – *Returns:* `Promise<ForumReportListResponse Type>` – reports and pagination. – Throws `ForumReportForbiddenException` or `FailedToLoadForumReportsException`.
06	`updateReportStatus( reportId: number, status: string, adminId: number, role Name: string)`	A method to authorize an administrator and update a post report status. – `reportId: number` – report identifier. – `status: string` – target status. – `adminId: number` – administrator identifier. – `roleName: string` – must equal `ADMIN`. – *Returns:* `Promise<UpdateReportStatusResponse Type>` – the updated report. – Throws `ForumReportForbiddenException`, `Forum ReportNotFoundException`, or `FailedToUpdate ReportStatusException`.

4.11.2 ClassAttribute

No	Attribute	Type
01	`forumReportRepository`	`ForumReportRepository` – private readonly persistence dependency.

4.12 ForumReportRepository Class

4.12.1 ClassMethod

No	Method	Description
01	`ForumReportRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findCommentById(commentId: number)`	A method to check that a non-deleted comment exists and belongs to a post. – `commentId`: `number` – comment identifier. – *Returns:* `Promise<{ id: number; postId: number } | null>` – comment id and parent post id.
04	`findExistingReport( reporterId: number, postId: number | null, commentId: number | null)`	A method to find a duplicate report by the same user and target. – `reporterId`: `number` – reporter identifier. – `postId`: `number | null` – post target, or `null`. – `commentId`: `number | null` – comment target, or `null`. – *Returns:* `Promise<{ id: number } | null>` – the report id when one exists.
05	`createReport( reporterId: number, postId: number | null, commentId: number | null, reason: string)`	A method to insert a report. – `reporterId`: `number` – reporter identifier. – `postId`, `commentId`: `number | null` – exactly one target is populated by the service. – `reason`: `string` – report reason. – *Returns:* `Promise<ForumReportType>` – the created report.
06	`findReportById(id: number)`	A method to load a report by identifier. – `id`: `number` – report identifier. – *Returns:* `Promise<ForumReportType | null>` – the report or `null`.

No	Method	Description
07	`getReportList( page: number, limit: number, status?: string, search?: string)`	A method to query a page of reports and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – `status`: `string | undefined` – optional status filter. – `search`: `string | undefined` – optional case-insensitive reason search. – *Returns:* `Promise<{ reports: ForumReportList ResponseType['reports']; total: number }>` – report rows and total.
08	`updateReportStatus( id: number, status: string, adminId: number | null)`	A method to update a report status and record the reviewing administrator. – `id`: `number` – report identifier. – `status`: `string` – target status, converted to upper case. – `adminId`: `number | null` – reviewing administrator. – *Returns:* `Promise<ForumReportType>` – the updated report.

4.12.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for reports

Class / Element	Attributes or Signature	Responsibility
`CreateForumReportDto`	`reason: string`.	Report-creation body.
`CreateForumReport ResponseDto`	`ForumReportType`: `id`, `reporterId`, `post Id`, `commentId`, `reason`, `status`, `adminId`, `createdAt`.	Report-creation response.
`UpdateReportStatusDto`	`status: PENDING | REVIEWED | RESOL VED | DISMISSED`.	Report-status body.
`UpdateReportStatus ResponseDto`	`ForumReportType`.	Report-status response.

Class / Element	Attributes or Signature	Responsibility
ForumReportList ResponseDto	reports: ForumReportType[] with reporter, post, and comment summaries, plus pagination.	Administrator report queue.

4.13 ProfileController Class

4.13.1 ClassMethod

No	Method	Description
01	ProfileController(profileService: ProfileService)	A method to initialize the controller with its dependencies. – profileService: ProfileService – constructor-injected dependency. – *Returns:* void – constructor, no return value.
02	viewProfile(id: number)	A method to view profile. The implementation delegates to profileService.viewProfile. – id: number – target resource identifier. – *Returns:* FreelancerProfileDetailDto – serialized controller response. – Route: GET /profiles/:id.
03	updateProfile(id: number, currentUserId: number, body: UpdateFreelancerProfileDto)	A method to update profile. The implementation delegates to profileService.updateProfile. – id: number – target resource identifier. – currentUserId: number – authenticated user identifier used for ownership checks. – body: UpdateFreelancerProfileDto – validated HTTP request body. – *Returns:* ProfileRevisionSubmissionDto – serialized controller response. – Route: PUT /profiles/:id.
04	getSkills(id: number)	A method to get skills. The implementation delegates to profileService.getSkills. – id: number – target resource identifier. – *Returns:* Promise of the selected database projection / response object – operation result. – Route: GET /profiles/:id/skills.

No	Method	Description
05	`searchSkillSuggestions( search?: string)`	A method to search skill suggestions. The implementation delegates to `profileService.searchSkill Suggestions`. – `search`: `string` – optional search text. – *Returns:* `Promise<{ id: number; name: string; }[]>` – operation result. – Route: `GET /profiles/skills/suggestions`.
06	`addSkill( id: number, currentUserId: number, body: AddFreelancerSkill Dto)`	A method to add skill. The implementation delegates to `profileService.addSkill`. – `id`: `number` – target resource identifier. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `body`: `AddFreelancerSkillDto` – validated HTTP request body. – *Returns:* `AddFreelancerSkillResponseDto` – serialized controller response. – Route: `POST /profiles/:id/skills`.
07	`deleteSkill( id: number, currentUserId: number)`	A method to delete skill. The implementation delegates to `profileService.deleteSkill`. – `id`: `number` – target resource identifier. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – *Returns:* `Promise<{ id: number; freelancer ProfileId: number; skillId: number; proficiencyLevel: number; }>` – operation result. – Route: `DELETE /profiles/skills/:id`.

4.13.2 ClassAttribute

No	Attribute	Type
01	`profileService`	`ProfileService` – injected application dependency.

4.14 ProfileService Class

4.14.1 ClassMethod

No	Method	Description
01	`ProfileService( profileRepository: ProfileRepository, cloudinary: CloudinaryService, aiCvAnalyzer: AiCvAnalyzerService, skillResolver: SkillResolverService, profileRevisionService: ProfileRevisionService)`	A method to initialize the service with its dependencies. – `profileRepository`: `ProfileRepository` – constructor-injected dependency. – `cloudinary`: `CloudinaryService` – constructor-injected dependency. – `aiCvAnalyzer`: `AiCvAnalyzerService` – constructor-injected dependency. – `skillResolver`: `SkillResolverService` – constructor-injected dependency. – `profileRevisionService`: `ProfileRevisionService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`getOwnedFreelancerProfile( profileId: number, userId: number)`	A method to get owned freelancer profile as an internal helper. The implementation delegates to `profileRepository.findFreelancerProfileById`. – `profileId`: `number` – target profile identifier. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
03	`viewProfile( profileId: number)`	A method to view profile. The implementation delegates to `profileRepository.findFreelancerProfileById`. – `profileId`: `number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
04	`updateProfile( profileId: number, currentUserId: number, dto: UpdateFreelancer ProfileType)`	A method to update profile. The implementation delegates to `profileRepository.findFreelancerProfileBy Id`, `profileRevisionService.requiresManual Review`, `profileRevisionService.submit Freelancer`, `profileRepository.update FreelancerProfile`, `profileRevisionService. directUpdateResult`. – `profileId`: `number` – target profile identifier. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `dto`: `UpdateFreelancerProfileType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.
05	`getSkills( profileId: number)`	A method to get skills. The implementation delegates to `profileRepository.findFreelancerProfileById`, `profileRepository.findSkillsByProfileId`. – `profileId`: `number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
06	`searchSkillSuggestions( search?: string)`	A method to search skill suggestions. The implementation delegates to `profileRepository.searchActive CatalogSkills`. – `search`: `string` – optional search text. – *Returns:* `Promise<{ id: number; name: string; }[]>` – operation result.
07	`addSkill( profileId: number, currentUserId: number, dto: AddFreelancerSkill Type)`	A method to add skill. The implementation delegates to `profileRepository.findFreelancerProfileById`, `profileRepository.findOrCreateCatalogSkill`, `profileRepository.findSkillByProfileIdAnd SkillId`, `profileRepository.addSkillToProfile`. – `profileId`: `number` – target profile identifier. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `dto`: `AddFreelancerSkillType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
08	deleteSkill(skillId: number, currentUserId: number)	A method to delete skill. The implementation delegates to profileRepository.findSkillById, profile Repository.deleteSkill. – skillId: number – catalog or freelancer-skill identifier as required by the method. – currentUserId: number – authenticated user identifier used for ownership checks. – *Returns:* Promise<{ id: number; freelancer ProfileId: number; skillId: number; proficiencyLevel: number; }> – operation result.

4.14.2 ClassAttribute

No	Attribute	Type
01	profileRepository	ProfileRepository – injected persistence gateway.
02	cloudinary	CloudinaryService – injected application dependency.
03	aiCvAnalyzer	AiCvAnalyzerService – injected application dependency.
04	skillResolver	SkillResolverService – injected application dependency.
05	profileRevisionService	ProfileRevisionService – injected application dependency.

4.15 ProfileRepository Class

4.15.1 ClassMethod

No	Method	Description
01	ProfileRepository(prisma: PrismaService)	A method to initialize the repository with its dependencies. – prisma: PrismaService – constructor-injected dependency. – *Returns:* void – constructor, no return value.

No	Method	Description
02	`findFreelancerProfileById( profileId: number)`	A method to find freelancer profile by id. – `profileId: number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
03	`updateFreelancerProfile( profileId: number, data: { displayName: string; title: string; bio?: string | null; availabilityStatus?: AvailabilityStatus; education?: string[] | null; certifications?: string[] | null; languages?: string[] | null; experience?: string[] | null; })`	A method to update freelancer profile. – `profileId: number` – target profile identifier. – `data: { displayName: string; title: string; bio?: string | null; availability Status?: AvailabilityStatus; education?: string[] | null; certifications?: string[] | null; languages?: string[] | null; experience?: string[] | null; }` – values to persist. – *Returns:* `Promise of the selected database projection / response object` – operation result.
04	`findSkillsByProfileId( profileId: number)`	A method to find skills by profile id. – `profileId: number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
05	`searchActiveCatalog Skills( search?: string)`	A method to search active catalog skills. – `search: string` – optional search text. – *Returns:* `Promise<{ id: number; name: string; }[]>` – operation result.
06	`findOrCreateCatalogSkill( name: string)`	A method to find or create catalog skill. – `name: string` – catalog name to resolve. – *Returns:* `Promise<{ id: number; createdAt: Date; updatedAt: Date; deletedAt: Date | null; name: string; description: string | null; slug: string; }>` – operation result.
07	`findSkillByProfileIdAnd SkillId( profileId: number, skillId: number)`	A method to find skill by profile id and skill id. – `profileId: number` – target profile identifier. – `skillId: number` – catalog or freelancer-skill identifier as required by the method. – *Returns:* `Promise<{ id: number; freelancer ProfileId: number; skillId: number; proficiencyLevel: number; } | null>` – operation result.

No	Method	Description
08	addSkillToProfile(profileId: number, skillId: number, proficiencyLevel: number)	A method to add skill to profile. – profileId: number – target profile identifier. – skillId: number – catalog or freelancer-skill identifier as required by the method. – proficiencyLevel: number – freelancer proficiency level. – *Returns:* Promise of the selected database projection / response object – operation result.
09	slugify(value: string)	A method to slugify as an internal helper. – value: string – operation parameter used by the implementation. – *Returns:* string – operation result.
10	generateUniqueSlug(name: string)	A method to generate unique slug as an internal helper. – name: string – catalog name to resolve. – *Returns:* Promise<string> – operation result.
11	findSkillById(skillId: number)	A method to find skill by id. – skillId: number – catalog or freelancer-skill identifier as required by the method. – *Returns:* Promise of the selected database projection / response object – operation result.
12	deleteSkill(skillId: number)	A method to delete skill. – skillId: number – catalog or freelancer-skill identifier as required by the method. – *Returns:* Promise<{ id: number; freelancer ProfileId: number; skillId: number; proficiencyLevel: number; }> – operation result.

4.15.2 ClassAttribute

No	Attribute	Type
01	prisma	PrismaService – injected persistence gateway.

4.16 PortfolioController Class

4.16.1 ClassMethod

No	Method	Description
01	`PortfolioController( portfolioService: PortfolioService)`	A method to initialize the controller with its dependencies. – `portfolioService`: `PortfolioService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`getPortfolios( id: number)`	A method to get portfolios. The implementation delegates to `portfolioService.getPortfolios`. – `id`: `number` – target resource identifier. – *Returns:* `PortfolioItemListResponseDto` – serialized controller response. – Route: `GET /profiles/:id/portfolios`.
03	`addPortfolio( currentUserId: number, id: number, body: AddPortfolioDto)`	A method to add portfolio. The implementation delegates to `portfolioService.addPortfolio`. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `id`: `number` – target resource identifier. – `body`: `AddPortfolioDto` – validated HTTP request body. – *Returns:* `AddPortfolioResponseDto` – serialized controller response. – Route: `POST /profiles/:id/portfolios`.
04	`getPortfolioDetail( id: number)`	A method to get portfolio detail. The implementation delegates to `portfolioService.getPortfolio Detail`. – `id`: `number` – target resource identifier. – *Returns:* `PortfolioItemResponseDto` – serialized controller response. – Route: `GET /profiles/portfolios/:id`.
05	`updatePortfolio( currentUserId: number, id: number, body: UpdatePortfolio Dto)`	A method to update portfolio. The implementation delegates to `portfolioService.updatePortfolio`. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `id`: `number` – target resource identifier. – `body`: `UpdatePortfolioDto` – validated HTTP request body. – *Returns:* `UpdatePortfolioResponseDto` – serialized controller response. – Route: `PUT /profiles/portfolios/:id`.

No	Method	Description
06	`deletePortfolio( currentUserId: number, id: number)`	A method to delete portfolio. The implementation delegates to `portfolioService.deletePortfolio`. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `id`: `number` – target resource identifier. – *Returns:* `Promise<{ message: "Success. PortfolioDeleted"; }>` – operation result. – Route: `DELETE /profiles/portfolios/:id`.

4.16.2 ClassAttribute

No	Attribute	Type
01	`portfolioService`	`PortfolioService` – injected application dependency.

4.17 PortfolioService Class

4.17.1 ClassMethod

No	Method	Description
01	`PortfolioService( portfolioRepository: PortfolioRepository)`	A method to initialize the service with its dependencies. – `portfolioRepository`: `PortfolioRepository` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`getPortfolios( profileId: number)`	A method to get portfolios. The implementation delegates to `portfolioRepository.findFreelancerProfileById`, `portfolioRepository.findPortfoliosByFreelancerProfileId`. – `profileId`: `number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
03	`addPortfolio( profileId: number, currentUserId: number, data: AddPortfolioType)`	A method to add portfolio. The implementation delegates to `portfolioRepository.findFreelancerProfileById`, `portfolioRepository.addPortfolioToProfile`. – `profileId`: `number` – target profile identifier. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `data`: `AddPortfolioType` – values to persist. – *Returns:* `Promise of the selected database projection / response object` – operation result.
04	`getPortfolioDetail( portfolioId: number)`	A method to get portfolio detail. The implementation delegates to `portfolioRepository.findPortfolioById`. – `portfolioId`: `number` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
05	`updatePortfolio( portfolioId: number, currentUserId: number, data: UpdatePortfolioType)`	A method to update portfolio. The implementation delegates to `portfolioRepository.findPortfolioWithProfile`, `portfolioRepository.updatePortfolio`. – `portfolioId`: `number` – operation parameter used by the implementation. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – `data`: `UpdatePortfolioType` – values to persist. – *Returns:* `Promise of the selected database projection / response object` – operation result.
06	`deletePortfolio( portfolioId: number, currentUserId: number)`	A method to delete portfolio. The implementation delegates to `portfolioRepository.findPortfolioWithProfile`, `portfolioRepository.deletePortfolio`. – `portfolioId`: `number` – operation parameter used by the implementation. – `currentUserId`: `number` – authenticated user identifier used for ownership checks. – *Returns:* `Promise<{ message: "Success.PortfolioDeleted"; }>` – operation result.

4.17.2 ClassAttribute

No	Attribute	Type
01	`portfolioRepository`	`PortfolioRepository` – injected persistence gateway.

4.18 PortfolioRepository Class

4.18.1 ClassMethod

No	Method	Description
01	`PortfolioRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma`: `PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`findFreelancerProfileBy Id( profileId: number)`	A method to find freelancer profile by id. – `profileId`: `number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
03	`findPortfoliosBy FreelancerProfileId( freelancerProfileId: number)`	A method to find portfolios by freelancer profile id. – `freelancerProfileId`: `number` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
04	`addPortfolioToProfile( profileId: number, data: AddPortfolioType)`	A method to add portfolio to profile. – `profileId`: `number` – target profile identifier. – `data`: `AddPortfolioType` – values to persist. – *Returns:* `Promise of the selected database projection / response object` – operation result.
05	`findPortfolioById( portfolioId: number)`	A method to find portfolio by id. – `portfolioId`: `number` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
06	`findPortfolioWithProfile( portfolioId: number)`	A method to find portfolio with profile. – `portfolioId: number` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
07	`updatePortfolio( portfolioId: number, data: UpdatePortfolio Type)`	A method to update portfolio. – `portfolioId: number` – operation parameter used by the implementation. – `data: UpdatePortfolioType` – values to persist. – *Returns:* `Promise of the selected database projection / response object` – operation result.
08	`deletePortfolio( portfolioId: number)`	A method to delete portfolio. – `portfolioId: number` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.

4.18.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.19 AuthController Class

4.19.1 ClassMethod

No	Method	Description
01	`AuthController(auth Service: AuthService)`	A method to initialize the controller with the authentication service. – `authService: AuthService` – service that owns the authentication use cases. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`register(body: RegisterDto)`	A method to create an email/password account after OTP verification. – `body`: `RegisterDto` – email, OTP code, password, full name, and initial role. – *Returns:* `Promise<RegisterResponseType>` – the created account without password fields. – Route: `POST /auth/register`; public endpoint.
03	`sendOTP(body: SendOTPBodyDTO)`	A method to generate and email an OTP for registration or password reset. – `body`: `SendOTPBodyDTO` – email and verification-code type. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/otp`; public endpoint.

04	`login(`
	`body: LoginBodyDTO,`
	`userAgent: string, ipAddress: string)`
		A method to authenticate an email/password account and create a session.
		– `body`: `LoginBodyDTO` – email and password.
		– `userAgent`: `string` – device information used as session metadata.
		– `ipAddress`: `string` – client IP stored with the session.
		– *Returns:* `Promise<LoginResponseType>` – access and refresh tokens.
		– Route: `POST /auth/login`; public endpoint.

05	`refreshToken(`
	`body: RefreshTokenBodyDTO,`
	`userAgent: string, ipAddress: string)`
		A method to rotate a valid refresh token into a new token pair.
		– `body.refreshToken`: `string` – the current refresh token.
		– `userAgent`: `string` – device information for the replacement session.
		– `ipAddress`: `string` – client IP for the replacement session.
		– *Returns:* `Promise<LoginResponseType>` – the new access and refresh tokens.
		– Route: `POST /auth/refresh-token`; public endpoint.

No	Method	Description
06	`logout(body: LogoutBodyDTO)`	A method to terminate the session identified by a refresh token. – `body.refreshToken`: `string` – refresh token of the session to remove. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/logout`; public endpoint.
07	`forgotPassword(body: ForgotPasswordBodyDTO)`	A method to reset a password after validating a password-reset OTP. – `body`: `ForgotPasswordBodyDTO` – email, OTP code, and new password. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/forgot-password`; public endpoint.
08	`getGoogleLink( userAgent: string, ip: string)`	A method to build the Google OAuth consent URL. – `userAgent`: `string` – encoded in the OAuth state. – `ip`: `string` – encoded in the OAuth state. – *Returns:* `GetAuthorizationUrlResType` – the authorization URL. – Route: `GET /auth/google-link`; public endpoint.
09	`googleCallback( code: string, state: string, res: Response)`	A method to exchange the Google authorization code and redirect the browser. – `code`: `string` – authorization code returned by Google. – `state`: `string` – base64url payload with device and IP information. – `res`: `Response` – Express response used for the redirect. – *Returns:* `Promise<void>` – redirects to the web application with tokens or an error flag. – Route: `GET /auth/google/callback`; public endpoint.
10	`getMe(userId: number)`	A method to return the authenticated account and its roles. – `userId`: `number` – current user injected by `@UserActive('userId')`. – *Returns:* `Promise<GetMeResType>` – account, profile summary, and roles. – Route: `GET /auth/me`; authenticated endpoint.

No	Method	Description
11	`switchRole( userId: number, sessionId: number, body: SwitchRoleBodyDto)`	A method to make an already assigned role the primary role. – `userId`: `number` – current user. – `sessionId`: `number` – current session identifier. – `body.role`: role name to activate. – *Returns:* `Promise<SwitchRoleResponseType>` – a new access token for that role. – Route: `POST /auth/switch-role`; authenticated endpoint.
12	`joinRole( userId: number, sessionId: number, body: JoinRoleBodyDto)`	A method to assign an additional Client or Freelancer role. – `userId`: `number` – current user. – `sessionId`: `number` – current session identifier. – `body.role`: role name to join. – *Returns:* `Promise<JoinRoleResponseType>` – a new access token for the joined role. – Route: `POST /auth/join-role`; authenticated endpoint.

4.19.2 ClassAttribute

No	Attribute	Type
01	`authService`	`AuthService` – private readonly authentication service.
02	`logger`	`Logger` – private readonly NestJS logger for OAuth callback errors.

4.20 AuthService Class

4.20.1 ClassMethod

No	Method	Description
01	`AuthService( hashingService: HashingService, emailService: EmailService, sharedRoleRepository: SharedRoleRepository, authRepository: AuthRepository, tokenService: TokenService)`	A method to initialize the service and its Google OAuth client. – `hashingService`: `HashingService` – password hashing and verification. – `emailService`: `EmailService` – OTP delivery. – `sharedRoleRepository`: `SharedRoleRepository` – resolves system role identifiers. – `authRepository`: `AuthRepository` – account, OTP, session, and OAuth persistence. – `tokenService`: `TokenService` – JWT signing and verification. – *Returns:* `void` – constructor, no return value.
02	`register(body: RegisterBodyType)`	A method to validate registration data and create an account. – `body`: `RegisterBodyType` – email, OTP, password, full name, and role. – *Returns:* `Promise<UserType>` – the created account without sensitive fields. – Throws `EmailAlreadyExistsException`, `UserBannedException`, `InvalidVerificationCodeException`, or `OTPExpiredException`.
03	`sendOTP(body: SendOTPBodyType)`	A method to issue an OTP while enforcing the attempt window. – `body`: `SendOTPBodyType` – email and verification-code type. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `EmailAlreadyExistsException`, `EmailNotFoundException`, or `TooManyAttemptsException`.
04	`login(body: LoginBodyType and session metadata)`	A method to verify credentials, ban state, and the primary role before issuing tokens. – `body`: `LoginBodyType` – email, password, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the token pair. – Throws `IncorrectEmailException`, `IncorrectPasswordException`, `UserBannedException`, or `RoleNotFoundException`.

No	Method	Description
05	`generateAccessAndRefreshTokens(payload)`	A private method to sign a refresh token, persist a session, and sign the access token. – `payload`: object – user id, role id, role name, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the new token pair. – The access token carries the persisted session id.
06	`refreshToken(payload)`	A method to validate and rotate a refresh token. – `payload`: object – refresh token, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the replacement token pair. – Throws `RefreshTokenRevokedException` or `UserBannedException`.
07	`logout(refreshToken: string)`	A method to remove the session that owns a refresh token. – `refreshToken`: `string` – token of the session to terminate. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `RefreshTokenRevokedException` when the session is absent.
08	`forgotPassword(payload: ForgotPasswordBodyType)`	A method to validate a reset OTP, hash the new password, and remove the OTP. – `payload`: `ForgotPasswordBodyType` – email, OTP, and new password. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `EmailNotFoundException`, `UserBannedException`, or OTP validation errors.
09	`validateVerificationCode(uniqueValue)`	A method to load an OTP and reject missing or expired codes. – `uniqueValue`: object – email, code, and verification type. – *Returns:* `Promise<EmailVerificationType>` – the matching OTP record. – Throws `InvalidVerificationCodeException` or `OTPExpiredException`.

No	Method	Description
10	`getMe(userId: number)`	A method to return the current account, profile summary, and roles. – `userId`: `number` – current account identifier. – *Returns:* `Promise<GetMeResType>` – the authenticated account view. – Throws `NotFoundException` when absent or `UserBannedException` when banned.
11	`switchRole(userId, sessionId, body)`	A method to change the primary role and issue a new access token. – `userId`: `number` – current account. – `sessionId`: `number` – current session. – `body`: `SwitchRoleBodyType` – target role. – *Returns:* `Promise<{ accessToken: string }>` – the role-scoped access token. – Throws `RoleNotFoundException`.
12	`joinRole(userId, sessionId, body)`	A method to assign an additional role and issue a new access token. – `userId`: `number` – current account. – `sessionId`: `number` – current session. – `body`: `JoinRoleBodyType` – role to join. – *Returns:* `Promise<{ accessToken: string }>` – the role-scoped access token. – Throws `RoleNotFoundException` or `RoleAlreadyAssignedException`.
13	`getAuthorizationUrl(payload)`	A method to build the Google OAuth consent URL with a base64url state. – `payload`: object – user agent and IP address. – *Returns:* `GetAuthorizationUrlResType` – the authorization URL.
14	`googleCallback(payload)`	A method to exchange a Google code, create or load the account, and issue tokens. – `payload`: object – Google authorization code and state. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the token pair. – Creates a Client account and an OAuth account on first login; rejects banned accounts.

4.20.2 ClassAttribute

No	Attribute	Type
01	`hashingService`	`HashingService` – private readonly password utility.
02	`emailService`	`EmailService` – private readonly OTP delivery utility.
03	`sharedRoleRepository`	`SharedRoleRepository` – private readonly system-role lookup.
04	`authRepository`	`AuthRepository` – private readonly authentication persistence.
05	`tokenService`	`TokenService` – private readonly JWT utility.
06	`oauth2Client`	`OAuth2Client` – public Google OAuth client.
07	`logger`	`Logger` – private readonly NestJS logger.

4.21 AuthRepository Class

4.21.1 ClassMethod

No	Method	Description
01	`AuthRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findUserByEmail(email: string)`	A method to load a non-deleted account by email. – `email`: `string` – unique account email. – *Returns:* `Promise<UserType | null>` – the account without password timestamps, or `null`.
03	`findUserByEmailIncludeRoles(email: string)`	A method to load an account with its role assignments. – `email`: `string` – unique account email. – *Returns:* `Promise<UserType and roles | null>` – the account and ordered roles, or `null`.

No	Method	Description
04	`findVerificationCode(uniqueValue)`	A method to find an OTP by email, code, and type. – `uniqueValue`: object – email, code, and verification type. – *Returns:* `Promise<EmailVerificationType | null>` – the matching OTP, or `null`.
05	`findVerificationCodeByEmailAndType(email, type)`	A method to load the current OTP record for an email and type. – `email`: `string` – account email. – `type`: `VerificationCodeType` – registration or password-reset code. – *Returns:* `Promise<EmailVerificationType | null>` – the OTP record, or `null`.
06	`createUserAndRegister(deleteOtp, user)`	A method to delete the consumed OTP and create the account in one transaction. – `deleteOtp`: object – OTP code, type, and email. – `user`: object – email, password, full name, and initial role. – *Returns:* `Promise<UserType>` – the created account.
07	`createUserAndRegisterGoogle(user)`	A method to create a Google account with a Client role and profile. – `user`: object – email, full name, and role identifier. – *Returns:* `Promise<UserType and roles>` – the created account with roles.
08	`upsertVerificationCode(data)`	A method to create or replace an OTP record. – `data`: object – email, type, code, expiry, and attempt count. – *Returns:* `Promise<EmailVerificationType>` – the persisted OTP.
09	`findUserForLogin(email: string)`	A method to load an account and ordered roles for authentication. – `email`: `string` – account email. – *Returns:* `Promise<UserType and roles | null>` – the account, or `null`.
10	`createSession(data)`	A method to persist a refresh-token session. – `data`: object – user id, refresh token, device, IP address, and expiry. – *Returns:* `Promise<Session>` – the created session with its identifier.

No	Method	Description
11	`deleteSessionByRefresh Token(data)`	A method to delete a session by refresh token and user id. – `data`: object – refresh token and user id. – *Returns:* `Promise<Session>` – the deleted session.
12	`findUniqueRefreshToken IncludeUserRole(data)`	A method to load a session, its account, and the account roles by refresh token. – `data.refreshToken`: `string` – refresh token to resolve. – *Returns:* `Promise<Session and user roles | null>` – the session graph, or `null`.
13	`update(where, data)`	A method to update a non-deleted account. – `where`: object – account identifier. – `data`: `Partial<UserType>` – fields to change. – *Returns:* `Promise<UserType | null>` – the updated account.
14	`deleteVerifycationCode( uniqueValue)`	A method to delete an OTP after a successful password reset. – `uniqueValue`: object – email and verification type. – *Returns:* `Promise<EmailVerificationType>` – the deleted OTP.
15	`findUserById(userId: number)`	A method to load an account with profile and ordered roles. – `userId`: `number` – account identifier. – *Returns:* `Promise<UserType and profile and roles | null>` – the account view, or `null`.
16	`switchPrimaryRole(userId, roleName)`	A method to make an assigned role primary in one transaction. – `userId`: `number` – account identifier. – `roleName`: `string` – role to activate. – *Returns:* `Promise<RoleType | null>` – the activated role, or `null` when not assigned.
17	`joinRole(userId, role Name)`	A method to assign a Client or Freelancer role and create its profile row. – `userId`: `number` – account identifier. – `roleName`: `JoinRoleBodyType['role']` – role to join. – *Returns:* `Promise<{ status: 'joined' | ' already-assigned'; role: RoleType } | null>` – the assignment result, or `null` when the role is absent.

No	Method	Description
18	`createOauthAccount(data)`	A method to link a Google identity to an account. – `data`: object – user id, provider, and provider user id. – *Returns:* `Promise<OauthAccount>` – the created OAuth link.

4.21.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.22 RolesController Class

4.22.1 ClassMethod

No	Method	Description
01	`RolesController(rolesService: RolesService)`	A method to initialize the controller with the role service. – `rolesService`: `RolesService` – service that owns role use cases. – *Returns:* `void` – constructor, no return value.
02	`getRoles()`	A method to return all active roles. – *Returns:* `Promise<RoleListResponseType>` – the role list. – Route: `GET /roles`.
03	`getRoleById(id: number)`	A method to return one role with its permissions. – `id`: `number` – role identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Route: `GET /roles/:id`.
04	`createRole(body: CreateRoleBodyDto)`	A method to create a custom role. – `body`: `CreateRoleBodyDto` – role name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role. – Route: `POST /roles`.

No	Method	Description
05	`updateRole(id: number, body: UpdateRoleBodyDto)`	A method to update a role's name or description. – `id`: `number` – role identifier. – `body`: `UpdateRoleBodyDto` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role. – Route: `PATCH /roles/:id`.
06	`setRolePermissions( id: number, body: SetRolePermissions BodyDto)`	A method to replace every permission assigned to a role. – `id`: `number` – role identifier. – `body.permissionIds`: `number[]` – the complete replacement set. – *Returns:* `Promise<SetRolePermissionsResponse Type>` – the updated role detail. – Route: `PUT /roles/:id/permissions`.
07	`deleteRole(id: number)`	A method to soft-delete an unused custom role. – `id`: `number` – role identifier. – *Returns:* `Promise<DeleteRoleResponseType>` – a success message. – Route: `DELETE /roles/:id`.

4.22.2 ClassAttribute

No	Attribute	Type
01	`rolesService`	`RolesService` – private readonly role service.

4.23 RolesService Class

4.23.1 ClassMethod

No	Method	Description
01	`RolesService(roles Repository: Roles Repository)`	A method to initialize the service with the role repository. – `rolesRepository`: `RolesRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`isSystemRoleName(name: string)`	A private method to detect the four built-in role names. – `name: string` – role name to test. – *Returns:* `boolean` – true for Admin, Client, Freelancer, or Expert.
03	`getRoles()`	A method to load the active role list. – *Returns:* `Promise<RoleListResponseType>` – the role list. – Failure is mapped to `FailedToLoadRolesException`.
04	`getRoleById(id: number)`	A method to load a role and its permissions. – `id: number` – role identifier. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Prisma errors are mapped to `FailedToLoadRoleDetailException`.
05	`createRole(body: CreateRoleBodyType)`	A method to validate the name and create a custom role. – `body: CreateRoleBodyType` – role name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role. – Throws `CannotModifySystemRoleException` or `RoleAlreadyExistsException`.
06	`updateRole(id: number, body: UpdateRoleBodyType)`	A method to validate system-role and duplicate-name rules before updating. – `id: number` – role identifier. – `body: UpdateRoleBodyType` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role. – Throws `CannotModifySystemRoleException` or `RoleAlreadyExistsException`.
07	`deleteRole(id: number)`	A method to soft-delete a custom role that is not assigned to any user. – `id: number` – role identifier. – *Returns:* `Promise<DeleteRoleResponseType>` – a success message. – Throws `CannotModifySystemRoleException` or `RoleInUseException`.

No	Method	Description
08	`setRolePermissions(id, body)`	A method to validate permission ids and replace the role permission set. – `id`: `number` – role identifier. – `body`: `SetRolePermissionsBodyType` – permission ids to assign. – *Returns:* `Promise<SetRolePermissionsResponseType>` – the updated role detail. – Throws `InvalidPermissionIdsException`.

4.23.2 ClassAttribute

No	Attribute	Type
01	`rolesRepository`	`RolesRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.24 RolesRepository Class

4.24.1 ClassMethod

No	Method	Description
01	`RolesRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAll()`	A method to load all non-deleted roles ordered by id. – *Returns:* `Promise<RoleListItemType[]>` – the role list.
03	`findById(id: number)`	A method to load a role and its active permissions. – `id`: `number` – role identifier. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Throws `RoleNotFoundException`.

No	Method	Description
04	`findActiveByName(name, excludeId?)`	A method to find an active role by case-insensitive name. – `name`: `string` – role name. – `excludeId`: `number | undefined` – optional role to exclude. – *Returns:* `Promise<RoleListItemType | null>` – the matching role, or `null`.
05	`createRole(body: CreateRoleBodyType)`	A method to insert a role. – `body`: `CreateRoleBodyType` – name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role.
06	`updateRole(id: number, body: UpdateRoleBodyType)`	A method to update the provided role fields. – `id`: `number` – role identifier. – `body`: `UpdateRoleBodyType` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role.
07	`softDeleteRole(id: number)`	A method to set `deletedAt` on a role. – `id`: `number` – role identifier. – *Returns:* `Promise<void>` – no value.
08	`countAssignedUsers(roleId: number)`	A method to count user-role assignments. – `roleId`: `number` – role identifier. – *Returns:* `Promise<number>` – the assignment count.
09	`findActivePermissionIds(ids: number[])`	A method to filter permission ids down to active permissions. – `ids`: `number[]` – requested permission ids. – *Returns:* `Promise<number[]>` – the active ids.
10	`replaceRolePermissions(roleId, permissionIds)`	A method to delete the old role permissions and insert the new set in a transaction. – `roleId`: `number` – role identifier. – `permissionIds`: `number[]` – replacement permission ids. – *Returns:* `Promise<void>` – no value.

4.24.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.25 ConversationController Class

4.25.1 ClassMethod

No	Method	Description
01	`ConversationController( conversationService: ConversationService)`	A method to initialize the controller with the conversation service. – `conversationService`: `ConversationService` – service that owns all conversation use cases. – *Returns:* `void` – constructor, no return value.
02	`getConversations(userId: number)`	A method to return the current user's conversation list. – `userId`: `number` – current user identifier injected by `@UserActive('userId')`. – *Returns:* `Promise<GetConversationsResponseType>` – conversation items with other user, last message, unread count, and pin time. – Route: `GET /conversations`; authenticated endpoint.
03	`createConversation( userId: number, body: CreateConversationBodyType)`	A method to create or reopen a conversation with another user. – `userId`: `number` – current user identifier. – `body.participantId`: `number` – the other participant, validated by `CreateConversationBodyDto`. – *Returns:* `Promise<CreateConversationResponseType>` – id, participant1, participant2, and createdAt. – Route: `POST /conversations`; authenticated endpoint.

No	Method	Description
04	`getMessages( userId: number, id: number)`	A method to return the message history of one conversation. – `userId`: `number` – current user identifier. – `id`: `number` – conversation identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<DirectMessageType[]>` – non-deleted messages in ascending creation order. – Route: `GET /conversations/:id/messages`; authenticated endpoint.
05	`sendMessage( userId: number, id: number, body: SendMessageBodyType)`	A method to send a message through the REST fallback. – `userId`: `number` – sender identifier. – `id`: `number` – conversation identifier. – `body`: `SendMessageBodyType` – message text and optional attachment, validated by `SendMessageBodyDto`. – *Returns:* `Promise<DirectMessageType>` – the saved message. – Route: `POST /conversations/:id/messages`; authenticated endpoint.
06	`uploadFile( userId: number, id: number, file?: Express.Multer.File)`	A method to upload a message attachment for a conversation participant. – `userId`: `number` – current user identifier. – `id`: `number` – conversation identifier. – `file`: `Express.Multer.File | undefined` – multipart file read into memory by `FileInterceptor`. – *Returns:* `Promise<MessageAttachmentType>` – Cloudinary URL, original name, byte size, and MIME type. – Route: `POST /conversations/:id/files`; authenticated endpoint.
07	`deleteMessage( userId: number, id: number, messageId: number)`	A method to soft-delete a message sent by the current user. – `userId`: `number` – current user identifier. – `id`: `number` – conversation identifier. – `messageId`: `number` – message identifier. – *Returns:* `Promise<DeleteMessageResponseType>` – the deleted message id. – Route: `DELETE /conversations/:id/messages/:messageId`; authenticated endpoint.

No	Method	Description
08	`markAsRead( userId: number, id: number)`	A method to mark all messages from the other participant as read. – `userId`: `number` – current user identifier. – `id`: `number` – conversation identifier. – *Returns:* `Promise<{ count: number }>` – number of updated messages. This route has no response DTO decorator. – Route: `PATCH /conversations/:id/read`; authenticated endpoint.
09	`hideConversation( userId: number, id: number)`	A method to hide a conversation only for the current user. – `userId`: `number` – current user identifier. – `id`: `number` – conversation identifier. – *Returns:* `Promise<HideConversationResponse Type>` – id and hidden flag. – Route: `PATCH /conversations/:id/hide`; authenticated endpoint.
10	`pinConversation( userId: number, id: number, body: PinConversation BodyType)`	A method to pin or unpin a conversation for the current user. – `userId`: `number` – current user identifier. – `id`: `number` – conversation identifier. – `body.pinned`: `boolean` – target pin state, validated by `PinConversationBodyDto`. – *Returns:* `Promise<PinConversationResponse Type>` – id and the new `pinnedAt` value. – Route: `PATCH /conversations/:id/pin`; authenticated endpoint.

4.25.2 ClassAttribute

No	Attribute	Type
01	`conversationService`	`ConversationService` – private readonly service dependency.

4.26 ConversationService Class

4.26.1 ClassMethod

No	Method	Description
01	`ConversationService( conversationRepository: ConversationRepository)`	A method to initialize the service with the conversation repository. – `conversationRepository`: `Conversation Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`buildConversationItem( conversation: RawConversationItem, userId: number)`	A private method to map a repository row to the public conversation item. – `conversation`: `RawConversationItem` – row with both participants, last message, and unread count. – `userId`: `number` – current user, used to select the other participant and pin time. – *Returns:* `ConversationListItemType` – normalized item with `otherUser`, `lastMessage`, `unreadCount`, and `pinnedAt`.
03	`getConversations(userId: number)`	A method to load and sort all visible conversations of a user. – `userId`: `number` – current user identifier. – *Returns:* `Promise<GetConversationsResponse Type>` – pinned conversations first, then by last-message time or creation time. – Any failure is mapped to `FailedToLoad ConversationsException`.
04	`getConversationItem( conversationId: number, userId: number)`	A method to rebuild one conversation item for a realtime update. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – recipient of the realtime item. – *Returns:* `Promise<ConversationListItemType | null>` – the item, or `null` when the conversation is not visible to that user.
05	`getMessages( userId: number, conversationId: number)`	A method to authorize a participant and return the message history. – `userId`: `number` – current user identifier. – `conversationId`: `number` – conversation identifier. – *Returns:* `Promise<DirectMessageType[]>` – non-deleted messages. – Throws `ConversationNotFoundException` when the user is not a participant; other failures become `FailedTo LoadConversationsException`.

No	Method	Description
06	createConversation(userId: number, participantId: number)	A method to create a conversation or return the existing pair. – userId: number – current user identifier. – participantId: number – the other user. – *Returns:* Promise<CreateConversationResponse Type> – id, participant1, participant2, and createdAt. – Rejects self-conversation with CannotStart ConversationWithSelfException; an existing hidden conversation is unhidden.
07	sendMessage(userId: number, conversationId: number, message: string, attachment?: Message AttachmentType | null)	A method to save a message and unhide the conversation for both participants. – userId: number – sender identifier. – conversationId: number – conversation identifier. – message: string – text; may be empty when an attachment exists. – attachment: MessageAttachmentType | null | undefined – optional file metadata. – *Returns:* Promise<DirectMessageType> – the saved message. – Throws ConversationNotFoundException or FailedToSendMessageException.
08	uploadFile(userId: number, conversationId: number, file?: Express.Multer. File)	A method to validate participation and upload a file to Cloudinary. – userId: number – current user identifier. – conversationId: number – conversation identifier. – file: Express.Multer.File | undefined – in-memory multipart file. – *Returns:* Promise<MessageAttachmentType> – secure URL, original name, byte size, and MIME type. – Throws FailedToUploadFileException, ConversationNotFoundException, or passes through a Nest HttpException.

No	Method	Description
09	`deleteMessage( userId: number, conversationId: number, messageId: number)`	A method to authorize a participant and soft-delete the sender's message. – `userId: number` – current user identifier; must be the sender. – `conversationId: number` – conversation identifier. – `messageId: number` – message identifier. – *Returns:* `Promise<{ id: number }>` – the deleted message id. – Throws `ConversationNotFoundException`, `MessageNotFoundException`, or `FailedToDeleteMessageException`.
10	`markAsRead( userId: number, conversationId: number)`	A method to mark messages from the other participant as read. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<{ count: number }>` – number of updated messages. – Throws `ConversationNotFoundException` or `FailedToMarkAsReadException`.
11	`hideConversation( userId: number, conversationId: number)`	A method to set the current participant's hidden timestamp. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<HideConversationResponseType>` – id and hidden flag. – Throws `ConversationNotFoundException` or `FailedToHideConversationException`.
12	`pinConversation( userId: number, conversationId: number, pinned: boolean)`	A method to set or clear the current participant's pin timestamp. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – `pinned: boolean` – true to pin, false to unpin. – *Returns:* `Promise<PinConversationResponseType>` – id and `pinnedAt`. – Throws `ConversationNotFoundException` or `FailedToPinConversationException`.

No	Method	Description
13	`ensureParticipant( conversationId: number, userId: number)`	A method used by the gateway to assert that a socket user belongs to a conversation. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – socket user identifier. – *Returns:* the non-deleted `Conversation` row. – Throws `ConversationAccessDeniedException` when the user is not a participant.

4.26.2 ClassAttribute

No	Attribute	Type
01	`conversationRepository`	`ConversationRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger; records attachment-upload failures.

4.27 ConversationRepository Class

4.27.1 ClassMethod

No	Method	Description
01	`ConversationRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findConversationBetween( userId1: number, user Id2: number)`	A method to find the non-deleted conversation for a user pair in either participant order. – `userId1`, `userId2`: `number` – the two users. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.

No	Method	Description
03	`getConversationsByUserId( userId: number)`	A method to query all conversations visible to a user with previews and unread counts. – `userId: number` – current user identifier. – *Returns:* conversation rows with both participant profiles, the latest non-deleted message, and a filtered message count. – A conversation is included only when the current participant's hidden timestamp is `null`.
04	`findConversationByIdFor User( conversationId: number, userId: number)`	A method to load a non-deleted conversation only when the user is a participant. – `conversationId: number` – conversation identifier. – `userId: number` – current user identifier. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.
05	`findConversationById( conversationId: number)`	A method to load a non-deleted conversation without a participant check. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.
06	`hideConversationForUser( conversationId: number, userId: number)`	A method to set the hidden timestamp of the participant who requested the hide. – `conversationId: number` – conversation identifier. – `userId: number` – participant identifier. – *Returns:* `Promise<boolean>` – true when the row was updated, false when the conversation or participant does not match.
07	`unhideConversation( conversationId: number)`	A method to clear both participants' hidden timestamps after new activity. – `conversationId: number` – conversation identifier. – *Returns:* the updated `Conversation` row.
08	`setPinnedForUser( conversationId: number, userId: number, pinned: boolean)`	A method to set or clear the participant's pin timestamp. – `conversationId: number` – conversation identifier. – `userId: number` – participant identifier. – `pinned: boolean` – true sets the current time, false clears it. – *Returns:* `Promise<Date | null>` – the new timestamp, or `null` when unpinned or not a participant.

No	Method	Description
09	`getMessages(conversationId: number)`	A method to query all non-deleted messages of a conversation in ascending time order. – `conversationId`: `number` – conversation identifier. – *Returns:* `Promise<DirectMessage[]>` – Prisma message rows.
10	`createConversation(participant1: number, participant2: number)`	A method to insert a conversation pair. – `participant1`: `number` – first participant (the creator in the service flow). – `participant2`: `number` – second participant. – *Returns:* `Promise<Conversation>` – the inserted row.
11	`createMessage(conversationId: number, senderId: number, message: string, attachment?: MessageAttachmentType)`	A method to insert a message and copy optional attachment metadata. – `conversationId`: `number` – conversation identifier. – `senderId`: `number` – sender identifier. – `message`: `string` – message text. – `attachment`: `MessageAttachmentType | undefined` – optional file metadata. – *Returns:* `Promise<DirectMessage>` – the inserted message.
12	`softDeleteMessage(conversationId: number, messageId: number, senderId: number)`	A method to set `deletedAt` only when the message belongs to the conversation and sender. – `conversationId`: `number` – conversation identifier. – `messageId`: `number` – message identifier. – `senderId`: `number` – sender identifier. – *Returns:* `Promise<boolean>` – true when one non-deleted row was updated.
13	`markMessagesAsRead(conversationId: number, userId: number)`	A method to mark messages from the other participant as read. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – current user; messages sent by this user are excluded. – *Returns:* the Prisma batch payload with the updated row count.
14	`countUnread(conversationId: number, userId: number)`	A method to count unread messages from the other participant. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – current user. – *Returns:* `Promise<number>` – count of non-deleted, unread messages not sent by the user.

4.27.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.28 ConversationsGateway Class

4.28.1 ClassMethod

No	Method	Description
01	`ConversationsGateway( conversationService: ConversationService, tokenService: TokenService)`	A method to initialize the gateway dependencies. – `conversationService`: `ConversationService` – shared use-case service. – `tokenService`: `TokenService` – verifies the access token during the socket handshake. – *Returns:* `void` – constructor, no return value.
02	`handleConnection(client: Socket)`	A method to authenticate a new socket and join its user room. – `client`: `Socket` – connecting Socket.IO client. – Reads `client.handshake.auth.token`, falling back to the bearer token in the authorization header; verifies it with `TokenService.verifyAccessToken`. – *Returns:* `Promise<void>` – stores `userId` in `client.data` and joins `user:<userId>`; disconnects on failure.
03	`handleDisconnect(client: Socket)`	A method to log a disconnected socket. – `client`: `Socket` – disconnected client. – *Returns:* `void` – no return value.
04	`handleSendMessage( client: Socket, payload: unknown)`	A method to handle the `message:send` event. – `client`: `Socket` – sender socket. – `payload`: `unknown` – validated by `SendMessage PayloadSchema`. – Ensures participation, saves the message, emits `message:new` to both user rooms, and refreshes both conversation lists. – *Returns:* `Promise<{ success: boolean; message?: DirectMessageType }>` – success flag and saved message; emits `message:error` on failure.

No	Method	Description
05	handleMarkAsRead(client: Socket, payload: unknown)	A method to handle the message:read event. – client: Socket – reader socket. – payload: unknown – validated by MarkAsRead PayloadSchema. – Calls markAsRead and emits conversation:updated to both participants. – *Returns:* Promise<{ success: boolean }> – success flag.
06	handleDeleteMessage(client: Socket, payload: unknown)	A method to handle the message:delete event. – client: Socket – sender socket. – payload: unknown – validated by DeleteMessage PayloadSchema. – Calls deleteMessage, emits message:deleted to both user rooms, and refreshes both conversation lists. – *Returns:* Promise<{ success: boolean }> – success flag; emits message:error on failure.
07	handleHideConversation(client: Socket, payload: unknown)	A method to handle the conversation:hide event. – client: Socket – current user's socket. – payload: unknown – validated by HideConversation PayloadSchema. – Calls hideConversation and emits conversation: hidden only to the current user's room. – *Returns:* Promise<{ success: boolean }> – success flag.
08	handleTyping(client: Socket, payload: unknown)	A method to handle the typing event. – client: Socket – sender socket. – payload: unknown – validated by TypingPayload Schema. – Ensures participation and forwards typing with conversationId and isTyping only to the other user's room. – *Returns:* Promise<{ success: boolean }> – success flag.

No	Method	Description
09	`emitConversationUpdated( conversationId: number, userId: number)`	A private method to push the latest conversation item to one user. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – target user. – *Returns:* `Promise<void>` – emits `conversation: updated` when `getConversationItem` returns a visible item.

4.28.2 ClassAttribute

No	Attribute	Type
01	`server`	`Server` – Socket.IO server injected by `@WebSocketServer()`; used to address user rooms.
02	`logger`	`Logger` – private readonly NestJS logger for connection and event failures.
03	`conversationService`	`ConversationService` – private readonly service dependency.
04	`tokenService`	`TokenService` – private readonly access-token verifier.

4.29 IdentityVerificationController Class

4.29.1 ClassMethod

No	Method	Description
01	`IdentityVerification Controller( service: AccountProfile Service)`	A method to initialize the controller with its dependencies. – `service`: `AccountProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`upload( userId: number, body: UploadIdentity DocumentDto, file?: Express.Multer. File)`	A method to upload. The implementation delegates to `service.uploadIdentityDocument`. – `userId`: `number` – account identifier used by this operation. – `body`: `UploadIdentityDocumentDto` – validated HTTP request body. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `IdentityVerificationDocumentDto` – serialized controller response. – Route: `POST /identity-verifications/ documents`.
03	`aiCheck( userId: number, body: UploadIdentity DocumentDto, files?: Express.Multer. File[])`	A method to ai check. The implementation delegates to `service.submitIdentityAiCheck`. – `userId`: `number` – account identifier used by this operation. – `body`: `UploadIdentityDocumentDto` – validated HTTP request body. – `files`: `Express.Multer.File[]` – multipart identity-verification images and liveness frames. – *Returns:* `IdentityVerificationDocumentDto` – serialized controller response. – Route: `POST /identity-verifications/ai-check`.
04	`livenessPose( userId: number, file?: Express.Multer. File)`	A method to liveness pose. The implementation delegates to `service.livenessPose`. – `userId`: `number` – account identifier used by this operation. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `Promise<IdentityAiPoseResult>` – operation result. – Route: `POST /identity-verifications/ liveness-pose`.

No	Method	Description
05	`getStatus( userId: number)`	A method to get status. The implementation delegates to `service.getIdentityStatus`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `IdentityVerificationStatusDto` – serialized controller response. – Route: `GET /identity-verifications/status`.
06	`getFile( userId: number, id: number, side: string | undefined, response: Response)`	A method to get file. The implementation delegates to `service.getIdentityDocumentFile`. – `userId`: `number` – account identifier used by this operation. – `id`: `number` – target resource identifier. – `side`: `string | undefined` – operation parameter used by the implementation. – `response`: `Response` – operation parameter used by the implementation. – *Returns:* `Promise<StreamableFile>` – operation result. – Route: `GET /identity-verifications/ documents/:id/file`.

4.29.2 ClassAttribute

No	Attribute	Type
01	`service`	`AccountProfileService` – injected application dependency.

4.30 ClientProfileController Class

4.30.1 ClassMethod

No	Method	Description
01	`ClientProfileController( service: AccountProfile Service)`	A method to initialize the controller with its dependencies. – `service`: `AccountProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getDetail( userId: number)`	A method to get detail. The implementation delegates to `service.getClientProfile`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `ClientProfileDetailDto` – serialized controller response. – Route: `GET /clients/:userId`.
03	`update( userId: number, body: UpdateClient ProfileDto)`	A method to update. The implementation delegates to `service.updateClientProfile`. – `userId`: `number` – account identifier used by this operation. – `body`: `UpdateClientProfileDto` – validated HTTP request body. – *Returns:* `ProfileRevisionSubmissionDto` – serialized controller response. – Route: `PUT /clients/me/profile`.

4.30.2 ClassAttribute

No	Attribute	Type
01	`service`	`AccountProfileService` – injected application dependency.

4.31 SocialLinkController Class

4.31.1 ClassMethod

No	Method	Description
01	`SocialLinkController( service: AccountProfile Service)`	A method to initialize the controller with its dependencies. – `service`: `AccountProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getAll( userId: number)`	A method to get all. The implementation delegates to `service.getSocialLinks`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `SocialLinkListDto` – serialized controller response. – Route: `GET /social-links`.
03	`add( userId: number, body: AddSocialLinkDto)`	A method to add. The implementation delegates to `service.addSocialLink`. – `userId`: `number` – account identifier used by this operation. – `body`: `AddSocialLinkDto` – validated HTTP request body. – *Returns:* `SocialLinkDto` – serialized controller response. – Route: `POST /social-links`.
04	`remove( userId: number, id: number)`	A method to remove. The implementation delegates to `service.deleteSocialLink`. – `userId`: `number` – account identifier used by this operation. – `id`: `number` – target resource identifier. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `DELETE /social-links/:id`.

4.31.2 ClassAttribute

No	Attribute	Type
01	`service`	`AccountProfileService` – injected application dependency.

4.32 FavoriteFreelancerController Class

4.32.1 ClassMethod

No	Method	Description
01	`FavoriteFreelancer Controller( service: AccountProfile Service)`	A method to initialize the controller with its dependencies. – `service`: `AccountProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`getAll( userId: number)`	A method to get all. The implementation delegates to `service.getFavorites`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `FavoriteFreelancerListDto` – serialized controller response. – Route: `GET /favorites/freelancers`.
03	`add( userId: number, freelancerId: number)`	A method to add. The implementation delegates to `service.addFavorite`. – `userId`: `number` – account identifier used by this operation. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `POST /favorites/freelancers/: freelancerId`.
04	`remove( userId: number, freelancerId: number)`	A method to remove. The implementation delegates to `service.removeFavorite`. – `userId`: `number` – account identifier used by this operation. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `DELETE /favorites/freelancers/: freelancerId`.

4.32.2 ClassAttribute

No	Attribute	Type
01	`service`	`AccountProfileService` – injected application dependency.

4.33 FollowingFreelancerController Class

4.33.1 ClassMethod

No	Method	Description
01	`FollowingFreelancer Controller( service: AccountProfile Service)`	A method to initialize the controller with its dependencies. – `service`: `AccountProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`followFreelancer( userId: number, freelancerId: number)`	A method to follow freelancer. The implementation delegates to `service.followFreelancer`. – `userId`: `number` – account identifier used by this operation. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `POST /following/freelancers/: freelancerId`.
03	`unfollowFreelancer( userId: number, freelancerId: number)`	A method to unfollow freelancer. The implementation delegates to `service.unfollowFreelancer`. – `userId`: `number` – account identifier used by this operation. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `DELETE /following/freelancers/: freelancerId`.

4.33.2 ClassAttribute

No	Attribute	Type
01	`service`	`AccountProfileService` – injected application dependency.

4.34 AccountProfileService Class

4.34.1 ClassMethod

No	Method	Description
01	`AccountProfileService( repository: AccountProfileRepository, profileRevisionService: ProfileRevisionService, cloudinary: CloudinaryService, hashing: HashingService, identityAi: IdentityAiService)`	A method to initialize the service with its dependencies. – `repository`: `AccountProfileRepository` – constructor-injected dependency. – `profileRevisionService`: `ProfileRevisionService` – constructor-injected dependency. – `cloudinary`: `CloudinaryService` – constructor-injected dependency. – `hashing`: `HashingService` – constructor-injected dependency. – `identityAi`: `IdentityAiService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`requireRole( userId: number, roleName: string)`	A method to require role as an internal helper. The implementation delegates to `repository.findUserWithRoles`. – `userId`: `number` – account identifier used by this operation. – `roleName`: `string` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
03	`normalizeIdentityNumber( value: unknown)`	A method to normalize identity number as an internal helper. – `value`: `unknown` – operation parameter used by the implementation. – *Returns:* `string | null` – operation result.
04	`presentQrConsistency( summary: unknown)`	A method to present qr consistency as an internal helper. – `summary`: `unknown` – operation parameter used by the implementation. – *Returns:* `{ status: string; score: number | null; matchedFields: string[]; mismatchedFields: string[]; notCheckedFields: string[]; } | null` – operation result.

No	Method	Description
05	`presentIdentityDocument( document: T)`	A method to present identity document as an internal helper. – `document`: `T` – operation parameter used by the implementation. – *Returns:* `Selected database projection / response object` – operation result.
06	`persistIdentityFile( userId: number, file: Express.Multer. File)`	A method to persist identity file as an internal helper. The implementation delegates to `cloudinary.is Configured`, `cloudinary.uploadFile`. – `userId`: `number` – account identifier used by this operation. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `Promise<string>` – operation result.
07	`uploadIdentityDocument( userId: number, documentType: Document TypeType, file?: Express.Multer. File)`	A method to upload identity document. The implementation delegates to `repository.create IdentityDocument`. – `userId`: `number` – account identifier used by this operation. – `documentType`: `DocumentTypeType` – operation parameter used by the implementation. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `Promise of the selected database projection / response object` – operation result.
08	`assertAiImage( file: Express.Multer. File, path: string)`	A method to assert ai image as an internal helper. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – `path`: `string` – operation parameter used by the implementation. – *Returns:* `void` – operation result.

No	Method	Description
09	`submitIdentityAiCheck( userId: number, documentType: DocumentTypeType, files: { front?: Express.Multer.File; back?: Express.Multer.File; selfie?: Express.Multer.File; liveness: Express.Multer.File[]; })`	A method to submit identity ai check. The implementation delegates to `repository.findIdentityDocuments`, `identityAi.verify`, `repository.findApprovedIdentityDocumentByIdentityNumber`, `repository.createIdentityAiDocument`. – `userId`: `number` – account identifier used by this operation. – `documentType`: `DocumentTypeType` – operation parameter used by the implementation. – `files`: `{ front?: Express.Multer.File; back?: Express.Multer.File; selfie?: Express.Multer.File; liveness: Express.Multer.File[]; }` – multipart identity-verification images and liveness frames. – *Returns:* `Promise of the selected database projection / response object` – operation result.
10	`livenessPose( userId: number, file?: Express.Multer.File)`	A method to liveness pose. The implementation delegates to `identityAi.livenessPose`. – `userId`: `number` – account identifier used by this operation. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `Promise<IdentityAiPoseResult>` – operation result.
11	`getIdentityStatus( userId: number)`	A method to get identity status. The implementation delegates to `repository.findIdentityDocuments`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
12	`getIdentityDocumentFile( userId: number, id: number, side: 'front' | 'back')`	A method to get identity document file. The implementation delegates to `repository.findIdentityDocumentById`. – `userId: number` – account identifier used by this operation. – `id: number` – target resource identifier. – `side: 'front' | 'back'` – operation parameter used by the implementation. – *Returns:* `Promise<{ absolutePath: string; fileName: string; }>` – operation result.
13	`getClientProfile( userId: number)`	A method to get client profile. The implementation delegates to `repository.findClientProfile`. – `userId: number` – account identifier used by this operation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
14	`updateClientProfile( userId: number, input: UpdateClientProfileType)`	A method to update client profile. The implementation delegates to `profileRevisionService.requiresManualReview`, `profileRevisionService.submitClient`, `repository.updateClientProfile`, `profileRevisionService.directUpdateResult`. – `userId: number` – account identifier used by this operation. – `input: UpdateClientProfileType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.
15	`getSocialLinks( userId: number)`	A method to get social links. The implementation delegates to `repository.findSocialLinks`. – `userId: number` – account identifier used by this operation. – *Returns:* `Promise<{ id: number; profileId: number; url: string; platform: SocialPlatform; }[]>` – operation result.

No	Method	Description
16	`addSocialLink( userId: number, input: AddSocialLink Type)`	A method to add social link. The implementation delegates to `repository.findSocialLinks`, `repository.findSocialLinkByPlatform`, `repository.createSocialLink`. – `userId: number` – account identifier used by this operation. – `input: AddSocialLinkType` – validated operation input. – *Returns:* `Promise<{ id: number; profileId: number; url: string; platform: Social Platform; }>` – operation result.
17	`deleteSocialLink( userId: number, id: number)`	A method to delete social link. The implementation delegates to `repository.findSocialLinkById`, `repository.deleteSocialLink`. – `userId: number` – account identifier used by this operation. – `id: number` – target resource identifier. – *Returns:* `Promise<{ message: string; }>` – operation result.
18	`getFavorites( clientId: number)`	A method to get favorites. The implementation delegates to `repository.findFavorites`. – `clientId: number` – client account identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
19	`addFavorite( clientId: number, freelancerId: number)`	A method to add favorite. The implementation delegates to `repository.findUserWithRoles`, `repository.findFavorite`, `repository.createFavorite`. – `clientId: number` – client account identifier. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result.

No	Method	Description
20	`removeFavorite( clientId: number, freelancerId: number)`	A method to remove favorite. The implementation delegates to `repository.findFavorite`, `repository.deleteFavorite`. – `clientId: number` – client account identifier. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result.
21	`followFreelancer( clientId: number, freelancerId: number)`	A method to follow freelancer. The implementation delegates to `repository.findFollow`, `repository.createFollowWithNotification`. – `clientId: number` – client account identifier. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result.
22	`unfollowFreelancer( clientId: number, freelancerId: number)`	A method to unfollow freelancer. The implementation delegates to `repository.findFollow`, `repository.deleteFollow`. – `clientId: number` – client account identifier. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result.

4.34.2 ClassAttribute

No	Attribute	Type
01	`repository`	`AccountProfileRepository` – injected persistence gateway.
02	`profileRevisionService`	`ProfileRevisionService` – injected application dependency.
03	`cloudinary`	`CloudinaryService` – injected application dependency.
04	`hashing`	`HashingService` – injected application dependency.

No	Attribute	Type
05	identityAi	IdentityAiService – injected application dependency.

4.35 AccountProfileRepository Class

4.35.1 ClassMethod

No	Method	Description
01	AccountProfileRepository(prisma: PrismaService)	A method to initialize the repository with its dependencies. – prisma: PrismaService – constructor-injected dependency. – *Returns:* void – constructor, no return value.
02	findUserWithRoles(userId: number)	A method to find user with roles. – userId: number – account identifier used by this operation. – *Returns:* Selected database projection / response object – operation result.
03	findClientProfile(userId: number)	A method to find client profile. – userId: number – account identifier used by this operation. – *Returns:* Selected database projection / response object – operation result.
04	updateClientProfile(userId: number, input: UpdateClient ProfileType)	A method to update client profile. – userId: number – account identifier used by this operation. – input: UpdateClientProfileType – validated operation input. – *Returns:* Promise of the selected database projection / response object – operation result.

No	Method	Description
05	`createIdentityDocument( userId: number, documentType: DocumentTypeType, fileUrl: string)`	A method to create identity document. – `userId`: `number` – account identifier used by this operation. – `documentType`: `DocumentTypeType` – operation parameter used by the implementation. – `fileUrl`: `string` – operation parameter used by the implementation. – *Returns:* `Selected database projection / response object` – operation result.
06	`createIdentityAiDocument(data: { userId: number; documentType: DocumentTypeType; fileUrl: string; backFileUrl: string | null; status: VerificationStatus; reviewNotes: string | null; duplicateOfId: number | null; aiStatus: string; aiScore: number | null; aiError: string | null; aiSummary: Prisma.InputJsonValue; aiSelfieSummary: Prisma.InputJsonValue; aiScannedAt: Date; })`	A method to create identity ai document. – `data`: `{ userId: number; documentType: DocumentTypeType; fileUrl: string; backFileUrl: string | null; status: VerificationStatus; reviewNotes: string | null; duplicateOfId: number | null; aiStatus: string; aiScore: number | null; aiError: string | null; aiSummary: Prisma.InputJsonValue; aiSelfieSummary: Prisma.InputJsonValue; aiScannedAt: Date; }` – values to persist. – *Returns:* `Selected database projection / response object` – operation result.
07	`findApprovedIdentityDocumentByIdentityNumber( userId: number, identityNumber: string)`	A method to find approved identity document by identity number. – `userId`: `number` – account identifier used by this operation. – `identityNumber`: `string` – operation parameter used by the implementation. – *Returns:* `Promise<{ id: number; }>` – operation result.
08	`findIdentityDocuments( userId: number)`	A method to find identity documents. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Selected database projection / response object` – operation result.

No	Method	Description
09	`findIdentityDocumentById( id: number)`	A method to find identity document by id. – `id`: `number` – target resource identifier. – *Returns:* `Selected database projection / response object` – operation result.
10	`findSocialLinks( userId: number)`	A method to find social links. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise<{ id: number; profileId: number; url: string; platform: Social Platform; }[] | null>` – operation result.
11	`findSocialLinkByPlatform( userId: number, platform: AddSocialLink Type['platform'])`	A method to find social link by platform. – `userId`: `number` – account identifier used by this operation. – `platform`: `AddSocialLinkType['platform']` – operation parameter used by the implementation. – *Returns:* `Promise<{ id: number; profileId: number; url: string; platform: Social Platform; } | null>` – operation result.
12	`createSocialLink( userId: number, input: AddSocialLink Type)`	A method to create social link. – `userId`: `number` – account identifier used by this operation. – `input`: `AddSocialLinkType` – validated operation input. – *Returns:* `Promise<{ id: number; profileId: number; url: string; platform: Social Platform; }>` – operation result.
13	`findSocialLinkById( id: number)`	A method to find social link by id. – `id`: `number` – target resource identifier. – *Returns:* `Selected database projection / response object` – operation result.
14	`deleteSocialLink( id: number)`	A method to delete social link. – `id`: `number` – target resource identifier. – *Returns:* `Prisma__SocialLinkClient<{ id: number; profileId: number; url: string; platform: SocialPlatform; }, never, Default Args, PrismaClientOptions>` – operation result.

No	Method	Description
15	`findFavorite( clientId: number, freelancerId: number)`	A method to find favorite. – `clientId`: `number` – client account identifier. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Prisma__FavoriteFreelancerClient<{ createdAt: Date; clientId: number; freelancerId: number; } | null, null, DefaultArgs, PrismaClientOptions>` – operation result.
16	`createFavorite( clientId: number, freelancerId: number)`	A method to create favorite. – `clientId`: `number` – client account identifier. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Prisma__FavoriteFreelancerClient<{ createdAt: Date; clientId: number; freelancerId: number; }, never, Default Args, PrismaClientOptions>` – operation result.
17	`deleteFavorite( clientId: number, freelancerId: number)`	A method to delete favorite. – `clientId`: `number` – client account identifier. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Prisma__FavoriteFreelancerClient<{ createdAt: Date; clientId: number; freelancerId: number; }, never, Default Args, PrismaClientOptions>` – operation result.
18	`findFavorites( clientId: number)`	A method to find favorites. – `clientId`: `number` – client account identifier. – *Returns:* `Selected database projection / response object` – operation result.
19	`findDiscoverable Freelancers( clientId: number)`	A method to find discoverable freelancers. – `clientId`: `number` – client account identifier. – *Returns:* `Selected database projection / response object` – operation result.

4.35.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.36 PermissionsController Class

4.36.1 ClassMethod

No	Method	Description
01	`PermissionsController( permissionsService: PermissionsService)`	A method to initialize the controller with the permission service. – `permissionsService`: `PermissionsService` – service that owns permission queries. – *Returns:* `void` – constructor, no return value.
02	`getPermissions(query: PermissionFilterDto)`	A method to return a filtered page of route permissions. – `query`: `PermissionFilterDto` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<PermissionListResponseType>` – permissions, pagination, and module names. – Route: `GET /permissions`.
03	`getPermissionById(id: number)`	A method to return one permission. – `id`: `number` – permission identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<PermissionDetailResponseType>` – the permission detail. – Route: `GET /permissions/:id`.

4.36.2 ClassAttribute

No	Attribute	Type
01	`permissionsService`	`PermissionsService` – private readonly permission service.

4.37 PermissionsService Class

4.37.1 ClassMethod

No	Method	Description
01	`PermissionsService( permissionsRepository: PermissionsRepository)`	A method to initialize the service with the permission repository. – `permissionsRepository`: `PermissionsRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getPermissions(filter: PermissionFilterType)`	A method to load a permission page and compute pagination metadata. – `filter`: `PermissionFilterType` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<PermissionListResponseType>` – permissions, pagination, and module names. – Failure is mapped to `FailedToLoadPermissions Exception`.
03	`getPermissionById(id: number)`	A method to load one permission. – `id`: `number` – permission identifier. – *Returns:* `Promise<PermissionDetailResponse Type>` – the permission detail. – Prisma errors are mapped to `FailedToLoad PermissionDetailException`.

4.37.2 ClassAttribute

No	Attribute	Type
01	`permissionsRepository`	`PermissionsRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.38 PermissionsRepository Class

4.38.1 ClassMethod

No	Method	Description
01	`PermissionsRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAll(filter: PermissionFilterType)`	A method to query permissions, the matching total, and the distinct module list. – `filter`: `PermissionFilterType` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<{ permissions: PermissionListItemType[]; total: number; modules: string[] }>` – the page and supporting data. – Search matches name, path, or module case-insensitively.
03	`findById(id: number)`	A method to load one active permission. – `id`: `number` – permission identifier. – *Returns:* `Promise<PermissionDetailResponseType>` – the permission detail. – Throws `PermissionNotFoundException`.

4.38.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.39 SessionsController Class

4.39.1 ClassMethod

No	Method	Description
01	`SessionsController( sessionsService: SessionsService)`	A method to initialize the controller with the session service. – `sessionsService`: `SessionsService` – service that owns session queries. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getMySessions( userId: number, sessionId: number, query: SessionFilterDto)`	A method to return the current user's sessions. – `userId`: `number` – current user. – `sessionId`: `number` – current session, used to mark the active row. – `query`: `SessionFilterDto` – page, limit, search, and sort options. – *Returns:* `Promise<SessionListResponseType>` – sessions and pagination. – Route: `GET /sessions`; authenticated endpoint.
03	`getMySessionById( userId: number, sessionId: number, id: number)`	A method to return one owned session. – `userId`: `number` – current user. – `sessionId`: `number` – current session, used to mark the active row. – `id`: `number` – session identifier. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Route: `GET /sessions/:id`; authenticated endpoint.
04	`revokeMySession( userId: number, sessionId: number, id: number)`	A method to revoke one owned session. – `userId`: `number` – current user. – `sessionId`: `number` – current session, used to compute `loggedOut`. – `id`: `number` – session identifier. – *Returns:* `Promise<RevokeSessionResponseType>` – a success message and logout flag. – Route: `DELETE /sessions/:id`; authenticated endpoint.

4.39.2 ClassAttribute

No	Attribute	Type
01	`sessionsService`	`SessionsService` – private readonly session service.

4.40 SessionsService Class

4.40.1 ClassMethod

No	Method	Description
01	`SessionsService(sessionsRepository: SessionsRepository)`	A method to initialize the service with the session repository. – `sessionsRepository`: `SessionsRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getMySessions(userId, filter, currentSessionId?)`	A method to load a page of the user's sessions. – `userId`: `number` – current user. – `filter`: `SessionFilterType` – page, limit, search, and sort options. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<SessionListResponseType>` – sessions and pagination. – Failure is mapped to `FailedToLoadSessionsException`.
03	`getMySessionById(id, userId, currentSessionId?)`	A method to load one session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – current user. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Prisma errors are mapped to `FailedToLoadSessionDetailException`.
04	`revokeMySession(id, userId, currentSessionId?)`	A method to delete an active session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – current user. – `currentSessionId`: `number | undefined` – current session used to compute `loggedOut`. – *Returns:* `Promise<RevokeSessionResponseType>` – a success message and logout flag. – Prisma errors are mapped to `FailedToRevokeSessionException`.

4.40.2 ClassAttribute

No	Attribute	Type
01	`sessionsRepository`	`SessionsRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.41 SessionsRepository Class

4.41.1 ClassMethod

No	Method	Description
01	`SessionsRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAllByUserId(userId, filter, currentSessionId? )`	A method to query the user's sessions and mark the current one. – `userId`: `number` – owner identifier. – `filter`: `SessionFilterType` – page, limit, search, and sort options. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<{ sessions: SessionListItem Type[]; total: number }>` – the page and total. – Search matches device information or IP address.
03	`findByIdForUser(id, user Id, currentSessionId?)`	A method to load one session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – owner identifier. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Throws `SessionNotFoundException`.

No	Method	Description
04	`deleteActiveByIdForUser(id, userId)`	A method to delete an unexpired session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – owner identifier. – *Returns:* `Promise<void>` – no value. – Throws `SessionNotFoundException` or `SessionAlreadyExpiredException`.

4.41.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.42 Forum Category Administration (Shared Classes)

UC-14.01–14.02 and UC-17.01–17.06 use the shared `ForumAdminController`, `ForumAdminServi` and `ForumAdminRepository`. Their complete specifications, including the category list, detail, create, update, delete, and restore endpoints, are given once in the shared forum administration class specifications for Sprint 3 (see the `ForumAdminController` subsection).

4.43 ForumAdminController Class

4.43.1 ClassMethod

No	Method	Description
01	`ForumAdminController(adminService: ForumAdminService)`	A method to initialize the controller with the administration service. – `adminService`: `ForumAdminService` – service dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getAdminStats(roleName: string)`	A method to return the forum dashboard counters. – `roleName: string` – current role name from `@User Active('roleName')`. – *Returns:* `Promise<ForumAdminStatsType>` – category, post, comment, report, pending-report, and user totals plus the five newest posts. – Route: `GET /forums/admin/stats`; administrator endpoint.
03	`getAdminCategoryLists( roleName: string, page: number, limit: number, search: string, deleted: string, sortBy: 'id' | 'name' | 'createdAt', sortOrder: 'asc' | 'desc')`	A method to list categories with search, deleted-state filtering, and sorting. – `roleName: string` – current role name. – `page, limit: number` – coerced with `Number()` and defaulted to 1 and 10. – `search: string` – optional case-insensitive name search. – `deleted: string` – optional `'true'` or `'false'` filter. – `sortBy, sortOrder: 'id' | 'name' | 'createdAt'` and `'asc' | 'desc'` – optional sort controls. – *Returns:* `Promise<ForumAdminCategoryListResponseType>` – categories and pagination. – Route: `GET /forums/admin/categories`; administrator endpoint.
04	`getAdminCategoryById( roleName: string, id: number)`	A method to return one active category. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with public post count. – Route: `GET /forums/admin/categories/:id`; administrator endpoint.
05	`createAdminCategory( roleName: string, body: CreateForumCategoryBodyDto)`	A method to create a category. – `roleName: string` – current role name. – `body: CreateForumCategoryBodyDto` – name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category. – Route: `POST /forums/admin/categories`; administrator endpoint.

No	Method	Description
06	`updateAdminCategory( roleName: string, id: number, body: UpdateForum CategoryBodyDto)`	A method to rename or re-describe a category. – `roleName: string` – current role name. – `id: number` – category identifier. – `body: UpdateForumCategoryBodyDto` – optional name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Route: `PATCH /forums/admin/categories/:id`; administrator endpoint.
07	`deleteAdminCategory( roleName: string, id: number)`	A method to soft-delete a category and detach its posts. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Route: `DELETE /forums/admin/categories/:id`; administrator endpoint.
08	`restoreAdminCategory( roleName: string, id: number)`	A method to restore a soft-deleted category. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Route: `PATCH /forums/admin/categories/:id/restore`; administrator endpoint.
09	`getAdminCommentLists( roleName: string, page: number, limit: number, search: string)`	A method to list active comments for administration. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – `search: string` – optional case-insensitive content search. – *Returns:* `Promise<ForumAdminCommentList ResponseType>` – comments with author and parent-post summaries, plus pagination. – Route: `GET /forums/admin/comments`; administrator endpoint.

No	Method	Description
10	`getPendingPosts( roleName: string, page: number, limit: number)`	A method to list posts waiting for moderation. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<PendingForumPostList ResponseType>` – non-deleted `PENDING` posts, oldest first, with pagination. – Route: `GET /forums/admin/pending-posts`; administrator endpoint.
11	`getTrashPosts( roleName: string, page: number, limit: number)`	A method to list deleted or rejected posts. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<ForumTrashPostListResponse Type>` – posts with `deletedAt` set or moderation status `R EJECTED`. – Route: `GET /forums/admin/trash/posts`; administrator endpoint.
12	`getTrashComments( roleName: string, page: number, limit: number)`	A method to list deleted comments. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<ForumTrashCommentList ResponseType>` – soft-deleted comments with author and parent-post summaries. – Route: `GET /forums/admin/trash/comments`; administrator endpoint.
13	`restoreTrashPost( roleName: string, id: number)`	A method to restore a post from the trash. – `roleName: string` – current role name. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – Route: `PATCH /forums/admin/trash/posts/:id/ restore`; administrator endpoint.

No	Method	Description
14	`restoreTrashComment(roleName: string, id: number)`	A method to restore a comment from the trash. – `roleName: string` – current role name. – `id: number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment. – Route: `PATCH /forums/admin/trash/comments/:id/restore`; administrator endpoint.
15	`approvePendingPost(roleName: string, userId: number, id: number)`	A method to approve a pending post. – `roleName: string` – current role name. – `userId: number` – reviewing administrator recorded in `reviewedById`. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the approved post. – Route: `PATCH /forums/admin/posts/:id/approve`; administrator endpoint.
16	`rejectPendingPost(roleName: string, userId: number, id: number)`	A method to reject a pending post and move it to the trash. – `roleName: string` – current role name. – `userId: number` – reviewing administrator. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the rejected post, now soft-deleted. – Route: `PATCH /forums/admin/posts/:id/reject`; administrator endpoint.

4.43.2 ClassAttribute

No	Attribute	Type
01	`adminService`	`ForumAdminService` – private readonly service dependency.

4.44 ForumAdminService Class

4.44.1 ClassMethod

No	Method	Description
01	`ForumAdminService( adminRepository: ForumAdminRepository)`	A method to initialize the service with the administration repository. – `adminRepository`: `ForumAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getAdminStats(roleName: string)`	A method to authorize an administrator and load dashboard counters. – `roleName`: `string` – must equal `RoleName.ADMIN`. – *Returns:* `Promise<ForumAdminStatsType>` – counters and the five newest posts. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
03	`getAdminCommentLists( roleName: string, page: number, limit: number, search?: string)`	A method to authorize an administrator and return a page of active comments. – `roleName`: `string` – must equal `RoleName.ADMIN`. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional content filter. – *Returns:* `Promise<ForumAdminCommentList ResponseType>` – comments and pagination with `total Pages`.
04	`getAdminCategoryLists( roleName: string, page: number, limit: number, search?: string, sortBy?: 'id' | 'name' | 'createdAt', sortOrder?: 'asc' | 'desc', deleted?: string)`	A method to authorize an administrator and return a page of categories. – `roleName`: `string` – must equal `RoleName.ADMIN`. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional name filter. – `sortBy`, `sortOrder`: optional sort controls; repository defaults are `id` and `desc`. – `deleted`: `string | undefined` – `'true'` lists only deleted, `'false'` only active. – *Returns:* `Promise<ForumAdminCategoryList ResponseType>` – categories and pagination.

No	Method	Description
05	`getAdminCategoryById( roleName: string, categoryId: number)`	A method to authorize an administrator and load one active category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponse Type>` – category detail. – Throws `ForumCategoryNotFoundException` when absent, or `ForumReportForbiddenException` for other failures.
06	`createAdminCategory( roleName: string, body: CreateForum CategoryBodyType)`	A method to authorize an administrator and create a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `body: CreateForumCategoryBodyType` – name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category. – Unknown repository errors are rethrown unchanged.
07	`updateAdminCategory( roleName: string, categoryId: number, body: UpdateForum CategoryBodyType)`	A method to authorize an administrator and update a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – `body: UpdateForumCategoryBodyType` – optional name and description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Unknown repository errors are rethrown unchanged.
08	`deleteAdminCategory( roleName: string, categoryId: number)`	A method to authorize an administrator and soft-delete a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Throws `FailedToDeleteForumCategoryException` for an unexpected failure.

No	Method	Description
09	`restoreAdminCategory( roleName: string, categoryId: number)`	A method to authorize an administrator and restore a deleted category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Throws `FailedToRestoreForumCategory Exception` for an unexpected failure.
10	`getPendingPosts( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the moderation queue. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<PendingForumPostList ResponseType>` – pending posts and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
11	`reviewPendingPost( roleName: string, post Id: number, status: 'APPROVED' | 'RE JECTED', adminId: number)`	A method to authorize an administrator and apply a moderation decision. – `roleName: string` – must equal `RoleName.ADMIN`. – `postId: number` – post identifier. – `status: 'APPROVED' | 'REJECTED'` – decision passed by the approve/reject routes. – `adminId: number` – reviewing administrator. – *Returns:* `Promise<ForumPostType>` – the reviewed post. – A Prisma `P2025` error is mapped to `ForumPostNot FoundException`; other Prisma errors become `FailedTo ReviewForumPostException`.
12	`getTrashPosts( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the post trash. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<ForumTrashPostListResponse Type>` – deleted or rejected posts and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.

No	Method	Description
13	`getTrashComments( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the comment trash. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<ForumTrashCommentList ResponseType>` – deleted comments and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
14	`restoreTrashPost( roleName: string, post Id: number)`	A method to authorize an administrator and restore a post. – `roleName: string` – must equal `RoleName.ADMIN`. – `postId: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – Throws `FailedToRestoreForumPostException` for an unexpected failure.
15	`restoreTrashComment( roleName: string, commentId: number)`	A method to authorize an administrator and restore a comment. – `roleName: string` – must equal `RoleName.ADMIN`. – `commentId: number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment. – A Prisma `P2025` error is mapped to `ForumCommentNot FoundException`; other Prisma errors become `FailedTo RestoreForumCommentException`.

4.44.2 ClassAttribute

No	Attribute	Type
01	`adminRepository`	`ForumAdminRepository` – private readonly persistence dependency.

4.45 ForumAdminRepository Class

4.45.1 ClassMethod

No	Method	Description
01	`ForumAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`getPendingPosts(page: number, limit: number)`	A method to query the moderation queue and its total in one transaction. – `page`, `limit`: `number` – pagination. – *Returns:* `Promise<{ posts: PendingForumPostType[]; total: number }>` – non-deleted `PENDING` posts, oldest first, with author summary. – `moderationCategories` is normalized to `string[] | null`.
03	`reviewForumPost( postId: number, status: 'APPROVED' | 'REJECTED', adminId: number)`	A method to apply a moderation decision only to a non-deleted `PENDING` post. – `postId`: `number` – post identifier. – `status`: `'APPROVED' | 'REJECTED'` – target moderation status. – `adminId`: `number` – reviewer recorded in `reviewedById`. – *Returns:* `Promise<ForumPostType>` – the reviewed post. – `REJECTED` also sets `deletedAt`; a zero-row update throws `ForumPostNotFoundException`.
04	`getAdminStats()`	A method to count forum entities and load the five newest posts in one transaction. – *Returns:* `Promise<ForumAdminStatsType>` – active category, post, comment, and user counts; total and pending reports; five recent posts. – Posts, comments, categories, and users exclude soft-deleted rows; report totals include all rows.
05	`getAdminCommentLists( page: number, limit: number, search?: string)`	A method to query a page of active comments and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional case-insensitive content filter. – *Returns:* `Promise<{ comments: ForumAdminCommentType[]; total: number }>` – comments with author and parent-post summaries, newest first.

No	Method	Description
06	`getAdminCategoryLists( page: number, limit: number, search?: string, sortBy?: 'id' | 'name' | 'createdAt', sortOrder?: 'asc' | 'desc', deleted?: string)`	A method to query a page of categories and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional case-insensitive name filter. – `sortBy`, `sortOrder`: optional sort controls; defaults are `'id'` and `'desc'`. – `deleted`: `string | undefined` – `'true'` selects only deleted rows, `'false'` only active rows, omitted selects both. – *Returns:* `Promise<{ categories: ForumAdminCategoryType[]; total: number }>` – categories with `postCount` and pagination total.
07	`getAdminCategoryById(id: number)`	A method to load one active category with its public post count. – `id`: `number` – category identifier. – *Returns:* `Promise<ForumCategoryType | null>` – the category detail or `null`.
08	`createAdminCategory(data: { name: string; description?: string | null })`	A method to create a category with a unique name and generated slug. – `data.name`: `string` – category name. – `data.description`: `string | null | undefined` – optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category with `postCount = 0`. – Throws `ForumCategoryAlreadyExistsException` for a case-insensitive active-name conflict; slug collisions receive a numeric suffix.
09	`updateAdminCategory( id: number, data: UpdateForumCategoryBodyType)`	A method to update a category and regenerate its slug when the name changes. – `id`: `number` – category identifier. – `data`: `UpdateForumCategoryBodyType` – optional name and description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Throws `ForumCategoryNotFoundException` or `ForumCategoryAlreadyExistsException`.

No	Method	Description
10	`deleteAdminCategory(id: number)`	A method to detach posts and soft-delete a category in one transaction. – `id`: `number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Non-deleted posts in the category are reassigned to `categoryId = null` before the category receives `deletedAt`.
11	`restoreAdminCategory(id: number)`	A method to restore a soft-deleted category after a name-conflict check. – `id`: `number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Throws `ForumCategoryNotFoundException` or `ForumCategoryAlreadyExistsException`.
12	`getTrashPosts(page: number, limit: number)`	A method to query deleted or rejected posts and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – *Returns:* `Promise<{ posts: ForumTrashPostType[]; total: number }>` – trash posts with `deletedAt` and author summary. – Ordering is by `deletedAt` descending with null values last.
13	`getTrashComments(page: number, limit: number)`	A method to query soft-deleted comments and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – *Returns:* `Promise<{ comments: ForumTrashCommentType[]; total: number }>` – trash comments with `deletedAt` and parent-post summary, newest deletion first.
14	`restoreTrashPost(postId: number)`	A method to restore a post from the trash. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – A deleted `APPROVED` post becomes visible again; a `REJECTED` post returns to `PENDING` and its reviewer fields are cleared.

No	Method	Description
15	`restoreTrashComment( commentId: number)`	A method to clear `deletedAt` on a trashed comment. – `commentId`: `number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment with author and parent-post summaries. – Throws `ForumCommentNotFoundException` when no deleted row matches.

4.45.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.46 IdentityVerificationsAdminController Class

4.46.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminController( service: Identity VerificationsAdmin Service)`	A method to initialize the controller with the administration service. – `service`: `IdentityVerificationsAdminService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`list(query: Identity VerificationAdminFilter Dto)`	A method to return a filtered, paginated document list. – `query`: `IdentityVerificationAdminFilterDto` – page, limit, optional status, and optional search. – *Returns:* `Promise<IdentityVerificationAdmin ListResponseType>` – documents and pagination. – Route: `GET /admin/identity-verifications`.

No	Method	Description
03	`detail(id: number)`	A method to return one document with user and reviewer detail. – `id: number` – document identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<IdentityVerificationAdminDetailType>` – document, user profile, freelancer verification state, and reviewing administrator. – Route: `GET /admin/identity-verifications/:id`.

04	`file(`
	`id: number, side: string | undefined,`
	`response: Response)`
		A method to stream a legacy `local://` document image.
		– `id: number` – document identifier.
		– `side: string | undefined` – `'back'` selects the back image; any other value selects the front.
		– `response: Response` – Express response used to set headers.
		– *Returns:* `Promise<StreamableFile>` – file stream; sets inline `Content-Disposition` and `application/octet-stream`.
		– Route: `GET /admin/identity-verifications/:id/file`.

05	`approve(`
	`id: number, adminId: number,`
	`body: ApproveIdentityVerificationBodyDto)`
		A method to approve a pending document.
		– `id: number` – document identifier.
		– `adminId: number` – reviewing administrator from `@UserActive('userId')`.
		– `body: ApproveIdentityVerificationBodyDto` – optional `reviewNotes` up to 1000 characters.
		– *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail.
		– Route: `PATCH /admin/identity-verifications/:id/approve`.

06	`reject(`
	`id: number, adminId: number,`
	`body: RejectIdentityVerificationBodyDto)`
		A method to reject a pending document.
		– `id: number` – document identifier.
		– `adminId: number` – reviewing administrator.
		– `body: RejectIdentityVerificationBodyDto` – optional trimmed `reviewNotes` up to 1000 characters.
		– *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail.
		– Route: `PATCH /admin/identity-verifications/:id/reject`.

4.46.2 ClassAttribute

No	Attribute	Type
01	`service`	`IdentityVerificationsAdmin Service` – private readonly service dependency.

4.47 IdentityVerificationsAdminService Class

4.47.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminService( repository: IdentityVerificationsAdmin Repository)`	A method to initialize the service with the identity-verification repository. – `repository`: `IdentityVerificationsAdmin Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`presentFileUrl(document: T)`	A private generic method to replace legacy `local://` file paths with API URLs. – `document`: `T` – object with `id`, `fileUrl`, and optional `backFileUrl`. – *Returns:* the same object with `fileUrl` set to the file route and, when needed, `backFileUrl` set to the same route with `?side=back`. – Non-local URLs are returned unchanged.
03	`presentList(documents: T[])`	A private method to map `presentFileUrl` over a document list. – `documents`: `T[]` – list of document objects. – *Returns:* the mapped list.
04	`list(params: { page: number; limit: number; status?: string; search?: string })`	A method to load a filtered page of documents. – `params`: object – page, limit, optional status string, and optional search. – *Returns:* `Promise<IdentityVerificationAdmin ListResponseType>` – documents with presentation URLs and pagination. – `status` is cast to the Prisma `VerificationStatus` union before the repository call.

No	Method	Description
05	`detail(id: number)`	A method to load one document and present its file URLs. – `id: number` – document identifier. – *Returns:* `Promise<IdentityVerificationAdminDetailType>` – the document detail. – Throws `IdentityVerificationNotFoundException`.
06	`review( id: number, adminId: number, status: ReviewableStatus, reviewNotes: string | null)`	A method to approve or reject a pending document. – `id: number` – document identifier. – `adminId: number` – reviewing administrator. – `status: ReviewableStatus` – `APPROVED` or `REJECTED`. – `reviewNotes: string | null` – optional reviewer note. – *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail. – Throws `IdentityVerificationNotFoundException` or `IdentityVerificationAlreadyReviewedException`.
07	`getLocalFile( id: number, side: 'front' | 'back' = 'front' )`	A method to resolve and validate a legacy local document file. – `id: number` – document identifier. – `side: 'front' | 'back'` – which stored URL to use; default front. – *Returns:* `Promise<{ absolutePath: string; fileName: string }>` – resolved path and base filename. – Throws `IdentityVerificationNotFoundException` or `IdentityVerificationFileInvalidException`; rejects paths escaping the uploads root.

4.47.2 ClassAttribute

No	Attribute	Type
01	`repository`	`IdentityVerificationsAdminRepository` – private readonly persistence dependency.

4.48 IdentityVerificationsAdminRepository Class

4.48.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findDocuments(params: IdentityVerificationAdmin ListParams)`	A method to query a filtered page of active documents and the matching total. – `params`: `IdentityVerificationAdminListParams` – page, limit, optional status, and optional search. – *Returns:* `Promise<{ documents: IdentityVerificationAdminListItemType[]; total: number }>` – documents with user summary, newest first. – Search matches user email or profile display name case-insensitively; `deletedAt` must be `null`.
03	`findDocumentById(id: number)`	A method to load the full detail projection of an active document. – `id`: `number` – document identifier. – *Returns:* `Promise<IdentityVerificationAdmin DetailType | null>` – document, reviewer, user, and freelancer summary, or `null`.
04	`findDocumentRaw(id: number)`	A method to load the minimal fields needed for review and file streaming. – `id`: `number` – document identifier. – *Returns:* `Promise<{ id: number; userId: number; fileUrl: string; backFileUrl: string | null; status: VerificationStatus } | null>` – raw document fields, or `null`.

No	Method	Description
05	`reviewDocument( id: number, adminId: number, status: VerificationStatus, reviewNotes: string | null)`	A method to persist a review and synchronize the freelancer verification flag in one transaction. – `id: number` – document identifier. – `adminId: number` – reviewing administrator. – `status: VerificationStatus` – `APPROVED` or `REJECTED`. – `reviewNotes: string | null` – reviewer note. – *Returns:* `Promise<IdentityVerificationAdminDetailType | null>` – the updated detail, or `null`. – Updates the document status, reviewer, notes, and `reviewedAt`; sets `FreelancerProfile.idVerified` to true only for `APPROVED`. Transaction limits are 10s wait and 15s timeout.

4.48.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.49 FollowingFreelancerController Class

4.49.1 ClassMethod

No	Method	Description
01	`FollowingFreelancerController( service: AccountProfileService)`	A method to initialize the controller with its dependencies. – `service: AccountProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`getFollowing( userId: number)`	A method to get following. The implementation delegates to `service.getFollowing`. – `userId: number` – account identifier used by this operation. – *Returns:* `FollowingFreelancerListDto` – serialized controller response. – Route: `GET /following/freelancers`.

No	Method	Description
03	`followFreelancer( userId: number, freelancerId: number)`	A method to follow freelancer. The implementation delegates to `service.followFreelancer`. – `userId: number` – account identifier used by this operation. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `POST /following/freelancers/:freelancerId`.
04	`unfollowFreelancer( userId: number, freelancerId: number)`	A method to unfollow freelancer. The implementation delegates to `service.unfollowFreelancer`. – `userId: number` – account identifier used by this operation. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `DELETE /following/freelancers/:freelancerId`.

4.49.2 ClassAttribute

No	Attribute	Type
01	`service`	`AccountProfileService` – injected application dependency.

4.50 AccountProfileService Class

4.50.1 ClassMethod

No	Method	Description
01	`AccountProfileService( repository: Account ProfileRepository, profileRevisionService: ProfileRevisionService, cloudinary: Cloudinary Service, hashing: HashingService, identityAi: IdentityAi Service)`	A method to initialize the service with its dependencies. – `repository`: `AccountProfileRepository` – constructor-injected dependency. – `profileRevisionService`: `ProfileRevision Service` – constructor-injected dependency. – `cloudinary`: `CloudinaryService` – constructor-injected dependency. – `hashing`: `HashingService` – constructor-injected dependency. – `identityAi`: `IdentityAiService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`requireRole( userId: number, roleName: string)`	A method to require role as an internal helper. The implementation delegates to `repository.findUser WithRoles`. – `userId`: `number` – account identifier used by this operation. – `roleName`: `string` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
03	`getFollowing( clientId: number)`	A method to get following. The implementation delegates to `repository.findFollowing`. – `clientId`: `number` – client account identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
04	`followFreelancer( clientId: number, freelancerId: number)`	A method to follow freelancer. The implementation delegates to `repository.findFollow`, `repository. createFollowWithNotification`. – `clientId`: `number` – client account identifier. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result.

No	Method	Description
05	`unfollowFreelancer( clientId: number, freelancerId: number)`	A method to unfollow freelancer. The implementation delegates to `repository.findFollow`, `repository.deleteFollow`. – `clientId`: `number` – client account identifier. – `freelancerId`: `number` – target freelancer account identifier. – *Returns:* `Promise<{ message: string; }>` – operation result.

4.50.2 ClassAttribute

No	Attribute	Type
01	`repository`	`AccountProfileRepository` – injected persistence gateway.
02	`profileRevisionService`	`ProfileRevisionService` – injected application dependency.
03	`cloudinary`	`CloudinaryService` – injected application dependency.
04	`hashing`	`HashingService` – injected application dependency.
05	`identityAi`	`IdentityAiService` – injected application dependency.

4.51 AccountProfileRepository Class

4.51.1 ClassMethod

No	Method	Description
01	`AccountProfileRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma`: `PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`findUserWithRoles( userId: number)`	A method to find user with roles. – `userId: number` – account identifier used by this operation. – *Returns:* `Selected database projection / response object` – operation result.
03	`findFollow( clientId: number, freelancerId: number)`	A method to find follow. – `clientId: number` – client account identifier. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Prisma__FollowFreelancerClient<{ createdAt: Date; clientId: number; freelancerId: number; } | null, null, DefaultArgs, PrismaClientOptions>` – operation result.
04	`createFollowWith Notification( clientId: number, freelancerId: number, followerName: string)`	A method to create follow with notification. – `clientId: number` – client account identifier. – `freelancerId: number` – target freelancer account identifier. – `followerName: string` – operation parameter used by the implementation. – *Returns:* `Promise<{ createdAt: Date; client Id: number; freelancerId: number; }>` – operation result.
05	`deleteFollow( clientId: number, freelancerId: number)`	A method to delete follow. – `clientId: number` – client account identifier. – `freelancerId: number` – target freelancer account identifier. – *Returns:* `Prisma__FollowFreelancerClient<{ createdAt: Date; clientId: number; freelancerId: number; }, never, Default Args, PrismaClientOptions>` – operation result.
06	`findFollowing( clientId: number)`	A method to find following. – `clientId: number` – client account identifier. – *Returns:* `Selected database projection / response object` – operation result.

4.51.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.52 NotificationController Class

4.52.1 ClassMethod

No	Method	Description
01	`NotificationController( service: NotificationService)`	A method to initialize the controller with its dependencies. – `service`: `NotificationService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`getAll( userId: number)`	A method to get all. The implementation delegates to `service.getAll`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `NotificationListDto` – serialized controller response. – Route: `GET /notifications`.
03	`markAllRead( userId: number)`	A method to mark all read. The implementation delegates to `service.markAllRead`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise<{ message: "Success. NotificationsMarkedAsRead"; count: number; }>` – operation result. – Route: `PATCH /notifications/read-all`.
04	`markRead( userId: number, id: number)`	A method to mark read. The implementation delegates to `service.markRead`. – `userId`: `number` – account identifier used by this operation. – `id`: `number` – target resource identifier. – *Returns:* `NotificationDto` – serialized controller response. – Route: `PATCH /notifications/:id/read`.

No	Method	Description
05	`delete( userId: number, id: number)`	A method to delete. The implementation delegates to `service.delete`. – `userId`: `number` – account identifier used by this operation. – `id`: `number` – target resource identifier. – *Returns:* `Promise<{ message: "Success. NotificationDeleted"; }>` – operation result. – Route: `DELETE /notifications/:id`.

4.52.2 ClassAttribute

No	Attribute	Type
01	`service`	`NotificationService` – injected application dependency.

4.53 NotificationService Class

4.53.1 ClassMethod

No	Method	Description
01	`NotificationService( repository: NotificationRepository)`	A method to initialize the service with its dependencies. – `repository`: `NotificationRepository` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`getAll( userId: number)`	A method to get all. The implementation delegates to `repository.findAll`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Selected database projection / response object` – operation result.

No	Method	Description
03	`markRead( userId: number, notificationId: number)`	A method to mark read. The implementation delegates to `repository.findById`, `repository.markRead`. – `userId`: `number` – account identifier used by this operation. – `notificationId`: `number` – notification identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
04	`markAllRead( userId: number)`	A method to mark all read. The implementation delegates to `repository.markAllRead`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise<{ message: "Success. NotificationsMarkedAsRead"; count: number; }>` – operation result.
05	`delete( userId: number, notificationId: number)`	A method to delete. The implementation delegates to `repository.findById`, `repository.softDelete`. – `userId`: `number` – account identifier used by this operation. – `notificationId`: `number` – notification identifier. – *Returns:* `Promise<{ message: "Success. NotificationDeleted"; }>` – operation result.

4.53.2 ClassAttribute

No	Attribute	Type
01	`repository`	`NotificationRepository` – injected persistence gateway.

4.54 NotificationRepository Class

4.54.1 ClassMethod

No	Method	Description
01	`NotificationRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma`: `PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`findAll( userId: number)`	A method to find all. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Selected database projection / response object` – operation result.
03	`findById( userId: number, notificationId: number)`	A method to find by id. – `userId`: `number` – account identifier used by this operation. – `notificationId`: `number` – notification identifier. – *Returns:* `Selected database projection / response object` – operation result.
04	`markRead( notificationId: number)`	A method to mark read. – `notificationId`: `number` – notification identifier. – *Returns:* `Selected database projection / response object` – operation result.
05	`markAllRead( userId: number)`	A method to mark all read. – `userId`: `number` – account identifier used by this operation. – *Returns:* `PrismaPromise<BatchPayload>` – operation result.
06	`softDelete( notificationId: number)`	A method to soft delete. – `notificationId`: `number` – notification identifier. – *Returns:* `Selected database projection / response object` – operation result.

4.54.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.55 AuthController Class

4.55.1 ClassMethod

No	Method	Description
01	`AuthController( authService: AuthService)`	A method to initialize the controller with its dependencies. – `authService`: `AuthService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`switchRole( userId: number, sessionId: number, body: SwitchRoleBodyDto)`	A method to switch role. The implementation delegates to `authService.switchRole`. – `userId`: `number` – account identifier used by this operation. – `sessionId`: `number` – current authenticated session identifier. – `body`: `SwitchRoleBodyDto` – validated HTTP request body. – *Returns:* `SwitchRoleResponseDto` – serialized controller response. – Route: `POST /auth/switch-role`.
03	`joinRole( userId: number, sessionId: number, body: JoinRoleBodyDto)`	A method to join role. The implementation delegates to `authService.joinRole`. – `userId`: `number` – account identifier used by this operation. – `sessionId`: `number` – current authenticated session identifier. – `body`: `JoinRoleBodyDto` – validated HTTP request body. – *Returns:* `JoinRoleResponseDto` – serialized controller response. – Route: `POST /auth/join-role`.

4.55.2 ClassAttribute

No	Attribute	Type
01	`authService`	`AuthService` – injected application dependency.

4.56 AuthService Class

4.56.1 ClassMethod

No	Method	Description
01	`AuthService( hashingService: HashingService, emailService: EmailService, sharedRoleRepository: SharedRoleRepository, authRepository: AuthRepository, tokenService: TokenService)`	A method to initialize the service with its dependencies. – `hashingService`: `HashingService` – constructor-injected dependency. – `emailService`: `EmailService` – constructor-injected dependency. – `sharedRoleRepository`: `SharedRoleRepository` – constructor-injected dependency. – `authRepository`: `AuthRepository` – constructor-injected dependency. – `tokenService`: `TokenService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`switchRole( userId: number, sessionId: number, body: SwitchRoleBodyType)`	A method to switch role. The implementation delegates to `authRepository.switchPrimaryRole`, `tokenService.signAccessToken`. – `userId`: `number` – account identifier used by this operation. – `sessionId`: `number` – current authenticated session identifier. – `body`: `SwitchRoleBodyType` – validated HTTP request body. – *Returns:* `Promise<{ accessToken: string; }>` – operation result.
03	`joinRole( userId: number, sessionId: number, body: JoinRoleBodyType)`	A method to join role. The implementation delegates to `authRepository.joinRole`, `tokenService.signAccessToken`. – `userId`: `number` – account identifier used by this operation. – `sessionId`: `number` – current authenticated session identifier. – `body`: `JoinRoleBodyType` – validated HTTP request body. – *Returns:* `Promise<{ accessToken: string; }>` – operation result.

4.56.2 ClassAttribute

No	Attribute	Type
01	`hashingService`	`HashingService` – injected application dependency.
02	`emailService`	`EmailService` – injected application dependency.
03	`sharedRoleRepository`	`SharedRoleRepository` – injected persistence gateway.
04	`authRepository`	`AuthRepository` – injected persistence gateway.
05	`tokenService`	`TokenService` – injected application dependency.

4.57 AuthRepository Class

4.57.1 ClassMethod

No	Method	Description
01	`AuthRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma`: `PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`switchPrimaryRole( userId: number, roleName: string)`	A method to switch primary role. – `userId`: `number` – account identifier used by this operation. – `roleName`: `string` – operation parameter used by the implementation. – *Returns:* `Promise<{ id: number; createdAt: Date; deletedAt: Date | null; name: string; description: string | null; } | null>` – operation result.

No	Method	Description
03	`joinRole( userId: number, roleName: JoinRoleBody Type['role'])`	A method to join role. – `userId`: `number` – account identifier used by this operation. – `roleName`: `JoinRoleBodyType['role']` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.

4.57.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.58 UsersController Class

4.58.1 ClassMethod

No	Method	Description
01	`UsersController(service: UsersService)`	A method to initialize the controller with the user administration service. – `service`: `UsersService` – service that owns admin user use cases. – *Returns:* `void` – constructor, no return value.
02	`createUser(body: AdminCreateUserBodyDto)`	A method to create an account with an initial role. – `body`: `AdminCreateUserBodyDto` – email, password, full name, and role. – *Returns:* `Promise<AdminCreateUserResponseType>` – the created account. – Route: `POST /users`.

No	Method	Description
03	`updateUser( id: number, actorId: number, body: AdminUpdateUser BodyDto)`	A method to update email, display name, or ban state. – `id`: `number` – target user identifier. – `actorId`: `number` – current administrator, used for the self-ban check. – `body`: `AdminUpdateUserBodyDto` – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponse Type>` – the updated account. – Route: `PATCH /users/:id`.
04	`updateClientProfile( id: number, body: AdminUpdateClient ProfileBodyDto)`	A method to update the Client profile attached to an account. – `id`: `number` – target user identifier. – `body`: `AdminUpdateClientProfileBodyDto` – company name, description, and website. – *Returns:* `Promise<AdminClientProfileResponse Type>` – the updated Client profile. – Route: `PATCH /users/:id/client-profile`.
05	`updateFreelancerProfile( id: number, body: AdminUpdate FreelancerProfileBodyDto)`	A method to update the Freelancer headline and biography. – `id`: `number` – target user identifier. – `body`: `AdminUpdateFreelancerProfileBodyDto` – title, bio, languages, education, and certifications. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/freelancer-profile`.
06	`replaceFreelancerSkills( id: number, body: AdminReplace FreelancerSkillsBodyDto)`	A method to replace every skill on a Freelancer profile. – `id`: `number` – target user identifier. – `body.skills`: array of skill name and proficiency pairs. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PUT /users/:id/freelancer-profile/ skills`.

No	Method	Description
07	`createPortfolioItem( id: number, body: AdminCreate PortfolioItemBodyDto)`	A method to add a portfolio item to a Freelancer profile. – `id: number` – target user identifier. – `body: AdminCreatePortfolioItemBodyDto` – title, description, technologies, and project URL. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /users/:id/freelancer-profile/ portfolio-items`.
08	`updatePortfolioItem( id: number, itemId: number, body: AdminUpdate PortfolioItemBodyDto)`	A method to update one portfolio item owned by the Freelancer profile. – `id: number` – target user identifier. – `itemId: number` – portfolio item identifier. – `body: AdminUpdatePortfolioItemBodyDto` – optional item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/freelancer-profile/ portfolio-items/:itemId`.
09	`deletePortfolioItem( id: number, itemId: number)`	A method to soft-delete one portfolio item. – `id: number` – target user identifier. – `itemId: number` – portfolio item identifier. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `DELETE /users/:id/freelancer-profile/ portfolio-items/:itemId`.
10	`getUsers(query: Admin UserQueryDto)`	A method to return a filtered, paginated user list. – `query: AdminUserQueryDto` – page, limit, search, role, sort, and status filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination. – Route: `GET /users`.
11	`getSkillCatalog()`	A method to return the active skill catalogue for the profile editor. – *Returns:* `Promise<AdminSkillCatalogListType>` – active skills ordered by name. – Route: `GET /users/skills-catalog`.

No	Method	Description
12	`getUserById(id: number)`	A method to return a consolidated user detail. – `id`: `number` – target user identifier. – *Returns:* `Promise<AdminUserDetailResponse Type>` – account, roles, and profile summaries. – Route: `GET /users/:id`.

4.58.2 ClassAttribute

No	Attribute	Type
01	`service`	`UsersService` – private readonly user administration service.

4.59 UsersService Class

4.59.1 ClassMethod

No	Method	Description
01	`UsersService( repository: Users Repository, hashingService: Hashing Service)`	A method to initialize the service with persistence and password hashing. – `repository`: `UsersRepository` – user persistence dependency. – `hashingService`: `HashingService` – password hashing utility. – *Returns:* `void` – constructor, no return value.
02	`createUser(body: Admin CreateUserBodyType)`	A method to validate the email and role, hash the password, and create the account. – `body`: `AdminCreateUserBodyType` – email, password, full name, and role id. – *Returns:* `Promise<AdminCreateUserResponse Type>` – the created account. – Throws `EmailAlreadyExistsException`, `Create UserRoleNotFoundException`, `CannotAssignAdmin RoleException`, or `FailedToCreateUserException`.

No	Method	Description
03	`getUsers(query: AdminUserQueryType)`	A method to load a user page. – `query`: `AdminUserQueryType` – page, limit, search, role, sort, and status filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination. – Failure is mapped to `FailedToGetUsersException`.
04	`getUserById(id: number)`	A method to load a consolidated account detail. – `id`: `number` – target user identifier. – *Returns:* `Promise<AdminUserDetailResponseType>` – account, roles, and profiles. – Throws `UserNotFoundException` when absent or `FailedToGetUserException` on an unexpected read failure.
05	`listSkillCatalog()`	A method to return the active skill catalogue. – *Returns:* `Promise<AdminSkillCatalogListType>` – active skills ordered by name.
06	`updateUser(id, actorId, body)`	A method to update account fields while enforcing the self-ban and email rules. – `id`: `number` – target user identifier. – `actorId`: `number` – current administrator. – `body`: `AdminUpdateUserBodyType` – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponseType>` – the updated account. – Throws `CannotBanSelfException`, `EmailAlreadyExistsException`, `UserNotFoundException`, or `FailedToUpdateUserException`.
07	`updateClientProfile(userId, body)`	A method to validate the Client profile context and upsert the profile. – `userId`: `number` – target user identifier. – `body`: `AdminUpdateClientProfileBodyType` – company fields. – *Returns:* `Promise<AdminClientProfileResponseType>` – the updated Client profile. – Throws `UserNotFoundException`, `NoClientRoleForClientProfileException`, or `FailedToUpdateClientProfileException`.

No	Method	Description
08	`updateFreelancerProfile(userId, body)`	A method to validate the Freelancer context and upsert profile fields. – `userId`: `number` – target user identifier. – `body`: `AdminUpdateFreelancerProfileBodyType` – title, bio, languages, education, and certifications. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `NoFreelancerRoleForProfileException` or `FailedToUpdateFreelancerProfileException`.
09	`replaceFreelancerSkills(userId, body)`	A method to replace all Freelancer skills and create missing catalogue skills. – `userId`: `number` – target user identifier. – `body`: `AdminReplaceFreelancerSkillsBodyType` – skill name and proficiency pairs. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `NoFreelancerRoleForProfileException` or `FailedToSaveSkillsException`.
10	`createPortfolioItem(userId, body)`	A method to create a portfolio item on the Freelancer profile. – `userId`: `number` – target user identifier. – `body`: `AdminCreatePortfolioItemBodyType` – item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `FailedToCreatePortfolioItemException`.
11	`updatePortfolioItem(userId, itemId, body)`	A method to update an owned portfolio item. – `userId`: `number` – target user identifier. – `itemId`: `number` – portfolio item identifier. – `body`: `AdminUpdatePortfolioItemBodyType` – optional item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `PortfolioItemNotFoundException` or `FailedToUpdatePortfolioItemException`.

No	Method	Description
12	`deletePortfolioItem(userId, itemId)`	A method to soft-delete an owned portfolio item. – `userId`: `number` – target user identifier. – `itemId`: `number` – portfolio item identifier. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `PortfolioItemNotFoundException` or `FailedToDeletePortfolioItemException`.
13	`getFreelancerProfileContext(userId)`	A private method to load the profile and role context for a Freelancer edit. – `userId`: `number` – target user identifier. – *Returns:* `Promise<context>` – profile id, Freelancer profile id, and role flag. – Throws `UserNotFoundException` or `NoFreelancerRoleForProfileException`.
14	`ensureFreelancerProfileRow(userId, context)`	A private method to create the Profile and FreelancerProfile rows when missing. – `userId`: `number` – target user identifier. – `context`: object – profile id and role context. – *Returns:* `Promise<number | null>` – the Freelancer profile id. – Used by skill and portfolio editors before writing child rows.

4.59.2 ClassAttribute

No	Attribute	Type
01	`repository`	`UsersRepository` – private readonly persistence dependency.
02	`hashingService`	`HashingService` – private readonly password utility.

4.60 UsersRepository Class

4.60.1 ClassMethod

No	Method	Description
01	`UsersRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`getUsers(query: AdminUserQueryType)`	A method to query users with search, role, status, sorting, and pagination. – `query`: `AdminUserQueryType` – list filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination.
03	`createUserByAdmin(data)`	A method to create the account, profile, and primary role in one write. – `data`: object – email, hashed password, full name, role id, and role name. – *Returns:* `Promise<AdminCreateUserResponseType>` – the created account. – Creates an empty Expert profile when the role is Expert.
04	`findUserByEmail(email)`	A method to find a non-deleted account by email. – `email`: `string` – account email. – *Returns:* `Promise<{ id: number } | null>` – the account id, or `null`.
05	`findActiveRoleById(id)`	A method to load an active role by id. – `id`: `number` – role identifier. – *Returns:* `Promise<{ id: number; name: string } | null>` – the role, or `null`.
06	`updateUserByAdmin(id, data)`	A method to update account fields, profile name, and sessions on ban. – `id`: `number` – target account. – `data`: object – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponseType | null>` – the updated account, or `null` when absent. – Setting `isBanned` to true deletes every session in the same transaction.
07	`findClientProfileEditContext(id)`	A method to load the profile context required by the Client editor. – `id`: `number` – target account. – *Returns:* `Promise<context | null>` – profile id, Client profile id, and role flag.

No	Method	Description
08	`upsertClientProfileBy Admin(userId, data)`	A method to create or update the Client profile. – `userId`: `number` – target account. – `data`: object – company name, description, and website. – *Returns:* `Promise<ClientProfile | null>` – the persisted Client profile.
09	`findFreelancerProfileEdit Context(id)`	A method to load the profile context required by the Freelancer editor. – `id`: `number` – target account. – *Returns:* `Promise<context | null>` – profile id, Freelancer profile id, and role flag.
10	`updateFreelancerProfileBy Admin(userId, data)`	A method to update Profile biography and Freelancer profile fields. – `userId`: `number` – target account. – `data`: object – title, bio, languages, education, and certifications. – *Returns:* `Promise<void>` – no value.
11	`ensureFreelancerProfile( userId, profileId)`	A method to create the missing Profile or FreelancerProfile row. – `userId`: `number` – target account. – `profileId`: `number | null` – existing profile id, or `null`. – *Returns:* `Promise<number | null>` – the Freelancer profile id.
12	`replaceFreelancerSkills( freelancerProfileId, skills)`	A method to replace all skills and create missing catalogue skills in a transaction. – `freelancerProfileId`: `number` – Freelancer profile identifier. – `skills`: array of skill name and proficiency pairs. – *Returns:* `Promise<void>` – no value. – Free-text names are matched case-insensitively; missing skills receive a generated slug.
13	`createPortfolioItem( freelancerProfileId, data)`	A method to insert a portfolio item. – `freelancerProfileId`: `number` – Freelancer profile identifier. – `data`: object – title, description, technologies, and project URL. – *Returns:* `Promise<PortfolioItem>` – the created item.

No	Method	Description
14	`findPortfolioItemOwned(id)`	A method to load a portfolio item with its owner and delete state. – `id`: `number` – portfolio item identifier. – *Returns:* `Promise<{ id: number; freelancerProfileId: number; deletedAt: Date | null } | null>` – the ownership record, or `null`.
15	`updatePortfolioItemByAdmin(id, data)`	A method to update only the provided portfolio item fields. – `id`: `number` – portfolio item identifier. – `data`: object – optional item fields. – *Returns:* `Promise<PortfolioItem>` – the updated item.
16	`softDeletePortfolioItem(id)`	A method to set the portfolio item delete timestamp. – `id`: `number` – portfolio item identifier. – *Returns:* `Promise<PortfolioItem>` – the soft-deleted item.
17	`listActiveSkillCatalog()`	A method to return active skills ordered by name. – *Returns:* `Promise<AdminSkillCatalogListType>` – the catalogue.
18	`getUserById(id)`	A method to load the full account graph for the detail page. – `id`: `number` – target account. – *Returns:* `Promise<AdminUserDetailResponseType | null>` – the detail view, or `null`.

4.60.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.61 ReviewController Class

4.61.1 ClassMethod

No	Method	Description
01	`ReviewController( service: ReviewService)`	A method to initialize the controller with its dependencies. – `service`: `ReviewService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`list( userId: number, contractId: number)`	A method to list. The implementation delegates to `service.list`. – `userId`: `number` – account identifier used by this operation. – `contractId`: `number` – contract identifier. – *Returns:* `ReviewListDto` – serialized controller response. – Route: `GET /reviews/contracts/:contractId`.
03	`create( userId: number, contractId: number, body: CreateReviewDto)`	A method to create. The implementation delegates to `service.create`. – `userId`: `number` – account identifier used by this operation. – `contractId`: `number` – contract identifier. – `body`: `CreateReviewDto` – validated HTTP request body. – *Returns:* `ReviewDto` – serialized controller response. – Route: `POST /reviews/contracts/:contractId`.
04	`detail( userId: number, reviewId: number)`	A method to detail. The implementation delegates to `service.detail`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – *Returns:* `ReviewDto` – serialized controller response. – Route: `GET /reviews/:reviewId`.
05	`update( userId: number, reviewId: number, body: UpdateReviewDto)`	A method to update. The implementation delegates to `service.update`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – `body`: `UpdateReviewDto` – validated HTTP request body. – *Returns:* `ReviewDto` – serialized controller response. – Route: `PATCH /reviews/:reviewId`.

No	Method	Description
06	`remove( userId: number, reviewId: number)`	A method to remove. The implementation delegates to `service.remove`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – *Returns:* `Promise<{ message: "Success.Review Deleted"; }>` – operation result. – Route: `DELETE /reviews/:reviewId`.
07	`respond( userId: number, reviewId: number, body: CreateReview ResponseDto)`	A method to respond. The implementation delegates to `service.respond`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – `body`: `CreateReviewResponseDto` – validated HTTP request body. – *Returns:* `ReviewResponseDto` – serialized controller response. – Route: `POST /reviews/:reviewId/response`.
08	`updateResponse( userId: number, responseId: number, body: UpdateReview ResponseDto)`	A method to update response. The implementation delegates to `service.updateResponse`. – `userId`: `number` – account identifier used by this operation. – `responseId`: `number` – review response identifier. – `body`: `UpdateReviewResponseDto` – validated HTTP request body. – *Returns:* `ReviewResponseDto` – serialized controller response. – Route: `PATCH /reviews/responses/:responseId`.
09	`removeResponse( userId: number, responseId: number)`	A method to remove response. The implementation delegates to `service.removeResponse`. – `userId`: `number` – account identifier used by this operation. – `responseId`: `number` – review response identifier. – *Returns:* `Promise<{ message: "Success.Review ResponseDeleted"; }>` – operation result. – Route: `DELETE /reviews/responses/:responseId`.

4.61.2 ClassAttribute

No	Attribute	Type
01	service	ReviewService – injected application dependency.

4.62 ReviewService Class

4.62.1 ClassMethod

No	Method	Description
01	ReviewService(repository: Review Repository)	A method to initialize the service with its dependencies. – repository: ReviewRepository – constructor-injected dependency. – *Returns:* void – constructor, no return value.
02	requireContract Participant(userId: number, contractId: number)	A method to require contract participant as an internal helper. The implementation delegates to repository. findContract. – userId: number – account identifier used by this operation. – contractId: number – contract identifier. – *Returns:* Promise of the selected database projection / response object – operation result.
03	present(review: T)	A method to present as an internal helper. – review: T – operation parameter used by the implementation. – *Returns:* T & { response?: null | undefined; overallRating: number; } – operation result.
04	list(userId: number, contractId: number)	A method to list. The implementation delegates to repository.findManyByContract. – userId: number – account identifier used by this operation. – contractId: number – contract identifier. – *Returns:* Promise of the selected database projection / response object – operation result.

No	Method	Description
05	`detail( userId: number, reviewId: number)`	A method to detail. The implementation delegates to `repository.findById`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
06	`create( userId: number, contractId: number, input: CreateReviewType)`	A method to create. The implementation delegates to `repository.findByContractAndReviewer`, `repository.create`. – `userId`: `number` – account identifier used by this operation. – `contractId`: `number` – contract identifier. – `input`: `CreateReviewType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.
07	`update( userId: number, reviewId: number, input: UpdateReviewType)`	A method to update. The implementation delegates to `repository.findById`, `repository.update`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – `input`: `UpdateReviewType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.
08	`remove( userId: number, reviewId: number)`	A method to remove. The implementation delegates to `repository.findById`, `repository.softDelete`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – *Returns:* `Promise<{ message: "Success.Review Deleted"; }>` – operation result.

No	Method	Description
09	`respond( userId: number, reviewId: number, responseText: string)`	A method to respond. The implementation delegates to `repository.findById`, `repository.findResponseByReview`, `repository.createResponse`. – `userId`: `number` – account identifier used by this operation. – `reviewId`: `number` – review identifier. – `responseText`: `string` – response content. – *Returns:* `Promise of the selected database projection / response object` – operation result.
10	`updateResponse( userId: number, responseId: number, responseText: string)`	A method to update response. The implementation delegates to `repository.findResponseById`, `repository.updateResponse`. – `userId`: `number` – account identifier used by this operation. – `responseId`: `number` – review response identifier. – `responseText`: `string` – response content. – *Returns:* `Promise of the selected database projection / response object` – operation result.
11	`removeResponse( userId: number, responseId: number)`	A method to remove response. The implementation delegates to `repository.findResponseById`, `repository.softDeleteResponse`. – `userId`: `number` – account identifier used by this operation. – `responseId`: `number` – review response identifier. – *Returns:* `Promise<{ message: "Success.ReviewResponseDeleted"; }>` – operation result.

4.62.2 ClassAttribute

No	Attribute	Type
01	`repository`	`ReviewRepository` – injected persistence gateway.

4.63 ReviewRepository Class

4.63.1 ClassMethod

No	Method	Description
01	`ReviewRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma`: `PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`findContract( contractId: number)`	A method to find contract. – `contractId`: `number` – contract identifier. – *Returns:* `Selected database projection / response object` – operation result.
03	`findById( reviewId: number, includeDeleted: boolean)`	A method to find by id. – `reviewId`: `number` – review identifier. – `includeDeleted`: `boolean` – operation parameter used by the implementation. – *Returns:* `Selected database projection / response object` – operation result.
04	`findByContractAnd Reviewer( contractId: number, reviewerId: number)`	A method to find by contract and reviewer. – `contractId`: `number` – contract identifier. – `reviewerId`: `number` – operation parameter used by the implementation. – *Returns:* `Prisma__ReviewClient<{ id: number; } | null, null, DefaultArgs, PrismaClient Options>` – operation result.
05	`findManyByContract( contractId: number)`	A method to find many by contract. – `contractId`: `number` – contract identifier. – *Returns:* `Selected database projection / response object` – operation result.
06	`create( contractId: number, reviewerId: number, revieweeId: number, reviewerName: string, input: CreateReviewType)`	A method to create. – `contractId`: `number` – contract identifier. – `reviewerId`: `number` – operation parameter used by the implementation. – `revieweeId`: `number` – operation parameter used by the implementation. – `reviewerName`: `string` – operation parameter used by the implementation. – `input`: `CreateReviewType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
07	`update( reviewId: number, input: UpdateReviewType)`	A method to update. – `reviewId`: `number` – review identifier. – `input`: `UpdateReviewType` – validated operation input. – *Returns:* `Selected database projection / response object` – operation result.
08	`softDelete( reviewId: number)`	A method to soft delete. – `reviewId`: `number` – review identifier. – *Returns:* `Selected database projection / response object` – operation result.
09	`findResponseByReview( reviewId: number)`	A method to find response by review. – `reviewId`: `number` – review identifier. – *Returns:* `Prisma__ReviewResponseClient<{ id: number; deletedAt: Date | null; } | null, null, DefaultArgs, PrismaClientOptions>` – operation result.
10	`findResponseById( responseId: number)`	A method to find response by id. – `responseId`: `number` – review response identifier. – *Returns:* `Prisma__ReviewResponseClient<{ id: number; userId: number; reviewId: number; } | null, null, DefaultArgs, PrismaClient Options>` – operation result.
11	`createResponse( reviewId: number, userId: number, reviewerId: number, contractId: number, responderName: string, responseText: string, deletedResponseId?: number)`	A method to create response. – `reviewId`: `number` – review identifier. – `userId`: `number` – account identifier used by this operation. – `reviewerId`: `number` – operation parameter used by the implementation. – `contractId`: `number` – contract identifier. – `responderName`: `string` – operation parameter used by the implementation. – `responseText`: `string` – response content. – `deletedResponseId`: `number` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
12	`updateResponse( responseId: number, responseText: string)`	A method to update response. – `responseId`: `number` – review response identifier. – `responseText`: `string` – response content. – *Returns:* `Selected database projection / response object` – operation result.
13	`softDeleteResponse( responseId: number)`	A method to soft delete response. – `responseId`: `number` – review response identifier. – *Returns:* `Selected database projection / response object` – operation result.

4.63.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.64 GeneralAccountProfileController Class

4.64.1 ClassMethod

No	Method	Description
01	`GeneralAccountProfile Controller( service: AccountProfile Service)`	A method to initialize the controller with its dependencies. – `service`: `AccountProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`get( userId: number)`	A method to get. The implementation delegates to `service.getGeneralProfile`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `GeneralProfileDto` – serialized controller response. – Route: `GET /account-profile`.

No	Method	Description
03	`update( userId: number, body: UpdateGeneral ProfileDto)`	A method to update. The implementation delegates to `service.updateGeneralProfile`. – `userId: number` – account identifier used by this operation. – `body: UpdateGeneralProfileDto` – validated HTTP request body. – *Returns:* `ProfileRevisionSubmissionDto` – serialized controller response. – Route: `PUT /account-profile`.
04	`changePassword( userId: number, body: ChangePasswordDto)`	A method to change password. The implementation delegates to `service.changePassword`. – `userId: number` – account identifier used by this operation. – `body: ChangePasswordDto` – validated HTTP request body. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `PUT /account-profile/password`.
05	`uploadAvatar( userId: number, file?: Express.Multer. File)`	A method to upload avatar. The implementation delegates to `service.uploadAvatar`. – `userId: number` – account identifier used by this operation. – `file: Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `AvatarUploadResponseDto` – serialized controller response. – Route: `POST /account-profile/avatar`.
06	`getAvatarFile( userId: number, response: Response)`	A method to get avatar file. The implementation delegates to `service.getAvatarFile`. – `userId: number` – account identifier used by this operation. – `response: Response` – operation parameter used by the implementation. – *Returns:* `Promise<StreamableFile>` – operation result. – Route: `GET /account-profile/avatar/:userId/file`.

4.64.2 ClassAttribute

No	Attribute	Type
01	`service`	`AccountProfileService` – injected application dependency.

4.65 AccountProfileService Class

4.65.1 ClassMethod

No	Method	Description
01	`AccountProfileService( repository: AccountProfileRepository, profileRevisionService: ProfileRevisionService, cloudinary: CloudinaryService, hashing: HashingService, identityAi: IdentityAiService)`	A method to initialize the service with its dependencies. – `repository`: `AccountProfileRepository` – constructor-injected dependency. – `profileRevisionService`: `ProfileRevisionService` – constructor-injected dependency. – `cloudinary`: `CloudinaryService` – constructor-injected dependency. – `hashing`: `HashingService` – constructor-injected dependency. – `identityAi`: `IdentityAiService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`presentAvatarUrl( userId: number, avatarUrl: string | null, updatedAt?: Date)`	A method to present avatar url as an internal helper. – `userId`: `number` – account identifier used by this operation. – `avatarUrl`: `string | null` – operation parameter used by the implementation. – `updatedAt`: `Date` – operation parameter used by the implementation. – *Returns:* `string | null` – operation result.
03	`detectAvatarType( buffer: Buffer)`	A method to detect avatar type as an internal helper. – `buffer`: `Buffer` – operation parameter used by the implementation. – *Returns:* `{ mimeType: string; extension: string; } | null` – operation result.

No	Method	Description
04	`presentGeneralProfile( user: NonNullable< Awaited<ReturnType< AccountProfileRepository['findGeneralProfile']>> > )`	A method to present general profile as an internal helper. – `user`: `NonNullable< Awaited<ReturnType< AccountProfileRepository['findGeneral Profile']>> >` – operation parameter used by the implementation. – *Returns:* `Selected database projection / response object` – operation result.
05	`getGeneralProfile( userId: number)`	A method to get general profile. The implementation delegates to `repository.findGeneralProfile`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
06	`updateGeneralProfile( userId: number, input: UpdateGeneral ProfileType)`	A method to update general profile. The implementation delegates to `repository.findGeneralProfile`, `profileRevisionService.requiresManualReview`, `profileRevisionService.submitGeneral`, `repository.updateGeneralProfile`, `profile RevisionService.directUpdateResult`. – `userId`: `number` – account identifier used by this operation. – `input`: `UpdateGeneralProfileType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.
07	`changePassword( userId: number, input: ChangePassword Type)`	A method to change password. The implementation delegates to `repository.findPasswordCredential`, `hashing.verify`, `repository.updatePassword`, `hashing.hash`. – `userId`: `number` – account identifier used by this operation. – `input`: `ChangePasswordType` – validated operation input. – *Returns:* `Promise<{ message: string; }>` – operation result.

No	Method	Description
08	`uploadAvatar( userId: number, file?: Express.Multer.File)`	A method to upload avatar. The implementation delegates to `repository.findGeneralProfile`, `cloudinary.isConfigured`, `cloudinary.uploadFile`, `repository.updateAvatar`. – `userId`: `number` – account identifier used by this operation. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `Promise<{ avatarUrl: string; }>` – operation result.
09	`getAvatarFile( userId: number)`	A method to get avatar file. The implementation delegates to `repository.findGeneralProfile`. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise<{ absolutePath: string; fileName: string; contentType: string; }>` – operation result.

4.65.2 ClassAttribute

No	Attribute	Type
01	`repository`	`AccountProfileRepository` – injected persistence gateway.
02	`profileRevisionService`	`ProfileRevisionService` – injected application dependency.
03	`cloudinary`	`CloudinaryService` – injected application dependency.
04	`hashing`	`HashingService` – injected application dependency.
05	`identityAi`	`IdentityAiService` – injected application dependency.

4.66 AccountProfileRepository Class

4.66.1 ClassMethod

No	Method	Description
01	`AccountProfileRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma`: `PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`findGeneralProfile( userId: number)`	A method to find general profile. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Selected database projection / response object` – operation result.
03	`findPasswordCredential( userId: number)`	A method to find password credential. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Prisma__UserClient<{ profile: { id: number; } | null; password: string | null; } | null, null, DefaultArgs, Prisma ClientOptions>` – operation result.
04	`updateGeneralProfile( userId: number, input: UpdateGeneral ProfileType, primaryRole?: string)`	A method to update general profile. – `userId`: `number` – account identifier used by this operation. – `input`: `UpdateGeneralProfileType` – validated operation input. – `primaryRole`: `string` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
05	`updatePassword( userId: number, password: string)`	A method to update password. – `userId`: `number` – account identifier used by this operation. – `password`: `string` – operation parameter used by the implementation. – *Returns:* `Selected database projection / response object` – operation result.

No	Method	Description
06	`updateAvatar( userId: number, avatarUrl: string)`	A method to update avatar. – `userId`: `number` – account identifier used by this operation. – `avatarUrl`: `string` – operation parameter used by the implementation. – *Returns:* `Prisma__ProfileClient<{ avatarUrl: string | null; updatedAt: Date; }, never, DefaultArgs, PrismaClientOptions>` – operation result.

4.66.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.67 BannersAdminController Class

4.67.1 ClassMethod

No	Method	Description
01	`BannersAdminController( service: BannersAdmin Service)`	A method to initialize the controller with the banner administration service. – `service`: `BannersAdminService` – service that owns banner use cases. – *Returns:* `void` – constructor, no return value.
02	`listBanners(query: BannerAdminQueryDto)`	A method to return a filtered, paginated banner list. – `query`: `BannerAdminQueryDto` – page, limit, search, position, active, and sort options. – *Returns:* `Promise<BannerAdminListResponse Type>` – banners and pagination. – Route: `GET /admin/banners`.

No	Method	Description
03	`createBanner(body: CreateBannerBodyDto)`	A method to create an advertisement banner. – `body: CreateBannerBodyDto` – title, image URL, link, position, active state, and dates. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the created banner. – Route: `POST /admin/banners`.
04	`updateBanner(id: number, body: UpdateBannerBody Dto)`	A method to update a banner. – `id: number` – banner identifier. – `body: UpdateBannerBodyDto` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the updated banner. – Route: `PATCH /admin/banners/:id`.
05	`deleteBanner(id: number)`	A method to permanently delete a banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponse Type>` – a success message. – Route: `DELETE /admin/banners/:id`.
06	`uploadImage(file?: Express.Multer.File)`	A method to upload a banner image and return its URL. – `file: Express.Multer.File | undefined` – multipart image field named `file`. – *Returns:* `Promise<{ imageUrl: string; public Id: string }>` – the stored image reference. – Route: `POST /admin/banners/upload-image`; multipart field `file`; maximum 10 MB.
07	`getBannerDetail(id: number)`	A method to return one banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the banner detail. – Route: `GET /admin/banners/:id`.

4.67.2 ClassAttribute

No	Attribute	Type
01	`service`	`BannersAdminService` – private readonly banner administration service.

4.68 BannersAdminService Class

4.68.1 ClassMethod

No	Method	Description
01	`BannersAdminService( repository: BannersAdmin Repository, cloudinaryService: CloudinaryService)`	A method to initialize the service with persistence and image storage. – `repository`: `BannersAdminRepository` – banner persistence dependency. – `cloudinaryService`: `CloudinaryService` – remote image storage. – *Returns:* `void` – constructor, no return value.
02	`uploadImage(file?: Express.Multer.File)`	A method to validate an image and store it on Cloudinary or the local upload directory. – `file`: `Express.Multer.File | undefined` – multipart image. – *Returns:* `Promise<{ imageUrl: string; publicId: string }>` – the stored image reference. – Throws `BannerImageRequiredException`, `BannerImageInvalidTypeException`, `BannerImageTooLargeException`, or `FailedToUploadBannerImageException`.
03	`listBanners(query: BannerAdminQueryType)`	A method to load a banner page. – `query`: `BannerAdminQueryType` – list filters. – *Returns:* `Promise<BannerAdminListResponseType>` – banners and pagination. – Failure is mapped to `FailedToLoadBannerListException`.
04	`getBannerDetail(id: number)`	A method to load one banner. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponseType>` – the banner detail. – Throws `BannerAdminNotFoundException` or `FailedToLoadBannerDetailException`.

No	Method	Description
05	`createBanner(body: BannerCreateBodyType)`	A method to create a banner after validating its position. – `body`: `BannerCreateBodyType` – banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the created banner. – Throws `BannerPositionTakenException` or `Failed ToCreateBannerException`.
06	`updateBanner(id: number, body: BannerUpdateBody Type)`	A method to update a banner after validating its position. – `id`: `number` – banner identifier. – `body`: `BannerUpdateBodyType` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the updated banner. – Throws `BannerAdminNotFoundException`, `Banner PositionTakenException`, or `FailedToUpdate BannerException`.
07	`deleteBanner(id: number)`	A method to permanently delete a banner. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponse Type>` – a success message. – Throws `BannerAdminNotFoundException` or `Failed ToDeleteBannerException`.

4.68.2 ClassAttribute

No	Attribute	Type
01	`repository`	`BannersAdminRepository` – private readonly persistence dependency.
02	`cloudinaryService`	`CloudinaryService` – private readonly image storage dependency.

4.69 BannersAdminRepository Class

4.69.1 ClassMethod

No	Method	Description
01	`BannersAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listBanners(query: BannerAdminQueryType)`	A method to query banners with search, position, active-state, and sorting filters. – `query`: `BannerAdminQueryType` – list filters. – *Returns:* `Promise<BannerAdminListResponse Type>` – banners and pagination.
03	`getBannerDetail(id: number)`	A method to load one banner. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponse Type | null>` – the banner, or `null`.
04	`createBanner(data: BannerCreateBodyType)`	A method to validate the position constraint and insert a banner. – `data`: `BannerCreateBodyType` – banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the created banner. – Throws `BannerPositionTakenException`.
05	`updateBanner(id, data)`	A method to validate the position constraint and update a banner. – `id`: `number` – banner identifier. – `data`: `BannerUpdateBodyType` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the updated banner. – Throws `BannerAdminNotFoundException` or `Banner PositionTakenException`.
06	`deleteBanner(id: number)`	A method to delete a banner and report whether a row was removed. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponse Type>` – a success message. – Throws `BannerAdminNotFoundException`.

4.69.2 ClassAttribute

No	Attribute	Type
01	prisma	PrismaService – private readonly database client.

4.70 SkillsAdminController Class

4.70.1 ClassMethod

No	Method	Description
01	SkillsAdminController(service: SkillsAdmin Service)	A method to initialize the controller with the skill administration service. – service: SkillsAdminService – service that owns skill use cases. – *Returns:* void – constructor, no return value.
02	listSkills(query: Skill AdminQueryDto)	A method to return a filtered, paginated skill list. – query: SkillAdminQueryDto – page, limit, search, active, and sort options. – *Returns:* Promise<SkillAdminListResponseType> – skills and pagination. – Route: GET /admin/skills.
03	createSkill(body: Create SkillBodyDto)	A method to create a skill. – body: CreateSkillBodyDto – name and optional description. – *Returns:* Promise<SkillAdminDetailResponse Type> – the created skill. – Route: POST /admin/skills.
04	updateSkill(id: number, body: UpdateSkillBody Dto)	A method to update a skill. – id: number – skill identifier. – body: UpdateSkillBodyDto – optional name, description, and active state. – *Returns:* Promise<SkillAdminDetailResponse Type> – the updated skill. – Route: PATCH /admin/skills/:id.
05	restoreSkill(id: number)	A method to restore a soft-deleted skill. – id: number – skill identifier. – *Returns:* Promise<SkillAdminDetailResponse Type> – the restored skill. – Route: PATCH /admin/skills/:id/restore.

No	Method	Description
06	`deleteSkill(id: number)`	A method to soft-delete a skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponse Type>` – a success message. – Route: `DELETE /admin/skills/:id`.
07	`getSkillDetail(id: number)`	A method to return one skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the skill detail. – Route: `GET /admin/skills/:id`.

4.70.2 ClassAttribute

No	Attribute	Type
01	`service`	`SkillsAdminService` – private readonly skill administration service.

4.71 SkillsAdminService Class

4.71.1 ClassMethod

No	Method	Description
01	`SkillsAdminService( repository: SkillsAdmin Repository)`	A method to initialize the service with the skill repository. – `repository`: `SkillsAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`listSkills(query: Skill AdminQueryType)`	A method to load a skill page. – `query`: `SkillAdminQueryType` – list filters. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Failure is mapped to `FailedToLoadSkillList Exception`.

No	Method	Description
03	`getSkillDetail(id: number)`	A method to load one skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the skill detail. – Throws `SkillAdminNotFoundException` or `FailedToLoadSkillDetailException`.
04	`createSkill(body: AdminCreateSkillBodyType)`	A method to create a skill and handle a concurrent duplicate. – `body: AdminCreateSkillBodyType` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the created skill. – Throws `SkillNameAlreadyExistsException` or `FailedToCreateSkillException`.
05	`updateSkill(id, body)`	A method to update a skill and handle a concurrent duplicate. – `id: number` – skill identifier. – `body: AdminUpdateSkillBodyType` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the updated skill. – Throws `SkillAdminNotFoundException`, `SkillNameAlreadyExistsException`, or `FailedToUpdateSkillException`.
06	`deleteSkill(id: number)`	A method to soft-delete a skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponseType>` – a success message. – Throws `SkillAdminNotFoundException` or `FailedToDeleteSkillException`.
07	`restoreSkill(id: number)`	A method to restore a soft-deleted skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the restored skill. – Throws `SkillAdminNotFoundException`, `SkillNameAlreadyExistsException`, or `FailedToRestoreSkillException`.

4.71.2 ClassAttribute

No	Attribute	Type
01	`repository`	`SkillsAdminRepository` – private readonly persistence dependency.

4.72 SkillsAdminRepository Class

4.72.1 ClassMethod

No	Method	Description
01	`SkillsAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listSkills(query: SkillAdminQueryType)`	A method to query skills with search, active-state, sorting, and pagination. – `query`: `SkillAdminQueryType` – list filters. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Search matches name, slug, or description.
03	`getSkillDetail(id: number)`	A method to load one skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType | null>` – the skill, or `null`.
04	`createSkill(data: AdminCreateSkillBodyType)`	A method to generate a unique slug and insert a skill. – `data`: `AdminCreateSkillBodyType` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the created skill. – Throws `SkillNameAlreadyExistsException`.

No	Method	Description
05	`updateSkill(id, data)`	A method to update a skill and regenerate its slug when the name changes. – `id`: `number` – skill identifier. – `data`: `AdminUpdateSkillBodyType` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the updated skill. – Throws `SkillAdminNotFoundException` or `Skill NameAlreadyExistsException`.
06	`deleteSkill(id: number)`	A method to set the skill delete timestamp. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponse Type>` – a success message. – Throws `SkillAdminNotFoundException`.
07	`restoreSkill(id: number)`	A method to clear the skill delete timestamp after checking name conflicts. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the restored skill. – Throws `SkillAdminNotFoundException` or `Skill NameAlreadyExistsException`.
08	`generateUniqueSlug(name: string)`	A private method to derive a slug and append a numeric suffix on collision. – `name`: `string` – skill name. – *Returns:* `Promise<string>` – the unique slug.

4.72.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.73 ProfileRevisionController Class

4.73.1 ClassMethod

No	Method	Description
01	`ProfileRevision Controller( service: ProfileRevision Service)`	A method to initialize the controller with its dependencies. – `service`: `ProfileRevisionService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`latest( userId: number, profileType: Profile RevisionType)`	A method to latest. The implementation delegates to `service.latestForUser`. – `userId`: `number` – account identifier used by this operation. – `profileType`: `ProfileRevisionType` – profile role type. – *Returns:* `MyProfileRevisionDto` – serialized controller response. – Route: `GET /profile-revisions/me`.

4.73.2 ClassAttribute

No	Attribute	Type
01	`service`	`ProfileRevisionService` – injected application dependency.

4.74 ProfileRevisionAdminController Class

4.74.1 ClassMethod

No	Method	Description
01	`ProfileRevisionAdmin Controller( service: ProfileRevision Service)`	A method to initialize the controller with its dependencies. – `service`: `ProfileRevisionService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`list( query: ProfileRevision AdminFilterDto)`	A method to list. The implementation delegates to `service.list`. – `query`: `ProfileRevisionAdminFilterDto` – operation parameter used by the implementation. – *Returns:* `ProfileRevisionAdminListDto` – serialized controller response. – Route: `GET /admin/profile-revisions`.

No	Method	Description
03	`detail( id: number)`	A method to detail. The implementation delegates to `service.detail`. – `id`: `number` – target resource identifier. – *Returns:* `ProfileRevisionDetailDto` – serialized controller response. – Route: `GET /admin/profile-revisions/:id`.
04	`approve( id: number, adminId: number, body: ApproveProfile RevisionDto)`	A method to approve. The implementation delegates to `service.approve`. – `id`: `number` – target resource identifier. – `adminId`: `number` – administrator identifier recorded with the review decision. – `body`: `ApproveProfileRevisionDto` – validated HTTP request body. – *Returns:* `ProfileRevisionDetailDto` – serialized controller response. – Route: `PATCH /admin/profile-revisions/:id/ approve`.
05	`reject( id: number, adminId: number, body: RejectProfile RevisionDto)`	A method to reject. The implementation delegates to `service.reject`. – `id`: `number` – target resource identifier. – `adminId`: `number` – administrator identifier recorded with the review decision. – `body`: `RejectProfileRevisionDto` – validated HTTP request body. – *Returns:* `ProfileRevisionDetailDto` – serialized controller response. – Route: `PATCH /admin/profile-revisions/:id/ reject`.

4.74.2 ClassAttribute

No	Attribute	Type
01	`service`	`ProfileRevisionService` – injected application dependency.

4.75 ProfileRevisionService Class

4.75.1 ClassMethod

No	Method	Description
01	`ProfileRevisionService( repository: ProfileRevisionRepository)`	A method to initialize the service with its dependencies. – `repository`: `ProfileRevisionRepository` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`requiresManualReview( profileStrength: number)`	A method to requires manual review. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `boolean` – operation result.
03	`directUpdateResult( profileStrength: number)`	A method to direct update result. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `{ message: "Success.ProfileUpdated"; reviewRequired: boolean; profileStrength: number; revision: null; }` – operation result.
04	`submitFreelancer( userId: number, profileId: number, proposedData: UpdateFreelancerProfileType, profileStrength: number)`	A method to submit freelancer. – `userId`: `number` – account identifier used by this operation. – `profileId`: `number` – target profile identifier. – `proposedData`: `UpdateFreelancerProfileType` – operation parameter used by the implementation. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `Promise of the selected database projection / response object` – operation result.
05	`submitClient( userId: number, profileId: number, proposedData: UpdateClientProfileType, profileStrength: number)`	A method to submit client. – `userId`: `number` – account identifier used by this operation. – `profileId`: `number` – target profile identifier. – `proposedData`: `UpdateClientProfileType` – operation parameter used by the implementation. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
06	`submitExpert( userId: number, profileId: number, proposedData: UpdateExpertProfileType, profileStrength: number)`	A method to submit expert. – `userId`: `number` – account identifier used by this operation. – `profileId`: `number` – target profile identifier. – `proposedData`: `UpdateExpertProfileType` – operation parameter used by the implementation. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `Promise of the selected database projection / response object` – operation result.
07	`submitGeneral( userId: number, profileId: number, primaryRole: string, proposedData: UpdateGeneralProfileType, profileStrength: number)`	A method to submit general. – `userId`: `number` – account identifier used by this operation. – `profileId`: `number` – target profile identifier. – `primaryRole`: `string` – operation parameter used by the implementation. – `proposedData`: `UpdateGeneralProfileType` – operation parameter used by the implementation. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `Promise of the selected database projection / response object` – operation result.
08	`submit( userId: number, profileId: number, profileType: ProfileRevisionType, proposedData: | UpdateFreelancerProfileType | UpdateClientProfileType | UpdateExpertProfileType | UpdateGeneralProfileType, profileStrength: number)`	A method to submit as an internal helper. The implementation delegates to `repository.findProfileForSnapshot`, `repository.submit`. – `userId`: `number` – account identifier used by this operation. – `profileId`: `number` – target profile identifier. – `profileType`: `ProfileRevisionType` – profile role type. – `proposedData`: `| UpdateFreelancerProfileType | UpdateClientProfileType | UpdateExpertProfileType | UpdateGeneralProfileType` – operation parameter used by the implementation. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
09	`latestForUser( userId: number, profileType: ProfileRevisionType)`	A method to latest for user. The implementation delegates to `repository.findLatestForUser`. – `userId`: `number` – account identifier used by this operation. – `profileType`: `ProfileRevisionType` – profile role type. – *Returns:* `Promise of the selected database projection / response object` – operation result.
10	`list( filter: ProfileRevisionAdminFilterType)`	A method to list. The implementation delegates to `repository.syncLowStrengthProfiles`, `repository.findMany`. – `filter`: `ProfileRevisionAdminFilterType` – validated list filters and pagination. – *Returns:* `Promise of the selected database projection / response object` – operation result.
11	`detail( id: number)`	A method to detail. The implementation delegates to `repository.findById`. – `id`: `number` – target resource identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
12	`approve( id: number, adminId: number, reviewNotes?: string | null)`	A method to approve. – `id`: `number` – target resource identifier. – `adminId`: `number` – administrator identifier recorded with the review decision. – `reviewNotes`: `string | null` – administrator review notes or rejection reason. – *Returns:* `Promise of the selected database projection / response object` – operation result.
13	`reject( id: number, adminId: number, reviewNotes: string)`	A method to reject. – `id`: `number` – target resource identifier. – `adminId`: `number` – administrator identifier recorded with the review decision. – `reviewNotes`: `string` – administrator review notes or rejection reason. – *Returns:* `Promise of the selected database projection / response object` – operation result.

No	Method	Description
14	review(id: number, adminId: number, status: typeof RevisionStatus.APPROVED | typeof RevisionStatus.REJECTED, reviewNotes: string | null)	A method to review as an internal helper. The implementation delegates to repository.findById, repository.review. – id: number – target resource identifier. – adminId: number – administrator identifier recorded with the review decision. – status: typeof RevisionStatus.APPROVED | typeof RevisionStatus.REJECTED – operation parameter used by the implementation. – reviewNotes: string | null – administrator review notes or rejection reason. – *Returns:* Promise of the selected database projection / response object – operation result.

4.75.2 ClassAttribute

No	Attribute	Type
01	repository	ProfileRevisionRepository – injected persistence gateway.

4.76 ProfileRevisionRepository Class

4.76.1 ClassMethod

No	Method	Description
01	ProfileRevision Repository(prisma: PrismaService)	A method to initialize the repository with its dependencies. – prisma: PrismaService – constructor-injected dependency. – *Returns:* void – constructor, no return value.
02	findProfileForSnapshot(userId: number, profileId: number)	A method to find profile for snapshot. – userId: number – account identifier used by this operation. – profileId: number – target profile identifier. – *Returns:* Selected database projection / response object – operation result.

No	Method	Description
03	`submit( userId: number, profileId: number, profileType: ProfileRevisionType, currentData: Prisma.InputJsonValue, proposedData: Prisma.InputJsonValue, profileStrength: number)`	A method to submit. – `userId`: `number` – account identifier used by this operation. – `profileId`: `number` – target profile identifier. – `profileType`: `ProfileRevisionType` – profile role type. – `currentData`: `Prisma.InputJsonValue` – operation parameter used by the implementation. – `proposedData`: `Prisma.InputJsonValue` – operation parameter used by the implementation. – `profileStrength`: `number` – calculated profile completion strength. – *Returns:* `Promise of the selected database projection / response object` – operation result.
04	`findLatestForUser( userId: number, profileType: ProfileRevisionType)`	A method to find latest for user. – `userId`: `number` – account identifier used by this operation. – `profileType`: `ProfileRevisionType` – profile role type. – *Returns:* `Selected database projection / response object` – operation result.
05	`syncLowStrengthProfiles( threshold: number)`	A method to sync low strength profiles. – `threshold`: `number` – profile strength threshold. – *Returns:* `Promise<void>` – operation result.
06	`findMany( params: { page: number; limit: number; status?: RevisionStatus; profileType?: ProfileRevisionType; search?: string; } )`	A method to find many. – `params`: `{ page: number; limit: number; status?: RevisionStatus; profileType?: ProfileRevisionType; search?: string; }` – operation parameter used by the implementation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
07	`findById( id: number)`	A method to find by id. – `id`: `number` – target resource identifier. – *Returns:* `Selected database projection / response object` – operation result.

No	Method	Description
08	review(id: number, adminId: number, status: typeof RevisionStatus.APPROVED | typeof RevisionStatus.REJECTED, reviewNotes: string | null, proposedData: FreelancerProposal | ClientProposal | ExpertProposal, profileType: ProfileRevisionType)	A method to review. – id: number – target resource identifier. – adminId: number – administrator identifier recorded with the review decision. – status: typeof RevisionStatus.APPROVED | typeof RevisionStatus.REJECTED – operation parameter used by the implementation. – reviewNotes: string | null – administrator review notes or rejection reason. – proposedData: FreelancerProposal | ClientProposal | ExpertProposal – operation parameter used by the implementation. – profileType: ProfileRevisionType – profile role type. – *Returns:* Promise of the selected database projection / response object – operation result.

4.76.2 ClassAttribute

No	Attribute	Type
01	prisma	PrismaService – injected persistence gateway.

4.77 ExpertProfileController Class

4.77.1 ClassMethod

No	Method	Description
01	ExpertProfileController(service: ExpertProfileService)	A method to initialize the controller with its dependencies. – service: ExpertProfileService – constructor-injected dependency. – *Returns:* void – constructor, no return value.

No	Method	Description
02	`updateMine( userId: number, body: UpdateExpert ProfileDto)`	A method to update mine. The implementation delegates to `service.submitUpdate`. – `userId`: `number` – account identifier used by this operation. – `body`: `UpdateExpertProfileDto` – validated HTTP request body. – *Returns:* `ExpertProfileUpdateResponseDto` – serialized controller response. – Route: `PUT /expert-profile/me`.

4.77.2 ClassAttribute

No	Attribute	Type
01	`service`	`ExpertProfileService` – injected application dependency.

4.78 ExpertProfileService Class

4.78.1 ClassMethod

No	Method	Description
01	`ExpertProfileService( repository: Expert ProfileRepository, revisions: Profile RevisionService)`	A method to initialize the service with its dependencies. – `repository`: `ExpertProfileRepository` – constructor-injected dependency. – `revisions`: `ProfileRevisionService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`cleanList( values: string[])`	A method to clean list as an internal helper. – `values`: `string[]` – operation parameter used by the implementation. – *Returns:* `string[]` – operation result.
03	`sanitize( input: T)`	A method to sanitize as an internal helper. – `input`: `T` – validated operation input. – *Returns:* `T` – operation result.

No	Method	Description
04	`context( userId: number)`	A method to context as an internal helper. The implementation delegates to `repository.findByUserId`. – `userId: number` – account identifier used by this operation. – *Returns:* `Promise of the selected database projection / response object` – operation result.
05	`submitUpdate( userId: number, input: UpdateExpertProfileType)`	A method to submit update. The implementation delegates to `repository.hasPendingRevision`, `revisions.submitExpert`. – `userId: number` – account identifier used by this operation. – `input: UpdateExpertProfileType` – validated operation input. – *Returns:* `Promise of the selected database projection / response object` – operation result.

4.78.2 ClassAttribute

No	Attribute	Type
01	`repository`	`ExpertProfileRepository` – injected persistence gateway.
02	`revisions`	`ProfileRevisionService` – injected application dependency.

4.79 ExpertProfileRepository Class

4.79.1 ClassMethod

No	Method	Description
01	`ExpertProfileRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma: PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`findByUserId( userId: number)`	A method to find by user id. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Selected database projection / response object` – operation result.
03	`hasPendingRevision( userId: number)`	A method to has pending revision. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Prisma__ProfileRevisionClient<{ id: number; } | null, null, DefaultArgs, Prisma ClientOptions>` – operation result.

4.79.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.80 ProfileController Class

4.80.1 ClassMethod

No	Method	Description
01	`ProfileController( profileService: Profile Service)`	A method to initialize the controller with its dependencies. – `profileService`: `ProfileService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`uploadCv( id: number, currentUserId: number, file?: Express.Multer.File)`	A method to upload cv. The implementation delegates to `profileService.uploadCv`. – `id: number` – target resource identifier. – `currentUserId: number` – authenticated user identifier used for ownership checks. – `file: Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `CvUploadResponseDto` – serialized controller response. – Route: `POST /profiles/:id/cv`.
03	`getCvFile( id: number, currentUserId: number, response: Response)`	A method to get cv file. The implementation delegates to `profileService.getCvFile`. – `id: number` – target resource identifier. – `currentUserId: number` – authenticated user identifier used for ownership checks. – `response: Response` – operation parameter used by the implementation. – *Returns:* `Promise<void | StreamableFile>` – operation result. – Route: `GET /profiles/:id/cv/file`.
04	`deleteCv( id: number, currentUserId: number)`	A method to delete cv. The implementation delegates to `profileService.deleteCv`. – `id: number` – target resource identifier. – `currentUserId: number` – authenticated user identifier used for ownership checks. – *Returns:* `Promise<{ message: string; }>` – operation result. – Route: `DELETE /profiles/:id/cv`.

4.80.2 ClassAttribute

No	Attribute	Type
01	`profileService`	`ProfileService` – injected application dependency.

4.81 ProfileService Class

4.81.1 ClassMethod

No	Method	Description
01	`ProfileService( profileRepository: ProfileRepository, cloudinary: CloudinaryService, aiCvAnalyzer: AiCvAnalyzerService, skillResolver: SkillResolverService, profileRevisionService: ProfileRevisionService)`	A method to initialize the service with its dependencies. – `profileRepository`: `ProfileRepository` – constructor-injected dependency. – `cloudinary`: `CloudinaryService` – constructor-injected dependency. – `aiCvAnalyzer`: `AiCvAnalyzerService` – constructor-injected dependency. – `skillResolver`: `SkillResolverService` – constructor-injected dependency. – `profileRevisionService`: `ProfileRevisionService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`uploadCv( profileId: number, userId: number, file?: Express.Multer.File)`	A method to upload cv. The implementation delegates to `cloudinary.isConfigured`, `cloudinary.uploadFile`, `profileRepository.updateCv`. – `profileId`: `number` – target profile identifier. – `userId`: `number` – account identifier used by this operation. – `file`: `Express.Multer.File` – optional multipart file provided by the upload interceptor. – *Returns:* `Promise<{ cvUrl: string | null; cvFileName: string | null; }>` – operation result.
03	`deleteCv( profileId: number, userId: number)`	A method to delete cv. The implementation delegates to `cloudinary.deleteFile`, `profileRepository.clearCv`. – `profileId`: `number` – target profile identifier. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise<{ message: string; }>` – operation result.

No	Method	Description
04	`getCvFile( profileId: number, userId: number)`	A method to get cv file. – `profileId`: `number` – target profile identifier. – `userId`: `number` – account identifier used by this operation. – *Returns:* `Promise<{ remoteUrl: string; fileName: string; absolutePath?: undefined; } | { absolutePath: string; fileName: string; remoteUrl?: undefined; }>` – operation result.
05	`resolveLocalCvPath( cvUrl: string)`	A method to resolve local cv path as an internal helper. – `cvUrl`: `string` – operation parameter used by the implementation. – *Returns:* `string | null` – operation result.

4.81.2 ClassAttribute

No	Attribute	Type
01	`profileRepository`	`ProfileRepository` – injected persistence gateway.
02	`cloudinary`	`CloudinaryService` – injected application dependency.
03	`aiCvAnalyzer`	`AiCvAnalyzerService` – injected application dependency.
04	`skillResolver`	`SkillResolverService` – injected application dependency.
05	`profileRevisionService`	`ProfileRevisionService` – injected application dependency.

4.82 ProfileRepository Class

4.82.1 ClassMethod

No	Method	Description
01	`ProfileRepository( prisma: PrismaService)`	A method to initialize the repository with its dependencies. – `prisma`: `PrismaService` – constructor-injected dependency. – *Returns:* `void` – constructor, no return value.
02	`findFreelancerProfileBy Id( profileId: number)`	A method to find freelancer profile by id. – `profileId`: `number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.
03	`updateCv( profileId: number, data: { cvUrl: string; cvFileName: string; cv PublicId: string | null })`	A method to update cv. – `profileId`: `number` – target profile identifier. – `data`: `{ cvUrl: string; cvFileName: string; cvPublicId: string | null }` – values to persist. – *Returns:* `Selected database projection / response object` – operation result.
04	`clearCv( profileId: number)`	A method to clear cv. – `profileId`: `number` – target profile identifier. – *Returns:* `Promise of the selected database projection / response object` – operation result.

4.82.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – injected persistence gateway.

4.83 AdminExpertProfileController Class

4.83.1 ClassMethod

No	Method	Description
01	`AdminExpertProfile Controller(service: ExpertProfileService)`	A method to initialize the controller with the expert profile service. – `service`: `ExpertProfileService` – service that owns expert profile use cases. – *Returns:* `void` – constructor, no return value.
02	`update( userId: number, adminId: number, body: AdminUpdateExpert ProfileDto)`	A method for an administrator to update an expert profile directly. – `userId`: `number` – expert account identifier. – `adminId`: `number` – current administrator, recorded on superseded revisions. – `body`: `AdminUpdateExpertProfileDto` – display name, bio, title, expertise, experience, education, certifications, website, and active state. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/expert-profile`.

4.83.2 ClassAttribute

No	Attribute	Type
01	`service`	`ExpertProfileService` – private readonly expert profile service.

4.84 ExpertProfileService Class

4.84.1 ClassMethod

No	Method	Description
01	`ExpertProfileService( repository: Expert ProfileRepository, revisions: Profile RevisionService)`	A method to initialize the service with persistence and revision handling. – `repository`: `ExpertProfileRepository` – expert profile persistence. – `revisions`: `ProfileRevisionService` – self-service revision workflow. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`cleanList(values: string[])`	A private method to trim, de-duplicate, and preserve the first casing of a string list. – `values`: `string[]` – raw list values. – *Returns:* `string[]` – the cleaned list.
03	`sanitize(input)`	A private method to clean list fields and normalize the website value. – `input`: `UpdateExpertProfileType` – profile input. – *Returns:* `UpdateExpertProfileType` – the sanitized input.
04	`context(userId: number)`	A private method to load an expert account and verify its Expert role. – `userId`: `number` – target account. – *Returns:* `Promise<user context>` – account, profile, and roles. – Throws `ExpertProfileNotFoundException` or `NotAnExpertAccountException`.
05	`getMine(userId: number)`	A method to return the authenticated expert's profile. – `userId`: `number` – current expert account. – *Returns:* `Promise<ExpertProfileType>` – profile and expert fields. – Used by the self-service profile page, not by the admin endpoint.
06	`submitUpdate(userId, input)`	A method to submit a self-service expert profile revision. – `userId`: `number` – current expert account. – `input`: `UpdateExpertProfileType` – requested profile changes. – *Returns:* `Promise<revision result>` – the pending revision. – Throws `ExpertProfilePendingRevisionException` when a revision is already pending.

No	Method	Description
07	`updateByAdmin(userId, adminId, input)`	A method to update an expert profile directly and supersede pending revisions. – `userId`: `number` – target expert account. – `adminId`: `number` – current administrator. – `input`: `AdminUpdateExpertProfileType` – administrative profile changes. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `ExpertProfileNotFoundException` or `NotAnExpertAccountException`.

4.84.2 ClassAttribute

No	Attribute	Type
01	`repository`	`ExpertProfileRepository` – private readonly persistence dependency.
02	`revisions`	`ProfileRevisionService` – private readonly revision workflow dependency.

4.85 ExpertProfileRepository Class

4.85.1 ClassMethod

No	Method	Description
01	`ExpertProfileRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findByUserId(userId: number)`	A method to load an expert account with its profile and roles. – `userId`: `number` – target account. – *Returns:* `Promise<user context | null>` – the account graph, or `null`.

No	Method	Description
03	`updateDirect(userId, adminId, input, profileCompletionPercent)`	A method to supersede pending revisions, update the profile, and upsert the expert row in one transaction. – `userId`: `number` – target expert account. – `adminId`: `number` – reviewing administrator. – `input`: `AdminUpdateExpertProfileType` – sanitized profile changes. – `profileCompletionPercent`: `number` – recalculated completeness. – *Returns:* `Promise<void>` – no value. – Creates a system notification when a pending revision was superseded.
04	`hasPendingRevision(userId: number)`	A method to check for a pending expert revision. – `userId`: `number` – target account. – *Returns:* `Promise<{ id: number } | null>` – the pending revision, or `null`.
05	`findPublicMany(query: PublicExpertQueryType)`	A method to query active public expert profiles. – `query`: `PublicExpertQueryType` – expertise, search, and pagination filters. – *Returns:* `Promise<public expert page>` – public expert cards.
06	`findPublicById(id: number)`	A method to load one active public expert profile. – `id`: `number` – expert identifier. – *Returns:* `Promise<public expert | null>` – the public profile, or `null`.

4.85.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.86 JobCategoriesAdminController Class

4.86.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdmin Controller(service: Job CategoriesAdminService)`	A method to initialize the controller with the job category service. – `service`: `JobCategoriesAdminService` – service that owns job category use cases. – *Returns:* `void` – constructor, no return value.
02	`listJobCategories(query: JobCategoryAdminQueryDto)`	A method to return a filtered, paginated job category list. – `query`: `JobCategoryAdminQueryDto` – page, limit, search, status, deleted, and sort options. – *Returns:* `Promise<JobCategoryAdminList ResponseType>` – categories and pagination. – Route: `GET /admin/job-categories`.
03	`createJobCategory(body: CreateJobCategoryBodyDto)`	A method to create a job category. – `body`: `CreateJobCategoryBodyDto` – name, description, icon, and status. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the created category. – Route: `POST /admin/job-categories`.
04	`updateJobCategory(id: number, body: UpdateJob CategoryBodyDto)`	A method to update a job category. – `id`: `number` – category identifier. – `body`: `UpdateJobCategoryBodyDto` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the updated category. – Route: `PATCH /admin/job-categories/:id`.
05	`restoreJobCategory(id: number)`	A method to restore a soft-deleted job category. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the restored category. – Route: `PATCH /admin/job-categories/:id/ restore`.
06	`deleteJobCategory(id: number)`	A method to soft-delete a job category that has no active jobs. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDelete ResponseType>` – a success message. – Route: `DELETE /admin/job-categories/:id`.

No	Method	Description
07	`getJobCategoryDetail(id: number)`	A method to return one job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the category detail. – Route: `GET /admin/job-categories/:id`.

4.86.2 ClassAttribute

No	Attribute	Type
01	`service`	`JobCategoriesAdminService` – private readonly job category service.

4.87 JobCategoriesAdminService Class

4.87.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdminService( repository: JobCategoriesAdminRepository)`	A method to initialize the service with the job category repository. – `repository: JobCategoriesAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`listJobCategories(query: JobCategoryAdminQueryType)`	A method to load a job category page. – `query: JobCategoryAdminQueryType` – list filters. – *Returns:* `Promise<JobCategoryAdminListResponseType>` – categories and pagination. – Failure is mapped to `FailedToLoadJobCategoryListException`.
03	`getJobCategoryDetail(id: number)`	A method to load one job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the category detail. – Throws `JobCategoryAdminNotFoundException` or `FailedToLoadJobCategoryDetailException`.

No	Method	Description
04	`createJobCategory(body: AdminCreateJobCategory BodyType)`	A method to create a category and handle a concurrent duplicate. – `body: AdminCreateJobCategoryBodyType` – category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the created category. – Throws `JobCategoryNameAlreadyExists Exception` or `FailedToCreateJobCategory Exception`.
05	`updateJobCategory(id, body)`	A method to update a category and handle a concurrent duplicate. – `id: number` – category identifier. – `body: AdminUpdateJobCategoryBodyType` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the updated category. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryNameAlreadyExistsException`, or `FailedToUpdateJobCategoryException`.
06	`deleteJobCategory(id: number)`	A method to soft-delete a category after checking active job references. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDelete ResponseType>` – a success message. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryHasJobsException`, or `FailedToDelete JobCategoryException`.
07	`restoreJobCategory(id: number)`	A method to restore a soft-deleted category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the restored category. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryNameAlreadyExistsException`, or `FailedToRestoreJobCategoryException`.

4.87.2 ClassAttribute

No	Attribute	Type
01	`repository`	`JobCategoriesAdminRepository` – private readonly persistence dependency.

4.88 JobCategoriesAdminRepository Class

4.88.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdmin Repository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listJobCategories(query: JobCategoryAdminQuery Type)`	A method to query categories with search, status, deleted-state, sorting, and pagination. – `query`: `JobCategoryAdminQueryType` – list filters. – *Returns:* `Promise<JobCategoryAdminList ResponseType>` – categories and pagination. – Search matches name, slug, or description.
03	`getJobCategoryDetail(id: number)`	A method to load one category with its job count. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType | null>` – the category, or `null`.
04	`createJobCategory(data)`	A method to generate a unique slug and insert a category. – `data`: `AdminCreateJobCategoryBodyType` – category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the created category. – Throws `JobCategoryNameAlreadyExists Exception`.

No	Method	Description
05	`updateJobCategory(id, data)`	A method to update a category and regenerate its slug when the name changes. – `id`: `number` – category identifier. – `data`: `AdminUpdateJobCategoryBodyType` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the updated category. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryNameAlreadyExistsException`.
06	`deleteJobCategory(id: number)`	A method to check active job references and set the delete timestamp. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDeleteResponseType>` – a success message. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryHasJobsException`.
07	`restoreJobCategory(id: number)`	A method to clear the delete timestamp after checking name conflicts. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the restored category. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryNameAlreadyExistsException`.
08	`generateUniqueSlug(name: string)`	A private method to derive a slug and append a numeric suffix on collision. – `name`: `string` – category name. – *Returns:* `Promise<string>` – the unique slug.

4.88.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

5 Database Design

5.1 Profile, Account and Review Persistence – Nguyen Lu Nhat Huy

The following model fields and constraints support the five sprint modules; names are taken from the Prisma schema. Related global database diagrams remain in the preceding sections.

5.1.1 Profile

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
userId	Int @unique
displayName	String? @db.VarChar(255)
avatarUrl	String? @db.VarChar(500)
coverUrl	String? @db.VarChar(500)
bio	String?
onlineStatus	Boolean @default(false)
availabilityStatus	AvailabilityStatus @default(OFFLINE)
profileCompletionPercent	Int@default(0)
walletAddress	String? @db.VarChar(42)
createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt
user	User @relation(fields: [userId], references: [id], onDelete: Restrict, onUpdate: NoAction)
freelancerProfile	FreelancerProfile?
clientProfile	ClientProfile?
expertProfile	Expert?
socialLinks	SocialLink[]
revisions	ProfileRevision[]

5.1.2 FreelancerProfile

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
profileId	Int @unique
title	String? @db.VarChar(255)

education	String[] @default([])
certifications	String[] @default([])
languages	String[] @default([])
cvUrl	String? @db.VarChar(500)
cvFileName	String? @db.VarChar(255)
cvPublicId	String? @db.VarChar(255)
cvUploadedAt	DateTime?
experience	String[] @default([])
idVerified	Boolean @default(false)
createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt
profile	Profile @relation(fields: [profileId], references: [id], onDelete: Restrict, onUpdate: NoAction)
skills	FreelancerSkill[]
portfolioItems	PortfolioItem[]

5.1.3 FreelancerSkill

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
freelancerProfileId	Int
skillId	Int
proficiencyLevel	Int @db.SmallInt // Range 1-10.
freelancerProfile	FreelancerProfile @relation(fields: [freelancerProfileId], references: [id], onDelete: Restrict, onUpdate: NoAction)
skill	Skill @relation(fields: [skillId], references: [id], onDelete: Restrict, onUpdate: NoAction)
@@unique([freelancerProfileId, skillId])	
@@index([freelancerProfileId])	
@@index([skillId])	

5.1.4 PortfolioItem

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
freelancerProfileId	Int

title	String @db.VarChar(255)
description	String?
technologies	String[] @default([])
projectUrl	String? @db.VarChar(500)
createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt
deletedAt	DateTime?
freelancerProfile	FreelancerProfile @relation(fields: [freelancerProfileId], references: [id], onDelete: Restrict, onUpdate: NoAction)
likes	PortfolioLike[]

5.1.5 ClientProfile

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
profileId	Int @unique
companyName	String? @db.VarChar(255)
companyDescription	String?
website	String? @db.VarChar(255)
createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt
profile	Profile @relation(fields: [profileId], references: [id], onDelete: Restrict, onUpdate: NoAction)

5.1.6 Expert

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
profileId	Int @unique
title	String? @db.VarChar(255)
expertise	String[] @default([])
yearsOfExperience	Int @default(0)
education	String[] @default([])
certifications	String[] @default([])
website	String? @db.VarChar(500)
isActive	Boolean @default(false)

createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt
profile	Profile @relation(fields: [profileId], references: [id], onDelete: Restrict, onUpdate: NoAction)
@@index([isActive])	

5.1.7 SocialLink

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
profileId	Int
platform	SocialPlatform
url	String @db.VarChar(500)
profile	Profile @relation(fields: [profileId], references: [id], onDelete: Restrict, onUpdate: NoAction)

5.1.8 FavoriteFreelancer

Field / constraint	Type and mapping
clientId	Int
freelancerId	Int
createdAt	DateTime @default(now())
client	User @relation("FavoriteClient", fields: [clientId], references: [id], onDelete: Cascade, onUpdate: NoAction)
freelancer	User @relation("FavoriteTargetFreelancer", fields: [freelancerId], references: [id], onDelete: Cascade, onUpdate: NoAction)
@@id([clientId,	freelancerId])
@@index([freelancerId])	

5.1.9 FollowFreelancer

Field / constraint	Type and mapping
clientId	Int
freelancerId	Int
createdAt	DateTime @default(now())

client	User @relation("FollowClient", fields: [clientId], references: [id], onDelete: Cascade, onUpdate: NoAction)
freelancer	User @relation("FollowTargetFreelancer", fields: [freelancerId], references: [id], onDelete: Cascade, onUpdate: NoAction)
@@id([clientId,	freelancerId])
@@index([freelancer	Id])

5.1.10 Notification

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
userId	Int
type	NotificationType
title	String? @db.VarChar(255)
message	String?
data	Json?
isRead	Boolean @default(false)
createdAt	DateTime @default(now())
deletedAt	DateTime?
user	User @relation(fields: [userId], references: [id], onDelete: Restrict, onUpdate: NoAction)
@@index([userId,	isRead])
@@index([createdA	t])

5.1.11 Review

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
contractId	Int
reviewerId	Int
revieweeId	Int
overallRating	Decimal @db.Decimal(2, 1)
breakdown	Json?
comment	String?
createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt

deletedAt	DateTime?
contract	Contract @relation(fields: [contractId], references: [id], onDelete: Restrict, onUpdate: NoAction)
reviewer	User @relation("ReviewReviewer", fields: [reviewerId], references: [id], onDelete: Restrict, onUpdate: NoAction)
reviewee	User @relation("ReviewReviewee", fields: [revieweeId], references: [id], onDelete: Restrict, onUpdate: NoAction)
response	ReviewResponse?
@@unique([contractId, reviewerId])	
@@index([revieweeId])	
@@index([reviewerId])	

5.1.12 ReviewResponse

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
reviewId	Int @unique
userId	Int
responseText	String
createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt
deletedAt	DateTime?
review	Review @relation(fields: [reviewId], references: [id], onDelete: Restrict, onUpdate: NoAction)
user	User @relation(fields: [userId], references: [id], onDelete: Restrict, onUpdate: NoAction)

5.1.13 ProfileRevision

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
userId	Int
profileId	Int
profileType	ProfileRevisionType
status	RevisionStatus @default(PENDING)
currentData	Json
proposedData	Json

profileStrength	Int
adminId	Int?
reviewNotes	String?
createdAt	DateTime @default(now())
updatedAt	DateTime @updatedAt
reviewedAt	DateTime?
user	User @relation("ProfileRevisionUser", fields: [userId], references: [id], onDelete: Restrict, onUpdate: NoAction)
admin	User? @relation("ProfileRevisionAdmin", fields: [adminId], references: [id], onDelete: SetNull, onUpdate: NoAction)
profile	Profile @relation(fields: [profileId], references: [id], onDelete: Restrict, onUpdate: NoAction)
@@index([userId,	profileType, status])
@@index([status,	createdAt])
@@index([profileId])	

5.1.14 IdVerificationDocument

Field / constraint	Type and mapping
id	Int @id @default(autoincrement())
userId	Int
documentType	DocumentType
fileUrl	String @db.VarChar(500)
status	VerificationStatus @default(PENDING)
adminId	Int?
reviewNotes	String?
createdAt	DateTime @default(now())
reviewedAt	DateTime?
deletedAt	DateTime?
aiStatus	String? @db.VarChar(20)
aiScore	Float?
aiSummary	Json?
aiSelfieSummary	Json?
aiError	String?
aiScannedAt	DateTime?
backFileUrl	String? @db.VarChar(500)
identityNumber	String? @db.VarChar(50)

duplicateOfId	Int?
user	User @relation("IdVerificationUser", fields: [userId], references: [id], onDelete: Restrict, onUpdate: NoAction)
admin	User? @relation("IdVerificationAdmin", fields: [adminId], references: [id], onDelete: SetNull, onUpdate: NoAction)
duplicateOf	IdVerificationDocument? @relation("IdVerificationDuplicate", fields: [duplicateOfId], references: [id], onDelete: SetNull, onUpdate: NoAction)
duplicates	IdVerificationDocument[] @relation("IdVerificationDuplicate")
@@index([identityNumber])	

5.1.15 UserRole

Field / constraint	Type and mapping
userId	Int
roleId	Int
isPrimary	Boolean @default(false)
createdAt	DateTime @default(now())
user	User @relation(fields: [userId], references: [id], onDelete: Cascade, onUpdate: NoAction)
role	Role @relation(fields: [roleId], references: [id], onDelete: Cascade, onUpdate: NoAction)
@@id([userId,	roleId])